M5 hierarchical retail forecasting: blending the winner's LightGBM with zero-shot foundation models

Full project report: code, data, methods, every result, interpretations and the reasons behind each decision

Student	Alif Bagas Adiutomo, Department of Industrial Engineering and Management, Yuan Ze University
Advisor	Dr. Yogi Tri Prasetyo
Report date	24 October 2026 (deadline about 29 October 2026)
Repository	`m5-foundation-blend` (this report is `docs/REPORT.md`; the PDF with all code and raw results is `docs/REPORT_FULL_with_code_and_results.pdf`)
Status	Main experiments finished and checked. One overnight job on PC 2 (tuned shares, top-down alignment, new Chronos-2 variants) was launched on 21 October; its result files had not come back when this report was written, so they are described but not reported (Section 9).

0. The result in one paragraph

We rebuilt the winning M5 solution (six LightGBM model sets, averaged) on six forecast windows. We then replaced 20 percent of its forecast at the nine aggregate levels of the hierarchy with a zero-shot Chronos-2 forecast made directly on the summed series. The rule was fixed before the final window was scored. The blend scored better than the winner alone on all six windows, with a mean gain of 0.0129 WRMSSE (95 percent interval 0.0072 to 0.0182, sign test p = 0.031). On the competition's final window it moves our reproduction from 0.5228 to 0.5195, which is below the official winning score of 0.5204. That last gap is smaller than the noise in the test, so the safe claim is that the blend beats **our reproduction** consistently, and matches or slightly improves on the official winner. **We cannot say that we beat the winner with statistical confidence, and this report never claims it.**

How to read this report

Sections 1 to 4 explain the problem, the data, the score and the machines. Section 5 tells the story of the project in order and gives the reason for every decision. Section 6 explains each method. Section 7 gives every result with an interpretation. Section 8 states what is and is not proven. Section 9 lists limits, ideas that were considered, and work still open. Section 10 lists the literature we checked. Section 11 explains how to rerun everything from this repository. The appendices hold level-by-level tables, a file inventory, a glossary and our working rules. In the PDF version, the full source code and every raw result file follow as further appendices.

Every number comes from a saved result file in `results/`. Where a number comes from an estimate or from a log line rather than a file, the text says so.

1. Project goal and targets

The M5 Accuracy competition (Walmart sales, run on Kaggle in 2020) asked for 28-day sales forecasts for 42,840 related time series. Gradient-boosted trees did best, and the first-place solution by YeonJun In used LightGBM (Makridakis, Spiliotis and Assimakopoulos, 2022b; Januschowski et al., 2022). Since then, pretrained time series foundation models have appeared that forecast new data with no training on it (Ansari et al., 2024; Das et al., 2024; Ansari et al., 2025). A 2025 workshop preprint by Yang, Cao and Liu (arXiv 2507.22053, not peer reviewed) reports that an ensemble of LightGBM and PatchTST reaches 0.4989 on the M5 task, against 0.5230 for their LightGBM reference.

The research question is: **can newer models, especially pretrained foundation models used zero-shot, improve on the M5 winner without retraining the winner?**

Three targets were agreed, in this order of priority:

Target	WRMSSE	Meaning
Official winner	0.5204	The published winning score on the final window. Primary target.
Our reproduction	0.5228	The winner's own code run by us under the same protocol. The fair comparison: a blend must beat this, because it removes machine and library differences.
Stretch (Yang et al.)	0.4989	A preprint result. Stretch goal only, since it is not peer reviewed and its protocol may differ.

Why the reproduction differs from the official score. The official 0.5204 comes from the competition run. Our run uses the same code and settings but a different machine, library versions and compatibility patches, so a gap of 0.0024 is expected.

When we compare a blend to a winner, the fair comparison is always winner (our run) against blend (built on the same winner run).

2. The M5 data and how it is organised

The organiser's release has four tables. We used the full release, which includes the evaluation days that Kaggle hid at the time.

File	Rows	What it holds
`sales_train_evaluation.csv`	30,490 (one per item in one store)	Identifier columns (id, item_id, dept_id, cat_id, store_id, state_id) and daily unit sales in columns d_1 to d_1941.
`sales_test_evaluation.csv`	30,490	The 28 final days d_1942 to d_1969. Appending these gives 1,969 days. Final-window scoring needs this file.
`calendar.csv`	1,969 days	Date, weekday, month, year, week id (`wm_yr_wk`), up to two event names and types per day, and SNAP flags for CA, TX and WI (days when food-stamp purchases are allowed).
`sell_prices.csv`	about 6.8 million	Weekly price of each item in each store. A missing price means the item was not on sale that week.

The data are not included in this repository (see `data/README.md`).

The hierarchy

Sales are nested: an item sold in a store belongs to a department, a category, a state, and the whole company. M5 scores forecasts at twelve levels at once. The 30,490 item-store series are the bottom level (level 12). Every other level is a sum of bottom series.

Level	Name	Series	Made by summing
1	Total	1	All series
2	State	3	Stores in a state (CA, TX, WI)
3	Store	10	Each of 10 stores
4	Category	3	Each category (FOODS, HOBBIES, HOUSEHOLD)
5	Department	7	Each of 7 departments
6	State x Category	9	State by category
7	State x Department	21	State by department
8	Store x Category	30	Store by category
9	Store x Department	70	Store by department
10	Item	3,049	One item across all stores
11	Item x State	9,147	One item across a state
12	Item x Store	30,490	The raw bottom series

Total: 1 + 3 + 10 + 3 + 7 + 9 + 21 + 30 + 70 + 3,049 + 9,147 + 30,490 = 42,840 series.

Why this matters for the method. The top levels (1 to 9, only 154 series) are smooth, high-volume series. The bottom levels (10 to 12) are noisy and often zero. A model that works well at the top may do nothing at the bottom, and the score weights every level equally, so the bottom three levels, which are the hardest, make up a quarter of the score. In the final window the winner scores 0.20 to 0.57 at levels 1 to 9 and 0.88 to 0.97 at levels 10 to 12.

Time axis and the six windows

The data covers d_1 to d_1969. A forecast window is 28 days long. A model sees only days up to its origin and forecasts the next 28 days. The first four origins (cv5, cv2, cv3, cv4) are validation folds that the winner's own code defines, and the last two are the competition's public and final windows, so each window is exactly what the winner's code produces when trained up to that day.

Window	Trained on days up to	Forecast days	Role
cv5	d_1801	d_1802 to d_1829	Extra evidence window
cv2	d_1829	d_1830 to d_1857	Extra evidence window
cv3	d_1857	d_1858 to d_1885	Extra evidence window
cv4	d_1885	d_1886 to d_1913	Validation fold used earlier for design checks
public	d_1913	d_1914 to d_1941	The competition's public leaderboard window
final	d_1941	d_1942 to d_1969	The competition's final (private) window. Scored only after the rule was fixed.

The windows do not overlap in their forecast days, but each one trains on mostly the same history, so they are not fully independent. We say this wherever we count windows.

3. The score (WRMSSE) and how we checked our scorer

What the score measures. WRMSSE is the weighted root mean squared scaled error. For each of the 42,840 series the squared forecast error over the 28 days is divided by the series' typical day-to-day change in training (its scale, computed from the first non-zero sale onward), and its square root gives the RMSSE. Each series gets a weight equal to its share of dollar sales (units times price) in the last 28 training days, and weights add to one inside each level. A level's score is the weighted sum of its RMSSE values. The overall score is the plain average of the twelve level scores. Lower is better. A score near 1 is about as good as a simple naive forecast.

Why scale and weights matter. Dividing by each series' own scale puts slow and fast sellers on one footing, and weighting by dollars puts the focus on products that earn money. Because every level counts equally, the item-level scores (levels 10 to 12), which are hard and large, weigh as much as the total (level 1), which is easy.

Checking our scorer. We wrote our own evaluator (`src/m5_data_and_wrmsse.py`). On the final window the seasonal naive baseline (repeat the last week) scores 0.8470, matching the official value at all twelve levels. Our scorer also reproduces the winner's 0.5228 exactly from the saved forecast file, and the public-window figure 0.6415. Sanity checks built into the script (weights sum to one at each level, a perfect forecast scores zero, 42,840 series across 12 levels) all pass.

4. Machines, folders and workflow

Three computers shared the work. The cloud workspace had no GPU and could not hold the 1,969-day file for the final window, so final-window scoring and every heavy run happened on the user's machines. Files moved by zip.

Machine	Hardware	Used for
PC 1	Holds the original data and early LightGBM results	Data preparation and the first experiments
PC 2	NVIDIA RTX 3090 GPU, Python virtual environment	Chronos-2 and the other foundation models, PatchTST, final-window scoring, bootstrap analysis
PC 3	Intel i7-13700, 24 threads, 32 GB RAM, CPU only	Training the winner's LightGBM models (about 11 hours per window, 220 model files each) and SHAP

Workflow rules we kept. Each job was a single script with plain-text output. Rules for every job: do not delete files, do not change model settings, do not send anything anywhere, copy results back under new names. Heavy runs ran outside OneDrive. Experiments on the winner's code kept its settings unchanged; only compatibility patches (library versions, the window key) were applied and logged in the patch logs in `results/02_winner_reproduction/`.

Compute spent. Winner training: the final window, the public window and cv4 first, then cv3, cv2 and cv5, about 11.0, 11.0 and 10.8 hours on PC 3 respectively. Foundation-model runs take under two minutes per window each on the GPU for the 154 aggregate series.

5. The project in order, and why each decision was made

This section is the logic of the project. For each step it says what we did, what came out, and why the next step followed. Numbers in this section are from the result files named in each step.

5.1 Step 1: get the scorer right (before any model)

What. We wrote the 12-level WRMSSE evaluator and tested it on synthetic data and on the real data with the naive and seasonal naive baselines (`results/01_baselines_and_early_models/results_phase1_baselines.txt`). On the public window the naive forecast scores 1.4639 and the seasonal naive 0.8697. On the final window the seasonal naive scores 0.8470, which matches the official value at every level.

Why. Every later comparison is only as good as the scorer. If the scorer were wrong by a small amount, a blend that "beats" the winner could be a scoring artefact. Matching a published benchmark to four digits at all twelve levels removes that doubt.

5.2 Step 2: our own LightGBM, to learn what matters (E1 and E1b)

What. We trained a single global LightGBM model (Tweedie loss) on weekly forecast origins and added feature groups one at a time. On the public window (`results_E1_lightgbm.txt`, `results_E1b_lightgbm.txt`):

Step	Features	WRMSSE	Change
L0 base (lags, rolling means, calendar)	34	0.6141	
L1 + price	38	0.5779	-0.0362
L2 + event	43	0.5597	-0.0183
L3 + context (group level averages)	49	0.5422	-0.0174
L4 + sample weights	49	0.5888	+0.0465

Seasonal naive on the same window scored 0.8697. Price, events and group context each helped; sample weights hurt. **L4 was declared the final model in advance (0.5888). L0 to L3 are ablation rows scored on the holdout for information only, so 0.5422 must not be quoted as a result.** Gain was dominated by the 28-day mean features (68 percent of gain at L0; the non-zero mean at 51 percent in L4).

Why we did it. It told us which kinds of information help a tree model on this data, and it gave a first working reference before we spent days of compute on the winner's code. **Why we did not stop here.** Our own model is not the competition winner, and the goal is to improve on the winner. A claim that a foundation model helps *our* LightGBM would not say anything about the M5 winner. So the comparison had to be against the winner's own code.

5.3 Step 3: zero-shot foundation models alone (Chronos-2, PatchTST)

What. We forecast every one of the 30,490 bottom series with Chronos-2 (zero-shot) in three variants and trained PatchTST on M5. On the public window (`results_chronos2_public_window.txt`, `results_patchtst_public_window.txt`):

Model	WRMSSE	Reading
Seasonal naive	0.8697	Baseline
Chronos-2 A, history only	0.7501	Zero-shot, no covariates
Chronos-2 B, with covariates	0.6991	Covariates (weekday, event, SNAP, price) help by 0.051
Chronos-2 C, cross-learning (best grouping: item)	0.7046	Related series forecast together; no gain over B
PatchTST P0, history only	0.8962	Trained on M5; worse than seasonal naive
PatchTST P1, with covariates	0.7460	Covariates help by 0.15

On the final window, bottom-up Chronos-2 scored 0.6320 and PatchTST with covariates 0.6613 (`results_*_final_window.txt`), against 0.5228 for the winner.

Why. We needed to know whether foundation models can compete on their own. They cannot: even the best is far behind a tuned tree ensemble, in line with the M5 conclusions. It also showed that covariates are the main driver for both model types, which is why every later foundation-model run uses covariates.

5.4 Step 4: the key observation, direct aggregates instead of bottom-up sums

What. Looking at the levels, bottom-up Chronos-2 was weak at the top of the hierarchy (level 1 about 0.37 against 0.20 for the winner). The reason is that item-level errors are large and do not cancel when added up. We therefore forecast the **summed series directly**: the 154 aggregate series of levels 1 to 9, each given to Chronos-2 with its own covariates. Results over levels 1 to 9 on the final window (`results/03_chronos2_aggregate/results_aggregate.txt`, mean over the nine levels): winner 0.388, direct Chronos-2 B 0.480, bottom-up Chronos-2 0.525, bottom-up PatchTST 0.564.

Why. At the top of the hierarchy the series are smooth and high-volume, which is where a pretrained model is at its best. Forecasting there closes about half of the gap. Direct B (with covariates) was best in the three early windows; direct A (no covariates) was worse, and the bottom-up versions swung widely between windows. A stable candidate was what we needed for a blend.

5.5 Step 5: rebuild the winner (the hard part, 11 hours per window)

What. We ran the M5 first-place code (folder A1 of the M5 methods repository) on PC 3, first for the final window, the public window and cv4, later for cv3, cv2 and cv5. Only compatibility patches were applied (library versions, window selection), all listed in `PATCH_LOG_*.txt`. The result was 0.5228 on the final window against 0.5204 official.

Why. Three reasons. First, the question is about improving the winner, so we need the winner's forecasts. Second, using the winner's own code under the same protocol is the fair baseline: a blend must beat *this* run. Third, running six windows, not one, is what allows an honest statement about consistency (Section 7.3). One window can be luck. Four extra windows cost about 33 hours of compute and turned a one-window anecdote into a six-window test.

5.6 Step 6: fix the blend rule before looking at the final window

What. We named the headline rule in advance: 20 percent of direct Chronos-2 B (mean point forecast, covariates, context 1,024) mixed into the winner at levels 1 to 9; levels 10 to 12 stay as the winner forecast them.

Why each part of the rule.

- *Direct forecasts.* From step 4: stable at the top of the hierarchy.
- *Levels 1 to 9 only.* The foundation model's level scores at the item levels were no better than the winner's (Section 7.8), and the aggregates are where the winner's bias shows (Section 7.5).
- *Share 0.20.* A modest round value. A foundation model that is weaker alone should get a minority share. It was fixed before the final window and never tuned on it. Other shares are reported only as sensitivity (Section 7.6).
- *Covariates, mean point forecast.* Covariates matter (Section 5.3). WRMSSE is a squared-error measure, so the mean minimises it; the median is reported and is nearly the same.
- *Context 1,024 days.* Long enough to cover about three years of seasonality, short enough to stay cheap. Fixed, never changed between windows.
- *Pre-declaration.* More variants raise the chance of a lucky win. Naming the rule first is what makes the result honest.

5.7 Step 7: test the rule, then try to break it

We scored all six windows (Section 7.3), tested the final window against the official number with a bootstrap (7.4), explained where the gain comes from (7.5), and then ran extra checks: ablations (7.6), a share sweep (7.6), reconciliation (7.7), item-level forecasts at levels 10 and 11 (7.8), four other foundation models (7.9) and SHAP on the winner (7.10). **Why:** a single positive result invites the questions "is it just Chronos-2?", "is it just this window?", "does it depend on the share?", "is it explained by anything?". Each extra run answers one of those.

5.8 Decision log (summary)

Decision	Alternative	Reason
Score with our own evaluator, checked against the official seasonal naive	Trust a library	Every result depends on it; matching 0.8470 at 12 levels removes doubt
Compare against the reproduced winner, not only the official number	Compare only to 0.5204	Removes machine and library differences from the comparison
Six windows	Final window only	One window cannot show consistency; the winner's bias changes from window to window
Direct forecasts of aggregates	Bottom-up sums	Item errors do not cancel; bottom-up was unstable across windows
Covariates on	Univariate	Gained 0.05 (Chronos-2) and 0.15 (PatchTST) on the public window
Mean point forecast	Median	Squared-error metric; the median result is nearly the same
Levels 1 to 9 only	All levels	No evidence of gain at item levels in the first design; see 7.8 for the later test
Fixed 20 percent share	Tuned share	Tuning on the test window would make the result untrustworthy
Chronos-2 as headline	TimesFM 3.0 (best mean later)	Chronos-2 was named in advance; choosing after the results would be selection
Report the final window once	Iterate on it	The final window is the only unseen check
Do not claim to beat the winner	Claim	The margin is inside the bootstrap noise

6. Methods in detail

6.1 The winner: six LightGBM sets

The M5 winner trained gradient-boosted trees (LightGBM) with a Tweedie loss, which suits sales data with many zeros and a skewed shape. It made six model sets, each a collection of separate models by group, then averaged them with equal weights.

- **nr_*** = non-recursive. One model predicts each of the 28 days directly. Features use only sales at least 28 days old, so no forecast is fed back in.
- **r_*** = recursive. Models that use recent days and feed their own forecasts forward day by day.
- **store, store_cat, store_dept** = the grouping: separate models per store (10), per store and category (30), or per store and department (70). Each family has 220 models in total.

Features include lags, rolling means and standard deviations over several windows (7, 14, 30, 60, 180 days), target encodings (average sales by item, state, store and similar groups, rebuilt using only days before the forecast origin), price and price momentum, calendar fields, events and SNAP flags. Each model uses 3,000 trees. We changed no setting. For

every new window the script rebuilds target encodings on days up to the origin and retrains all six sets, which is why each window costs about 11 hours.

Why six sets and equal weights (the winner's design). The sets make different kinds of errors. Recursive sets are less biased when sales are rising; non-recursive ones are stable but under-forecast when sales jump. Averaging cancels part of this, but it also explains why the winner's bias varies so much between windows (Section 7.5).

6.2 Foundation models, zero-shot

A foundation model is a large model pretrained on many time series from many fields. **Zero-shot** means we do not train it on M5: we hand it a history and it returns a forecast. This is cheap, has no tuning, and is easy to explain. The main model is **Chronos-2** (Amazon, `amazon/chronos-2`), which can also use covariates (extra series known in advance).

For the aggregate series, the covariates are built per group: the weekday of each day, whether an event occurs, and the SNAP flag averaged over the group's states (a group in one state gets that state's flag; the total gets the weighted mix). Settings fixed in advance: context 1,024 days, horizon 28, point forecast equal to the mean implied by the model's quantiles (the integral of the quantile function), clipped at zero.

PatchTST cuts the history into patches (length 16, stride 8) and predicts all 28 days at once (Nie et al., 2023, preprint). We trained one global model on M5 with and without covariates (context 224 days, level scaling, masked loss, 12,000 steps, fixed settings, seed 0). It appears only as a comparison member.

6.3 The blend

For each of the nine aggregate levels the blended forecast of every series is

blend = 0.80 × winner aggregate + 0.20 × direct Chronos-2 B forecast

where the winner aggregate is the winner's 30,490 bottom forecasts summed up to that level. Levels 10, 11 and 12 stay as the winner forecast them.

An important consequence. Each level is blended on its own, so the blended levels no longer add up exactly (the state forecasts do not sum to the total, for example). This is called incoherence. We handle it two ways: we state it as a limitation, and we ran an OLS reconciliation (6.5) that makes all twelve levels add up, as a labelled secondary result.

6.4 How we judged results: uncertainty and honesty rules

- **Day-block bootstrap.** We resample the 28 forecast days in blocks of 7 days (to keep weekly patterns), 4 blocks per draw, 2,000 draws, and recompute the whole 12-level score each time. This gives a 95 percent interval for the change in WRMSSE and the share of draws where the blend is better. It captures day-to-day luck only, not which series or which model, so it is a rough guide. The same draws are used for every variant.
- **Across-window checks.** We count windows improved, run a two-sided exact sign test (6 of 6 gives p = 0.031; 5 of 6 would give 0.22), and bootstrap the mean change.
- **Pre-declaration.** The headline blend was named before the final window was scored. Everything else is labelled secondary or sensitivity.
- **Reporting all variants.** More variants raise the chance of a lucky win, so the report states how many were tried (Section 9).

6.5 Reconciliation (secondary)

Reconciliation adjusts the forecasts of all levels as little as possible (in the ordinary least squares sense, Hyndman et al., 2011) so they add up; Athanasopoulos et al. (2024) review the field. We solved it without building the huge matrix, using a conjugate-gradient solver on the stacked aggregation matrices (solver residual about 1e-9 on every window, which confirms the solution is accurate). A structural variant was tried on two windows and was worse (`results/07_reconciliation`).

6.6 Other foundation models (replication)

To test that the gain is not specific to Chronos-2 we ran four more zero-shot models with the same inputs, rule and windows: **t0-gamma** (The Forecasting Company), **Timestral-209M** (Mistral), **TimesFM 3.0** (Google; weights carry a non-commercial licence, fine for a thesis; see also Das et al., 2024) and **TiRex-2** (NXAI, the decontaminated variant released for zero-shot benchmark evaluation). Each ran in its own Python environment with the same covariates, context 1,024 and horizon 28, and was scored standalone and as a 20 percent blend. An extra row averages all five models.

6.7 SHAP explainability

To explain why the winner behaves as it does, we computed exact SHAP values (Lundberg et al., arXiv 1905.04610) for the winner's nr_store models in three stores (CA_1, TX_1, WI_1), sampling 4,000 rows each from the final window. The script reuses the winner's own feature code and does not retrain anything. Values are on the log scale of the Tweedie forecast.

7. Results

All scores are WRMSSE (lower is better). "Change" is blend minus winner, so negative means the blend is better.

7.1 Reproduction of the winner

On the final window our run gives 0.5228 against the official 0.5204. The individual sets differ widely, and averaging them is what produces a good score. The table lists each set with its bias (total forecast divided by total actual; 1.000 is unbiased). Source: `results/02_winner_reproduction/`.

Set	Final WRMSSE	Final bias	Public WRMSSE	Public bias	cv3 WRMSSE	cv3 bias	cv5 WRMSSE	cv5 bias
nr_store	0.5282	0.987	0.6950	0.920	0.7297	0.942	0.8232	0.884
nr_store_cat	0.5535	0.973	0.7678	0.905	0.7735	0.924	0.9155	0.862
nr_store_dept	0.6064	0.952	0.8629	0.883	0.8511	0.901	0.9757	0.848
r_store	0.6835	1.066	0.4734	0.987	0.6955	1.050	0.5451	1.030
r_store_cat	0.5802	1.029	0.5440	0.963	0.6629	1.022	0.5650	0.975
r_store_dept	0.5526	1.011	0.6184	0.942	0.6578	0.997	0.6142	0.952
average	**0.5228**	**1.003**	**0.6415**	**0.933**	**0.6562**	**0.973**	**0.6730**	**0.925**

Interpretation. The ranking of the sets flips between windows. On the final window the non-recursive sets are best; on the public window the recursive ones are far better (r_store 0.4734 against nr_store 0.6950). This window sensitivity is itself a finding: tuning anything on a single window would mislead. The winner's score also varies by window from 0.5167 (cv4) to 0.6730 (cv5). Level scores of the final-window reproduction: L1 0.2006, L2 0.3118, L3 0.3994, L4 0.2839, L5 0.3720, L6 0.3948, L7 0.4782, L8 0.4816, L9 0.5737, L10 0.9653, L11 0.9287, L12 0.8841. The item levels are by far the weakest.

A puzzle we flag but have not resolved. On the public window the single set r_store (0.4734) is much better than the equal-weight average of the six (0.6415), and our own simple global LightGBM with the pre-declared final design scored 0.5888 on the same window (`results_E1b_lightgbm.txt`). The winner's equal average was designed for its own final window, where it works well. Whether set-specific weights learned from earlier windows would help is an open idea (Section 9.3); we did not test it because per-set forecasts were kept for only three windows.

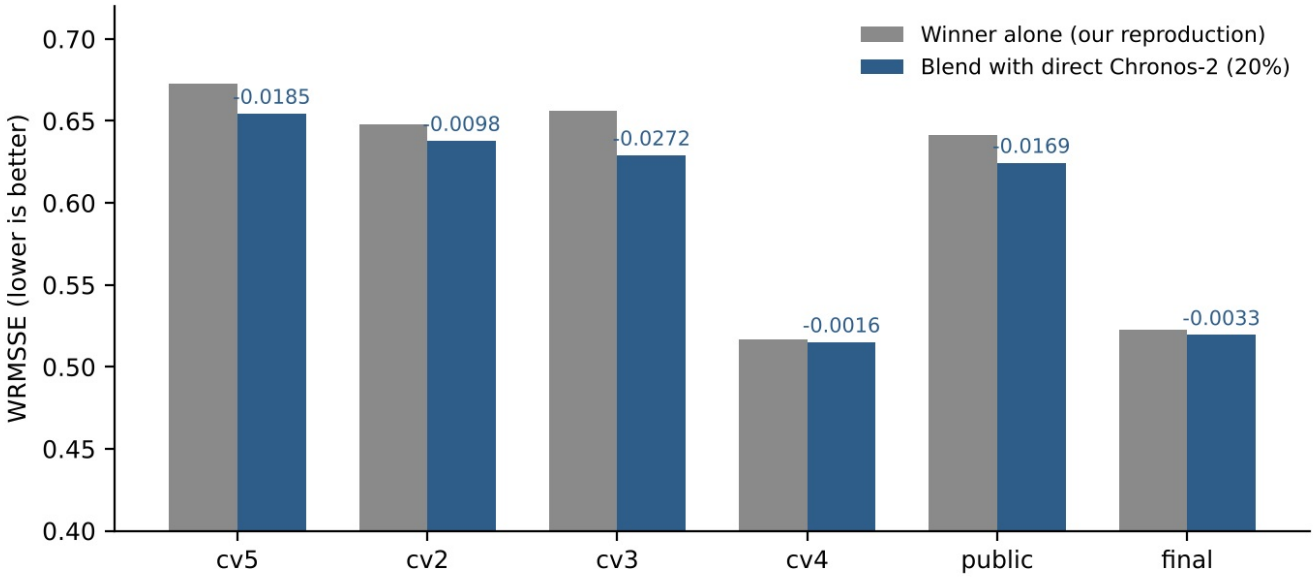
7.2 Early results recap

See Sections 5.2 to 5.4 for the early single-window results (own LightGBM ladder, Chronos-2 and PatchTST alone, direct aggregates).

7.3 The headline: six windows, the blend improves every one

Window	Winner	Blend	Change	95% interval of change	P(better)	Sales growth	Winner bias
cv5	0.6730	0.6545	-0.0185	[-0.0285, -0.0078]	1.00	+8.5%	0.925
cv2	0.6477	0.6379	-0.0098	[-0.0156, -0.0022]	0.99	+7.5%	0.951
cv3	0.6562	0.6291	-0.0272	[-0.0538, -0.0030]	1.00	0.0%	0.973
cv4	0.5167	0.5151	-0.0016	[-0.0074, +0.0031]	0.72	+0.6%	1.002
public	0.6415	0.6245	-0.0169	[-0.0280, -0.0085]	1.00	+4.1%	0.933
final	0.5228	0.5195	-0.0033	[-0.0135, +0.0060]	0.74	-0.2%	1.003
Mean	**0.6097**	**0.5968**	**-0.0129**	[-0.0182, -0.0072]			

Six of six windows improved (sign test p = 0.031). The mean change is -0.0129 with a 95 percent interval of [-0.0182, -0.0072] from the day bootstrap of the mean. The smallest gain is on cv4 (-0.0016) and the largest on cv3 (-0.0272). With only the five windows before the final one, the mean was -0.0148 and p = 0.062; adding the final window brought p to 0.031 but lowered the mean because the final-window gain is small. Windows share training history, so the six are not fully independent. Source: `results/04_six_window_blend/`, `results/05_foundation_models/`, `results/06_ablation_bootstrap/`.



Interpretation. The blend never hurts on average over windows, and helps clearly on four windows (cv5, cv2, cv3, public). On cv4 and the final window the gain is small and its interval includes zero. That is what the bias analysis in 7.5 predicts.

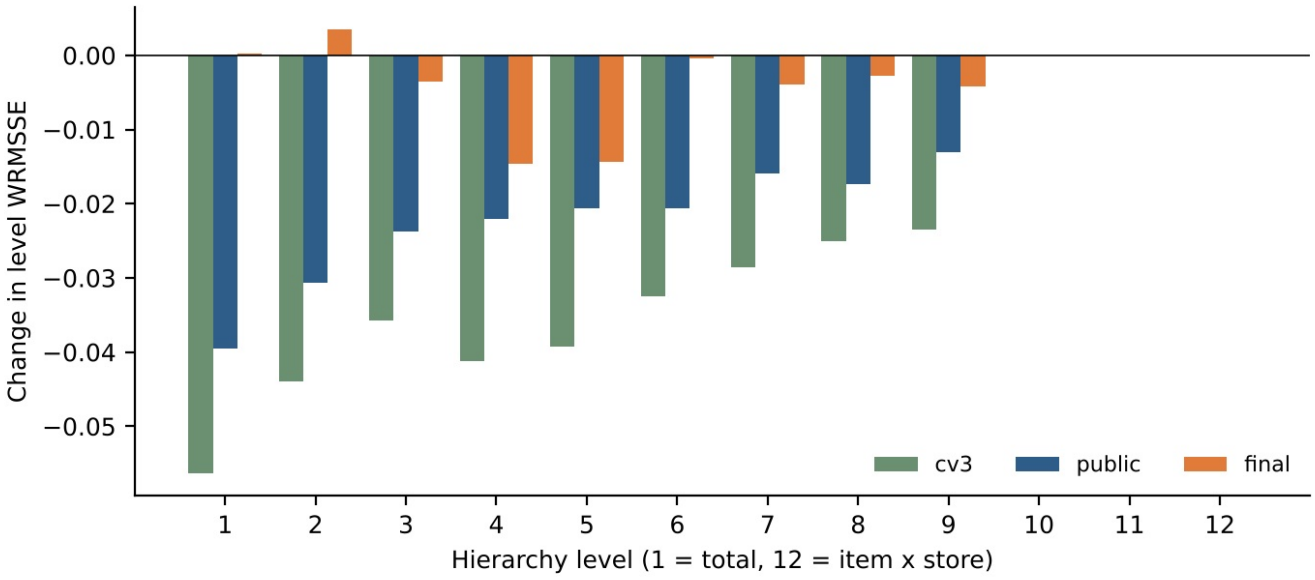
7.4 The final window against the official score

On the final window the winner alone scores 0.5228 and the blend 0.5195, a change of -0.0033 with a 95 percent interval of [-0.0132, +0.0061] and a 0.73 share of draws in which the blend is better. The official score is 0.5204. In the bootstrap draws the blend is below 0.5204 in 61 percent and the winner alone is below 0.5204 in 53 percent. In other words, the final window cannot separate our blend from the official winner, and even our reproduction of the winner is within noise of the official number.

This is why the safe claim is "consistently better than the reproduction on six windows, comparable to the official winner on the final window", and not "we beat the winner".

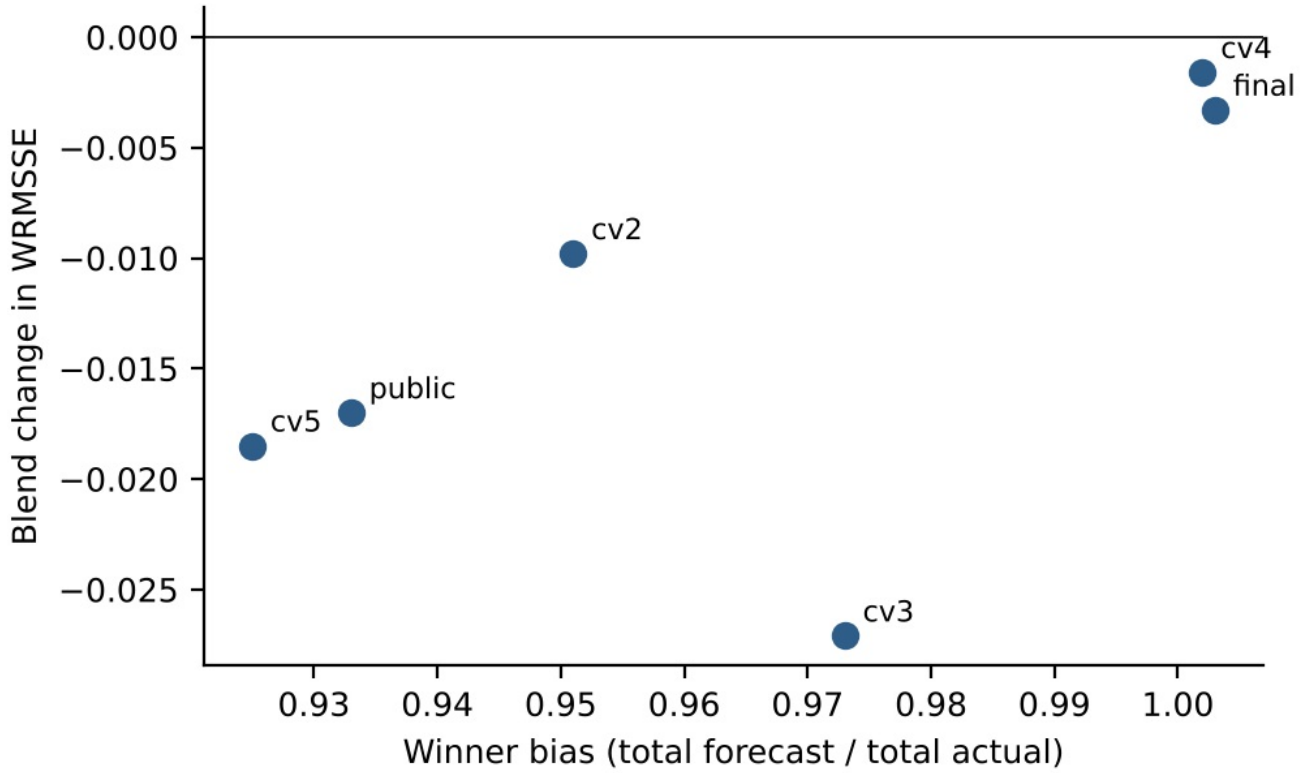
7.5 Where the gain comes from: levels, and the winner's bias

The blend changes levels 1 to 9 only; levels 10 to 12 are identical to the winner. The gain is therefore entirely from the aggregate levels. On the public window the gain is large and even (-0.0395 at the total, shrinking to -0.0131 at store by department); on cv4 and the final window it is small and mixed, because there the winner is already close to unbiased.



The winner is biased low on four of six windows (cv5, cv2, cv3, public): its total forecast is only 92.5 to 97.3 percent of actual sales, while on the final window and cv4 it is near 1.00. **The gain follows this bias.** Where the winner under-forecasts, the blend gains 0.010 to 0.027; where it is unbiased, only 0.002 to 0.003. A first explanation, "sales growth", fails:

cv3 has zero growth and the largest gain, and the correlation between growth and gain is -0.19 across the six windows.



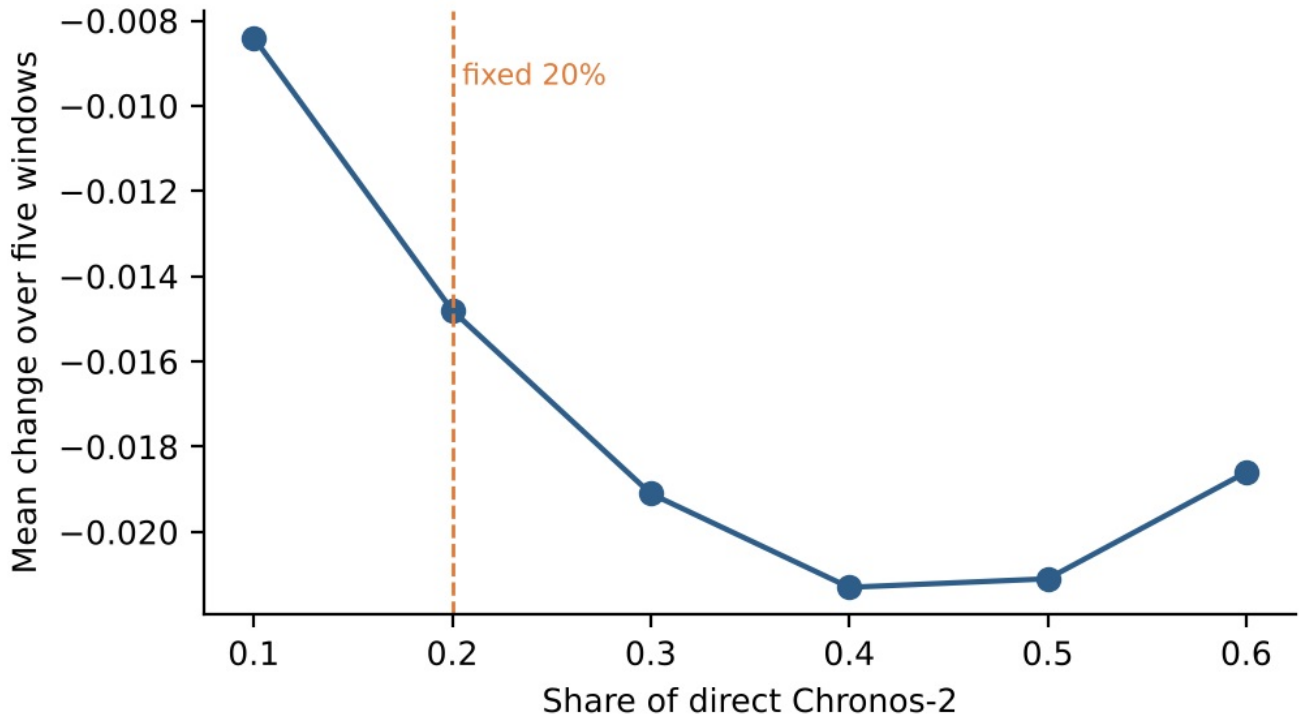
Interpretation. The reason the winner under-forecasts, per SHAP (7.10), is that it sets each item's forecast largely from the item's own recent average and its item-state average, with only a weak trend term; the foundation models see the overall level of aggregate series directly and so correct the level. This is a consistent explanation, not a proven cause.

A rule that adapts the share to the disagreement between the winner's total and Chronos-2's total lines up for four of six windows but fails for the final window (2.3 percent disagreement with best share 0.15, against cv5 at 2.1 percent with best share 0.50). With six windows we judged such a rule untrustworthy and rejected it.

7.6 Tuning the share, and the ablations

Averaged over the five windows before the final one, a larger share looks better than 20 percent:

Share	0.10	0.20	0.30	0.40	0.50	0.60
Mean change, five windows	-0.0084	-0.0148	-0.0191	**-0.0213**	-0.0211	-0.0186



Yet on the final window a 0.40 share gives 0.5259, worse than the winner alone (0.5228), because that window is nearly unbiased and wants a share near 0.15. A leave-one-window-out test (choose the share on four windows, apply to the fifth) helps cv5, cv3 and public, but hurts cv4 by +0.0053 (worse than the winner alone) and cv2 slightly. We therefore kept the fixed 0.20 as the primary result and report 0.40 only as a sensitivity result that is better on average and worse on the final window.

Ablations (change versus the winner at 20 percent; negative is better):

Variant	cv4	public	final	Comment
Direct B mean 20% (headline)	-0.0016	-0.0169	-0.0033	Fixed in advance
Direct B median 20%	-0.0023	-0.0155	-0.0034	Mean and median nearly equal
Direct B mean 10%	-0.0014	-0.0094	-0.0029	Safer, smaller
Direct B mean 30%	-0.0005	-0.0225	-0.0013	Better on public, worse on final
Direct A, no covariates, 20%	+0.0026	-0.0089	+0.0023	Covariates matter
Bottom-up Chronos-2 C, 20%	+0.0175	-0.0113	-0.0096	Best on final, worst on cv4: unstable, not used
Bottom-up PatchTST P1, 20%	+0.0086	+0.0009	-0.0055	Unstable

Bottom-up Chronos-2 C scores best on the final window (0.5133) yet hurts cv4 by 0.0175. Picking it because of the final window would be exactly the tuning on the answer that we avoid, and its instability is the reason direct B was chosen. Source: `results/06_ablation_bootstrap/results_final_analysis.txt`.

7.7 Reconciliation

Making the blend coherent with OLS reconciliation improves the unreconciled blend on five of six windows slightly and is worse on cv5. The six reconciled scores are 0.6577, 0.6361, 0.6281, 0.5144, 0.6229 and 0.5181 for cv5, cv2, cv3, cv4, public and final. Six of six windows improve against the winner (mean -0.0134, sign test p = 0.031), and the final window reaches 0.5181 (change -0.0047). This removes the incoherence limitation but is a secondary variant, because it was added after the main rule was set. The structural variant (cv4, public) was worse than OLS on both windows.

7.8 Item levels (levels 10 and 11), exploratory

Levels 10 to 12 are the weakest part of the score and are untouched by the headline blend. We ran direct zero-shot Chronos-2 B on level 10 (3,049 item series) and level 11 (9,147 item-by-state series) for all six windows and mixed it in at the same 20 percent share (`results/09_levels_10_11/`). Level 12 stays as the winner. The decision rule was fixed in advance: judge on the five windows before the final one, show the final window once as exploratory, and keep the headline unchanged unless the five windows clearly improve.

Window	Level 10 + 11 change versus headline

cv5	-0.0034
cv2	-0.0018
cv3	-0.0018
cv4	-0.0010
public	-0.0017
final	-0.0005 (0.5195 to 0.5190)
Five-window mean	-0.0019, 5 of 5 improved, p = 0.062
Six-window mean	-0.0017, 6 of 6 improved, p = 0.031

Level 10 alone gives a mean change of -0.0012 over five windows. On the public window the level 10 and 11 scores move from 0.8230 and 0.8261 (winner) to 0.8096 and 0.8191 (blend), even though direct Chronos-2 alone scores worse (0.8268 and 0.8455) than the winner at these levels.

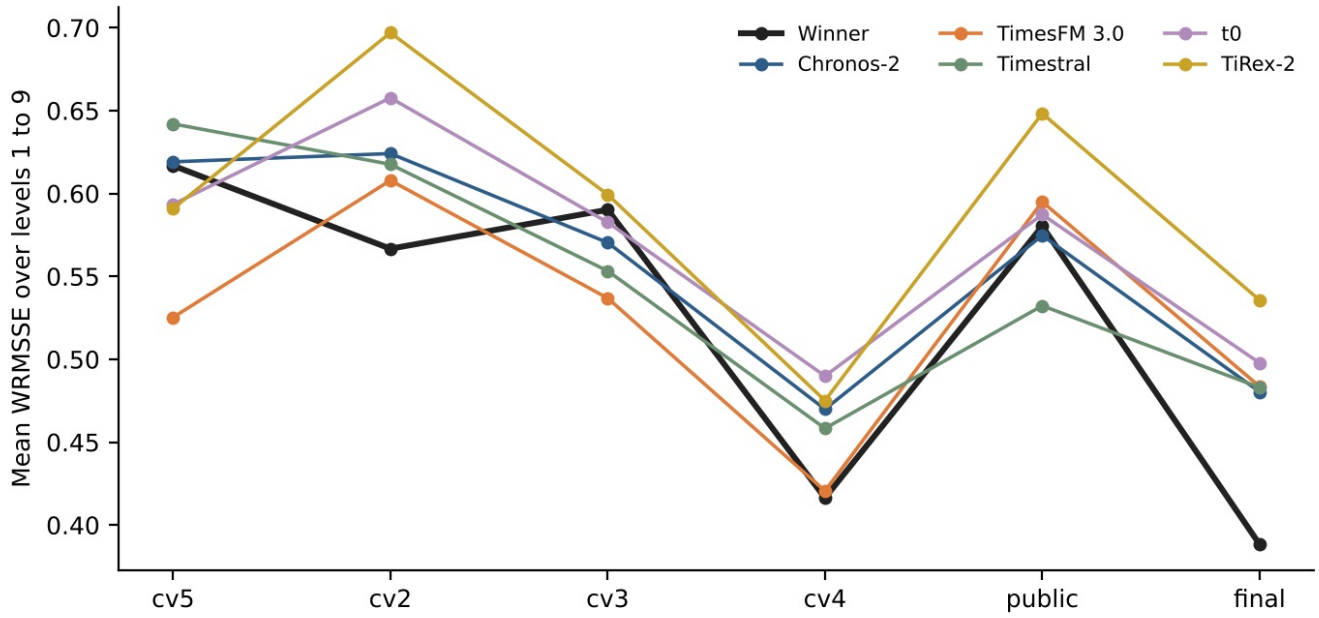
Interpretation. The gain is consistent but tiny (under 0.002), because these series are mostly zeros. A blend helps even where the foundation model is weaker alone, which is the classic combination effect (Wang et al., 2023). The five-window evidence does not clearly cross the pre-set bar, so the headline stays unchanged and this is reported as a secondary extension.

7.9 Other foundation models: replication

Mean WRMSSE over levels 1 to 9 for the winner and each model as a standalone forecast (`results/05_foundation_models/results_fm_PC2.txt`):

Window	Winner	Chronos-2 B	TimesFM 3.0	Timestral	t0	TiRex-2
cv5	0.6168	0.6191	**0.5253**	0.6420	0.5935	0.5909
cv2	**0.5667**	0.6241	0.6079	0.6176	0.6577	0.6970
cv3	0.5902	0.5704	**0.5369**	0.5533	0.5826	0.5994
cv4	**0.4164**	0.4699	0.4206	0.4583	0.4899	0.4751
public	0.5803	0.5747	0.5950	**0.5323**	0.5873	0.6483
final	**0.3884**	0.4804	0.4836	0.4830	0.4978	0.5357
Mean	0.5265	0.5564	0.5282	0.5478	0.5681	0.5911

Standalone, the winner is best on cv2, cv4 and the final window, and a foundation model is best on cv5, cv3 and public. No single foundation model is best everywhere. The foundation models win mainly on the earlier windows, where the winner is biased low; those windows are also where any pretraining overlap with M5 would matter most, and we could not rule that out (Section 9.1).

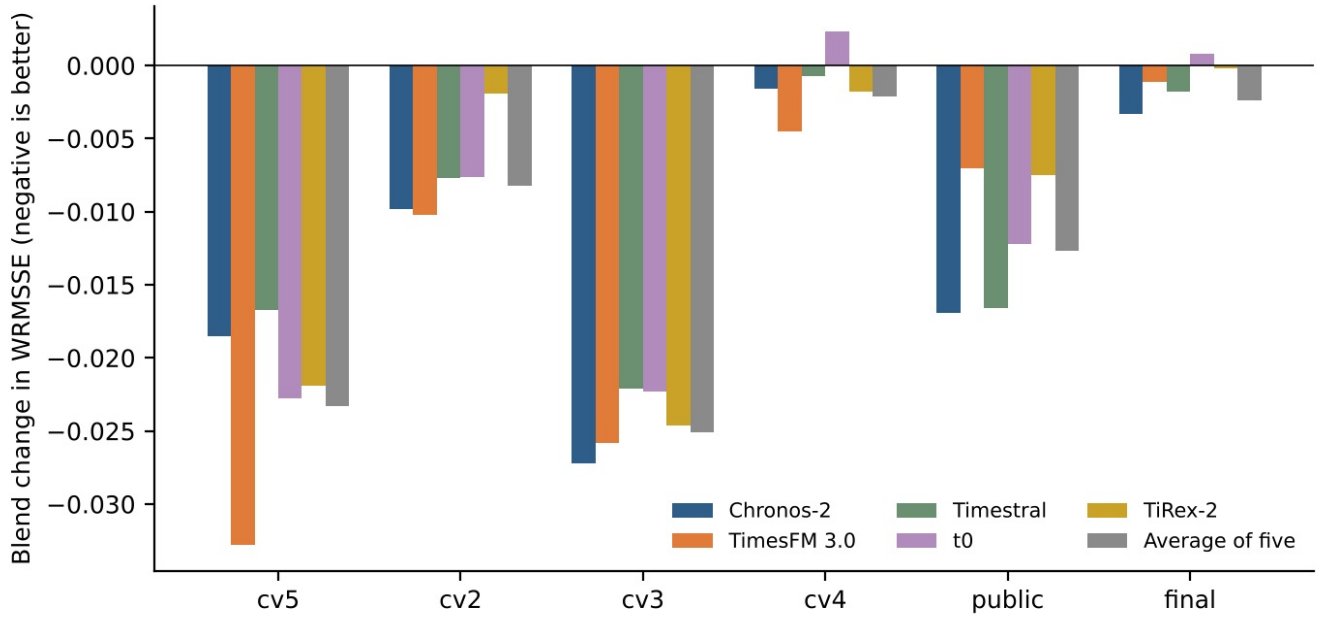


Blend of each model into the winner at the same fixed 20 percent share (negative is better):

Model	cv5	cv2	cv3	cv4	public	final	Mean	Improved
Chronos-2 B	-0.0185	-0.0098	-0.0272	-0.0016	-0.0169	-0.0033	-0.0129	6 of 6

TimesFM 3.0	-0.0328	-0.0102	-0.0258	-0.0045	-0.0070	-0.0011	-0.0136	6 of 6
Timestral	-0.0167	-0.0077	-0.0221	-0.0007	-0.0166	-0.0018	-0.0109	6 of 6
t0	-0.0228	-0.0076	-0.0223	+0.0023	-0.0122	+0.0008	-0.0103	4 of 6
TiRex-2	-0.0219	-0.0019	-0.0246	-0.0018	-0.0075	-0.0002	-0.0097	6 of 6
Average of all five	-0.0233	-0.0082	-0.0251	-0.0021	-0.0127	-0.0024	-0.0123	6 of 6

Six-window average WRMSSE of the whole system: winner 0.6097; with TimesFM 3.0 0.5961; Chronos-2 0.5968; average of five 0.5974; Timestral 0.5987; t0 0.5994; TiRex-2 0.6000.



Interpretation. (1) Every model improves the winner on average, and four of the five improve all six windows, so the gain is **not specific to Chronos-2**. (2) No other model clearly beats Chronos-2: TimesFM 3.0 has the best mean (-0.0136), but its advantage of 0.0007 is noise and its final-window gain is smaller. (3) The average of all five (-0.0123) does not beat Chronos-2 alone, because the models make similar errors, so combining models adds nothing. (4) t0 fails two windows. We keep Chronos-2 as the headline because it was named in advance; choosing TimesFM 3.0 now would mean choosing after the results.

7.10 SHAP: what the winner's models rely on

Mean absolute SHAP value measures how much a feature moves the forecast on average (`results/08_shap/`). Share of the total by feature family, averaged over three stores (the ranking was the same in all three):

Feature family	Share of total mean absolute SHAP
Rolling means and standard deviations	34.8%
Target encodings (item-state average etc.)	25.8%
Calendar, events, SNAP	19.4%
Item and group identity	10.9%
Lags	4.4%
Price	4.0%
Other	0.7%

Top feature	Family	Share %	Gain %	Direction
enc_item_id_state_id_mean	target encoding	18.5	12.5	higher raises forecast
rolling_mean_30	rolling	12.4	28.1	higher raises forecast
item_id	identity	10.2	13.2	categorical
enc_item_id_state_id_std	target encoding	6.8	1.6	higher raises forecast
tm_dw (day of week)	calendar	6.0	0.9	varies by day
rolling_mean_14	rolling	6.0	6.9	higher raises forecast
rolling_mean_60	rolling	4.2	16.0	higher raises forecast
tm_y (year)	calendar	3.5	0.2	trend term

tm_w_end (weekend)	calendar	2.8	0.4	weekend raises forecast
price_momentum_m	price	1.5	0.6	higher price momentum lowers forecast

Interpretation. The model rests on each item's recent average level (rolling means) and its long-run item-state average (target encodings), with weekday and weekend effects on top. It has no direct feature for the overall level of the chain and only a weak year term. This fits the finding that the winner under-forecasts when total sales are rising and that aggregate-level foundation models correct this. **Limits of the SHAP run:** it covers one of the six sets (nr_store), three stores and a sample of rows; the recursive sets are not covered; SHAP explains the model, not the world; direction signs for categorical features such as item_id are not meaningful.

8. What we can and cannot claim

Claim	Status	Support
The blend scores better than the reproduced winner on all six windows	Supported	6 of 6, sign test p = 0.031; mean change -0.0129, interval [-0.0182, -0.0072]
The gain does not depend on Chronos-2	Supported	Four other zero-shot models improve the winner on average; three of four improve all six windows
On the final window the blend scores 0.5195, below the official 0.5204	True as numbers, not as a statistical claim	Bootstrap: blend below 0.5204 in 61% of draws; the winner alone in 53%
We beat the M5 winner	**Do not claim**	Difference is within noise; our reproduction is itself 0.0024 from the official score
We reach the 0.4989 of Yang et al.	Not reached	Best result 0.5181 to 0.5195; their figure is a preprint with a possibly different protocol
The gain is explained by the winner's low bias	Plausible, consistent with SHAP	Gain tracks bias across windows; SHAP shows reliance on backward-looking levels; not a proven cause
Combining several foundation models helps	Not supported	Average of five (-0.0123) is not better than Chronos-2 alone (-0.0129)

Recommended wording. "A fixed-rule blend of the M5 winner's LightGBM forecasts with zero-shot foundation-model forecasts of the aggregate series improved our reproduction of the winner on all six forecast windows. On the competition's final window the blend scored 0.5195, compared with 0.5228 for the reproduction and 0.5204 officially; the difference to the official score is within the noise of the test."

9. Limitations, ideas considered, and open work

9.1 Limitations and threats to validity

- **Possible pretraining overlap.** We do not know whether M5 or similar Walmart data is in the pretraining data of Chronos-2 or the other models. We read the Chronos-2 paper's text: it lists the pretraining sources as the Chronos and GIFT-Eval corpora with the full list in Table 6 (Appendix A), and its retail case study is on the Rossmann data, not M5. **Table 6 itself could not be read, so the check is not complete** and must be done from the PDF (arXiv 2510.15821) before submission. TimesFM's large standalone gains on the two oldest windows could be a sign of overlap and are unverified. Replication across four models trained on different data weakens but does not remove this worry. A decontaminated TiRex-2 variant still improved six of six windows (mean -0.0097), which is partial reassurance only.
- **Only four weeks per window.** The bootstrap resamples days only. It does not capture uncertainty from which series or which model.
- **Windows share history.** The six windows are not independent evidence. The sign test treats them as if they were.
- **One data set, one winner.** The conclusions are for M5 and for this winner. A different base model that is unbiased may gain less.
- **Reproduction is not the official run.** Our winner is 0.0024 worse than the official 0.5204, from machine and library differences.
- **Incoherent forecasts.** The headline blend does not add up across levels. OLS reconciliation fixes this and is reported as a secondary variant.
- **Item levels barely improved.** Levels 10 to 12 are a quarter of the score and the hardest; the best extension gains under 0.002.
- **Exploration count.** Many variants were tried (shares, reconciliation, five models, averages, levels 10 and 11). The headline was named before the final window, but the number of variants should be reported.
- **SHAP scope.** One set of six, three stores, a sample of rows.
- **Preprint benchmark.** The 0.4989 figure is not peer reviewed. We checked its abstract and tables through the arXiv page: the reference model in that paper is itself a hierarchical LightGBM ensemble, and the paper does not explain why its Chronos M5 scores are far worse than the other models, so the numbers should not be ranked directly against ours.

- **Early single-window results are one window.** The ladders in Sections 5.2 and 5.3 use the public window only.

9.2 Work that was designed but whose results are not in this report

An overnight job (`src/m5_overnight.py`, launched on 21 October on PC 2, with `src/m5_tune.py` as a smaller version) tests the following with saved forecasts and a few short GPU runs. At the time of writing its result files (`results_overnight_A.txt`, `results_overnight_FINAL.txt`) had not been returned, so **none of these is reported here**.

Idea	What it does	Why it was tried
Tuned shares (global, per group, per level)	Choose the share on earlier windows	A larger share looked better on the five earlier windows but not on the final window; the honest test asks whether a share chosen without the scored window helps
Choice of direct forecast	Chronos-2, other models, averages, new Chronos-2 variants	Models are best on different windows
Horizon-dependent share	The share changes over the 28 days	Winner and foundation model may lose accuracy at different speeds
Top-down alignment	Scale the winner's item forecasts so they add up to the blended aggregate, at level 1, 3 or 9, with shrinkage	The second-place M5 team used this idea with a trained neural model on the totals (Anderer and Li, arXiv preprint). Ours would use a zero-shot model. It also spreads the correction in proportion to sales and reaches levels 10 to 12, which the headline blend never touches
New Chronos-2 variants	Context of 512 days and the whole history; cross-learning in groups	Chronos-2 was pretrained with long contexts and has a group mode for related series (Ansari et al., 2025); we used 1,024 days and forecast each series alone
Stack	The best group blend plus the best alignment	Combine the two ways to reduce bias

Rule for using the results. Every setting is picked without the window it is scored on: the five pre-final windows use the other four, and the final window uses those five and is scored once. A rule is adopted only if it beats the 20 percent headline on at least four of the five held-out windows and on the final window. The script also prints the best any per-level tuning could achieve on each window, so the gap to the honest rows shows how much of any gain would be overfitting. Whatever the result, the headline stays the pre-declared result, and tuned rules are reported as secondary.

9.3 Ideas considered and our view of them

- **Deep learning (transformers, N-BEATS) trained on M5.** On M5, trained deep models lost to tree ensembles in the competition (Makridakis et al., 2022b), and our own PatchTST was worse than the winner on two of three windows in the ablation. The exception is the top of the hierarchy: the second-place team trained N-BEATS on the totals only and used it to correct their LightGBM forecasts (Anderer and Li, arXiv preprint; the abstract-level details we verified: LightGBM at the bottom, N-BEATS at the top, forecasts aligned by a multiplier, WRMSSE 0.52816 on the evaluation window, second place). A model trained on our own 154 aggregate series could be one more blend candidate. We did not build it: it needs new code and validation-based tuning inside each window's history, and experiments close on 22 October.
- **Reinforcement learning (DQN).** Not recommended. Forecasting is not a sequence of decisions with rewards. The only natural use is choosing the blend share, which is a pick-one-of-N problem; with six windows a DQN would memorise them, and a direct search over shares does the same job more safely. It would also be hard to defend to an examiner.
- **Set weights for the six winner sets.** On the public window the set r_store alone scores 0.4734 against 0.6415 for the equal average (Section 7.1). Learning set weights from earlier windows is untested. Per-set forecasts were kept for only cv4, public and final.
- **Hyperparameter tuning of anything.** Fair only on data before the scored window. We never tune on the final window.
- **Rank stability caution.** Hewamalage et al. (arXiv preprint) report that M5 rankings are unstable because of hierarchical aggregation and scaling, and that "magic numbers" affect them. This is a further reason to claim only consistency across windows and to avoid any claim from a single score.

9.4 Still open

Item	Status
Overnight tuning, alignment and variants job	Launched 21 October; results not yet returned
Check Chronos-2 pretraining list (paper Table 6)	Open
Confirm which M5 overview paper the advisor wants cited (results article or background article)	Open

Add the overnight results to this report and to the paper draft	Open
Verify the SHAP journal version details (Nature Machine Intelligence)	Open; cited as the arXiv version

10. Literature we checked

Each entry was opened and its authors, venue and year read from the page, except where noted. Peer-reviewed unless marked preprint.

Source	Used for
Makridakis, Spiliotis and Assimakopoulos (2022a). The M5 competition: Background, organization, and implementation. *International Journal of Forecasting* 38(4), 1325-1336. Link	Design of M5: 42,840 hierarchical series, point and uncertainty tracks
Makridakis, Spiliotis and Assimakopoulos (2022b). M5 accuracy competition: Results, findings, and conclusions. *IJF* 38(4), 1346-1364. Link	Competition results, tree methods on top
Januschowski et al. (2022). Forecasting with trees. *IJF* 38(4), 1473-1481. Link	Why boosted trees dominated M5
Ke et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *NeurIPS 30*, 3146-3154. Link	The winner's model family
Ansari et al. (2024). Chronos: Learning the language of time series. *TMLR*. Link	Pretrained forecasting models
Ansari et al. (2025). Chronos-2: From univariate to universal forecasting. arXiv 2510.15821 (**preprint**). Link	Our headline foundation model; covariates and group mode
Das et al. (2024). A decoder-only foundation model for time-series forecasting. *ICML 2024*, PMLR 235, 10148-10167. Link	TimesFM
Nie et al. (2023). A time series is worth 64 words (PatchTST). arXiv 2211.14730 (**preprint**; venue not confirmed). Link	PatchTST
Hyndman et al. (2011). Optimal combination forecasts for hierarchical time series. *Computational Statistics and Data Analysis* 55(9), 2579-2589. Link	OLS reconciliation
Wickramasuriya, Athanasopoulos and Hyndman (2019). Optimal forecast reconciliation ... through trace minimization. *JASA* 114(526), 804-819. Link	Trace-minimising reconciliation (not used; context)
Athanasopoulos et al. (2024). Forecast reconciliation: A review. *IJF* 40(2), 430-456. Link	Reconciliation overview
Wang et al. (2023). Forecast combinations: An over 50-year review. *IJF* 39(4), 1518-1547. Link	Why simple fixed-share combinations work
Yang, Cao and Liu (2025). Foundation models for demand forecasting via dual-strategy ensembling. arXiv 2507.22053 (**preprint**; KDD 2025 AI for Supply Chain workshop). Link	The 0.4989 stretch target
Lundberg et al. (2019/2020). Explainable AI for trees. arXiv 1905.04610 (**preprint version checked**; journal version in *Nature Machine Intelligence*, details not checked). Link	Exact TreeSHAP
Anderer and Li. Hierarchical forecasting with a top-down alignment of independent level forecasts. arXiv 2103.08250 (**preprint**; venue not listed on the page). Link	M5 second place; top-down alignment idea
Hewamalage, Montero-Manso, Bergmeir and Hyndman (2021). A look at the evaluation setup of the M5 forecasting competition. arXiv 2108.03588 (**preprint**). Link	Caution about M5 rank stability
Sprangers, Wadman, Schelter and de Rijke (2024). Hierarchical forecasting at scale. *IJF* 40(4), 1689-1700. Link	Context only; only the abstract page was read, no M5 results reported there

A search of one citation index (RePEc, about 80 papers citing the M5 results article) found no paper that tests pretrained foundation models on M5 in this way. That is modest support for novelty, not an exhaustive review.

11. Reproducing the work from this repository

What is here. `src/` holds every script we wrote (flat, so imports resolve), `forecasts/` holds the saved forecasts of every window (winner, Chronos-2 aggregates, four other foundation models, levels 10 and 11), `results/` holds every result file, and `docs/` holds this report and its figures.

What is not here. The M5 data (download from Kaggle, competition `m5-forecasting-accuracy`, plus the evaluation file released later by the organisers; see `data/README.md`) and the winner's own code (M5 methods repository, folder A1, `github.com/Mcompetitions/M5-methods`), which has its own licence. `src/winner_runner/` holds our wrapper scripts for it and the list of compatibility patches is in `results/02_winner_reproduction/PATCH_LOG_*.txt`.

Environment. Python 3.10 or newer with `numpy`, `pandas`, `scipy`, `lightgbm`, `torch` (CUDA build for the GPU steps) and `chronos-forecasting`. Each other foundation model used its own virtual environment (see `src/m5_fm_forecast.py`).

Typical commands (replace `DATA` with the M5 folder holding 1,969 days):

```
python scripts/prepare_inputs.py                      # builds inputs/ from forecasts/
python src/m5_data_and_wrmsse.py --selftest            # checks the scorer on synthetic data
python src/m5_aggregate.py --data_dir DATA --out_dir out_agg            # direct Chronos-2 on aggregates (GPU)
python src/m5_multiwindow.py --data_dir DATA --in_dir inputs --out_dir out_mw --stage score
```

```
python src/m5_fm_score.py --data_dir DATA --in_dir inputs --fm_dir forecasts/foundation_models --out_dir out_fm
python src/m5_lvl1011.py --data_dir DATA --in_dir inputs --out_dir forecasts/level10_11 --stage score
python src/m5_final_analysis.py --data_dir DATA --in_dir inputs --out_dir out_analysis
python src/m5_overnight.py --data_dir DATA --in_dir inputs --fm_dir forecasts/foundation_models --lvl_dir forecasts/leve
l10_11 --out_dir out_overnight
```

Scoring the final window needs the 1,969-day data. The winner's forecasts are included, so no 11-hour training run is needed to reproduce the blend results.

12. Status and timeline

Item	Status
Scorer built and checked against the official seasonal naive	Done
Own LightGBM ladder, Chronos-2 and PatchTST alone	Done (public window; final-window bottom-up runs also saved)
Winner reproduction on six windows	Done
Direct Chronos-2 B on six windows, blend scoring, bootstrap, sign test, reconciliation	Done and checked: reference numbers match to within 0.0002
Four other foundation models, standalone and blend	Done
SHAP on nr_store (3 stores)	Done
Levels 10 and 11	Done (exploratory)
Overnight tuning, alignment and variants	Launched; results not yet returned
Chronos-2 Table 6 check, M5 overview citation question	Open
Paper draft (introduction to recommendations, with references)	Drafted in a shared document; to be updated with open items

Experiments stop about 22 October, writing runs to 29 October. Late reruns of the 11-hour winner training are no longer possible, so any further work uses saved forecasts only.

Appendix A. Score by level and window

Each table gives the WRMSSE of the winner alone and of the headline blend (20 percent direct Chronos-2 B at levels 1 to 9, winner at levels 10 to 12) for one window. Lower is better. Levels 10 to 12 are identical by construction. The overall score is the equal mean of the 12 levels.

Window cv5 (d_1802 to d_1829; trained to d_1801)

Level	Name	Series	Winner	Blend	Change
1	Total	1	0.5700	0.5575	-0.0125
2	State	3	0.5876	0.5703	-0.0173
3	Store	10	0.6222	0.5886	-0.0336
4	Category	3	0.5714	0.5487	-0.0227
5	Department	7	0.6120	0.5871	-0.0249
6	State x Category	9	0.5964	0.5710	-0.0254
7	State x Department	21	0.6412	0.6184	-0.0228
8	Store x Category	30	0.6505	0.6185	-0.0320
9	Store x Department	70	0.6999	0.6688	-0.0311
10	Item	3049	0.8531	0.8531	+0.0000
11	Item x State	9147	0.8418	0.8418	+0.0000
12	Item x Store	30490	0.8298	0.8298	+0.0000
	Mean of 12		0.6730	0.6545	-0.0185

Window cv2 (d_1830 to d_1857; trained to d_1829)

Level	Name	Series	Winner	Blend	Change
1	Total	1	0.4973	0.4825	-0.0148
2	State	3	0.5182	0.5085	-0.0097
3	Store	10	0.5662	0.5588	-0.0074
4	Category	3	0.5098	0.4964	-0.0134
5	Department	7	0.5755	0.5511	-0.0244

6	State x Category	9	0.5411	0.5324	-0.0087
7	State x Department	21	0.6096	0.5991	-0.0105
8	Store x Category	30	0.6042	0.5907	-0.0135
9	Store x Department	70	0.6787	0.6632	-0.0155
10	Item	3049	0.9090	0.9090	+0.0000
11	Item x State	9147	0.8943	0.8943	+0.0000
12	Item x Store	30490	0.8691	0.8691	+0.0000
	Mean of 12		0.6477	0.6379	-0.0098

Window cv3 (d_1858 to d_1885; trained to d_1857)

Level	Name	Series	Winner	Blend	Change
1	Total	1	0.5189	0.4625	-0.0564
2	State	3	0.5628	0.5188	-0.0440
3	Store	10	0.6339	0.5982	-0.0357
4	Category	3	0.5215	0.4803	-0.0412
5	Department	7	0.5330	0.4938	-0.0392
6	State x Category	9	0.5770	0.5445	-0.0325
7	State x Department	21	0.6033	0.5747	-0.0286
8	Store x Category	30	0.6634	0.6384	-0.0250
9	Store x Department	70	0.6980	0.6745	-0.0235
10	Item	3049	0.8638	0.8638	+0.0000
11	Item x State	9147	0.8575	0.8575	+0.0000
12	Item x Store	30490	0.8419	0.8419	+0.0000
	Mean of 12		0.6562	0.6291	-0.0272

Window cv4 (d_1886 to d_1913; trained to d_1885)

Level	Name	Series	Winner	Blend	Change
1	Total	1	0.2320	0.2363	+0.0043
2	State	3	0.3286	0.3301	+0.0015
3	Store	10	0.4448	0.4359	-0.0089
4	Category	3	0.3181	0.3205	+0.0024
5	Department	7	0.4114	0.4102	-0.0012
6	State x Category	9	0.4177	0.4158	-0.0019
7	State x Department	21	0.4858	0.4882	+0.0024
8	Store x Category	30	0.5254	0.5149	-0.0105
9	Store x Department	70	0.5842	0.5766	-0.0076
10	Item	3049	0.8061	0.8061	+0.0000
11	Item x State	9147	0.8228	0.8228	+0.0000
12	Item x Store	30490	0.8239	0.8239	+0.0000
	Mean of 12		0.5167	0.5151	-0.0016

Window public (d_1914 to d_1941; trained to d_1913)

Level	Name	Series	Winner	Blend	Change
1	Total	1	0.5513	0.5118	-0.0395
2	State	3	0.5517	0.5211	-0.0306
3	Store	10	0.5845	0.5608	-0.0237
4	Category	3	0.5521	0.5301	-0.0220
5	Department	7	0.5654	0.5448	-0.0206
6	State x Category	9	0.5664	0.5458	-0.0206
7	State x Department	21	0.5907	0.5748	-0.0159
8	Store x Category	30	0.6092	0.5919	-0.0173
9	Store x Department	70	0.6513	0.6383	-0.0130
10	Item	3049	0.8230	0.8230	+0.0000
11	Item x State	9147	0.8261	0.8261	+0.0000

12	Item x Store	30490	0.8260	0.8260	+0.0000
	Mean of 12		0.6415	0.6245	-0.0169

Window final (d_1942 to d_1969; trained to d_1941)

Level	Name	Series	Winner	Blend	Change
1	Total	1	0.2006	0.2009	+0.0003
2	State	3	0.3118	0.3153	+0.0035
3	Store	10	0.3994	0.3958	-0.0036
4	Category	3	0.2839	0.2693	-0.0146
5	Department	7	0.3720	0.3576	-0.0144
6	State x Category	9	0.3948	0.3944	-0.0004
7	State x Department	21	0.4782	0.4743	-0.0039
8	Store x Category	30	0.4816	0.4789	-0.0027
9	Store x Department	70	0.5737	0.5695	-0.0042
10	Item	3049	0.9653	0.9653	+0.0000
11	Item x State	9147	0.9287	0.9287	+0.0000
12	Item x Store	30490	0.8841	0.8841	+0.0000
	Mean of 12		0.5228	0.5195	-0.0033

Appendix B. Repository file inventory

Path	Purpose
`src/m5_data_and_wrmsse.py`	Data loading, hierarchy aggregators, the WRMSSE evaluator, baselines and checks
`src/m5_chronos2.py`	Chronos-2 zero-shot ladder (A, B, C), quantile-to-mean conversion, data inputs
`src/m5_aggregate.py`	Direct zero-shot forecasts of the 154 aggregate series (levels 1 to 9), builds covariates
`src/m5_patchtst.py`	PatchTST global model with and without covariates
`src/m5_multiwindow.py`	Direct forecasts for new origins and scoring: winner, blend, OLS reconciled, bootstrap, sign test
`src/m5_multiwindow_auto.py`	Unattended driver that waits for winner files and scores new windows
`src/m5_final_analysis.py`	Ablation, day-block bootstrap and level-1 bias analysis
`src/sweep_lowo.py`	Share sweep and leave-one-window-out test
`src/recon_test.py`	OLS and structural reconciliation test (matrix-free conjugate gradient)
`src/m5_fm_forecast.py`, `src/m5_fm_score.py`	Adapters for t0, Timestral, TimesFM 3.0, TiRex-2, and the standalone and blend scoring
`src/m5_shap_nr.py`	Read-only exact TreeSHAP on the nr_store models
`src/m5_lvl1011.py`	Direct Chronos-2 at levels 10 and 11 and scoring
`src/m5_tune.py`, `src/m5_overnight.py`	Tuned shares, horizon share, top-down alignment, new Chronos-2 variants (results pending)
`src/m5_lightgbm_e1.py`, `e1b`, `e1c`, `e1d`	Our own LightGBM experiments. E1 and E1b ran on the real data (results in `results/01`). E1c and E1d (per-group LightGBM) were developed and smoke-tested on small synthetic data; their real-data result files are not in this repository, so no real-data number from them is reported here
`src/m5_compare_levels.py`, `src/m5_blend_check.py`, `src/m5_blend_final.py`	Helper scripts for early level comparison and early blend checks (synthetic smoke tests only in this repository)
`src/winner_runner/`	Wrapper scripts used to run the winner's code on PC 3 and make window packages
`scripts/prepare_inputs.py`	Builds the `inputs/` folder the scoring scripts expect from `forecasts/`
`forecasts/`	Saved forecasts: `winner_lightgbm` (30,490 x 28 per window), `chronos2_aggregate` (154 x 28), `foundation_models` (154 x 28 per model and window), `level10_11`
`results/`	Every result file, in nine folders by step

Appendix C. Glossary

Term	Meaning
WRMSSE	Weighted root mean squared scaled error; the M5 accuracy score. Lower is better.
Window	A 28-day forecast period, with the model trained only on earlier days.
Zero-shot	Using a pretrained model without training it on M5.
Foundation model	A large model pretrained on many time series (Chronos-2, TimesFM, and others).
Covariate	An extra series known in advance (weekday, event flag, SNAP).

Direct vs bottom-up	Direct: forecast the summed series. Bottom-up: forecast each item and add up.
Bias	Total forecast divided by total actual. Below 1 means under-forecasting.
Blend / share	Weighted mix of two forecasts; share is the weight on the new forecast (here 0.20).
Coherent / reconciliation	Forecasts add up across levels / the adjustment that makes them add up.
Bootstrap	Repeated random resampling of the 28 days to see how much the score could vary.
Sign test	Tests whether improvement on 6 of 6 windows could be chance (p = 0.031).
SHAP	A measure of how much each feature moved each prediction.
Tweedie loss	A loss function for non-negative data with many zeros, used by the winner.
SNAP	US food-stamp programme; SNAP days raise sales of some foods.
Alignment (top-down)	Scaling item forecasts so they add up to a chosen aggregate forecast.

Appendix D. Rules we set for ourselves

- Never tune anything on the final window. The final window is scored once, after the rule is fixed.
- The headline blend is named in advance: direct Chronos-2 B, mean point, 20 percent, levels 1 to 9.
- Every other result (shares, reconciliation, other models, averages, item levels) is labelled secondary or sensitivity.
- Do not claim to beat the winner when the difference is inside the noise.
- Winner settings are never changed; only logged compatibility patches are allowed.
- Jobs on the lab machines never delete files, change model settings, or send anything anywhere; results are copied back under new names.
- No credentials, no logins, no emails to the authors of the preprint, no work around site blocks.

Appendix E. All code

src/m5_aggregate.py

```
"""
E7: Direct forecasts on the aggregate series (M5 levels 1 to 9) with zero-shot Chronos-2.

Idea (hypothesis, not a result): the bottom-up sums of Chronos-2 / PatchTST bottom forecasts are weak at the top
of the hierarchy (Level 1: 0.37 against 0.20 for the winner average). Forecasting the aggregate series directly,
where the data are smooth, may do better. This script produces those direct forecasts and scores them.

Three origins, all scored on the competition-style 28-day window that follows the origin:
  final  : origin d_1941, forecast d_1942..d_1969   (D-28 where D = number of days available)
  public : origin d_1913, forecast d_1914..d_1941
  cv4    : origin d_1885, forecast d_1886..d_1913
The weights of the evaluator for each window are computed only from data before the origin.

Variants: A_univariate (no covariates), B_covariates (weekday, event, SNAP share of the group).
No training on M5 is done. Level 10 to 12 are not forecast here (they stay bottom-up).

Outputs (plain text and npy, in --out_dir):
  agg_chronos2_{variant}_{origin}.npy   shape (154, 28): levels 1 to 9 stacked in level order
  results_aggregate.txt                 per level scores, direct vs optional bottom-up references

Usage:
  python m5_aggregate.py --data_dir m5_data --out_dir m5_outputs_agg
  optional bottom-up reference (repeatable), all three files are (30490, 28) arrays:
  --bu "name:final.npy:public.npy:cv4.npy"
  python m5_aggregate.py --selftest
"""
import argparse
import os
import sys
import tempfile
import time

import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from m5_data_and_wrmsse import (  # noqa: E402
    HORIZON, LEVELS, WRMSSEEvaluator, load_m5, load_calendar, make_synthetic,
)
from m5_chronos2 import MockPipeline, augment_calendar, quantile_mean_weights, to_np  # noqa: E402

N_AGG_LEVELS = 9  # levels 1..9


def level_slices(sizes):
    out, s = [], 0
    for g in sizes[:N_AGG_LEVELS]:
        out.append((s, s + g))
        s += g
    return out
```

```
def build_inputs(ev, meta, values, cal_full, dcols, origin, ctx_len, with_cov):
    """Aggregated target + covariates for every group at levels 1..9, using days before `origin` only for the target."""
    cal = cal_full.set_index("d").loc[dcols]
    states = sorted(meta["state_id"].unique())
    state_code = np.searchsorted(states, meta["state_id"].to_numpy())
    onehot = np.zeros((len(meta), len(states)))
    onehot[np.arange(len(meta)), state_code] = 1.0
    snap = np.stack([cal[f"snap_{s}"].to_numpy().astype(np.float32) for s in states])  # (S, D)
    wd = np.array([f"w{x}" for x in cal["wday"].to_numpy().astype(int)])
    event = cal["event_name_1"].notna().to_numpy().astype(np.float32)
    a = max(origin - ctx_len, 0)
    inputs = []
    for A in ev.A[:N_AGG_LEVELS]:
        Y = np.asarray(A @ values[:, :origin], dtype=np.float32)  # (G, origin)
        size = np.asarray(A.sum(axis=1)).ravel()
        share = np.asarray(A @ onehot) / size[:, None]  # (G, S) share of members per state
        snap_g = (share @ snap).astype(np.float32)  # (G, D)
        for g in range(A.shape[0]):
            tgt = Y[g, a:origin]
            if not with_cov:
                inputs.append(tgt)
                continue
            inputs.append({
                "target": tgt,
                "past_covariates": {"snap": snap_g[g, a:origin], "wday": wd[a:origin], "event": event[a:origin]},
                "future_covariates": {"snap": snap_g[g, origin:origin + HORIZON],
                                      "wday": wd[origin:origin + HORIZON],
                                      "event": event[origin:origin + HORIZON]},
            })
    return inputs


def forecast(pipe, inputs, weights, med_idx):
    preds = pipe.predict(inputs, prediction_length=HORIZON, batch_size=64, cross_learning=False)
    arr = np.stack([to_np(t)[0] for t in preds])  # (n, n_q, H)
    mean = np.tensordot(arr, weights, axes=([1], [0]))
    return np.clip(mean, 0, None).astype(np.float32), np.clip(arr[:, med_idx, :], 0, None).astype(np.float32)


def score_direct(ev, agg_pred, slices):
    """RMSSE-weighted score per level for direct forecasts of levels 1..9."""
    out = []
    for i, (s, e) in enumerate(slices):
        P = agg_pred[s:e]
        Y, sc, w = ev.y_agg[i], ev.scale[i], ev.weight[i]
        valid = w > 0
        r = np.zeros(len(sc))
        r[valid] = np.sqrt(np.mean((Y[valid] - P[valid]) ** 2, axis=1) / sc[valid])
        out.append(float(np.sum(w * r)))
    return out


def bottom_up_levels(ev, bottom_pred):
    P = np.asarray(bottom_pred, dtype=np.float64)
    return ev.score(P)[1]


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default=None)
    ap.add_argument("--out_dir", default="m5_outputs_agg")
    ap.add_argument("--model", default="amazon/chronos-2")
    ap.add_argument("--context_length", type=int, default=1024)
    ap.add_argument("--bu", action="append", default=[], help="name:final.npy:public.npy:cv4.npy")
    ap.add_argument("--selftest", action="store_true")
    args = ap.parse_args()
    t0 = time.time()

    if args.selftest:
        tmp = tempfile.mkdtemp()
        make_synthetic(tmp, n_days=300)
        augment_calendar(tmp, 300)
        data_dir, args.context_length = tmp, 120
    else:
        data_dir = args.data_dir or "m5_data"
    os.environ.setdefault("M5_WINDOW", "final")
    meta, values, dcols, calendar, prices = load_m5(data_dir)
    cal_full = load_calendar(data_dir)
    n, D = values.shape
    dcols = list(dcols)
    origins = {"final": D - HORIZON, "public": D - 2 * HORIZON, "cv4": D - 3 * HORIZON}
    print(f"Series {n}, days {D}, origins {origins}", flush=True)

    if args.selftest:
        pipe = MockPipeline()
    else:
        import torch
        from chronos import Chronos2Pipeline
        cuda = torch.cuda.is_available()
        print("Loading", args.model, "cuda" if cuda else "CPU", flush=True)
        pipe = Chronos2Pipeline.from_pretrained(args.model, device_map="cuda" if cuda else "cpu")
    weights = quantile_mean_weights(pipe.quantiles)
    med_idx = int(np.argmin(np.abs(np.asarray(pipe.quantiles) - 0.5)))
```

```
    os.makedirs(args.out_dir, exist_ok=True)
    bu = {}
    for spec in args.bu:
        name, *paths = spec.split(":")
        # allow Windows drive letters: rejoin pieces like "C" + "\\path"
        fixed, buf = [], ""
        for p in paths:
            if len(p) == 1 and not buf:
                buf = p + ":"
            else:
                fixed.append(buf + p)
                buf = ""
        assert len(fixed) == 3, f"--bu needs name and three files, got {spec}"
        bu[name] = {k: np.load(f) for k, f in zip(["final", "public", "cv4"], fixed)}

    rows = {}  # (label, origin) -> list of 9 level scores
    for oname, o in origins.items():
        ev = WRMSSEEvaluator(meta, values[:, : o + HORIZON], dcols[: o + HORIZON], calendar, prices)
        slices = level_slices(ev.sizes)
        for variant, with_cov in [("A_univariate", False), ("B_covariates", True)]:
            inputs = build_inputs(ev, meta, values, cal_full, dcols[: o + HORIZON], o, args.context_length, with_cov)
            mean, med = forecast(pipe, inputs, weights, med_idx)
            np.save(os.path.join(args.out_dir, f"agg_chronos2_{variant}_{oname}.npy"), mean)
            np.save(os.path.join(args.out_dir, f"agg_chronos2_{variant}_{oname}_median.npy"), med)
            rows[(f"direct_{variant}", oname)] = score_direct(ev, mean, slices)
            print(f"{oname} {variant}: L1 {rows[(f'direct_{variant}', oname)][0]:.4f}", flush=True)
        for name, d in bu.items():
            rows[(f"bottomup_{name}", oname)] = bottom_up_levels(ev, d[oname])[:N_AGG_LEVELS]

    labels = sorted({k[0] for k in rows})
    lines = ["AGGREGATE-LEVEL DIRECT FORECASTS (Chronos-2 zero-shot), WRMSSE per level, lower is better",
             f"Origins: {origins} | context {args.context_length} | days available {D}",
             "Direct rows come from forecasting the summed series; bottomup rows come from the supplied bottom forecasts.",
             "Hypothesis test only; no weights fitted here.", ""]
    for oname in origins:
        lines.append(f"WINDOW: {oname} (forecast d_{origins[oname] + 1}..d_{origins[oname] + HORIZON})")
        head = f"{'Level':<24}" + "".join(f"{l[:26]:>28}" for l in labels)
        lines += [head, "-" * len(head)]
        for i in range(N_AGG_LEVELS):
            lines.append(f"{LEVELS[i][0]:>2} {LEVELS[i][1]:<21}" +
                         "".join(f"{rows[(l, oname)][i]:>28.4f}" if (l, oname) in rows else f"{'':>28}" for l in labels))
        lines.append(f"{'mean of L1..L9':<24}" +
                     "".join(f"{np.mean(rows[(l, oname)]):>28.4f}" if (l, oname) in rows else f"{'':>28}" for l in labels))
        lines.append("")
    lines.append(f"Runtime {(time.time() - t0) / 60:.1f} min")
    text = "\n".join(lines)
    with open(os.path.join(args.out_dir, "results_aggregate.txt"), "w") as f:
        f.write(text + "\n")
    print("\n" + text)


if __name__ == "__main__":
    main()
```

src/m5_blend_check.py

```
#!/usr/bin/env python3
"""
Blend check: how much does mixing saved forecasts help, with weights chosen on the validation window only?

For 2 to 4 saved forecasts (pred_<name>.npy for the holdout, predval_<name>.npy for the validation window), the
script
  1. scores every model alone on both windows,
  2. searches a grid of non-negative weights summing to 1 and picks the weights with the lowest VALIDATION WRMSSE,
  3. reports the holdout WRMSSE of those weights, of equal weights, and of each variant after a single global
     calibration factor (validation actual total / validation forecast total),
  4. prints the 12-level holdout table for the single models and the chosen blend.
Nothing is selected on the holdout. Training is not involved; it only reads files and takes a few minutes at most.

Caveat: LightGBM validation forecasts are mildly optimistic (their early stopping or round count saw that window),
so validation-fitted weights can lean slightly toward LightGBM.

Usage
  python m5_blend_check.py --names E1c_lightgbm_cal,chronos2_B_covariates
  python m5_blend_check.py --names E1c_lightgbm_cal,E1d_lightgbm_cal,chronos2_B_covariates --step 0.1
Output: printed report and blend_check.txt in --out_dir.
"""
import argparse
import itertools
import os
import sys

import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from m5_data_and_wrmsse import HORIZON, WRMSSEEvaluator, format_table, load_m5  # noqa: E402


def simplex_grid(k, step):
    n = int(round(1 / step))
```

```
    for c in itertools.product(range(n + 1), repeat=k - 1):
        if sum(c) <= n:
            yield np.array(list(c) + [n - sum(c)], dtype=float) / n


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default="m5_data")
    ap.add_argument("--out_dir", default="m5_outputs")
    ap.add_argument("--names", required=True, help="2 to 4 forecast names, comma separated")
    ap.add_argument("--step", type=float, default=0.1, help="weight grid step")
    args = ap.parse_args()

    names = [s.strip() for s in args.names.split(",") if s.strip()]
    if not 2 <= len(names) <= 4:
        raise SystemExit("Give 2 to 4 names")
    H, V = [], []
    for nm in names:
        ph = os.path.join(args.out_dir, f"pred_{nm}.npy")
        pv = os.path.join(args.out_dir, f"predval_{nm}.npy")
        for p in (ph, pv):
            if not os.path.exists(p):
                raise SystemExit(f"Missing file: {p}")
        H.append(np.load(ph).astype(np.float64))
        V.append(np.load(pv).astype(np.float64))

    meta, values, dcols, calendar, prices = load_m5(args.data_dir)
    n, D = values.shape
    T = D - HORIZON
    ev_hold = WRMSSEEvaluator(meta, values, dcols, calendar, prices)
    ev_val = WRMSSEEvaluator(meta, values[:, :T], dcols[:T], calendar, prices)

    def calib(pv):
        return float(ev_val.y_test.sum() / max(pv.sum(), 1e-9))

    lines = [f"{'Strategy':<46}{'Weights':<26}{'Validation':>11}{'Holdout':>9}{'HoldBias':>9}"]

    def add(label, w, plain=True):
        pv = sum(wi * v for wi, v in zip(w, V))
        ph = sum(wi * h for wi, h in zip(w, H))
        f = 1.0 if plain else calib(pv)
        sv, sh = ev_val.score(pv * f), ev_hold.score(ph * f)
        wt = " ".join(f"{x:.2f}" for x in w)
        lines.append(f"{label:<46}{wt:<26}{sv[0]:>11.4f}{sh[0]:>9.4f}{(ph * f).sum() / ev_hold.y_test.sum():>9.3f}")
        return sv[0], sh[0], sh

    for i, nm in enumerate(names):
        w = np.eye(len(names))[i]
        add(f"{nm} (alone)", w, True)
        add(f"{nm} (alone, calibrated)", w, False)

    eq = np.ones(len(names)) / len(names)
    add("equal weights", eq, True)
    add("equal weights, calibrated", eq, False)

    best = {True: (9e9, None), False: (9e9, None)}
    for w in simplex_grid(len(names), args.step):
        pv = sum(wi * v for wi, v in zip(w, V))
        for plain in (True, False):
            f = 1.0 if plain else calib(pv)
            s = ev_val.score(pv * f)[0]
            if s < best[plain][0]:
                best[plain] = (s, w)
    out = {}
    for plain, label in ((True, "best on validation"), (False, "best on validation, calibrated")):
        out[plain] = add(label, best[plain][1], plain)

    # level table for the calibrated validation-best blend
    wb = best[False][1]
    pvb = sum(wi * v for wi, v in zip(wb, V))
    phb = sum(wi * h for wi, h in zip(wb, H)) * calib(pvb)
    table = {nm[:14]: ev_hold.score(h) for nm, h in zip(names, H)}
    table["blend_cal"] = ev_hold.score(phb)

    text = "\n".join([
        "BLEND CHECK (weights chosen on the validation window only)",
        f"Validation window: d_{T - HORIZON + 1} to d_{T} | Holdout window: d_{T + 1} to d_{D} | grid step {args.step}",
        "Models: " + ", ".join(names),
        "Calibrated = one global multiplier fitted on validation (actual total / forecast total).",
        "Lower WRMSSE is better. HoldBias = total forecast / total actual on the holdout.",
        "",
        *lines,
        "",
        "WRMSSE BY LEVEL (holdout; single models, uncalibrated, and the validation-best calibrated blend)",
        format_table(ev_hold, table),
        "",
        "Reference: Yang et al. Kaggle Top-1 reproduction 0.5230 | LightGBM + PatchTST 0.4989 (window unconfirmed)",
        "Our E1b global LightGBM L3 on this holdout: 0.5422 (no validation forecasts saved).",
    ])
    print(text)
    with open(os.path.join(args.out_dir, "blend_check.txt"), "w") as f:
        f.write(text + "\n")
    print(f"\nSaved: {os.path.join(args.out_dir, 'blend_check.txt')}")
```

```
if __name__ == "__main__":
    main()
```

src/m5_blend_final.py

```
"""
E8: Winner's six-set LightGBM average + direct Chronos-2 forecasts of the summed series (levels 1 to 9).

Rule fixed BEFORE the final window is scored (written here so it cannot be changed after seeing the result):
  * Base: the winner's equal-weight six-set average, unchanged at levels 10 to 12.
  * At levels 1 to 9 the forecast of each group is (1 - s) * winner + s * direct Chronos-2 with covariates (variant B).
  * Headline share s = 0.20. It is chosen on the cv4 window, whose sales growth (0.6 percent) is the closest to the
    final window (0.2 percent). The public window (4.1 percent growth) is reported but not used to choose.
  * All other shares are reported as sensitivity only. The final window is scored once.

Windows (28 days each): final d_1942..d_1969, public d_1914..d_1941, cv4 d_1886..d_1913.
Scoring uses the same WRMSSE code that reproduced the seasonal-naive baseline of the official results.

Usage:
  python m5_blend_final.py --data_dir m5_data --in_dir blend_inputs --out_dir blend_out
Expected files in --in_dir:
  winner_final.npy winner_public.npy winner_cv4.npy                    (30490 x 28 each)
  agg_chronos2_B_covariates_final.npy / _public.npy / _cv4.npy         (154 x 28 each)
Windows that need actuals which are not in the data folder are skipped with a message.
"""
import argparse
import os
import sys

import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from m5_data_and_wrmsse import HORIZON, LEVELS, WRMSSEEvaluator, load_m5  # noqa: E402

HEADLINE = 0.20
SHARES = [0.0, 0.05, 0.10, 0.15, 0.20, 0.30, 0.40, 0.50]
ORIGIN = {"final": 1941, "public": 1913, "cv4": 1885}


def blend_levels(ev, winner, direct, s):
    sizes = ev.sizes[:9]
    off = np.cumsum([0] + sizes)
    out = []
    for i, (A, sc, wt, Y) in enumerate(zip(ev.A, ev.scale, ev.weight, ev.y_agg)):
        P = np.asarray(A @ winner)
        if i < 9 and s > 0:
            P = (1 - s) * P + s * direct[off[i]:off[i + 1]]
        v = wt > 0
        r = np.zeros(len(sc))
        r[v] = np.sqrt(np.mean((Y[v] - P[v]) ** 2, axis=1) / sc[v])
        out.append(float(np.sum(wt * r)))
    return float(np.mean(out)), out


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default="m5_data")
    ap.add_argument("--in_dir", default="blend_inputs")
    ap.add_argument("--out_dir", default="blend_out")
    args = ap.parse_args()
    os.makedirs(args.out_dir, exist_ok=True)

    meta, values, dcols, calendar, prices = load_m5(args.data_dir)
    dcols = list(dcols)
    D = values.shape[1]
    lines = ["BLEND OF THE WINNER (SIX-SET AVERAGE) WITH DIRECT CHRONOS-2 FORECASTS AT LEVELS 1 TO 9",
             f"Days available in data folder: {D}. Headline share fixed in advance: {HEADLINE:.2f} (chosen on cv4).",
             "WRMSSE over all 12 levels, lower is better. Share 0.00 is the winner alone.", ""]
    res = {}
    for win in ["cv4", "public", "final"]:
        end = ORIGIN[win] + HORIZON
        if D < end:
            lines.append(f"Window {win}: skipped, the data folder has {D} days and {end} are needed for the actuals.\n")
            continue
        w = np.load(os.path.join(args.in_dir, f"winner_{win}.npy"))
        d = np.load(os.path.join(args.in_dir, f"agg_chronos2_B_covariates_{win}.npy"))
        ev = WRMSSEEvaluator(meta, values[:, :end], dcols[:end], calendar, prices)
        bias = float(w.sum() / ev.y_test.sum())
        lines.append(f"Window {win} (forecast d_{ORIGIN[win] + 1}..d_{end}), winner bias {bias:.3f}")
        lines.append(f"{'Share':>8}{'WRMSSE':>10}{'Change':>10}")
        base = None
        for s in SHARES:
            m, lv = blend_levels(ev, w, d, s)
            if s == 0:
                base = m
            mark = "  <- headline" if abs(s - HEADLINE) < 1e-9 else ""
            lines.append(f"{s:>8.2f}{m:>10.4f}{m - base:>+10.4f}{mark}")
            res[(win, s)] = (m, lv)
        lines.append("")
```

```
        lines.append(f"{'Level':<24}{'Winner alone':>14}{'Headline blend':>16}{'Change':>10}")
        for i, (lv, name, _) in enumerate(LEVELS):
            a, b = res[(win, 0.0)][1][i], res[(win, HEADLINE)][1][i]
            lines.append(f"{lv:>2} {name:<21}{a:>14.4f}{b:>16.4f}{b - a:>+10.4f}")
        lines.append("")
    if ("final", 0.0) in res:
        m0, mh = res[("final", 0.0)][0], res[("final", HEADLINE)][0]
        lines += ["FINAL WINDOW VERDICT (headline share, scored once)",
                  f"  Winner alone (our reproduction): {m0:.4f} | blend: {mh:.4f} | change {mh - m0:+.4f}",
                  f"  Official M5 winner: 0.5204 -> blend {'beats' if mh < 0.5204 else 'does not beat'} it",
                  f"  Our reproduction 0.5228 -> blend {'beats' if mh < m0 else 'does not beat'} it",
                  "  Note: the blend changes levels 1 to 9 only, so the levels are no longer exactly coherent."]
    text = "\n".join(lines)
    with open(os.path.join(args.out_dir, "results_blend_final.txt"), "w") as f:
        f.write(text + "\n")
    print(text)


if __name__ == "__main__":
    main()
```

src/m5_chronos2.py

```
#!/usr/bin/env python3
"""
E4, E5, E6: Chronos-2 on M5 as one ablation ladder (zero-shot, no training on M5).

  A_univariate     E4  sales history only
  B_covariates     E5  + price, SNAP, weekday, event flag (all known in advance)
  C_crosslearn_*   E6  + cross-learning inside groups of related series

Each configuration is run twice with the same settings:
  holdout     context ends at d_1913, forecasts d_1914..d_1941 (the 28-day test window)
  validation  context ends 28 days earlier, forecasts d_1886..d_1913

The validation window gives a clean basis for choosing models and for fitting
ensemble weights later, so nothing is ever chosen on the holdout.

Point forecast: Chronos-2 returns quantiles. WRMSSE is a squared-error metric, so
the primary point forecast is the mean implied by the quantiles (integral of the
quantile function). The median forecast is also scored and saved for transparency.
This choice is fixed in advance.

Data handling: days before an item's first price (not yet on sale) are set to
missing in the context, and missing prices are back-filled with the first price.
Scoring always uses the raw sales.

Outputs (in --out_dir)
  pred_chronos2_<cfg>.npy          holdout forecast, mean point, shape (30490, 28)
  pred_chronos2_<cfg>_median.npy   holdout forecast, median point
  predval_chronos2_<cfg>.npy       validation-window forecast, mean point
  results_chronos2.txt             plain-text log: ladder, 12-level tables, timings

Usage
  python m5_chronos2.py --quick            # 512 series, checks the setup and prints a time estimate
  python m5_chronos2.py                    # full run
  python m5_chronos2.py --groupings store_dept,item,store_cat    # try several cross-learning groupings
  python m5_chronos2.py --selftest         # synthetic data and a mock model, no GPU needed

Requires m5_data_and_wrmsse.py in the same folder. A full run needs chronos-forecasting and a CUDA PyTorch.
"""
import argparse
import os
import sys
import tempfile
import time

import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from m5_data_and_wrmsse import (  # noqa: E402
    HORIZON,
    WRMSSEEvaluator,
    find_file,
    format_table,
    load_m5,
    load_calendar,
    make_synthetic,
    seasonal_naive_forecast,
)

COVARIATES = ["price", "snap", "wday", "event"]


# ---------------------------------------------------------------------------
# Helpers
# ---------------------------------------------------------------------------
def to_np(x):
    if hasattr(x, "detach"):
        x = x.detach().cpu().float().numpy()
```

```
    return np.asarray(x)


def quantile_mean_weights(levels):
    """Weights w such that sum_j w_j * Q(p_j) approximates the mean (integral of Q over [0, 1])."""
    p = np.concatenate([[0.0], np.asarray(levels, dtype=np.float64), [1.0]])
    dp = np.diff(p)
    w = np.zeros(len(p))
    w[:-1] += dp / 2
    w[1:] += dp / 2
    out = w[1:-1].copy()
    out[0] += w[0]
    out[-1] += w[-1]
    return out


def augment_calendar(folder, n_days, seed=1):
    rng = np.random.default_rng(seed)
    cal = pd.read_csv(os.path.join(folder, "calendar.csv"))
    dates = pd.date_range("2015-01-01", periods=n_days)
    cal["date"] = dates.strftime("%Y-%m-%d")
    cal["wday"] = dates.dayofweek + 1
    cal["month"] = dates.month
    ev = rng.random(n_days) < 0.05
    cal["event_name_1"] = np.where(ev, "EventX", None)
    cal["event_type_1"] = np.where(ev, "Cultural", None)
    for st in ["CA", "TX", "WI"]:
        cal[f"snap_{st}"] = (rng.random(n_days) < 0.3).astype(int)
    cal.to_csv(os.path.join(folder, "calendar.csv"), index=False)


class MockPipeline:
    """Stands in for Chronos2Pipeline in --selftest: seasonal-naive quantiles, same interface."""

    quantiles = [0.01, 0.05, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.95, 0.99]

    def predict(self, inputs, prediction_length=28, batch_size=256, cross_learning=False, **kw):
        out = []
        for item in inputs:
            tgt = item["target"] if isinstance(item, dict) else item
            tgt = np.nan_to_num(np.asarray(tgt, dtype=np.float64))
            base = np.tile(tgt[-7:], 5)[:prediction_length]
            q = np.stack([base * f for f in np.linspace(0.6, 1.6, len(self.quantiles))])
            out.append(q[None, :, :].astype(np.float32))
        return out


# ---------------------------------------------------------------------------
# Data
# ---------------------------------------------------------------------------
class M5Inputs:
    def __init__(self, meta, values, dcols, cal_full, prices):
        self.n, self.D = values.shape
        cal = cal_full.set_index("d").loc[dcols]
        self.meta = meta

        # price matrix (n x D), NaN before an item is on sale
        wk = cal["wm_yr_wk"].to_numpy()
        weeks = np.unique(wk)
        widx = np.searchsorted(weeks, wk)
        p = prices[prices["wm_yr_wk"].isin(weeks)]
        key = pd.MultiIndex.from_frame(meta[["item_id", "store_id"]])
        r = key.get_indexer(pd.MultiIndex.from_frame(p[["item_id", "store_id"]]))
        c = np.searchsorted(weeks, p["wm_yr_wk"].to_numpy())
        ok = r >= 0
        Pw = np.full((self.n, len(weeks)), np.nan, dtype=np.float32)
        Pw[r[ok], c[ok]] = p["sell_price"].to_numpy(dtype=np.float32)[ok]
        P = Pw[:, widx]
        valid = ~np.isnan(P)
        first = np.where(valid.any(axis=1), valid.argmax(axis=1), 0)
        fill = P[np.arange(self.n), first]
        fill = np.where(np.isnan(fill), 0.0, fill).astype(np.float32)
        self.P = np.where(valid, P, fill[:, None]).astype(np.float32)

        # target with pre-launch days masked as missing
        Y = values.astype(np.float32)
        Y[~valid] = np.nan
        self.Y = Y

        # shared calendar covariates
        states = sorted(meta["state_id"].unique())
        self.state_code = pd.factorize(meta["state_id"], sort=True)[0]
        self.snap = [cal[f"snap_{s}"].to_numpy().astype(np.float32) for s in states]
        wd = cal["wday"].to_numpy().astype(int)
        self.wday = np.array([f"w{x}" for x in wd])
        self.event = cal["event_name_1"].notna().to_numpy().astype(np.float32)

        # groupings for cross-learning
        m = meta
        self.groupings = {
            "store_dept": pd.factorize(m["store_id"] + "|" + m["dept_id"], sort=True)[0],
            "store_cat": pd.factorize(m["store_id"] + "|" + m["cat_id"], sort=True)[0],
            "store": pd.factorize(m["store_id"], sort=True)[0],
```

```
            "item": pd.factorize(m["item_id"], sort=True)[0],
            "cat_state": pd.factorize(m["cat_id"] + "|" + m["state_id"], sort=True)[0],
        }

    def build(self, i, origin, ctx_len, with_cov):
        a = max(origin - ctx_len, 0)
        tgt = self.Y[i, a:origin]
        if not with_cov:
            return tgt
        sn = self.snap[self.state_code[i]]
        past = {"price": self.P[i, a:origin], "snap": sn[a:origin],
                "wday": self.wday[a:origin], "event": self.event[a:origin]}
        fut = {"price": self.P[i, origin:origin + HORIZON], "snap": sn[origin:origin + HORIZON],
               "wday": self.wday[origin:origin + HORIZON], "event": self.event[origin:origin + HORIZON]}
        return {"target": tgt, "past_covariates": past, "future_covariates": fut}


# ---------------------------------------------------------------------------
# Inference
# ---------------------------------------------------------------------------
def run_config(pipe, data, idx_series, origin, ctx_len, with_cov, group_ids, batch_size, cl_tasks, weights, quantiles, log):
    """Returns (mean_forecast, median_forecast), each (len(idx_series), HORIZON)."""
    n = len(idx_series)
    q_levels = np.asarray(quantiles)
    med_idx = int(np.argmin(np.abs(q_levels - 0.5)))
    mean_out = np.zeros((n, HORIZON), dtype=np.float32)
    med_out = np.zeros((n, HORIZON), dtype=np.float32)

    def store(pos, preds):
        arr = np.stack([to_np(t)[0] for t in preds])  # (m, n_q, H)
        mean_out[pos] = np.tensordot(arr, weights, axes=([1], [0]))
        med_out[pos] = arr[:, med_idx, :]

    if group_ids is None:
        block = 2048
        for s in range(0, n, block):
            pos = np.arange(s, min(s + block, n))
            inputs = [data.build(idx_series[j], origin, ctx_len, with_cov) for j in pos]
            store(pos, pipe.predict(inputs, prediction_length=HORIZON, batch_size=batch_size, cross_learning=False))
            if (s // block) % 5 == 0:
                log(f"    {min(s + block, n)}/{n} series")
    else:
        nv = 1 + len(COVARIATES) if with_cov else 1
        chunk = max(4, cl_tasks // nv)
        g = group_ids[idx_series]
        order = np.argsort(g, kind="stable")
        chunks = []
        for gid in np.unique(g):
            members = order[g[order] == gid]
            for s in range(0, len(members), chunk):
                chunks.append(members[s:s + chunk])
        for ci, pos in enumerate(chunks):
            inputs = [data.build(idx_series[j], origin, ctx_len, with_cov) for j in pos]
            preds = pipe.predict(inputs, prediction_length=HORIZON,
                                 batch_size=len(pos) * nv + 8, cross_learning=True)
            store(pos, preds)
            if ci % 200 == 0:
                log(f"    chunk {ci + 1}/{len(chunks)}")
    return np.clip(mean_out, 0, None), np.clip(med_out, 0, None)


# ---------------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------------
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default=None)
    ap.add_argument("--out_dir", default="m5_outputs")
    ap.add_argument("--model", default="amazon/chronos-2")
    ap.add_argument("--context_length", type=int, default=1024)
    ap.add_argument("--batch_size", type=int, default=256, help="tasks per batch when cross-learning is off")
    ap.add_argument("--cl_tasks", type=int, default=100, help="approx. tasks per cross-learning batch")
    ap.add_argument("--groupings", default="store_dept", help="comma list: store_dept,store_cat,store,item,cat_state")
    ap.add_argument("--no_val", action="store_true", help="skip validation-window forecasts")
    ap.add_argument("--quick", action="store_true", help="512 series, no WRMSSE, prints a time estimate")
    ap.add_argument("--selftest", action="store_true")
    args = ap.parse_args()

    t0 = time.time()
    logs = []

    def log(msg):
        print(msg, flush=True)

    if args.selftest:
        tmp = tempfile.mkdtemp()
        make_synthetic(tmp, n_days=240)
        augment_calendar(tmp, 240)
        data_dir = tmp
        args.context_length = 120
    elif args.data_dir:
        data_dir = args.data_dir
    elif os.path.isdir("m5_data"):
```

```
    data_dir = "m5_data"
else:
    import kagglehub
    data_dir = kagglehub.competition_download("m5-forecasting-accuracy")
log(f"Data folder: {data_dir}")

meta, values, dcols, calendar, prices = load_m5(data_dir)
cal_full = load_calendar(data_dir)
n, D = values.shape
T = D - HORIZON
log(f"Series: {n} | days: {D} | holdout origin: {T} | validation origin: {T - HORIZON}")

data = M5Inputs(meta, values, dcols, cal_full, prices)

if args.selftest:
    pipe = MockPipeline()
    device_note = "mock model (selftest)"
else:
    import torch
    from chronos import Chronos2Pipeline
    use_cuda = torch.cuda.is_available()
    device_note = f"cuda: {torch.cuda.get_device_name(0)}" if use_cuda else "CPU (no CUDA found, this will be slow)"
    log(f"Loading {args.model} on {device_note}")
    pipe = Chronos2Pipeline.from_pretrained(args.model, device_map="cuda" if use_cuda else "cpu")
weights = quantile_mean_weights(pipe.quantiles)

groupings = [g.strip() for g in args.groupings.split(",") if g.strip()]
for g in groupings:
    if g not in data.groupings:
        raise SystemExit(f"Unknown grouping {g}. Choose from {list(data.groupings)}")
configs = [("A_univariate", False, None), ("B_covariates", True, None)]
configs += [(f"C_crosslearn_{g}", True, g) for g in groupings]

if args.quick:
    sub = np.random.default_rng(0).choice(n, size=min(512, n), replace=False)
    sub.sort()
    log(f"QUICK MODE: {len(sub)} series, timing only")
    for name, with_cov, grp in configs:
        t1 = time.time()
        gid = data.groupings[grp] if grp else None
        mean_f, med_f = run_config(pipe, data, sub, T, args.context_length, with_cov, gid,
                                   args.batch_size, args.cl_tasks, weights, pipe.quantiles, log)
        dt = time.time() - t1
        est = dt / len(sub) * n / 60
        bad = int(np.isnan(mean_f).sum())
        log(f"{name}: {dt:.1f} s for {len(sub)} series | NaN count {bad} | "
            f"estimated {est:.1f} min per window for all {n} series "
            f"({'x2 with validation' if not args.no_val else 'holdout only'})")
    return

ev_hold = WRMSSEEvaluator(meta, values, dcols, calendar, prices)
ev_val = None
if not args.no_val:
    ev_val = WRMSSEEvaluator(meta, values[:, :T], dcols[:T], calendar, prices)

os.makedirs(args.out_dir, exist_ok=True)
allidx = np.arange(n)
results, ladder = {}, []
for name, with_cov, grp in configs:
    t1 = time.time()
    gid = data.groupings[grp] if grp else None
    log(f"\n{name}: holdout window")
    mean_f, med_f = run_config(pipe, data, allidx, T, args.context_length, with_cov, gid,
                               args.batch_size, args.cl_tasks, weights, pipe.quantiles, log)
    s_mean, s_med = ev_hold.score(mean_f), ev_hold.score(med_f)
    np.save(os.path.join(args.out_dir, f"pred_chronos2_{name}.npy"), mean_f)
    np.save(os.path.join(args.out_dir, f"pred_chronos2_{name}_median.npy"), med_f)
    val_score = None
    if ev_val is not None:
        log(f"{name}: validation window")
        vmean, _ = run_config(pipe, data, allidx, T - HORIZON, args.context_length, with_cov, gid,
                              args.batch_size, args.cl_tasks, weights, pipe.quantiles, log)
        val_score = ev_val.score(vmean)
        np.save(os.path.join(args.out_dir, f"predval_chronos2_{name}.npy"), vmean)
    mins = (time.time() - t1) / 60
    results[name] = s_mean
    ladder.append((name, s_mean[0], s_med[0], None if val_score is None else val_score[0], mins))
    log(f"{name}: holdout WRMSSE (mean point) {s_mean[0]:.4f} | median point {s_med[0]:.4f} | {mins:.1f} min")

snaive = ev_hold.score(seasonal_naive_forecast(ev_hold.X_train))
table_results = {"SeasonalNaive": snaive}
for name, _, _ in configs:
    table_results[name.replace("crosslearn", "cl")] = results[name]

lines = [f"{'Config':<28}{'Holdout mean':>14}{'Holdout median':>16}{'Validation mean':>17}{'Min':>7}"]
for name, hm, hmed, v, mins in ladder:
    vs = "n/a" if v is None else f"{v:.4f}"
    lines.append(f"{name:<28}{hm:>14.4f}{hmed:>16.4f}{vs:>17}{mins:>7.1f}")

report = [
    "CHRONOS-2 ON M5 (zero-shot ablation ladder)",
    f"Data folder: {data_dir}",
    f"Device: {device_note} | model: {args.model}",
```

```
        f"Context length: {args.context_length} days | cross-learning tasks per batch: about {args.cl_tasks}",
        f"Training window: d_1 to d_{T} | Holdout (28 days): d_{T + 1} to d_{D}",
        "Point forecast: mean implied by the quantiles (primary, fixed in advance); median also reported.",
        "No training on M5 was done. Whether M5 or similar retail data was in the Chronos-2 pretraining corpus",
        "is not checked here and should be addressed in the limitations section.",
        "",
        "LADDER (WRMSSE, lower is better; validation window is d_%d to d_%d)" % (T - HORIZON + 1, T),
        *lines,
        "",
        "WRMSSE BY LEVEL (holdout, mean point)",
        format_table(ev_hold, table_results),
        "",
        "REFERENCE VALUES (published, window to be confirmed)",
        "  Original M5 winner: 0.520 | Yang et al. Top-1 reproduction: 0.5230 | Yang et al. LightGBM + PatchTST: 0.4989",
        "  Our E1b L3 (LightGBM, unweighted, all features): 0.5422",
        "",
        f"Runtime: {(time.time() - t0) / 60:.1f} min",
    ]
    text = "\n".join(report)
    out_path = os.path.join(args.out_dir, "results_chronos2.txt")
    with open(out_path, "w") as f:
        f.write(text + "\n")
    print("\n" + text)
    print(f"\nSaved: {out_path}")


if __name__ == "__main__":
    main()
```

src/m5_compare_levels.py

```
#!/usr/bin/env python3
"""
Level-by-level comparison of our saved forecasts with the published Yang et al. rows.

Purpose: Yang et al. do not state which 28-day window they used. Our models produce forecasts for two windows:
  holdout     d_1914 to d_1941
  validation  d_1886 to d_1913 (forecasts saved as predval_*.npy)
This script scores each saved forecast on both windows and prints all 12 WRMSSE levels next to the published
rows, so the window whose level pattern is closer to the paper can be judged. It trains nothing and only reads
files. It does not select any model; it only helps decide which window the published numbers are comparable to.

Note: validation forecasts are mildly optimistic for LightGBM (early stopping used that window), so a closer
match on validation is suggestive, not proof.

Usage
  python m5_compare_levels.py --names E1c_lightgbm,E1c_lightgbm_cal,E1c_store
  python m5_compare_levels.py --names E1c_lightgbm,chronos2_B_covariates --out_dir m5_outputs
Looks for pred_<name>.npy and predval_<name>.npy in --out_dir. A name with no predval file is scored on the holdout only.

Output: printed table and compare_levels.txt in --out_dir.
"""
import argparse
import os
import sys

import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from m5_data_and_wrmsse import (  # noqa: E402
    HORIZON,
    WRMSSEEvaluator,
    load_m5,
    seasonal_naive_forecast,
)

# Published rows (Yang et al., Table 1 and Table 2): Avg, then Levels 1 to 12
YANG = {
    "Yang Kaggle-Top1": [0.5230, 0.2006, 0.3123, 0.3992, 0.2843, 0.3725, 0.3953, 0.4792, 0.4815, 0.5742, 0.9649, 0.9283, 0.8838],
    "Yang AE LGB+PatchTST": [0.4989, 0.1455, 0.2907, 0.3830, 0.2244, 0.3392, 0.3688, 0.4631, 0.4625, 0.5623, 0.9528, 0.9188, 0.8758],
}


def row(label, avg, levels):
    return f"{label:<30}{avg:>7.4f}" + "".join(f"{v:>7.3f}" for v in levels)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default="m5_data")
    ap.add_argument("--out_dir", default="m5_outputs")
    ap.add_argument("--names", required=True, help="comma list of forecast names, files pred_<name>.npy / predval_<name>.npy")
    args = ap.parse_args()

    meta, values, dcols, calendar, prices = load_m5(args.data_dir)
    n, D = values.shape
    T = D - HORIZON
    ev_hold = WRMSSEEvaluator(meta, values, dcols, calendar, prices)
    ev_val = WRMSSEEvaluator(meta, values[:, :T], dcols[:T], calendar, prices)

    head = f"{'Model / window':<30}{'Avg':>7}" + "".join(f"{'L' + str(i):>7}" for i in range(1, 13))
```

```
    lines = [head, "-" * len(head)]
    for label, v in YANG.items():
        lines.append(row(label + " (paper)", v[0], v[1:]))
    lines.append("-" * len(head))

    sn_v = ev_val.score(seasonal_naive_forecast(ev_val.X_train))
    sn_h = ev_hold.score(seasonal_naive_forecast(ev_hold.X_train))
    lines.append(row("SeasonalNaive | validation", sn_v[0], sn_v[1]))
    lines.append(row("SeasonalNaive | holdout", sn_h[0], sn_h[1]))
    lines.append("-" * len(head))

    for nm in [s.strip() for s in args.names.split(",") if s.strip()]:
        ph = os.path.join(args.out_dir, f"pred_{nm}.npy")
        pv = os.path.join(args.out_dir, f"predval_{nm}.npy")
        if os.path.exists(pv):
            s = ev_val.score(np.load(pv))
            lines.append(row(f"{nm} | validation", s[0], s[1]))
        else:
            lines.append(f"{nm} | validation: no predval file")
        if os.path.exists(ph):
            s = ev_hold.score(np.load(ph))
            lines.append(row(f"{nm} | holdout", s[0], s[1]))
        else:
            lines.append(f"{nm} | holdout: no pred file")
        lines.append("")

    text = "\n".join([
        "LEVEL COMPARISON WITH PUBLISHED ROWS",
        "Validation window: d_%d to d_%d | Holdout window: d_%d to d_%d" % (T - HORIZON + 1, T, T + 1, D),
        "Lower is better. Level 1 = total, 9 = store x department, 10 = item, 11 = item x state, 12 = item x store.",
        "",
        *lines,
    ])
    print(text)
    with open(os.path.join(args.out_dir, "compare_levels.txt"), "w") as f:
        f.write(text + "\n")
    print(f"\nSaved: {os.path.join(args.out_dir, 'compare_levels.txt')}")


if __name__ == "__main__":
    main()
```

src/m5_data_and_wrmsse.py

```
#!/usr/bin/env python3
"""
M5 data preparation + official-style 12-level WRMSSE evaluator (Phase 1 and 2).

What this single script does
  1. Downloads M5 via kagglehub (or reads a local folder with --data_dir).
  2. Defines ONE fixed 28-day holdout: the last 28 observed days of
     sales_train_evaluation.csv (d_1914..d_1941 on the real data).
  3. Builds the 12 hierarchical aggregation levels (42,840 series).
  4. Computes WRMSSE exactly as in the M5 protocol:
       - RMSSE scale = mean squared first difference of the training series,
         starting from its first non-zero observation
       - weights = dollar sales over the last 28 training days, normalised
         within each level
       - overall WRMSSE = mean of the 12 level scores
  5. Runs Naive and Seasonal Naive as pipeline sanity checks.
  6. Writes a plain-text results file (AI-readable) to --out_dir.

Usage
  python m5_data_and_wrmsse.py                     # downloads M5 with kagglehub
  python m5_data_and_wrmsse.py --data_dir /path/to/m5
  python m5_data_and_wrmsse.py --selftest          # synthetic data, no download

Later phases only need: ev = WRMSSEEvaluator(...); ev.score(pred) where pred has
shape (30490, 28) in the same row order as sales_train_evaluation.csv.
"""
import argparse
import os
import tempfile
import time

import numpy as np
import pandas as pd
from scipy import sparse

HORIZON = 28
WEIGHT_WINDOW = 28
ID_COLS = ["item_id", "dept_id", "cat_id", "store_id", "state_id"]

LEVELS = [
    (1, "Total", []),
    (2, "State", ["state_id"]),
    (3, "Store", ["store_id"]),
    (4, "Category", ["cat_id"]),
    (5, "Department", ["dept_id"]),
    (6, "State x Category", ["state_id", "cat_id"]),
    (7, "State x Department", ["state_id", "dept_id"]),
```

```
    (8, "Store x Category", ["store_id", "cat_id"]),
    (9, "Store x Department", ["store_id", "dept_id"]),
    (10, "Item", ["item_id"]),
    (11, "Item x State", ["item_id", "state_id"]),
    (12, "Item x Store", ["item_id", "store_id"]),
]
EXPECTED_SIZES = [1, 3, 10, 3, 7, 9, 21, 30, 70, 3049, 9147, 30490]


# ---------------------------------------------------------------------------
# Data loading
# ---------------------------------------------------------------------------
def find_file(root, name):
    for dirpath, _, files in os.walk(root):
        if name in files:
            return os.path.join(dirpath, name)
    raise FileNotFoundError(f"{name} not found under {root}")


def load_calendar(data_dir):
    """Full calendar with a 'd' column (rebuilt as d_1, d_2, ... when the full organiser release has none)."""
    calendar = pd.read_csv(find_file(data_dir, "calendar.csv"))
    if "d" not in calendar.columns:
        calendar["d"] = ["d_%d" % (i + 1) for i in range(len(calendar))]
    return calendar


def load_m5(data_dir, with_test=None):
    """Load M5 sales, calendar and prices (Kaggle files or the full organiser release).

    Window: if sales_test_evaluation.csv (actual sales d_1942..d_1969) is found, its 28 days are appended, so the
    last 28 days (the holdout) are the competition's FINAL window d_1942..d_1969 and the 28 days before that
    (d_1914..d_1941) are the validation window. Without that file, or with environment variable M5_WINDOW=public
    (or with_test=False), the holdout is d_1914..d_1941 and the validation window is d_1886..d_1913.
    """
    sales = pd.read_csv(find_file(data_dir, "sales_train_evaluation.csv"))
    calendar = load_calendar(data_dir)[["d", "wm_yr_wk"]]
    prices = pd.read_csv(find_file(data_dir, "sell_prices.csv"))
    # Full organiser release: no 'id' in the sales file. Rebuild it in the Kaggle layout.
    if "id" not in sales.columns:
        sales.insert(0, "id", sales["item_id"] + "_" + sales["store_id"] + "_evaluation")
    dcols = sorted([c for c in sales.columns if c.startswith("d_")], key=lambda c: int(c[2:]))
    meta = sales[["id"] + ID_COLS].copy()
    values = sales[dcols].to_numpy(dtype=np.float64)
    if with_test is None:
        with_test = os.environ.get("M5_WINDOW", "final").lower() != "public"
    if with_test:
        try:
            tpath = find_file(data_dir, "sales_test_evaluation.csv")
        except FileNotFoundError:
            tpath = None
        if tpath is not None:
            tdf = pd.read_csv(tpath)
            tcols = sorted([c for c in tdf.columns if c.startswith("d_")], key=lambda c: int(c[2:]))
            if len(tcols) == HORIZON and int(tcols[0][2:]) == len(dcols) + 1:
                tdf = tdf.set_index(tdf["item_id"] + "|" + tdf["store_id"])
                tv = tdf.reindex(meta["item_id"] + "|" + meta["store_id"])[tcols].to_numpy(dtype=np.float64)
                if not np.isnan(tv).any():
                    values = np.concatenate([values, tv], axis=1)
                    dcols = list(dcols) + tcols
    return meta, values, dcols, calendar, prices


# ---------------------------------------------------------------------------
# Hierarchy
# ---------------------------------------------------------------------------
def build_aggregators(meta):
    """One sparse (n_groups x n_bottom) summing matrix per level."""
    n = len(meta)
    mats = []
    for _, _, keys in LEVELS:
        if keys:
            gid = meta.groupby(keys, sort=True).ngroup().to_numpy()
        else:
            gid = np.zeros(n, dtype=np.int64)
        g = int(gid.max()) + 1
        A = sparse.csr_matrix((np.ones(n), (gid, np.arange(n))), shape=(g, n))
        mats.append(A)
    return mats


# ---------------------------------------------------------------------------
# Evaluator
# ---------------------------------------------------------------------------
def rmsse_scale(block):
    """Mean squared first difference starting at the first non-zero value."""
    m, T = block.shape
    nz = block > 0
    first = np.where(nz.any(axis=1), nz.argmax(axis=1), T)
    d2 = np.diff(block, axis=1) ** 2
    idx = np.arange(1, T)[None, :]
    mask = idx > first[:, None]
    cnt = mask.sum(axis=1)
```

```
    total = (d2 * mask).sum(axis=1)
    return np.where(cnt > 0, total / np.maximum(cnt, 1), np.nan)


class WRMSSEEvaluator:
    def __init__(self, meta, values, dcols, calendar, prices, horizon=HORIZON):
        self.meta = meta
        self.horizon = horizon
        self.train_cols = dcols[:-horizon]
        self.test_cols = dcols[-horizon:]
        self.X_train = values[:, : -horizon]
        self.y_test = values[:, -horizon:]
        self.A = build_aggregators(meta)
        self.sizes = [A.shape[0] for A in self.A]

        dollars = self.X_train[:, -WEIGHT_WINDOW:] * self._price_matrix(
            calendar, prices, self.train_cols[-WEIGHT_WINDOW:]
        )

        self.scale, self.weight, self.y_agg = [], [], []
        for A in self.A:
            G = A.shape[0]
            sc = np.empty(G)
            for s in range(0, G, 4000):
                blk = A[s : s + 4000] @ self.X_train
                sc[s : s + 4000] = rmsse_scale(blk)
            dol = np.asarray(A @ dollars).sum(axis=1)
            w = dol / dol.sum()
            w = np.where(np.isfinite(sc) & (sc > 0), w, 0.0)
            self.scale.append(sc)
            self.weight.append(w)
            self.y_agg.append(np.asarray(A @ self.y_test))

    def _price_matrix(self, calendar, prices, cols):
        wk_of_day = calendar.set_index("d")["wm_yr_wk"]
        weeks = sorted(set(wk_of_day.loc[cols]))
        keys = pd.MultiIndex.from_frame(self.meta[["item_id", "store_id"]])
        per_week = {}
        for w in weeks:
            p = prices[prices["wm_yr_wk"] == w].set_index(["item_id", "store_id"])["sell_price"]
            per_week[w] = p.reindex(keys).to_numpy(dtype=np.float64)
        out = np.zeros((len(self.meta), len(cols)))
        for j, c in enumerate(cols):
            out[:, j] = np.nan_to_num(per_week[wk_of_day.loc[c]], nan=0.0)
        return out

    def score(self, pred):
        """pred: (n_bottom_series, horizon). Returns (overall, list of level scores)."""
        pred = np.asarray(pred, dtype=np.float64)
        assert pred.shape == self.y_test.shape, f"pred shape {pred.shape} != {self.y_test.shape}"
        out = []
        for A, sc, w, Y in zip(self.A, self.scale, self.weight, self.y_agg):
            P = np.asarray(A @ pred)
            valid = w > 0
            rmsse = np.zeros(len(sc))
            rmsse[valid] = np.sqrt(np.mean((Y[valid] - P[valid]) ** 2, axis=1) / sc[valid])
            out.append(float(np.sum(w * rmsse)))
        return float(np.mean(out)), out


# ---------------------------------------------------------------------------
# Baselines
# ---------------------------------------------------------------------------
def naive_forecast(X_train, h=HORIZON):
    return np.repeat(X_train[:, -1:], h, axis=1)


def seasonal_naive_forecast(X_train, h=HORIZON, period=7):
    last = X_train[:, -period:]
    reps = int(np.ceil(h / period))
    return np.tile(last, (1, reps))[:, :h]


# ---------------------------------------------------------------------------
# Reporting
# ---------------------------------------------------------------------------
def format_table(ev, results):
    names = list(results.keys())
    lines = []
    head = f"{'Level':<6}{'Name':<22}{'Series':>8}" + "".join(f"{n:>16}" for n in names)
    lines.append(head)
    lines.append("-" * len(head))
    for i, (lv, name, _) in enumerate(LEVELS):
        row = f"{lv:<6}{name:<22}{ev.sizes[i]:>8}"
        row += "".join(f"{results[n][1][i]:>16.4f}" for n in names)
        lines.append(row)
    lines.append("-" * len(head))
    lines.append(f"{'':<6}{'OVERALL WRMSSE':<22}{sum(ev.sizes):>8}" + "".join(f"{results[n][0]:>16.4f}" for n in names))
    return "\n".join(lines)


def run_checks(ev, real_data):
    notes = []
```

```
    for i, (lv, name, _) in enumerate(LEVELS):
        s = ev.weight[i].sum()
        if abs(s - 1.0) > 1e-6:
            notes.append(f"WARNING level {lv} weights sum to {s:.6f}, expected 1")
    perfect, _ = ev.score(ev.y_test)
    if perfect > 1e-9:
        notes.append(f"WARNING perfect forecast scored {perfect:.3e}, expected 0")
    if real_data:
        if ev.sizes != EXPECTED_SIZES:
            notes.append(f"WARNING level sizes {ev.sizes} differ from expected {EXPECTED_SIZES}")
        if sum(ev.sizes) != 42840:
            notes.append(f"WARNING total series {sum(ev.sizes)} != 42840")
    if not notes:
        notes.append("All sanity checks passed (weights sum to 1 per level, perfect forecast = 0"
                     + (", 42,840 series across 12 levels)." if real_data else ")."))
    return notes


# ---------------------------------------------------------------------------
# Synthetic data for --selftest
# ---------------------------------------------------------------------------
def make_synthetic(folder, n_days=140, seed=0):
    rng = np.random.default_rng(seed)
    depts = {"FOODS_1": "FOODS", "FOODS_2": "FOODS", "FOODS_3": "FOODS",
             "HOBBIES_1": "HOBBIES", "HOBBIES_2": "HOBBIES",
             "HOUSEHOLD_1": "HOUSEHOLD", "HOUSEHOLD_2": "HOUSEHOLD"}
    stores = {"CA_1": "CA", "CA_2": "CA", "CA_3": "CA", "CA_4": "CA",
              "TX_1": "TX", "TX_2": "TX", "TX_3": "TX",
              "WI_1": "WI", "WI_2": "WI", "WI_3": "WI"}
    rows = []
    for dept, cat in depts.items():
        for k in range(1, 4):
            item = f"{dept}_{k:03d}"
            for store, state in stores.items():
                base = rng.uniform(0.2, 6)
                dow = 1 + 0.4 * np.sin(np.arange(n_days) * 2 * np.pi / 7 + rng.uniform(0, 6))
                y = rng.poisson(base * dow)
                y[: rng.integers(0, 30)] = 0
                rows.append([f"{item}_{store}_evaluation", item, dept, cat, store, state] + list(y))
    cols = ["id", "item_id", "dept_id", "cat_id", "store_id", "state_id"] + [f"d_{i}" for i in range(1, n_days + 1)]
    pd.DataFrame(rows, columns=cols).to_csv(os.path.join(folder, "sales_train_evaluation.csv"), index=False)
    cal = pd.DataFrame({"d": [f"d_{i}" for i in range(1, n_days + 1)],
                        "wm_yr_wk": [11101 + (i // 7) for i in range(n_days)]})
    cal.to_csv(os.path.join(folder, "calendar.csv"), index=False)
    pr = []
    for r in rows:
        for wk in sorted(cal["wm_yr_wk"].unique()):
            pr.append([r[4], r[1], wk, round(float(rng.uniform(1, 10)), 2)])
    pd.DataFrame(pr, columns=["store_id", "item_id", "wm_yr_wk", "sell_price"]).to_csv(
        os.path.join(folder, "sell_prices.csv"), index=False)


# ---------------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------------
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default=None, help="Folder containing the M5 csv files")
    ap.add_argument("--out_dir", default="m5_outputs")
    ap.add_argument("--selftest", action="store_true", help="Use small synthetic M5-format data")
    args = ap.parse_args()

    t0 = time.time()
    tmp = None
    if args.selftest:
        tmp = tempfile.mkdtemp()
        make_synthetic(tmp)
        data_dir = tmp
    elif args.data_dir:
        data_dir = args.data_dir
    else:
        import kagglehub
        data_dir = kagglehub.competition_download("m5-forecasting-accuracy")
    print(f"Data folder: {data_dir}")

    meta, values, dcols, calendar, prices = load_m5(data_dir)
    print(f"Loaded {values.shape[0]} bottom series x {values.shape[1]} days")

    ev = WRMSSEEvaluator(meta, values, dcols, calendar, prices)
    print(f"Train: {ev.train_cols[0]}..{ev.train_cols[-1]} | Holdout: {ev.test_cols[0]}..{ev.test_cols[-1]}")

    notes = run_checks(ev, real_data=not args.selftest)

    results = {
        "Naive": ev.score(naive_forecast(ev.X_train)),
        "SeasonalNaive": ev.score(seasonal_naive_forecast(ev.X_train)),
    }
    table = format_table(ev, results)

    report = [
        "M5 DATA AND WRMSSE PIPELINE RESULTS (Phase 1 and 2)",
        f"Data folder: {data_dir}",
        f"Bottom series: {values.shape[0]} | Days: {values.shape[1]}",
```

```
        f"Training window: {ev.train_cols[0]} to {ev.train_cols[-1]}",
        f"Holdout window (28 days): {ev.test_cols[0]} to {ev.test_cols[-1]}",
        f"Weight window: last {WEIGHT_WINDOW} training days, dollar sales (units x price)",
        "Overall WRMSSE = mean of the 12 level scores",
        "",
        "SANITY CHECKS",
        *notes,
        "",
        "WRMSSE BY LEVEL",
        table,
        "",
        f"Runtime: {time.time() - t0:.1f} s",
    ]
    text = "\n".join(report)
    os.makedirs(args.out_dir, exist_ok=True)
    out_path = os.path.join(args.out_dir, "results_phase1_baselines.txt")
    with open(out_path, "w") as f:
        f.write(text + "\n")
    print("\n" + text)
    print(f"\nSaved: {out_path}")


if __name__ == "__main__":
    main()
```

src/m5_final_analysis.py

```
"""
E9: Uncertainty and ablation for the blend of the winner (six-set LightGBM average) with direct Chronos-2 forecasts.

Nothing here is tuned. All variants use the same fixed share (0.20) unless the label says otherwise, and the headline
blend is the one fixed in advance (direct Chronos-2 with covariates, mean point, 20 percent at levels 1 to 9).

Part 1, ablation: what each piece adds, scored on every window that has actuals (cv4, public, final).
Part 2, uncertainty: circular block bootstrap over the 28 forecast days (blocks of 7 days, 4 blocks per draw, 2000 draws),
  giving an interval for the change in WRMSSE against the winner alone and for the WRMSSE itself.
  This resamples days only. It does not capture uncertainty from the choice of series or the model, so treat the
  intervals as a rough guide with only four weeks of data per window.
Part 3, secondary outcomes: stability across windows and level-1 bias (total forecast divided by total actual).

Usage:
  python m5_final_analysis.py --data_dir m5_data --in_dir analysis_inputs --out_dir analysis_out
"""
import argparse
import os
import sys

import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from m5_data_and_wrmsse import HORIZON, LEVELS, WRMSSEEvaluator, load_m5  # noqa: E402

SHARE = 0.20
ORIGIN = {"cv4": 1885, "public": 1913, "final": 1941}
N_DRAWS, BLOCK, N_BLOCKS = 2000, 7, 4
OFFICIAL = 0.5204


def level_preds(ev, winner, kind, extra, s):
    """Aggregated forecasts for all 12 levels. Levels 1 to 9 are mixed with `extra`; levels 10 to 12 stay as the winner."""
    off = np.cumsum([0] + ev.sizes[:9])
    out = []
    for i, A in enumerate(ev.A):
        P = np.asarray(A @ winner)
        if i < 9 and kind is not None and s > 0:
            other = extra[off[i]:off[i + 1]] if kind == "direct" else np.asarray(A @ extra)
            P = (1 - s) * P + s * other
        out.append(P)
    return out


def sq_errors(ev, preds):
    return [(Y - P) ** 2 for Y, P in zip(ev.y_agg, preds)]


def point_score(ev, errs):
    lv = []
    for E, sc, w in zip(errs, ev.scale, ev.weight):
        v = w > 0
        r = np.zeros(len(sc))
        r[v] = np.sqrt(E[v].mean(axis=1) / sc[v])
        lv.append(float(np.sum(w * r)))
    return float(np.mean(lv)), lv


def boot_scores(ev, errs, counts):
    """counts: (B, 28) day counts. Returns (B,) overall WRMSSE per draw."""
    tot = np.zeros(counts.shape[0])
    for E, sc, w in zip(errs, ev.scale, ev.weight):
        v = np.where(w > 0)[0]
        for s in range(0, counts.shape[0], 250):
```

```
            M = (counts[s:s + 250] / counts.shape[1]) @ E[v].T  # (b, G)
            tot[s:s + 250] += (np.sqrt(M / sc[v]) * w[v]).sum(axis=1)
    return tot / len(ev.A)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default="m5_data")
    ap.add_argument("--in_dir", default="analysis_inputs")
    ap.add_argument("--out_dir", default="analysis_out")
    args = ap.parse_args()
    os.makedirs(args.out_dir, exist_ok=True)
    rng = np.random.default_rng(12345)

    meta, values, dcols, calendar, prices = load_m5(args.data_dir)
    dcols = list(dcols)
    D = values.shape[1]
    L = lambda n: np.load(os.path.join(args.in_dir, n))  # noqa: E731

    # fixed bootstrap draws, shared by every window and variant
    starts = rng.integers(0, HORIZON, size=(N_DRAWS, N_BLOCKS))
    counts = np.zeros((N_DRAWS, HORIZON))
    for b in range(N_DRAWS):
        for st in starts[b]:
            for j in range(BLOCK):
                counts[b, (st + j) % HORIZON] += 1

    lines = ["ABLATION, UNCERTAINTY AND SECONDARY OUTCOMES FOR THE BLEND (winner + direct Chronos-2 at levels 1 to 9)",
             f"Fixed share {SHARE:.2f} unless stated. WRMSSE over 12 levels, lower is better. Bootstrap: {N_DRAWS} draws, "
             f"circular blocks of {BLOCK} days, {N_BLOCKS} blocks per draw (days only, rough guide).",
             f"Days in data folder: {D}.", ""]
    summary = {}
    for win in ["cv4", "public", "final"]:
        end = ORIGIN[win] + HORIZON
        if D < end:
            lines += [f"WINDOW {win}: skipped, data folder has {D} days and {end} are needed.", ""]
            continue
        ev = WRMSSEEvaluator(meta, values[:, :end], dcols[:end], calendar, prices)
        w = L(f"winner_{win}.npy")
        variants = {
            "winner alone": (None, None, 0.0),
            "HEADLINE direct B mean 20%": ("direct", L(f"agg_chronos2_B_covariates_{win}.npy"), SHARE),
            "direct B mean 10%": ("direct", L(f"agg_chronos2_B_covariates_{win}.npy"), 0.10),
            "direct B mean 30%": ("direct", L(f"agg_chronos2_B_covariates_{win}.npy"), 0.30),
            "direct B median 20%": ("direct", L(f"agg_chronos2_B_covariates_{win}_median.npy"), SHARE),
            "direct A (no covariates) 20%": ("direct", L(f"agg_chronos2_A_univariate_{win}.npy"), SHARE),
            "bottom-up Chronos-2 C 20%": ("bu", L(f"bu_chronosC_{win}.npy"), SHARE),
            "bottom-up PatchTST P1 20%": ("bu", L(f"bu_patchtstP1_{win}.npy"), SHARE),
        }
        res = {}
        for name, (kind, extra, s) in variants.items():
            preds = level_preds(ev, w, kind, extra, s)
            errs = sq_errors(ev, preds)
            pt, lv = point_score(ev, errs)
            bs = boot_scores(ev, errs, counts)
            res[name] = (pt, lv, bs, preds)
        base_pt, _, base_bs, base_preds = res["winner alone"]
        lines.append(f"WINDOW {win} (forecast d_{ORIGIN[win] + 1}..d_{end})")
        lines.append(f"{'Variant':<32}{'WRMSSE':>9}{'Change':>9}{'95% interval of change':>26}{'P(better)':>11}")
        for name, (pt, lv, bs, _) in res.items():
            d = bs - base_bs
            lo, hi = np.percentile(d, [2.5, 97.5])
            pb = float((d < 0).mean()) if name != "winner alone" else float("nan")
            lines.append(f"{name:<32}{pt:>9.4f}{pt - base_pt:>+9.4f}{f'[{lo:+.4f}, {hi:+.4f}]':>26}"
                         f"{'' if name == 'winner alone' else f'{pb:>11.2f}'}")
        pt_h, _, bs_h, preds_h = res["HEADLINE direct B mean 20%"]
        lo_w, hi_w = np.percentile(base_bs, [2.5, 97.5])
        lo_h, hi_h = np.percentile(bs_h, [2.5, 97.5])
        lines.append(f"  Winner alone WRMSSE 95% interval [{lo_w:.4f}, {hi_w:.4f}] | headline blend [{lo_h:.4f}, {hi_h:.4f}]")
        if win == "final":
            lines.append(f"  Share of draws where the headline blend is below the official {OFFICIAL}: {(bs_h < OFFICIAL).mean():.2f} "
                         f"(the official score comes from a different run of the winner's code, so this is a rough reference)")
            lines.append(f"  Share of draws where the winner alone is below the official {OFFICIAL}: {(base_bs < OFFICIAL).mean():.2f}")
        # level-1 bias
        tot_act = float(ev.y_agg[0].sum())
        lines.append(f"  Level 1 bias (total forecast / total actual): winner {base_preds[0].sum() / tot_act:.3f} | headline blend "
                     f"{preds_h[0].sum() / tot_act:.3f}")
        lines.append(f"  {'Level':<24}{'Winner':>9}{'Headline':>10}{'Change':>9}")
        for i, (lv_no, lv_name, _) in enumerate(LEVELS):
            a, b = res["winner alone"][1][i], res["HEADLINE direct B mean 20%"][1][i]
            lines.append(f"  {lv_no:>2} {lv_name:<21}{a:>9.4f}{b:>10.4f}{b - a:>+9.4f}")
        lines.append("")
        summary[win] = (base_pt, pt_h)

    if summary:
        lines.append("STABILITY ACROSS WINDOWS (headline blend minus winner alone, WRMSSE; negative is better)")
        for win, (b0, h) in summary.items():
            lines.append(f"  {win:<8} winner {b0:.4f} | blend {h:.4f} | change {h - b0:+.4f}")
        ch = [h - b0 for b0, h in summary.values()]
        lines.append(f"  Windows improved: {sum(c < 0 for c in ch)} of {len(ch)}. Worst-case change: {max(ch):+.4f}.")
    text = "\n".join(lines)
    with open(os.path.join(args.out_dir, "results_final_analysis.txt"), "w") as f:
        f.write(text + "\n")
```

```
    print(text)


if __name__ == "__main__":
    main()
```

src/m5_fm_forecast.py

```
"""
E11: zero-shot foundation-model forecasts of the summed series (M5 levels 1 to 9, 154 series) for six windows.
One model per run (each in its own Python environment). Same inputs for every model as the Chronos-2 run:
  target = summed sales, context 1024 days; covariates known for the whole context and horizon:
  SNAP share of the group, event flag, weekday (7 one-hot columns). No training on M5.
usage: python m5_fm_forecast.py --model t0|timestral|timesfm3|tirex2g|mock --data_dir <M5 data> --out_dir fm_out
Output: fm_out/fm_<model>_<window>.npy (154 x 28 mean-like point), fm_<model>_<window>_median.npy, fm_<model>_log.txt
Windows: cv5 origin 1801, cv2 1829, cv3 1857, cv4 1885, public 1913, final 1941 (forecast the 28 days after the origin).
Existing output files are never overwritten.
"""
import argparse
import os
import sys
import time

import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from m5_data_and_wrmsse import HORIZON, WRMSSEEvaluator, load_calendar, load_m5  # noqa: E402

ORIGINS = {"cv5": 1801, "cv2": 1829, "cv3": 1857, "cv4": 1885, "public": 1913, "final": 1941}
CTX = 1024
N_AGG_LEVELS = 9


def mean_weights(levels):
    p = np.concatenate([[0.0], np.asarray(levels, dtype=np.float64), [1.0]])
    dp = np.diff(p)
    w = np.zeros(len(p))
    w[:-1] += dp / 2
    w[1:] += dp / 2
    out = w[1:-1].copy()
    out[0] += w[0]
    out[-1] += w[-1]
    return out


def build(ev, meta, values, cal_full, o):
    """Targets (154 arrays of length CTX) and covariates (154 arrays (9, CTX + HORIZON)) for origin o."""
    a = max(o - CTX, 0)
    cal = cal_full.set_index("d").iloc[a:o + HORIZON]
    states = sorted(meta["state_id"].unique())
    onehot = np.zeros((len(meta), len(states)))
    onehot[np.arange(len(meta)), np.searchsorted(states, meta["state_id"].to_numpy())] = 1.0
    snap = np.stack([cal[f"snap_{s}"].to_numpy().astype(np.float32) for s in states])
    wd = cal["wday"].to_numpy().astype(int)
    wd_oh = np.stack([(wd == k).astype(np.float32) for k in range(1, 8)])
    event = cal["event_name_1"].notna().to_numpy().astype(np.float32)[None]
    T = o - a
    assert snap.shape[1] == T + HORIZON, (snap.shape, T)
    targets, covs = [], []
    for A in ev.A[:N_AGG_LEVELS]:
        Y = np.asarray(A @ values[:, a:o], dtype=np.float32)
        size = np.asarray(A.sum(axis=1)).ravel()
        share = np.asarray(A @ onehot) / size[:, None]
        snap_g = (share @ snap).astype(np.float32)
        for g in range(A.shape[0]):
            targets.append(Y[g])
            covs.append(np.concatenate([snap_g[g][None], event, wd_oh], axis=0))
    return targets, covs


# ------------------------------------------------------------------ adapters: return (mean_point, median) each (n, HORIZON)
class Mock:
    name, used_covariates = "mock", True

    def predict(self, targets, covs):
        out = np.stack([np.tile(t[-7:], 4) for t in targets]).astype(np.float32)
        return out, out


class T0:
    name, used_covariates = "t0", True

    def __init__(self):
        import torch
        from t0 import T0Forecaster
        self.torch = torch
        self.dev = "cuda" if torch.cuda.is_available() else "cpu"
        self.model = T0Forecaster.from_pretrained("theforecastingcompany/t0-gamma").eval().to(self.dev)
        self.levels = [0.01, 0.05, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.95, 0.99]

    def predict(self, targets, covs):
```

```
        t = self.torch
        means, meds = [], []
        for i in range(0, len(targets), 16):
            ctx = t.tensor(np.stack(targets[i:i + 16]), dtype=t.float32, device=self.dev)
            fut = t.tensor(np.stack(covs[i:i + 16]), dtype=t.float32, device=self.dev)
            with t.no_grad():
                out = self.model.predict(ctx, horizon=HORIZON, quantile_levels=self.levels, future_covariates=fut)
            q = out.quantiles.float().cpu().numpy()
            means.append(np.tensordot(q, mean_weights(self.levels), axes=([2], [0])))
            meds.append(out.median.float().cpu().numpy())
        return np.concatenate(means), np.concatenate(meds)


class Timestral:
    name, used_covariates = "timestral", True

    def __init__(self):
        import torch
        from timestral import TimestralForecaster
        self.dev = "cuda" if torch.cuda.is_available() else "cpu"
        self.model = TimestralForecaster.from_pretrained("mistralai/Timestral-209M", device=self.dev)

    def predict(self, targets, covs):
        means, meds = [], []
        for i in range(0, len(targets), 16):
            ctx = np.stack(targets[i:i + 16])[:, None, :].astype(np.float32)  # (B, 1, T)
            kn = np.stack(covs[i:i + 16]).astype(np.float32)  # (B, C, T + H)
            fc = self.model.forecast(ctx, prediction_length=HORIZON, known_covariates=kn)
            q = np.asarray(fc.quantiles)[:, 0]  # (B, H, 21)
            lv = np.asarray(fc.quantile_levels, dtype=np.float64)
            means.append(np.tensordot(q, mean_weights(lv), axes=([2], [0])))
            meds.append(np.asarray(fc.median)[:, 0])
        return np.concatenate(means), np.concatenate(meds)


class TimesFM3:
    name, used_covariates = "timesfm3", True

    def __init__(self):
        from timesfm3 import ModelConfig, TimesFM3Evaluator
        cfg = ModelConfig(checkpoint_path="google/timesfm-3.0-pytorch", per_core_batch_size=16, device="cuda")
        self.fc = TimesFM3Evaluator(cfg)
        self.mode = None

    def _run(self, targets, covs, one_d):
        ctx = [t if one_d else t[None, :] for t in targets]
        outs = list(self.fc.predict_batch(contexts=ctx, horizon=HORIZON, past_future_covariates=list(covs), return_quantiles=True))
        pts = np.stack([np.asarray(o.forecast, dtype=np.float64).reshape(-1)[:HORIZON] for o in outs])
        qs = np.stack([np.asarray(o.quantiles, dtype=np.float64).reshape(HORIZON, -1) for o in outs])
        return pts, qs[:, :, qs.shape[2] // 2]

    def predict(self, targets, covs):
        pts, meds = [], []
        for i in range(0, len(targets), 32):
            tg, cv = targets[i:i + 32], covs[i:i + 32]
            if self.mode is None:  # the README is unclear whether a single variate is (1, T) or (T,): try both once
                try:
                    p, m = self._run(tg, cv, False)
                    self.mode = False
                except Exception as e:  # noqa: BLE001
                    print("  timesfm3: (1, T) input failed:", repr(e)[:200], "- trying 1-D input", flush=True)
                    p, m = self._run(tg, cv, True)
                    self.mode = True
            else:
                p, m = self._run(tg, cv, self.mode)
            pts.append(p)
            meds.append(m)
        return np.concatenate(pts), np.concatenate(meds)


class TiRex2G:
    name = "tirex2g"

    def __init__(self):
        import torch
        from tirex2 import TimeseriesType, load_model
        self.torch, self.TS = torch, TimeseriesType
        self.dev = "cuda" if torch.cuda.is_available() else "cpu"
        try:
            self.model = load_model("NX-AI/TiRex-2-gifteval-zs", device=self.dev)
            self.repo = "NX-AI/TiRex-2-gifteval-zs"
        except Exception as e:  # noqa: BLE001
            print("  tirex2g: decontaminated variant failed to load:", repr(e)[:200], "- using NX-AI/TiRex-2", flush=True)
            self.model = load_model("NX-AI/TiRex-2", device=self.dev)
            self.repo = "NX-AI/TiRex-2"
        self.used_covariates = True
        self.variant = None
        self.levels = np.arange(1, 10) / 10.0

    def _ts(self, t, c, variant):
        T = self.torch
        tg = T.tensor(t, dtype=T.float32)[None, :]
        if variant == "none":
```

```
            return self.TS(target=tg, past_covariates=None, future_covariates=None)
        L = len(t)
        c = T.tensor(c, dtype=T.float32)
        if variant == "past_ctx_future_h":
            return self.TS(target=tg, past_covariates=c[:, :L], future_covariates=c[:, L:])
        return self.TS(target=tg, past_covariates=None, future_covariates=c)  # "future_all"

    def _forecast(self, targets, covs, variant):
        res = []
        for i in range(0, len(targets), 16):
            ts = [self._ts(t, c, variant) for t, c in zip(targets[i:i + 16], covs[i:i + 16])]
            out = self.model.forecast(timeseries=ts, prediction_length=HORIZON, output_type="numpy")
            res += [np.asarray(o).reshape(-1, 9, HORIZON)[0] for o in out]  # (9, H) for the first target row
        q = np.stack(res).transpose(0, 2, 1)  # (n, H, 9)
        assert np.isfinite(q).all()
        return np.tensordot(q, mean_weights(self.levels), axes=([2], [0])), q[:, :, 4]

    def predict(self, targets, covs):
        if self.variant is None:
            for v in ["past_ctx_future_h", "future_all", "none"]:
                try:
                    out = self._forecast(targets[:16], covs[:16], v)
                    self.variant = v
                    self.used_covariates = v != "none"
                    print(f"  tirex2g: covariate layout that worked: {v} ({self.repo})", flush=True)
                    break
                except Exception as e:  # noqa: BLE001
                    print(f"  tirex2g: layout {v} failed:", repr(e)[:200], flush=True)
            if self.variant is None:
                raise RuntimeError("TiRex-2 failed with every covariate layout")
        return self._forecast(targets, covs, self.variant)


MODELS = {"mock": Mock, "t0": T0, "timestral": Timestral, "timesfm3": TimesFM3, "tirex2g": TiRex2G}


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--model", required=True, choices=list(MODELS))
    ap.add_argument("--data_dir", default="m5_data")
    ap.add_argument("--out_dir", default="fm_out")
    ap.add_argument("--windows", default=",".join(ORIGINS))
    args = ap.parse_args()
    os.makedirs(args.out_dir, exist_ok=True)
    os.environ.setdefault("M5_WINDOW", "final")
    meta, values, dcols, calendar, prices = load_m5(args.data_dir)
    dcols = list(dcols)
    cal_full = load_calendar(args.data_dir)
    log = os.path.join(args.out_dir, f"fm_{args.model}_log.txt")

    def say(m):
        print(m, flush=True)
        with open(log, "a", encoding="utf-8") as f:
            f.write(m + "\n")

    say(f"[{time.strftime('%Y-%m-%d %H:%M:%S')}] model {args.model}, series {values.shape[0]}, days {values.shape[1]}, calendar days {len(cal_fu
ll)}")
    wins = [w for w in args.windows.split(",") if w in ORIGINS]
    todo = [w for w in wins if not os.path.exists(os.path.join(args.out_dir, f"fm_{args.model}_{w}.npy")) and values.shape[1] >= ORIGINS[w]]
    if not todo:
        say("nothing to do")
        return
    t0 = time.time()
    model = MODELS[args.model]()
    say(f"model loaded in {(time.time() - t0) / 60:.1f} min")
    for w in todo:
        o = ORIGINS[w]
        t1 = time.time()
        ev = WRMSSEEvaluator(meta, values[:, :o], dcols[:o], calendar, prices)  # only the aggregation matrices are used here
        targets, covs = build(ev, meta, values, cal_full, o)
        mean, med = model.predict(targets, covs)
        mean = np.clip(np.asarray(mean, dtype=np.float32), 0, None)
        med = np.clip(np.asarray(med, dtype=np.float32), 0, None)
        assert mean.shape == (154, HORIZON) and np.isfinite(mean).all(), mean.shape
        tag = args.model if getattr(model, "used_covariates", True) else args.model + "_nocov"
        np.save(os.path.join(args.out_dir, f"fm_{tag}_{w}.npy"), mean)
        np.save(os.path.join(args.out_dir, f"fm_{tag}_{w}_median.npy"), med)
        ratio = float(mean[0].sum() / values[:, o - HORIZON:o].sum())
        say(f"  {w} (origin d_{o}) done in {(time.time() - t1) / 60:.1f} min | saved as fm_{tag}_{w}.npy | level 1 forecast total / previous 28
days actual {ratio:.3f}")
    say("done")


if __name__ == "__main__":
    main()
```

src/m5_fm_score.py

```
"""
Scores the foundation-model forecasts, first on their own, then blended into the winner.
Part 1 (STANDALONE): WRMSSE per level 1 to 9 of each model's direct forecast, next to the winner (summed bottom-up forecast) and Chronos-2 B.
```

```
  This is a sanity and quality check, not a selection step. Chronos-2 alone was also worse than the winner.
Part 2 (BLEND): the fixed rule (share 0.20, levels 1 to 9, levels 10 to 12 stay as the winner), each model separately, per window with a
  bootstrap interval, then a summary across windows. An average of all foundation models is an extra, clearly labelled.
usage: python m5_fm_score.py --data_dir m5_data --in_dir inputs --fm_dir fm_out --out_dir fm_score_out
  in_dir needs winner_<window>.npy and agg_chronos2_B_covariates_<window>.npy for the windows to be scored.
"""
import argparse
import glob
import os
import re
import sys

import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from m5_data_and_wrmsse import HORIZON, LEVELS, WRMSSEEvaluator, load_m5  # noqa: E402
from m5_multiwindow import (ORIGINS, SHARE, boot_scores, make_counts, mix_levels, point_score,  # noqa: E402
                            sign_test_p, sq_errors)

ORDER = ["cv5", "cv2", "cv3", "cv4", "public", "final"]


def direct_levels(ev, winner, direct):
    """12 level forecasts where levels 1 to 9 are the direct forecast and 10 to 12 are the winner (for standalone scoring of 1 to 9)."""
    off = np.cumsum([0] + ev.sizes[:9])
    return [direct[off[i]:off[i + 1]] if i < 9 else np.asarray(A @ winner) for i, A in enumerate(ev.A)]


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default="m5_data")
    ap.add_argument("--in_dir", default="inputs")
    ap.add_argument("--fm_dir", default="fm_out")
    ap.add_argument("--out_dir", default="fm_score_out")
    args = ap.parse_args()
    os.makedirs(args.out_dir, exist_ok=True)
    os.environ.setdefault("M5_WINDOW", "final")
    meta, values, dcols, calendar, prices = load_m5(args.data_dir)
    dcols = list(dcols)
    D = values.shape[1]
    tags = sorted({re.match(r"fm_(.+)_(cv5|cv2|cv3|cv4|public|final)\.npy$", os.path.basename(p)).group(1)
                   for p in glob.glob(os.path.join(args.fm_dir, "fm_*.npy")) if re.match(r"fm_(.+)_(cv5|cv2|cv3|cv4|public|final)\.npy$", os.pat
h.basename(p))})
    lines = ["FOUNDATION MODELS: standalone quality, then blend into the winner",
             f"Models found: {', '.join(tags) if tags else 'none'} | Chronos-2 B is the earlier reference. Days in data folder: {D}.",
             f"Blend rule fixed in advance: share {SHARE:.2f}, levels 1 to 9 only, levels 10 to 12 stay as the winner. Lower WRMSSE is better.",
""]
    standalone, blend_rows = [], {}
    all_direct = {}
    for win in ORDER:
        o = ORIGINS[win]
        end = o + HORIZON
        wp = os.path.join(args.in_dir, f"winner_{win}.npy")
        cp = os.path.join(args.in_dir, f"agg_chronos2_B_covariates_{win}.npy")
        if D < end or not os.path.exists(wp) or not os.path.exists(cp):
            standalone.append(f"WINDOW {win}: skipped (needs actuals up to d_{end}, winner and Chronos-2 files).\n")
            continue
        ev = WRMSSEEvaluator(meta, values[:, :end], dcols[:end], calendar, prices)
        w = np.load(wp).astype(np.float64)
        directs = {"Chronos-2 B": np.load(cp).astype(np.float64)}
        for t in tags:
            p = os.path.join(args.fm_dir, f"fm_{t}_{win}.npy")
            if os.path.exists(p):
                directs[t] = np.load(p).astype(np.float64)
        all_direct[win] = directs
        base_levels = [np.asarray(A @ w) for A in ev.A]
        # ---- standalone
        cols = {"winner": point_score(ev, sq_errors(ev, base_levels))[1][:9]}
        for name, d in directs.items():
            cols[name] = point_score(ev, sq_errors(ev, direct_levels(ev, w, d)))[1][:9]
        head = f"{'Level':<24}" + "".join(f"{c[:15]:>16}" for c in cols)
        standalone += [f"WINDOW {win} (forecast d_{o + 1}..d_{end}), standalone WRMSSE per level, direct forecasts of the summed series", head,
"-" * len(head)]
        for i in range(9):
            standalone.append(f"{LEVELS[i][0]:>2} {LEVELS[i][1]:<21}" + "".join(f"{v[i]:>16.4f}" for v in cols.values()))
        standalone.append(f"{'mean of L1..L9':<24}" + "".join(f"{np.mean(v):>16.4f}" for v in cols.values()))
        standalone.append("")
        # ---- blend
        counts = make_counts(1000 + o)
        b_pt, _ = point_score(ev, sq_errors(ev, base_levels))
        b_bs = boot_scores(ev, sq_errors(ev, base_levels), counts)
        names = list(directs)
        fm_only = [n for n in names if n != "Chronos-2 B"]
        extra = {}
        if fm_only:
            extra["AVERAGE of Chronos-2 B + all models (extra)"] = np.mean([directs[n] for n in names], axis=0)
        for name, d in list(directs.items()) + list(extra.items()):
            lv = mix_levels(ev, w, d, SHARE)
            er = sq_errors(ev, lv)
            pt, _ = point_score(ev, er)
            bs = boot_scores(ev, er, counts)
            blend_rows.setdefault(name, {})[win] = (pt - b_pt, bs - b_bs, pt, b_pt)
    lines += ["PART 1: STANDALONE (direct forecasts of levels 1 to 9, scored as in the earlier Chronos-2 table)", ""] + standalone
```

```
    lines += ["PART 2: BLEND INTO THE WINNER (change in 12-level WRMSSE vs the winner alone, negative is better)", ""]
    for name, per in blend_rows.items():
        lines.append(f"{name}")
        lines.append(f"  {'Window':<8}{'Winner':>9}{'Blend':>9}{'Change':>9}{'95% interval of change':>26}{'P(better)':>11}")
        for win in ORDER:
            if win in per:
                ch, dd, pt, b0 = per[win]
                lo, hi = np.percentile(dd, [2.5, 97.5])
                lines.append(f"  {win:<8}{b0:>9.4f}{pt:>9.4f}{ch:>+9.4f}{f'[{lo:+.4f}, {hi:+.4f}]':>26}{float((dd < 0).mean()):>11.2f}")
        chs = np.array([per[w_][0] for w_ in ORDER if w_ in per])
        draws = np.mean([per[w_][1] for w_ in ORDER if w_ in per], axis=0)
        lo, hi = np.percentile(draws, [2.5, 97.5])
        k = int((chs < 0).sum())
        lines.append(f"  Across {len(chs)} windows: mean change {chs.mean():+.4f} | improved {k} of {len(chs)} | sign test p = {sign_test_p(k, l
en(chs)):.3f} | "
                     f"95% interval of the mean change [{lo:+.4f}, {hi:+.4f}] | worst window {chs.max():+.4f}")
        lines.append("")
    lines.append("Notes: models are reported separately and none is picked by its result. The AVERAGE row is an extra, not the main claim. Windo
ws share training history, "
                 "so the intervals (days resampled in blocks of 7) are a rough guide.")
    text = "\n".join(lines)
    with open(os.path.join(args.out_dir, "results_fm.txt"), "w") as f:
        f.write(text + "\n")
    print(text)


if __name__ == "__main__":
    main()
```

src/m5_lightgbm_e1.py

```
#!/usr/bin/env python3
"""
E1: global LightGBM baseline for M5 (memory-aware, single script).

Design
  - One global LightGBM model with a horizon feature h (1..28), so no recursive
    forecasting is needed.
  - Training rows are (series, forecast origin, horizon). Features are computed
    only from data observed before the origin, plus known future inputs for the
    target day (calendar, SNAP, events, price).
  - Origins step back every 7 days. For each origin, n_h random horizons per
    series are sampled, which keeps the training table small enough for 16 GB RAM.
  - Validation: the origin 28 days before the training end. Its targets lie after
    every training target, and the 28-day test holdout is never touched.
  - Objective: Tweedie (variance power 1.1), as commonly used on M5.

Outputs (in --out_dir)
  pred_E1_lightgbm.npy        forecasts, shape (30490, 28), same row order as the sales file
  results_E1_lightgbm.txt     plain-text log with 12-level WRMSSE table and settings

Usage
  python m5_lightgbm_e1.py                 # full run
  python m5_lightgbm_e1.py --quick         # smaller, faster first run
  python m5_lightgbm_e1.py --selftest      # synthetic data, no download

Requires m5_data_and_wrmsse.py in the same folder.
"""
import argparse
import os
import sys
import tempfile
import time

import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from m5_data_and_wrmsse import (  # noqa: E402
    HORIZON,
    WRMSSEEvaluator,
    find_file,
    format_table,
    load_m5,
    make_synthetic,
    seasonal_naive_forecast,
)

import lightgbm as lgb  # noqa: E402

CAT_FEATURES = ["dept", "cat", "store", "state", "event1", "event_type1", "wday"]


# ---------------------------------------------------------------------------
# Feature construction
# ---------------------------------------------------------------------------
def build_price_matrix(meta, calendar, prices, dcols):
    wk = calendar.set_index("d")["wm_yr_wk"].loc[dcols].to_numpy()
    weeks = np.unique(wk)
    widx = np.searchsorted(weeks, wk)
    p = prices[prices["wm_yr_wk"].isin(weeks)]
```

```
    key = pd.MultiIndex.from_frame(meta[["item_id", "store_id"]])
    r = key.get_indexer(pd.MultiIndex.from_frame(p[["item_id", "store_id"]]))
    c = np.searchsorted(weeks, p["wm_yr_wk"].to_numpy())
    ok = r >= 0
    Pw = np.full((len(meta), len(weeks)), np.nan, dtype=np.float32)
    Pw[r[ok], c[ok]] = p["sell_price"].to_numpy(dtype=np.float32)[ok]
    return Pw[:, widx]


class FeatureBuilder:
    """Builds feature rows for (origin o, horizons H). Uses only columns < T of the sales matrix."""

    def __init__(self, meta, values, dcols, cal_full, prices, T):
        self.n = values.shape[0]
        self.T = T
        S = values[:, :T]
        self.S32 = S.astype(np.float32)
        self.cs = np.zeros((self.n, T + 1))
        self.cs[:, 1:] = np.cumsum(S, axis=1)
        self.cs2 = np.zeros((self.n, T + 1))
        self.cs2[:, 1:] = np.cumsum(S * S, axis=1)
        self.cn = np.zeros((self.n, T + 1), dtype=np.int32)
        self.cn[:, 1:] = np.cumsum(S > 0, axis=1)
        idx = np.where(S > 0, np.arange(T)[None, :], -1).astype(np.int32)
        self.lastnz = np.maximum.accumulate(idx, axis=1)
        del idx

        self.P = build_price_matrix(meta, cal_full, prices, dcols)

        cal = cal_full.set_index("d").loc[dcols]
        self.wday = cal["wday"].to_numpy()
        self.month = cal["month"].to_numpy()
        self.mday = pd.to_datetime(cal["date"]).dt.day.to_numpy()
        self.ev1 = pd.factorize(cal["event_name_1"].fillna("none"))[0]
        self.evt1 = pd.factorize(cal["event_type_1"].fillna("none"))[0]
        states = sorted(meta["state_id"].unique())
        self.snap = np.stack([cal[f"snap_{s}"].to_numpy() for s in states])
        self.state_code = pd.factorize(meta["state_id"], sort=True)[0]
        self.dept_code = pd.factorize(meta["dept_id"], sort=True)[0]
        self.cat_code = pd.factorize(meta["cat_id"], sort=True)[0]
        self.store_code = pd.factorize(meta["store_id"], sort=True)[0]
        self.names = None

    def _win(self, o, w):
        a = max(o - w, 0)
        m = o - a
        s = self.cs[:, o] - self.cs[:, a]
        s2 = self.cs2[:, o] - self.cs2[:, a]
        c = self.cn[:, o] - self.cn[:, a]
        mean = s / m
        var = np.maximum(s2 / m - mean ** 2, 0)
        mean_nz = np.where(c > 0, s / np.maximum(c, 1), 0.0)
        return mean, np.sqrt(var), c / m, mean_nz

    def _gather(self, rows, idx):
        ok = idx >= 0
        v = self.S32[rows, np.clip(idx, 0, self.T - 1)]
        return np.where(ok, v, np.nan)

    def build(self, o, H):
        n, k = H.shape
        rows = np.arange(n)[:, None]
        tc = o + H - 1  # target column index
        cols, names = [], []

        def add(name, arr):
            names.append(name)
            cols.append(np.broadcast_to(np.asarray(arr, dtype=np.float32), (n, k)))

        with np.errstate(all="ignore"):
            # origin-level history features
            m7, _, _, _ = self._win(o, 7)
            m14, _, _, _ = self._win(o, 14)
            m28, sd28, nz28, mnz28 = self._win(o, 28)
            m56, _, _, _ = self._win(o, 56)
            m112, sd112, nz112, _ = self._win(o, 112)
            add("mean7", m7[:, None])
            add("mean14", m14[:, None])
            add("mean28", m28[:, None])
            add("mean56", m56[:, None])
            add("mean112", m112[:, None])
            add("std28", sd28[:, None])
            add("std112", sd112[:, None])
            add("nzfrac28", nz28[:, None])
            add("nzfrac112", nz112[:, None])
            add("meannz28", mnz28[:, None])
            add("ratio7_28", (m7 / (m28 + 0.01))[:, None])
            add("ratio28_112", (m28 / (m112 + 0.01))[:, None])
            dsl = np.where(self.lastnz[:, o - 1] >= 0, o - 1 - self.lastnz[:, o - 1], o)
            add("days_since_sale", dsl[:, None])
            add("last1", self.S32[:, o - 1][:, None])

            # same-weekday lags aligned to the target day
```

```
            kmin = -(-H // 7)
            j1 = tc - 7 * kmin
            sw = [self._gather(rows, j1 - 7 * q) for q in range(4)]
            for q, arr in enumerate(sw, start=1):
                add(f"sw_lag{q}", arr)
            stack = np.stack(sw)
            cnt = np.sum(~np.isnan(stack), axis=0)
            add("sw_mean", np.where(cnt > 0, np.nansum(stack, axis=0) / np.maximum(cnt, 1), np.nan))
            add("lag364", self._gather(rows, tc - 364))

            # price (known in advance)
            Pt = self.P[rows, tc]
            Po = self.P[:, o - 1][:, None]
            Po28 = self.P[:, max(o - 29, 0)][:, None]
            add("price", Pt)
            add("price_ratio_origin", Pt / Po)
            add("price_chg28", Po / Po28)

            # calendar (known in advance)
            add("h", H)
            add("wday", self.wday[tc])
            add("month", self.month[tc])
            add("mday", self.mday[tc])
            add("snap", self.snap[self.state_code[:, None], tc])
            add("event1", self.ev1[tc])
            add("event_type1", self.evt1[tc])

            # static
            add("dept", self.dept_code[:, None])
            add("cat", self.cat_code[:, None])
            add("store", self.store_code[:, None])
            add("state", self.state_code[:, None])

        X = np.stack(cols, axis=-1).reshape(n * k, len(cols))
        if self.names is None:
            self.names = names
        return X

    def targets(self, o, H):
        rows = np.arange(self.n)[:, None]
        tc = o + H - 1
        assert tc.max() <= self.T - 1, "target would touch the holdout"
        return self.S32[rows, tc].reshape(-1)


# ---------------------------------------------------------------------------
# Synthetic calendar for --selftest
# ---------------------------------------------------------------------------
def augment_calendar(folder, n_days, seed=1):
    rng = np.random.default_rng(seed)
    cal = pd.read_csv(os.path.join(folder, "calendar.csv"))
    dates = pd.date_range("2015-01-01", periods=n_days)
    cal["date"] = dates.strftime("%Y-%m-%d")
    cal["wday"] = dates.dayofweek + 1
    cal["month"] = dates.month
    ev = rng.random(n_days) < 0.05
    cal["event_name_1"] = np.where(ev, "EventX", None)
    cal["event_type_1"] = np.where(ev, "Cultural", None)
    for st in ["CA", "TX", "WI"]:
        cal[f"snap_{st}"] = (rng.random(n_days) < 0.3).astype(int)
    cal.to_csv(os.path.join(folder, "calendar.csv"), index=False)


# ---------------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------------
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default=None)
    ap.add_argument("--out_dir", default="m5_outputs")
    ap.add_argument("--origins", type=int, default=52, help="number of training origins (weekly steps)")
    ap.add_argument("--n_h", type=int, default=6, help="horizons sampled per series per origin")
    ap.add_argument("--rounds", type=int, default=2000)
    ap.add_argument("--threads", type=int, default=-1)
    ap.add_argument("--seed", type=int, default=42)
    ap.add_argument("--quick", action="store_true", help="smaller, faster first run")
    ap.add_argument("--selftest", action="store_true")
    args = ap.parse_args()

    t0 = time.time()
    params = dict(
        objective="tweedie", tweedie_variance_power=1.1, metric="rmse",
        learning_rate=0.05, num_leaves=127, min_data_in_leaf=200,
        feature_fraction=0.8, bagging_fraction=0.8, bagging_freq=1,
        lambda_l2=0.1, max_bin=255, num_threads=args.threads, seed=args.seed, verbose=-1,
    )
    if args.quick:
        args.origins, args.n_h, args.rounds = 12, 4, 400
    if args.selftest:
        args.origins, args.n_h, args.rounds = 8, 4, 40
        params.update(num_leaves=15, min_data_in_leaf=10)
        tmp = tempfile.mkdtemp()
        make_synthetic(tmp, n_days=240)
```

```
    augment_calendar(tmp, 240)
    data_dir = tmp
elif args.data_dir:
    data_dir = args.data_dir
elif os.path.isdir("m5_data"):
    data_dir = "m5_data"
else:
    import kagglehub
    data_dir = kagglehub.competition_download("m5-forecasting-accuracy")
print(f"Data folder: {data_dir}")

meta, values, dcols, calendar, prices = load_m5(data_dir)
cal_full = pd.read_csv(find_file(data_dir, "calendar.csv"))
n, D = values.shape
T = D - HORIZON
print(f"Series: {n} | days: {D} | train days: {T}")

ev = WRMSSEEvaluator(meta, values, dcols, calendar, prices)
fb = FeatureBuilder(meta, values, dcols, cal_full, prices, T)
rng = np.random.default_rng(args.seed)

val_o = T - HORIZON
last_train_o = val_o - HORIZON
origins = [last_train_o - 7 * k for k in range(args.origins)]
origins = [o for o in origins if o >= 28]
print(f"Training origins: {len(origins)} (from {origins[-1]} to {origins[0]}) | validation origin: {val_o}")

Xs, ys = [], []
for o in origins:
    H = rng.random((n, HORIZON)).argsort(axis=1)[:, : args.n_h] + 1
    Xs.append(fb.build(o, H))
    ys.append(fb.targets(o, H))
X = np.concatenate(Xs)
y = np.concatenate(ys)
del Xs, ys
print(f"Training table: {X.shape[0]:,} rows x {X.shape[1]} features ({X.nbytes / 1e9:.2f} GB)")

Hv = np.tile(np.arange(1, HORIZON + 1), (n, 1))
Xv = fb.build(val_o, Hv)
yv = fb.targets(val_o, Hv)

names = fb.names
cats = [c for c in CAT_FEATURES if c in names]
dtrain = lgb.Dataset(X, label=y, feature_name=names, categorical_feature=cats, free_raw_data=True)
dval = lgb.Dataset(Xv, label=yv, reference=dtrain)
print("Training LightGBM ...")
model = lgb.train(
    params, dtrain, num_boost_round=args.rounds, valid_sets=[dval], valid_names=["val"],
    callbacks=[lgb.early_stopping(100), lgb.log_evaluation(50)],
)
best_it = model.best_iteration
val_rmse = model.best_score["val"]["rmse"]
del X, y, dtrain

Hp = np.tile(np.arange(1, HORIZON + 1), (n, 1))
Xp = fb.build(T, Hp)
pred = np.clip(model.predict(Xp, num_iteration=best_it), 0, None).reshape(n, HORIZON)

os.makedirs(args.out_dir, exist_ok=True)
np.save(os.path.join(args.out_dir, "pred_E1_lightgbm.npy"), pred.astype(np.float32))

results = {
    "SeasonalNaive": ev.score(seasonal_naive_forecast(ev.X_train)),
    "E1_LightGBM": ev.score(pred),
}
overall = results["E1_LightGBM"][0]

imp = model.feature_importance("gain")
order = np.argsort(imp)[::-1][:15]
imp_lines = [f"  {names[i]:<22}{100 * imp[i] / imp.sum():>6.1f}%" for i in order]

report = [
    "E1 GLOBAL LIGHTGBM RESULTS",
    f"Data folder: {data_dir}",
    f"Training window: d_1 to d_{T} | Holdout (28 days): d_{T + 1} to d_{D}",
    f"Training origins: {len(origins)} weekly steps | horizons sampled per series per origin: {args.n_h}",
    f"Training rows: {len(origins) * n * args.n_h:,} | features: {len(names)}",
    f"Objective: tweedie (variance power 1.1) | best iteration: {best_it} | validation RMSE: {val_rmse:.4f}",
    "Early stopping validation uses an origin whose targets lie before the holdout (no test leakage).",
    "",
    "WRMSSE BY LEVEL",
    format_table(ev, results),
    "",
    "REFERENCE VALUES (published, window to be confirmed)",
    "  Original M5 winner (hidden test window): 0.520",
    "  Yang et al. Kaggle Top-1 reproduction: 0.5230",
    "  Yang et al. LightGBM + PatchTST: 0.4989",
    f"  This run, E1 LightGBM: {overall:.4f}",
    "",
    "TOP FEATURES BY GAIN",
    *imp_lines,
    "",
    f"Runtime: {(time.time() - t0) / 60:.1f} min",
```

```
    ]
    text = "\n".join(report)
    out_path = os.path.join(args.out_dir, "results_E1_lightgbm.txt")
    with open(out_path, "w") as f:
        f.write(text + "\n")
    print("\n" + text)
    print(f"\nSaved: {out_path}")
    print(f"Saved: {os.path.join(args.out_dir, 'pred_E1_lightgbm.npy')}")


if __name__ == "__main__":
    main()
```

src/m5_lightgbm_e1b.py

```
#!/usr/bin/env python3
"""
E1b: stronger global LightGBM for M5, with a cumulative ablation ladder.

Builds on m5_lightgbm_e1.py (same origins, horizon feature, validation scheme,
no recursion, no holdout leakage) and adds four changes, one step at a time:

  L0_base      the 34 features of E1
  L1_+price    price history: price vs 365-day max and mean, number of price
               changes, days on sale
  L2_+event    event proximity: days to next and since last event, second event
               slot, days to next and since last SNAP day for the state
  L3_+context  context from the same store and department, the same item across
               stores, and the same store: relative scale, group trend, group
               weekday profile
  L4_+weights  all features, training rows weighted by recent dollar sales

Every step is trained, early-stopped on the validation origin, and scored on the
28-day holdout, so the log shows what each change contributes.

Selection note: the final model is L4, fixed in advance. The holdout scores of
L0 to L3 are reported for ablation only and are not used to choose the final model.

Outputs (in --out_dir)
  pred_E1b_lightgbm.npy       forecasts of the final model L4, shape (30490, 28)
  pred_E1b_L0.npy ... L3.npy  forecasts of the intermediate steps
  results_E1b_lightgbm.txt    plain-text log with ladder and 12-level tables

Usage
  python m5_lightgbm_e1b.py               # full run
  python m5_lightgbm_e1b.py --quick       # smaller, faster first run
  python m5_lightgbm_e1b.py --selftest    # synthetic data, no download

Requires m5_data_and_wrmsse.py and m5_lightgbm_e1.py in the same folder.
"""
import argparse
import gc
import os
import sys
import tempfile
import time
import warnings

import numpy as np
import pandas as pd
from scipy import sparse

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from m5_data_and_wrmsse import (  # noqa: E402
    HORIZON,
    WRMSSEEvaluator,
    find_file,
    format_table,
    load_m5,
    make_synthetic,
    seasonal_naive_forecast,
)
from m5_lightgbm_e1 import CAT_FEATURES, FeatureBuilder, augment_calendar  # noqa: E402

import lightgbm as lgb  # noqa: E402

PRICE_NAMES = ["price_max365_ratio", "price_mean365_ratio", "n_price_changes365", "days_on_sale"]
EVENT_NAMES = ["ev_next", "ev_prev", "ev2_flag", "snap_next", "snap_prev"]
CONTEXT_NAMES = ["ctx_rel_scale", "ctx_group_trend", "ctx_group_dow",
                 "ctx_item_mean28", "ctx_item_ratio7_28", "ctx_store_trend"]


def day_distances(flag, cap=30):
    """Days to the next flagged day and since the last flagged day, capped."""
    D = len(flag)
    ar = np.arange(D)
    nxt = np.where(flag, ar, 10 ** 6)
    nxt = np.minimum.accumulate(nxt[::-1])[::-1]
    prv = np.where(flag, ar, -(10 ** 6))
    prv = np.maximum.accumulate(prv)
    return np.minimum(nxt - ar, cap).astype(np.float32), np.minimum(ar - prv, cap).astype(np.float32)
```

```
class FeatureBuilderB(FeatureBuilder):
    def __init__(self, meta, values, dcols, cal_full, prices, T):
        super().__init__(meta, values, dcols, cal_full, prices, T)
        D = values.shape[1]
        cal = cal_full.set_index("d").loc[dcols]

        # event proximity (calendar is known in advance)
        self.ev_next, self.ev_prev = day_distances(cal["event_name_1"].notna().to_numpy())
        if "event_name_2" in cal.columns:
            self.ev2 = cal["event_name_2"].notna().to_numpy().astype(np.float32)
        else:
            self.ev2 = np.zeros(D, dtype=np.float32)
        states = sorted(meta["state_id"].unique())
        nx, pv = [], []
        for s in states:
            a, b = day_distances(cal[f"snap_{s}"].to_numpy() > 0)
            nx.append(a)
            pv.append(b)
        self.snap_next = np.stack(nx)
        self.snap_prev = np.stack(pv)

        # first day with a price (for days on sale)
        valid = ~np.isnan(self.P)
        self.first_price = np.where(valid.any(axis=1), valid.argmax(axis=1), D)
        del valid

        # group indicator matrices
        n = self.n
        self.gid_sd = pd.factorize(meta["store_id"] + "|" + meta["dept_id"], sort=True)[0]
        self.gid_st = self.store_code
        self.gid_it = pd.factorize(meta["item_id"], sort=True)[0]

        def indicator(gid):
            g = int(gid.max()) + 1
            M = sparse.csr_matrix((np.ones(n), (gid, np.arange(n))), shape=(g, n))
            cnt = np.asarray(M.sum(axis=1)).ravel()
            return M, cnt

        self.G_sd, self.cnt_sd = indicator(self.gid_sd)
        self.G_st, self.cnt_st = indicator(self.gid_st)
        self.G_it, self.cnt_it = indicator(self.gid_it)
        self.extra_names = None

    def build(self, o, H):
        X = super().build(o, H)
        n, k = H.shape
        rows = np.arange(n)[:, None]
        tc = o + H - 1
        cols, names = [], []

        def add(name, arr):
            names.append(name)
            cols.append(np.broadcast_to(np.asarray(arr, dtype=np.float32), (n, k)))

        with np.errstate(all="ignore"), warnings.catch_warnings():
            warnings.simplefilter("ignore")

            # ---- price history (uses prices before the origin plus the target price)
            a = max(o - 365, 0)
            Pw = self.P[:, a:o]
            pmax = np.nanmax(Pw, axis=1)
            pmean = np.nanmean(Pw, axis=1)
            d = np.diff(Pw, axis=1)
            nchg = np.sum(np.isfinite(d) & (d != 0), axis=1)
            Pt = self.P[rows, tc]
            add("price_max365_ratio", Pt / pmax[:, None])
            add("price_mean365_ratio", Pt / pmean[:, None])
            add("n_price_changes365", nchg[:, None])
            add("days_on_sale", np.where(self.first_price <= o - 1, o - self.first_price, -1)[:, None])

            # ---- event proximity
            add("ev_next", self.ev_next[tc])
            add("ev_prev", self.ev_prev[tc])
            add("ev2_flag", self.ev2[tc])
            sc = self.state_code[:, None]
            add("snap_next", self.snap_next[sc, tc])
            add("snap_prev", self.snap_prev[sc, tc])

            # ---- context from groups (origin data only)
            m7 = self._win(o, 7)[0]
            m28 = self._win(o, 28)[0]
            gm28_sd = (self.G_sd @ m28) / self.cnt_sd
            add("ctx_rel_scale", (m28 / (gm28_sd[self.gid_sd] + 0.01))[:, None])
            add("ctx_group_trend", ((self.G_sd @ m7) / (self.G_sd @ m28 + 0.01))[self.gid_sd][:, None])

            a56 = max(o - 56, 0)
            tot = np.asarray(self.G_sd @ self.S32[:, a56:o])
            wd = self.wday[a56:o]
            fac = np.ones((tot.shape[0], 7))
            overall = tot.mean(axis=1) + 1e-6
            for w in range(1, 8):
```

```
                m = wd == w
                if m.any():
                    fac[:, w - 1] = tot[:, m].mean(axis=1) / overall
            add("ctx_group_dow", fac[self.gid_sd[:, None], self.wday[tc] - 1])

            add("ctx_item_mean28", (((self.G_it @ m28) / self.cnt_it)[self.gid_it])[:, None])
            add("ctx_item_ratio7_28", (((self.G_it @ m7) / (self.G_it @ m28 + 0.01))[self.gid_it])[:, None])
            add("ctx_store_trend", (((self.G_st @ m7) / (self.G_st @ m28 + 0.01))[self.gid_st])[:, None])

        extra = np.stack(cols, axis=-1).reshape(n * k, len(cols))
        if self.extra_names is None:
            self.extra_names = names
        return np.concatenate([X, extra], axis=1)

    def row_weights(self, o, H):
        n, k = H.shape
        a = max(o - 28, 0)
        with warnings.catch_warnings():
            warnings.simplefilter("ignore")
            dollars = np.nansum(self.S32[:, a:o] * self.P[:, a:o], axis=1)
        w = np.clip(dollars / (dollars.mean() + 1e-9), 0.05, 20.0).astype(np.float32)
        return np.repeat(w, k)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default=None)
    ap.add_argument("--out_dir", default="m5_outputs")
    ap.add_argument("--origins", type=int, default=52)
    ap.add_argument("--n_h", type=int, default=6)
    ap.add_argument("--rounds", type=int, default=2000)
    ap.add_argument("--threads", type=int, default=-1)
    ap.add_argument("--seed", type=int, default=42)
    ap.add_argument("--quick", action="store_true")
    ap.add_argument("--selftest", action="store_true")
    args = ap.parse_args()

    t0 = time.time()
    params = dict(
        objective="tweedie", tweedie_variance_power=1.1, metric="rmse",
        learning_rate=0.05, num_leaves=127, min_data_in_leaf=200,
        feature_fraction=0.8, bagging_fraction=0.8, bagging_freq=1,
        lambda_l2=0.1, max_bin=255, num_threads=args.threads, seed=args.seed, verbose=-1,
    )
    if args.quick:
        args.origins, args.n_h, args.rounds = 12, 4, 400
    if args.selftest:
        args.origins, args.n_h, args.rounds = 8, 4, 40
        params.update(num_leaves=15, min_data_in_leaf=10)
        tmp = tempfile.mkdtemp()
        make_synthetic(tmp, n_days=240)
        augment_calendar(tmp, 240)
        data_dir = tmp
    elif args.data_dir:
        data_dir = args.data_dir
    elif os.path.isdir("m5_data"):
        data_dir = "m5_data"
    else:
        import kagglehub
        data_dir = kagglehub.competition_download("m5-forecasting-accuracy")
    print(f"Data folder: {data_dir}")

    meta, values, dcols, calendar, prices = load_m5(data_dir)
    cal_full = pd.read_csv(find_file(data_dir, "calendar.csv"))
    n, D = values.shape
    T = D - HORIZON
    print(f"Series: {n} | days: {D} | train days: {T}")

    ev = WRMSSEEvaluator(meta, values, dcols, calendar, prices)
    fb = FeatureBuilderB(meta, values, dcols, cal_full, prices, T)
    del prices
    gc.collect()
    rng = np.random.default_rng(args.seed)

    val_o = T - HORIZON
    last_train_o = val_o - HORIZON
    origins = [o for o in (last_train_o - 7 * k for k in range(args.origins)) if o >= 28]
    print(f"Training origins: {len(origins)} (from {origins[-1]} to {origins[0]}) | validation origin: {val_o}")

    Xs, ys, ws = [], [], []
    for o in origins:
        H = rng.random((n, HORIZON)).argsort(axis=1)[:, : args.n_h] + 1
        Xs.append(fb.build(o, H))
        ys.append(fb.targets(o, H))
        ws.append(fb.row_weights(o, H))
    X = np.concatenate(Xs)
    y = np.concatenate(ys)
    w = np.concatenate(ws)
    del Xs, ys, ws
    gc.collect()
    print(f"Training table: {X.shape[0]:,} rows x {X.shape[1]} features ({X.nbytes / 1e9:.2f} GB)")

    Hv = np.tile(np.arange(1, HORIZON + 1), (n, 1))
```

```
Xv = fb.build(val_o, Hv)
yv = fb.targets(val_o, Hv)
wv = fb.row_weights(val_o, Hv)
Xp = fb.build(T, Hv)

names = fb.names + fb.extra_names
idx = {nm: i for i, nm in enumerate(names)}
base_cols = [idx[nm] for nm in fb.names]
price_cols = [idx[nm] for nm in PRICE_NAMES]
event_cols = [idx[nm] for nm in EVENT_NAMES]
ctx_cols = [idx[nm] for nm in CONTEXT_NAMES]
steps = [
    ("L0_base", base_cols, False),
    ("L1_+price", base_cols + price_cols, False),
    ("L2_+event", base_cols + price_cols + event_cols, False),
    ("L3_+context", base_cols + price_cols + event_cols + ctx_cols, False),
    ("L4_+weights", base_cols + price_cols + event_cols + ctx_cols, True),
]

results, ladder, preds = {}, [], {}
final_model, final_names = None, None
total_actual = ev.y_test.sum()
for label, cols, weighted in steps:
    t1 = time.time()
    sub_names = [names[i] for i in cols]
    cats = [c for c in CAT_FEATURES if c in sub_names]
    dtr = lgb.Dataset(X[:, cols], label=y, weight=w if weighted else None,
                      feature_name=sub_names, categorical_feature=cats, free_raw_data=True)
    dva = lgb.Dataset(Xv[:, cols], label=yv, weight=wv if weighted else None, reference=dtr)
    print(f"\nTraining {label} ({len(cols)} features) ...")
    model = lgb.train(
        params, dtr, num_boost_round=args.rounds, valid_sets=[dva], valid_names=["val"],
        callbacks=[lgb.early_stopping(100, verbose=False), lgb.log_evaluation(100)],
    )
    pred = np.clip(model.predict(Xp[:, cols], num_iteration=model.best_iteration), 0, None).reshape(n, HORIZON)
    score = ev.score(pred)
    results[label] = score
    preds[label] = pred
    ladder.append((label, len(cols), score[0], model.best_iteration,
                   model.best_score["val"]["rmse"], pred.sum() / total_actual, (time.time() - t1) / 60))
    print(f"{label}: WRMSSE {score[0]:.4f} | best iteration {model.best_iteration} | {ladder[-1][-1]:.1f} min")
    if label == "L4_+weights":
        final_model, final_names = model, sub_names
    del dtr, dva
    gc.collect()

os.makedirs(args.out_dir, exist_ok=True)
for i, label in enumerate(["L0_base", "L1_+price", "L2_+event", "L3_+context"]):
    np.save(os.path.join(args.out_dir, f"pred_E1b_L{i}.npy"), preds[label].astype(np.float32))
np.save(os.path.join(args.out_dir, "pred_E1b_lightgbm.npy"), preds["L4_+weights"].astype(np.float32))

snaive = ev.score(seasonal_naive_forecast(ev.X_train))
table_results = {"SeasonalNaive": snaive, "L0_base": results["L0_base"], "L4_final": results["L4_+weights"]}

ladder_lines = [f"{'Step':<14}{'Feats':>6}{'WRMSSE':>9}{'Change':>9}{'BestIt':>8}{'ValRMSE':>9}{'Bias':>8}{'Min':>6}"]
prev = None
for label, nf, sc, bi, vr, bias, mins in ladder:
    ch = "" if prev is None else f"{sc - prev:+.4f}"
    ladder_lines.append(f"{label:<14}{nf:>6}{sc:>9.4f}{ch:>9}{bi:>8}{vr:>9.4f}{bias:>8.3f}{mins:>6.1f}")
    prev = sc

imp = final_model.feature_importance("gain")
order = np.argsort(imp)[::-1][:20]
imp_lines = [f"  {final_names[i]:<22}{100 * imp[i] / imp.sum():>6.1f}%" for i in order]
final = results["L4_+weights"][0]

report = [
    "E1b GLOBAL LIGHTGBM RESULTS (ablation ladder)",
    f"Data folder: {data_dir}",
    f"Training window: d_1 to d_{T} | Holdout (28 days): d_{T + 1} to d_{D}",
    f"Training origins: {len(origins)} weekly steps | horizons sampled per series per origin: {args.n_h}",
    f"Training rows: {len(origins) * n * args.n_h:,} | total features available: {len(names)}",
    "Objective: tweedie (variance power 1.1). Early stopping on a validation origin before the holdout.",
    "Final model is L4, fixed in advance. L0 to L3 holdout scores are for ablation only.",
    "Bias = total forecast / total actual over the holdout (1.000 means unbiased overall).",
    "",
    "ABLATION LADDER (each step adds to the previous one)",
    *ladder_lines,
    "",
    "WRMSSE BY LEVEL",
    format_table(ev, table_results),
    "",
    "REFERENCE VALUES (published, window to be confirmed)",
    "  Original M5 winner (hidden test window): 0.520",
    "  Yang et al. Kaggle Top-1 reproduction: 0.5230",
    "  Yang et al. LightGBM + PatchTST: 0.4989",
    "  E1 (previous script, full run): 0.6141",
    f"  This run, E1b final (L4): {final:.4f}",
    "",
    "TOP FEATURES BY GAIN (final model)",
    *imp_lines,
    "",
    f"Runtime: {(time.time() - t0) / 60:.1f} min",
```

```
    ]
    text = "\n".join(report)
    out_path = os.path.join(args.out_dir, "results_E1b_lightgbm.txt")
    with open(out_path, "w") as f:
        f.write(text + "\n")
    print("\n" + text)
    print(f"\nSaved: {out_path}")


if __name__ == "__main__":
    main()
```

src/m5_lightgbm_e1c.py

```
#!/usr/bin/env python3
"""
E1c: hierarchical-ensemble LightGBM for M5 (the per-group modelling idea of the M5 winner and of Yang et al.).

Same data table as E1b L3 (52 weekly origins, 6 sampled horizons per series and origin, the 34 base features
plus price, event and context features, all computed from data before the origin, no recursion, no holdout
leakage). The difference is that models are trained per group instead of one global model:

  store         10 models, one per store
  store_cat     30 models, one per store x category
  store_dept    70 models, one per store x department

Each model is early-stopped on a validation origin (28 days before the end of the training window). The three
levels each give a full forecast for all 30,490 series, and the final E1c forecast is the equal-weight average of
the three levels (as in the M5 winner, who averaged equally). Nothing is selected on the holdout.

Settings fixed in advance (not tuned): tweedie 1.1, learning rate 0.05, early stopping 100, at most 2000 rounds,
leaves / min rows per leaf: store 127 / 200, store_cat 63 / 100, store_dept 31 / 100. Features are the E1b L3 set.
Rows are unweighted. Note for the write-up: the choice of L3 features over the pre-specified L4 (dollar weights)
came from the E1b ablation and is disclosed as a post hoc decision.

Calibration variant (reported separately): a single global multiplier = total actual / total forecast on the
validation window, applied to the holdout forecast. It uses validation data only. Whether the calibrated or the
plain forecast goes into later ensembles is decided by validation WRMSSE only.

Validation-window forecasts (predval_*) come from the same models that were early-stopped on that window, so only
the number of boosting rounds per model has seen it. This is a small optimism for weight fitting and should be
mentioned in the limitations.

Outputs (in --out_dir)
  pred_E1c_store.npy, pred_E1c_store_cat.npy, pred_E1c_store_dept.npy   holdout forecasts of each level
  pred_E1c_lightgbm.npy                                                equal-weight average of the three levels
  pred_E1c_lightgbm_cal.npy                                            same, times the validation calibration factor
  predval_E1c_<same names>.npy                                         validation-window forecasts (d_1886 to d_1913)
  results_E1c_lightgbm.txt                                             plain-text log
  e1c_cache/                                                           per-group results; a rerun skips finished groups

Usage
  python m5_lightgbm_e1c.py              # full run (resumes from e1c_cache if interrupted)
  python m5_lightgbm_e1c.py --quick      # smaller table and fewer rounds, for timing
  python m5_lightgbm_e1c.py --selftest   # synthetic data, no download

Requires m5_data_and_wrmsse.py, m5_lightgbm_e1.py and m5_lightgbm_e1b.py in the same folder.
"""
import argparse
import gc
import os
import sys
import tempfile
import time

import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from m5_data_and_wrmsse import (  # noqa: E402
    HORIZON,
    WRMSSEEvaluator,
    find_file,
    format_table,
    load_m5,
    load_calendar,
    make_synthetic,
    seasonal_naive_forecast,
)
from m5_lightgbm_e1 import CAT_FEATURES, augment_calendar  # noqa: E402
from m5_lightgbm_e1b import (  # noqa: E402
    CONTEXT_NAMES,
    EVENT_NAMES,
    PRICE_NAMES,
    FeatureBuilderB,
)

import lightgbm as lgb  # noqa: E402

LEVEL_PARAMS = {
    "store": dict(num_leaves=127, min_data_in_leaf=200),
```

```
    "store_cat": dict(num_leaves=63, min_data_in_leaf=100),
    "store_dept": dict(num_leaves=31, min_data_in_leaf=100),
}


def series_groups(meta, fb):
    return {
        "store": fb.store_code,
        "store_cat": pd.factorize(meta["store_id"] + "|" + meta["cat_id"], sort=True)[0],
        "store_dept": fb.gid_sd,
    }


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default=None)
    ap.add_argument("--out_dir", default="m5_outputs")
    ap.add_argument("--origins", type=int, default=52)
    ap.add_argument("--n_h", type=int, default=6)
    ap.add_argument("--rounds", type=int, default=2000)
    ap.add_argument("--threads", type=int, default=-1)
    ap.add_argument("--seed", type=int, default=42)
    ap.add_argument("--quick", action="store_true")
    ap.add_argument("--selftest", action="store_true")
    args = ap.parse_args()

    t0 = time.time()
    base = dict(
        objective="tweedie", tweedie_variance_power=1.1, metric="rmse", learning_rate=0.05,
        feature_fraction=0.8, bagging_fraction=0.8, bagging_freq=1, lambda_l2=0.1, max_bin=255,
        num_threads=args.threads, seed=args.seed, verbose=-1,
    )
    level_params = {k: dict(v) for k, v in LEVEL_PARAMS.items()}
    if args.quick:
        args.origins, args.n_h, args.rounds = 12, 4, 400
    if args.selftest:
        args.origins, args.n_h, args.rounds = 8, 4, 40
        for v in level_params.values():
            v.update(num_leaves=7, min_data_in_leaf=5)
        tmp = tempfile.mkdtemp()
        make_synthetic(tmp, n_days=240)
        augment_calendar(tmp, 240)
        data_dir = tmp
    elif args.data_dir:
        data_dir = args.data_dir
    elif os.path.isdir("m5_data"):
        data_dir = "m5_data"
    else:
        import kagglehub
        data_dir = kagglehub.competition_download("m5-forecasting-accuracy")
    print(f"Data folder: {data_dir}", flush=True)

    meta, values, dcols, calendar, prices = load_m5(data_dir)
    cal_full = load_calendar(data_dir)
    n, D = values.shape
    T = D - HORIZON
    print(f"Series: {n} | days: {D} | train days: {T}", flush=True)

    ev_hold = WRMSSEEvaluator(meta, values, dcols, calendar, prices)
    ev_val = WRMSSEEvaluator(meta, values[:, :T], dcols[:T], calendar, prices)
    fb = FeatureBuilderB(meta, values, dcols, cal_full, prices, T)
    del prices
    gc.collect()
    rng = np.random.default_rng(args.seed)
    groups = series_groups(meta, fb)

    val_o = T - HORIZON
    last_train_o = val_o - HORIZON
    origins = [o for o in (last_train_o - 7 * k for k in range(args.origins)) if o >= 28]
    print(f"Training origins: {len(origins)} (from {origins[-1]} to {origins[0]}) | validation origin: {val_o}", flush=True)

    Xs, ys = [], []
    for o in origins:
        H = rng.random((n, HORIZON)).argsort(axis=1)[:, : args.n_h] + 1
        Xs.append(fb.build(o, H))
        ys.append(fb.targets(o, H))
    X = np.concatenate(Xs)
    y = np.concatenate(ys)
    del Xs, ys
    gc.collect()
    sid = np.tile(np.repeat(np.arange(n, dtype=np.int32), args.n_h), len(origins))
    print(f"Training table: {X.shape[0]:,} rows x {X.shape[1]} features ({X.nbytes / 1e9:.2f} GB)", flush=True)

    Hv = np.tile(np.arange(1, HORIZON + 1), (n, 1))
    Xv = fb.build(val_o, Hv)
    yv = fb.targets(val_o, Hv)
    Xp = fb.build(T, Hv)

    names = fb.names + fb.extra_names
    idx = {nm: i for i, nm in enumerate(names)}
    cols = [idx[nm] for nm in fb.names + PRICE_NAMES + EVENT_NAMES + CONTEXT_NAMES]
    sub_names = [names[i] for i in cols]
    cats = [c for c in CAT_FEATURES if c in sub_names]
```

```
X, Xv, Xp = X[:, cols], Xv[:, cols], Xp[:, cols]

cache = os.path.join(args.out_dir, "e1c_cache")
os.makedirs(cache, exist_ok=True)
pred_h = {lv: np.zeros((n, HORIZON), dtype=np.float32) for lv in groups}
pred_v = {lv: np.zeros((n, HORIZON), dtype=np.float32) for lv in groups}
stats = {lv: [] for lv in groups}  # (best_iter, minutes)

for lv, gser in groups.items():
    gid_row = gser[sid]
    order = np.argsort(gid_row, kind="stable")
    sorted_g = gid_row[order]
    n_groups = int(gser.max()) + 1
    bounds = np.searchsorted(sorted_g, np.arange(n_groups + 1))
    print(f"\n=== Level {lv}: {n_groups} models ===", flush=True)
    for g in range(n_groups):
        cfile = os.path.join(cache, f"{lv}_{g:03d}.npz")
        ser = np.where(gser == g)[0]
        if os.path.exists(cfile):
            z = np.load(cfile)
            pred_h[lv][ser] = z["hold"]
            pred_v[lv][ser] = z["val"]
            stats[lv].append((int(z["best_it"]), float(z["mins"])))
            continue
        t1 = time.time()
        rows = order[bounds[g]:bounds[g + 1]]
        vrows = (ser[:, None] * HORIZON + np.arange(HORIZON)[None, :]).ravel()
        dtr = lgb.Dataset(X[rows], label=y[rows], feature_name=sub_names,
                          categorical_feature=cats, free_raw_data=True)
        dva = lgb.Dataset(Xv[vrows], label=yv[vrows], reference=dtr)
        params = dict(base, **level_params[lv])
        model = lgb.train(params, dtr, num_boost_round=args.rounds, valid_sets=[dva], valid_names=["val"],
                          callbacks=[lgb.early_stopping(100, verbose=False)])
        ph = np.clip(model.predict(Xp[vrows], num_iteration=model.best_iteration), 0, None).reshape(len(ser), HORIZON)
        pv = np.clip(model.predict(Xv[vrows], num_iteration=model.best_iteration), 0, None).reshape(len(ser), HORIZON)
        mins = (time.time() - t1) / 60
        np.savez(cfile, hold=ph.astype(np.float32), val=pv.astype(np.float32),
                 best_it=model.best_iteration, mins=mins)
        pred_h[lv][ser] = ph
        pred_v[lv][ser] = pv
        stats[lv].append((model.best_iteration, mins))
        del dtr, dva, model
        gc.collect()
        if g % max(1, n_groups // 10) == 0 or g == n_groups - 1:
            print(f"  {lv} {g + 1}/{n_groups} | best iteration {stats[lv][-1][0]} | {mins:.1f} min | "
                  f"elapsed {(time.time() - t0) / 60:.0f} min", flush=True)

# ---- combine, score, calibrate
avg_h = np.mean([pred_h[lv] for lv in groups], axis=0)
avg_v = np.mean([pred_v[lv] for lv in groups], axis=0)
cal = float(ev_val.y_test.sum() / max(avg_v.sum(), 1e-9))
cal_h = avg_h * cal
cal_v = avg_v * cal

results_h = {lv: ev_hold.score(pred_h[lv]) for lv in groups}
results_v = {lv: ev_val.score(pred_v[lv]) for lv in groups}
results_h["average"] = ev_hold.score(avg_h)
results_v["average"] = ev_val.score(avg_v)
results_h["average_cal"] = ev_hold.score(cal_h)
results_v["average_cal"] = ev_val.score(cal_v)

os.makedirs(args.out_dir, exist_ok=True)
for lv in groups:
    np.save(os.path.join(args.out_dir, f"pred_E1c_{lv}.npy"), pred_h[lv])
    np.save(os.path.join(args.out_dir, f"predval_E1c_{lv}.npy"), pred_v[lv])
np.save(os.path.join(args.out_dir, "pred_E1c_lightgbm.npy"), avg_h.astype(np.float32))
np.save(os.path.join(args.out_dir, "predval_E1c_lightgbm.npy"), avg_v.astype(np.float32))
np.save(os.path.join(args.out_dir, "pred_E1c_lightgbm_cal.npy"), cal_h.astype(np.float32))
np.save(os.path.join(args.out_dir, "predval_E1c_lightgbm_cal.npy"), cal_v.astype(np.float32))

tot_h, tot_v = ev_hold.y_test.sum(), ev_val.y_test.sum()
lines = [f"{'Model':<14}{'Models':>7}{'Holdout':>10}{'Validation':>12}{'HoldBias':>10}{'ValBias':>9}{'AvgIter':>9}{'Min':>7}"]
order_names = list(groups) + ["average", "average_cal"]
for nm in order_names:
    ph = avg_h if nm == "average" else cal_h if nm == "average_cal" else pred_h[nm]
    pv = avg_v if nm == "average" else cal_v if nm == "average_cal" else pred_v[nm]
    if nm in groups:
        nm_models = len(stats[nm])
        it = f"{np.mean([s[0] for s in stats[nm]]):.0f}"
        mn = f"{sum(s[1] for s in stats[nm]):.1f}"
    else:
        nm_models, it, mn = "", "", ""
    lines.append(f"{nm:<14}{nm_models!s:>7}{results_h[nm][0]:>10.4f}{results_v[nm][0]:>12.4f}"
                 f"{ph.sum() / tot_h:>10.3f}{pv.sum() / tot_v:>9.3f}{it:>9}{mn:>7}")

table = {"SeasonalNaive": ev_hold.score(seasonal_naive_forecast(ev_hold.X_train))}
for lv in groups:
    table[lv] = results_h[lv]
table["average"] = results_h["average"]
table["average_cal"] = results_h["average_cal"]

use_cal = results_v["average_cal"][0] < results_v["average"][0]
report = [
```

```
        "E1c HIERARCHICAL-ENSEMBLE LIGHTGBM RESULTS",
        f"Data folder: {data_dir}",
        f"Training window: d_1 to d_{T} | Holdout (28 days): d_{T + 1} to d_{D} | validation window: d_{val_o + 1} to d_{T}",
        f"Training origins: {len(origins)} weekly steps | horizons sampled per series per origin: {args.n_h}",
        f"Features: {len(sub_names)} (E1b L3 set) | rows unweighted | objective tweedie 1.1",
        "Models per level: store 10, store_cat 30, store_dept 70. Final forecast = equal-weight average of the three levels.",
        "Each model is early-stopped on the validation origin. Nothing is selected on the holdout.",
        f"Calibration factor (validation actual / forecast total): {cal:.4f}",
        f"Rule: ensembles use the calibrated forecast only if its validation WRMSSE is lower. Result: "
        f"{'use calibrated' if use_cal else 'use plain average'}.",
        "Bias = total forecast / total actual (1.000 is unbiased).",
        "",
        "RESULTS (WRMSSE, lower is better)",
        *lines,
        "",
        "WRMSSE BY LEVEL (holdout)",
        format_table(ev_hold, table),
        "",
        "REFERENCE VALUES (published, window to be confirmed)",
        "  Original M5 winner (hidden test window): 0.520",
        "  Yang et al. Kaggle Top-1 reproduction: 0.5230",
        "  Yang et al. LightGBM + PatchTST: 0.4989",
        "  Our E1b global LightGBM, L3 (unweighted, all features): 0.5422",
        f"  This run, E1c average: {results_h['average'][0]:.4f} | calibrated: {results_h['average_cal'][0]:.4f}",
        "",
        f"Runtime: {(time.time() - t0) / 60:.1f} min",
    ]
    text = "\n".join(report)
    out_path = os.path.join(args.out_dir, "results_E1c_lightgbm.txt")
    with open(out_path, "w") as f:
        f.write(text + "\n")
    print("\n" + text)
    print(f"\nSaved: {out_path}")


if __name__ == "__main__":
    main()
```

src/m5_lightgbm_e1d.py

```
#!/usr/bin/env python3
"""
E1d: winner-style non-recursive per-group LightGBM for M5 (a faithful port of the non-recursive half of the
1st place solution, M5-methods folder A1), written to run on a 16 GB machine and to score on our fixed windows.

How it follows the winner
  - one row per series and target day, from d_710 up to the end of the training window (about 3.7 million rows per store)
  - features for target day t use sales only up to day t-28: lags 28 to 42, rolling mean and std (windows 7, 14, 30, 60,
    180) shifted by 28 days, so one model serves all 28 horizons without recursion
  - price features (11), calendar features (14), item_id / dept / cat / event categoricals, mean encodings from the
    training days only (store x department, and item x state for store models or item x store for group models)
  - rows before an item's first price week are dropped, as in the winner's grid
  - settings: tweedie 1.1, learning rate 0.015, 3000 trees, 255 leaves, min 255 rows per leaf, subsample 0.5,
    feature fraction 0.5, max_bin 100, boost_from_average off, NO early stopping
  - levels: store (10 models), store x category (30), store x department (70); equal-weight average of the levels run

How it differs (state these in the write-up)
  - only the non-recursive half of the winner's six model sets is implemented, not the recursive models
  - training window ends at d_1913, so the target window is d_1914 to d_1941, not the hidden final window
  - the price aggregates (max, min, std, mean, number of price levels) use the full price file as in the winner's code;
    prices are planned inputs published ahead of time, not sales
  - tm_y, tm_m and similar calendar variables follow the winner

Two windows are produced, each from its own model trained only on days before the window:
  holdout     trained to d_1913, forecasts d_1914 to d_1941
  validation  trained to d_1885, forecasts d_1886 to d_1913 (clean, because there is no early stopping)
Nothing is selected on the holdout. As in E1c, a single global calibration factor is fitted on the validation window
only, and the calibrated forecast is used in ensembles only if its validation WRMSSE is lower.

Outputs (in --out_dir)
  pred_E1d_<level>.npy, predval_E1d_<level>.npy   per level (30490 x 28)
  pred_E1d_lightgbm.npy, predval_E1d_lightgbm.npy plain average of the levels run
  pred_E1d_lightgbm_cal.npy, predval_E1d_lightgbm_cal.npy   same, times the validation calibration factor
  results_E1d_lightgbm.txt                        plain-text log
  e1d_cache/                                      one file per finished model; a rerun skips finished models

Usage
  python m5_lightgbm_e1d.py --quick                # one store, 60 trees, prints a time estimate for the full run
  python m5_lightgbm_e1d.py                        # store level only (10 models x 2 windows)
  python m5_lightgbm_e1d.py --levels store,store_cat,store_dept
  python m5_lightgbm_e1d.py --selftest             # synthetic data, no download

Requires m5_data_and_wrmsse.py in the same folder. CPU only. A full-size store model with 3000 trees takes a while
(run --quick first to see the estimate).
"""
import argparse
import gc
import os
import sys
import tempfile
import time
```

```
import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from m5_data_and_wrmsse import (  # noqa: E402
    HORIZON,
    WRMSSEEvaluator,
    find_file,
    format_table,
    load_m5,
    load_calendar,
    make_synthetic,
    seasonal_naive_forecast,
)

import lightgbm as lgb  # noqa: E402

PAD = 250
LAGS = list(range(28, 43))
ROLLS = [7, 14, 30, 60, 180]
PRICE_COLS = ["sell_price", "price_max", "price_min", "price_std", "price_mean", "price_norm",
              "price_nunique", "item_nunique", "price_momentum", "price_momentum_m", "price_momentum_y"]
CAL_COLS = ["event_name_1", "event_type_1", "event_name_2", "event_type_2", "snap_CA", "snap_TX", "snap_WI",
            "tm_d", "tm_w", "tm_m", "tm_y", "tm_wm", "tm_dw", "tm_w_end"]
LAG_COLS = [f"sales_lag_{k}" for k in LAGS]
ROLL_COLS = [f"rolling_{s}_{w}" for w in ROLLS for s in ("mean", "std")]
ENC_COLS = ["enc_store_dept_mean", "enc_store_dept_std", "enc_item_grp_mean", "enc_item_grp_std"]
CAT_COLS = ["item_id", "dept_id", "cat_id", "event_name_1", "event_type_1", "event_name_2", "event_type_2"]
ALL_COLS = ["item_id", "dept_id", "cat_id"] + PRICE_COLS + CAL_COLS + LAG_COLS + ROLL_COLS + ENC_COLS
COL = {c: i for i, c in enumerate(ALL_COLS)}


def augment_calendar(folder, n_days, seed=1):
    rng = np.random.default_rng(seed)
    cal = pd.read_csv(os.path.join(folder, "calendar.csv"))
    dates = pd.date_range("2015-01-01", periods=n_days)
    cal["date"] = dates.strftime("%Y-%m-%d")
    cal["wday"] = dates.dayofweek + 1
    cal["month"] = dates.month
    ev = rng.random(n_days) < 0.05
    cal["event_name_1"] = np.where(ev, "EventX", None)
    cal["event_type_1"] = np.where(ev, "Cultural", None)
    for st in ["CA", "TX", "WI"]:
        cal[f"snap_{st}"] = (rng.random(n_days) < 0.3).astype(int)
    cal.to_csv(os.path.join(folder, "calendar.csv"), index=False)


# ---------------------------------------------------------------------------
# Global preparation
# ---------------------------------------------------------------------------
def prepare_calendar(cal_full, D):
    cal = cal_full.set_index("d")
    cal = cal.iloc[:D] if len(cal) >= D else cal
    dt = pd.to_datetime(cal["date"])
    out = pd.DataFrame(index=np.arange(D))
    for c in ["event_name_1", "event_type_1", "event_name_2", "event_type_2"]:
        if c in cal.columns:
            codes = pd.factorize(cal[c])[0].astype(np.float32)
            codes[codes < 0] = np.nan
        else:
            codes = np.full(D, np.nan, dtype=np.float32)
        out[c] = codes[:D]
    for st in ["CA", "TX", "WI"]:
        col = f"snap_{st}"
        out[col] = cal[col].to_numpy()[:D].astype(np.float32) if col in cal.columns else 0.0
    day = dt.dt.day.to_numpy()[:D]
    out["tm_d"] = day
    out["tm_w"] = dt.dt.isocalendar().week.to_numpy().astype(np.int16)[:D]
    out["tm_m"] = dt.dt.month.to_numpy()[:D]
    out["tm_y"] = dt.dt.year.to_numpy()[:D] - dt.dt.year.min()
    out["tm_wm"] = np.ceil(day / 7).astype(np.int8)
    out["tm_dw"] = dt.dt.dayofweek.to_numpy()[:D]
    out["tm_w_end"] = (out["tm_dw"] >= 5).astype(np.int8)
    return out[CAL_COLS].to_numpy(dtype=np.float32)


def price_table(prices, cal_full, meta):
    """Weekly price features per (store, item), computed as in the winner's preprocessing."""
    store_cat = pd.Categorical(meta["store_id"]).categories
    item_cat = pd.Categorical(meta["item_id"]).categories
    p = prices.copy()
    p["sid"] = pd.Categorical(p["store_id"], categories=store_cat).codes
    p["iid"] = pd.Categorical(p["item_id"], categories=item_cat).codes
    p = p[(p["sid"] >= 0) & (p["iid"] >= 0)].sort_values(["sid", "iid", "wm_yr_wk"]).reset_index(drop=True)
    p["sell_price"] = p["sell_price"].astype(np.float32)
    g = p.groupby(["sid", "iid"])["sell_price"]
    p["price_max"] = g.transform("max")
    p["price_min"] = g.transform("min")
    p["price_std"] = g.transform("std")
    p["price_mean"] = g.transform("mean")
    p["price_norm"] = p["sell_price"] / p["price_max"]
    p["price_nunique"] = g.transform("nunique").astype(np.float32)
```

```
    p["item_nunique"] = p.groupby(["sid", "sell_price"])["iid"].transform("nunique").astype(np.float32)
    p["price_momentum"] = p["sell_price"] / g.shift(1)
    dt = pd.to_datetime(cal_full["date"])
    wk = pd.DataFrame({"wm_yr_wk": cal_full["wm_yr_wk"], "month": dt.dt.month, "year": dt.dt.year}).drop_duplicates("wm_yr_wk")
    p = p.merge(wk, on="wm_yr_wk", how="left")
    p["price_momentum_m"] = p["sell_price"] / p.groupby(["sid", "iid", "month"])["sell_price"].transform("mean")
    p["price_momentum_y"] = p["sell_price"] / p.groupby(["sid", "iid", "year"])["sell_price"].transform("mean")
    keep = ["sid", "iid", "wm_yr_wk"] + PRICE_COLS
    for c in PRICE_COLS:
        p[c] = p[c].astype(np.float32)
    return p[keep], store_cat, item_cat


def series_triples(values, valid, te):
    """Per-series sum, sum of squares, count over valid days before te."""
    n = values.shape[0]
    s = np.zeros(n)
    s2 = np.zeros(n)
    c = np.zeros(n)
    for a in range(0, n, 4000):
        b = min(a + 4000, n)
        v = valid[a:b, :te]
        x = np.where(v, values[a:b, :te], 0.0)
        s[a:b] = x.sum(axis=1)
        s2[a:b] = (x * x).sum(axis=1)
        c[a:b] = v.sum(axis=1)
    return s, s2, c


def group_stats(key, s, s2, c):
    k = int(key.max()) + 1
    gs = np.bincount(key, weights=s, minlength=k)
    gs2 = np.bincount(key, weights=s2, minlength=k)
    gc_ = np.bincount(key, weights=c, minlength=k)
    with np.errstate(all="ignore"):
        mean = gs / gc_
        var = (gs2 - gs * gs / gc_) / (gc_ - 1)
    std = np.sqrt(np.maximum(var, 0))
    mean = np.where(gc_ > 0, mean, np.nan)
    std = np.where(gc_ > 1, std, np.nan)
    return mean[key], std[key]


def encodings(values, valid, te, meta_codes):
    """Mean encodings from days before te only. Returns dict of per-series arrays."""
    s, s2, c = series_triples(values, valid, te)
    sd_key = meta_codes["store"] * (meta_codes["dept"].max() + 1) + meta_codes["dept"]
    is_key = meta_codes["item"] * (meta_codes["state"].max() + 1) + meta_codes["state"]
    sd_m, sd_s = group_stats(sd_key, s, s2, c)
    is_m, is_s = group_stats(is_key, s, s2, c)
    with np.errstate(all="ignore"):
        it_m = s / c
        it_s = np.sqrt(np.maximum((s2 - s * s / c) / (c - 1), 0))
    it_m = np.where(c > 0, it_m, np.nan)
    it_s = np.where(c > 1, it_s, np.nan)
    return {"sd": (sd_m, sd_s), "item_state": (is_m, is_s), "item_store": (it_m, it_s)}


# ---------------------------------------------------------------------------
# Per-store feature matrix
# ---------------------------------------------------------------------------
def build_store(ser, meta_codes, values, valid, wk_idx, ptab, weeks, cal_arr, first_idx, D):
    """X3: (n_s, n_days, F) float32 for target days first_idx..D-1 of the series in `ser`."""
    n_s = len(ser)
    n_days = D - first_idx
    t_all = np.arange(first_idx, D)
    X3 = np.full((n_s, n_days, len(ALL_COLS)), np.nan, dtype=np.float32)

    X3[:, :, COL["item_id"]] = meta_codes["item"][ser][:, None]
    X3[:, :, COL["dept_id"]] = meta_codes["dept"][ser][:, None]
    X3[:, :, COL["cat_id"]] = meta_codes["cat"][ser][:, None]

    # prices: dense (n_s, n_weeks) per feature, expanded to days
    store_code = meta_codes["store"][ser[0]]
    sub = ptab[ptab["sid"] == store_code]
    pos = {int(it): j for j, it in enumerate(meta_codes["item"][ser])}
    r = np.array([pos.get(int(i), -1) for i in sub["iid"].to_numpy()])
    ok = r >= 0
    cidx = np.searchsorted(weeks, sub["wm_yr_wk"].to_numpy())
    day_week = wk_idx[t_all]
    for cname in PRICE_COLS:
        M = np.full((n_s, len(weeks)), np.nan, dtype=np.float32)
        M[r[ok], cidx[ok]] = sub[cname].to_numpy()[ok]
        X3[:, :, COL[cname]] = M[:, day_week]
    del sub

    # calendar
    for k, cname in enumerate(CAL_COLS):
        X3[:, :, COL[cname]] = cal_arr[t_all, k][None, :]

    # sales history with pre-release days as missing
    S = np.where(valid[ser], values[ser], np.nan).astype(np.float32)
    Sp = np.concatenate([np.full((n_s, PAD), np.nan, dtype=np.float32), S], axis=1)
```

```
    for L, cname in zip(LAGS, LAG_COLS):
        X3[:, :, COL[cname]] = Sp[:, PAD + first_idx - L: PAD + D - L]
    Z = np.nan_to_num(Sp).astype(np.float64)
    V = (~np.isnan(Sp)).astype(np.float64)
    cs = np.concatenate([np.zeros((n_s, 1)), np.cumsum(Z, axis=1)], axis=1)
    cs2 = np.concatenate([np.zeros((n_s, 1)), np.cumsum(Z * Z, axis=1)], axis=1)
    cv = np.concatenate([np.zeros((n_s, 1)), np.cumsum(V, axis=1)], axis=1)
    del Z, V, Sp
    end = PAD + t_all - 28 + 1
    for w in ROLLS:
        a = end - w
        with np.errstate(all="ignore"):
            sm = cs[:, end] - cs[:, a]
            sm2 = cs2[:, end] - cs2[:, a]
            cnt = cv[:, end] - cv[:, a]
            mean = sm / w
            var = (sm2 - w * mean * mean) / (w - 1)
        full = cnt == w
        X3[:, :, COL[f"rolling_mean_{w}"]] = np.where(full, mean, np.nan)
        X3[:, :, COL[f"rolling_std_{w}"]] = np.where(full, np.sqrt(np.maximum(var, 0)), np.nan)
    del cs, cs2, cv
    return X3, S[:, first_idx:]


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default=None)
    ap.add_argument("--out_dir", default="m5_outputs")
    ap.add_argument("--levels", default="store", help="comma list: store,store_cat,store_dept")
    ap.add_argument("--rounds", type=int, default=3000)
    ap.add_argument("--lr", type=float, default=0.015)
    ap.add_argument("--threads", type=int, default=-1)
    ap.add_argument("--seed", type=int, default=1995)
    ap.add_argument("--first_day", type=int, default=710)
    ap.add_argument("--quick", action="store_true", help="one store, 60 trees, prints a time estimate")
    ap.add_argument("--selftest", action="store_true")
    args = ap.parse_args()

    t0 = time.time()
    log = lambda m: print(m, flush=True)  # noqa: E731
    params = dict(
        objective="tweedie", tweedie_variance_power=1.1, metric="rmse", boosting_type="gbdt",
        subsample=0.5, subsample_freq=1, learning_rate=args.lr, num_leaves=2 ** 8 - 1,
        min_data_in_leaf=2 ** 8 - 1, feature_fraction=0.5, max_bin=100, boost_from_average=False,
        verbose=-1, seed=args.seed, num_threads=args.threads,
    )
    first_idx = args.first_day - 1
    if args.selftest:
        args.rounds, params["learning_rate"] = 30, 0.1
        params.update(num_leaves=7, min_data_in_leaf=5)
        first_idx = 60
        tmp = tempfile.mkdtemp()
        make_synthetic(tmp, n_days=240)
        augment_calendar(tmp, 240)
        data_dir = tmp
    elif args.data_dir:
        data_dir = args.data_dir
    elif os.path.isdir("m5_data"):
        data_dir = "m5_data"
    else:
        import kagglehub
        data_dir = kagglehub.competition_download("m5-forecasting-accuracy")
    log(f"Data folder: {data_dir}")

    levels = [x.strip() for x in args.levels.split(",") if x.strip()]
    for lv in levels:
        if lv not in ("store", "store_cat", "store_dept"):
            raise SystemExit(f"Unknown level {lv}")

    meta, values, dcols, calendar, prices = load_m5(data_dir)
    cal_full = load_calendar(data_dir)
    n, D = values.shape
    T = D - HORIZON
    val_o = T - HORIZON
    log(f"Series: {n} | days: {D} | holdout origin: {T} | validation origin: {val_o} | first training day: d_{first_idx + 1}")

    wk_of_day = cal_full.set_index("d").loc[dcols, "wm_yr_wk"].to_numpy()
    weeks = np.unique(cal_full["wm_yr_wk"].to_numpy())
    wk_idx = np.searchsorted(weeks, wk_of_day)
    cal_arr = prepare_calendar(cal_full.assign(date=cal_full["date"]).set_index("d").loc[dcols].reset_index(), D)

    log("Preparing price features ...")
    ptab, store_cat_levels, item_cat_levels = price_table(prices, cal_full, meta)
    meta_codes = {
        "store": pd.Categorical(meta["store_id"], categories=store_cat_levels).codes.astype(np.int64),
        "item": pd.Categorical(meta["item_id"], categories=item_cat_levels).codes.astype(np.int64),
        "dept": pd.factorize(meta["dept_id"], sort=True)[0].astype(np.int64),
        "cat": pd.factorize(meta["cat_id"], sort=True)[0].astype(np.int64),
        "state": pd.factorize(meta["state_id"], sort=True)[0].astype(np.int64),
    }
    first_week = ptab.groupby(["sid", "iid"])["wm_yr_wk"].min()
    rel = np.full(n, np.iinfo(np.int64).max, dtype=np.int64)
    key = pd.MultiIndex.from_arrays([meta_codes["store"], meta_codes["item"]])
```

```
    ridx = first_week.index.get_indexer(key)
    rel_ok = ridx >= 0
    rel[rel_ok] = first_week.to_numpy()[ridx[rel_ok]]
    valid = wk_of_day[None, :] >= rel[:, None]
    values32 = values.astype(np.float32)

    ev_hold = WRMSSEEvaluator(meta, values, dcols, calendar, prices)
    ev_val = WRMSSEEvaluator(meta, values[:, :T], dcols[:T], calendar, prices)
    del prices
    gc.collect()

    windows = {"hold": T, "val": val_o}
    enc = {w: encodings(values, valid, te, meta_codes) for w, te in windows.items()}

    cache = os.path.join(args.out_dir, "e1d_cache")
    os.makedirs(cache, exist_ok=True)
    pred = {w: {lv: np.zeros((n, HORIZON), dtype=np.float32) for lv in levels} for w in windows}
    stats = {lv: [] for lv in levels}  # minutes per model

    store_ids = list(store_cat_levels)
    if args.quick:
        store_ids = store_ids[:1]
        args.rounds_full = args.rounds
        args.rounds = 60
    n_total_models = 0

    for st_i, store_name in enumerate(store_ids):
        scode = list(store_cat_levels).index(store_name)
        ser = np.where(meta_codes["store"] == scode)[0]
        groups = {}
        for lv in levels:
            if lv == "store":
                groups[lv] = {"all": np.arange(len(ser))}
            else:
                key_arr = meta_codes["cat" if lv == "store_cat" else "dept"][ser]
                groups[lv] = {int(k): np.where(key_arr == k)[0] for k in np.unique(key_arr)}

        def cfile(lv, g, w):
            return os.path.join(cache, f"{lv}_{store_name}_{g}_{w}.npz")

        X3 = None
        t_build = 0.0
        for lv in levels:
            for g, jmask in groups[lv].items():
                for w, te in windows.items():
                    cf = cfile(lv, g, w)
                    gser = ser[jmask]
                    if os.path.exists(cf) and not args.quick:
                        z = np.load(cf)
                        pred[w][lv][gser] = z["pred"]
                        stats[lv].append(float(z["mins"]))
                        continue
                    if X3 is None:
                        tb = time.time()
                        X3, Yst = build_store(ser, meta_codes, values32, valid, wk_idx, ptab, weeks, cal_arr, first_idx, D)
                        t_build = (time.time() - tb) / 60
                        log(f"{store_name}: feature matrix {X3.shape[0]} x {X3.shape[1]} x {X3.shape[2]} "
                            f"({X3.nbytes / 1e9:.2f} GB) built in {t_build:.1f} min")
                    t1 = time.time()
                    e = enc[w]
                    X3[:, :, COL["enc_store_dept_mean"]] = e["sd"][0][ser][:, None]
                    X3[:, :, COL["enc_store_dept_std"]] = e["sd"][1][ser][:, None]
                    grp = e["item_state"] if lv == "store" else e["item_store"]
                    X3[:, :, COL["enc_item_grp_mean"]] = grp[0][ser][:, None]
                    X3[:, :, COL["enc_item_grp_std"]] = grp[1][ser][:, None]

                    ntr = te - first_idx
                    vtr = valid[ser][jmask][:, first_idx:te]
                    Xtr = X3[jmask][:, :ntr, :][vtr]
                    ytr = Yst[jmask][:, :ntr][vtr]
                    dtr = lgb.Dataset(Xtr, label=ytr, feature_name=ALL_COLS, categorical_feature=CAT_COLS,
                                      free_raw_data=True)
                    del Xtr

                    def cb(env, tag=f"{store_name} {lv} {g} {w}", t_start=t1):
                        if (env.iteration + 1) % 500 == 0:
                            log(f"    {tag}: {env.iteration + 1} trees | {(time.time() - t_start) / 60:.1f} min")

                    model = lgb.train(params, dtr, num_boost_round=args.rounds, callbacks=[cb])
                    Xpr = X3[jmask][:, ntr:ntr + HORIZON, :].reshape(-1, X3.shape[2])
                    ph = np.clip(model.predict(Xpr), 0, None).reshape(len(gser), HORIZON)
                    on_sale = valid[gser][:, te:te + HORIZON]
                    ph = np.where(on_sale, ph, 0.0)
                    mins = (time.time() - t1) / 60
                    del dtr, model, ytr, vtr
                    gc.collect()
                    if args.quick:
                        full_models = len(store_cat_levels) * len(windows) * len(levels)
                        per_tree = mins * 60 / args.rounds
                        est = (t_build * len(store_cat_levels) + mins / args.rounds * args.rounds_full * full_models) / 60
                        log(f"\nQUICK RESULT: build {t_build:.1f} min, {args.rounds} trees in {mins:.1f} min "
                            f"({per_tree:.2f} s per tree incl. dataset construction)")
                        log(f"Rough estimate for this setup ({', '.join(levels)} | {len(windows)} windows | "
```

```
                            f"{args.rounds_full} trees): {est:.1f} hours "
                            f"(the first trees include fixed setup cost, so treat this as an upper-leaning guide)")
                        log(f"NaN in predictions: {int(np.isnan(ph).sum())}")
                        return
                    np.savez(cf, pred=ph.astype(np.float32), mins=mins)
                    pred[w][lv][gser] = ph
                    stats[lv].append(mins)
                    n_total_models += 1
                    log(f"  done {store_name} | {lv} {g} | {w} | {mins:.1f} min | elapsed {(time.time() - t0) / 60:.0f} min")
        del X3
        gc.collect()

    # ---- combine, score, calibrate
    avg = {w: np.mean([pred[w][lv] for lv in levels], axis=0) for w in windows}
    cal = float(ev_val.y_test.sum() / max(avg["val"].sum(), 1e-9))
    res_h, res_v = {}, {}
    for lv in levels:
        res_h[lv] = ev_hold.score(pred["hold"][lv])
        res_v[lv] = ev_val.score(pred["val"][lv])
    res_h["average"], res_v["average"] = ev_hold.score(avg["hold"]), ev_val.score(avg["val"])
    res_h["average_cal"], res_v["average_cal"] = ev_hold.score(avg["hold"] * cal), ev_val.score(avg["val"] * cal)

    os.makedirs(args.out_dir, exist_ok=True)
    for lv in levels:
        np.save(os.path.join(args.out_dir, f"pred_E1d_{lv}.npy"), pred["hold"][lv])
        np.save(os.path.join(args.out_dir, f"predval_E1d_{lv}.npy"), pred["val"][lv])
    np.save(os.path.join(args.out_dir, "pred_E1d_lightgbm.npy"), avg["hold"].astype(np.float32))
    np.save(os.path.join(args.out_dir, "predval_E1d_lightgbm.npy"), avg["val"].astype(np.float32))
    np.save(os.path.join(args.out_dir, "pred_E1d_lightgbm_cal.npy"), (avg["hold"] * cal).astype(np.float32))
    np.save(os.path.join(args.out_dir, "predval_E1d_lightgbm_cal.npy"), (avg["val"] * cal).astype(np.float32))

    tot_h, tot_v = ev_hold.y_test.sum(), ev_val.y_test.sum()
    lines = [f"{'Model':<14}{'Models':>7}{'Holdout':>10}{'Validation':>12}{'HoldBias':>10}{'ValBias':>9}{'Minutes':>9}"]
    for nm in levels + ["average", "average_cal"]:
        if nm in levels:
            ph, pv = pred["hold"][nm], pred["val"][nm]
            mods, mn = str(len(stats[nm]) // len(windows)), f"{sum(stats[nm]):.0f}"
        else:
            f = cal if nm == "average_cal" else 1.0
            ph, pv, mods, mn = avg["hold"] * f, avg["val"] * f, "", ""
        lines.append(f"{nm:<14}{mods:>7}{res_h[nm][0]:>10.4f}{res_v[nm][0]:>12.4f}"
                     f"{ph.sum() / tot_h:>10.3f}{pv.sum() / tot_v:>9.3f}{mn:>9}")
    table = {"SeasonalNaive": ev_hold.score(seasonal_naive_forecast(ev_hold.X_train))}
    for lv in levels:
        table[lv] = res_h[lv]
    table["average"], table["average_cal"] = res_h["average"], res_h["average_cal"]
    use_cal = res_v["average_cal"][0] < res_v["average"][0]

    report = [
        "E1d WINNER-STYLE NON-RECURSIVE PER-GROUP LIGHTGBM RESULTS",
        f"Data folder: {data_dir}",
        f"Holdout: trained on days up to d_{T}, forecasts d_{T + 1} to d_{D}. "
        f"Validation: trained up to d_{val_o}, forecasts d_{val_o + 1} to d_{T}.",
        f"Levels run: {', '.join(levels)} | rounds: {args.rounds} | learning rate: {args.lr} | no early stopping",
        f"Training rows start at d_{first_idx + 1}; features use sales only up to 28 days before each target day.",
        "Only the non-recursive half of the winner's design is implemented (no recursive models).",
        f"Calibration factor (validation actual / forecast total): {cal:.4f}",
        f"Rule: ensembles use the calibrated forecast only if its validation WRMSSE is lower. Result: "
        f"{'use calibrated' if use_cal else 'use plain average'}.",
        "Bias = total forecast / total actual (1.000 is unbiased).",
        "",
        "RESULTS (WRMSSE, lower is better)",
        *lines,
        "",
        "WRMSSE BY LEVEL (holdout)",
        format_table(ev_hold, table),
        "",
        "REFERENCE VALUES (published, window to be confirmed)",
        "  Original M5 winner (hidden test window): 0.520",
        "  Yang et al. Kaggle Top-1 reproduction: 0.5230",
        "  Yang et al. LightGBM + PatchTST: 0.4989",
        "  Our E1b global LightGBM, L3: 0.5422 | E1c per-group (sampled rows): 0.6137, calibrated 0.5935",
        f"  This run, E1d average: {res_h['average'][0]:.4f} | calibrated: {res_h['average_cal'][0]:.4f}",
        "",
        f"Runtime: {(time.time() - t0) / 60:.1f} min",
    ]
    text = "\n".join(report)
    out_path = os.path.join(args.out_dir, "results_E1d_lightgbm.txt")
    with open(out_path, "w") as f:
        f.write(text + "\n")
    print("\n" + text)
    print(f"\nSaved: {out_path}")


if __name__ == "__main__":
    main()
```

src/m5_lvl1011.py

```
"""
E11 (exploratory): add direct zero-shot Chronos-2 forecasts of the item-level aggregates, level 10 (item, 3049 series)
```

```
and level 11 (item x state, 9147 series), to the blend that already uses levels 1 to 9.

Rules fixed in advance, nothing is tuned here:
  - Same model and settings as the levels 1 to 9 run: amazon/chronos-2, covariates (weekday, event, SNAP share of the group),
    context 1024, horizon 28, mean implied by the quantiles.
  - Same share 0.20 of the direct forecast at each level it is applied to. The winner's bottom forecasts are not changed,
    so level 12 is untouched.
  - Variants: winner alone | HEADLINE (levels 1 to 9, the result already reported) | HEADLINE + level 10 | HEADLINE + levels 10 and 11.
  - The decision uses the five windows before the final one. The final window is scored once and labelled exploratory.

Stage forecast: writes lvl10_chronos2_B_<win>.npy (3049 x 28) and lvl11_chronos2_B_<win>.npy (9147 x 28) per window.
  Existing files are never overwritten, so the stage can be restarted.
Stage score: needs winner_<win>.npy and agg_chronos2_B_covariates_<win>.npy in --in_dir.

Usage:
  python m5_lvl1011.py --data_dir <M5 data, 1969 days> --in_dir inputs --out_dir lvl1011_out --stage all
"""
import argparse
import os
import sys
import time

import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from m5_data_and_wrmsse import HORIZON, LEVELS, WRMSSEEvaluator, load_calendar, load_m5  # noqa: E402
from m5_multiwindow import (N_DRAWS, boot_scores, make_counts, point_score, sign_test_p,  # noqa: E402
                            sq_errors)

ORDER = ["cv5", "cv2", "cv3", "cv4", "public", "final"]
ORIGINS = {"cv5": 1801, "cv2": 1829, "cv3": 1857, "cv4": 1885, "public": 1913, "final": 1941}
SHARE = 0.20
L10, L11 = 9, 10  # zero-based level indices
CHUNK = 1500


def build_inputs_levels(ev, meta, values, cal_full, dcols, origin, ctx_len, level_idx):
    cal = cal_full.set_index("d").loc[dcols]
    states = sorted(meta["state_id"].unique())
    state_code = np.searchsorted(states, meta["state_id"].to_numpy())
    onehot = np.zeros((len(meta), len(states)))
    onehot[np.arange(len(meta)), state_code] = 1.0
    snap = np.stack([cal[f"snap_{s}"].to_numpy().astype(np.float32) for s in states])
    wd = np.array([f"w{x}" for x in cal["wday"].to_numpy().astype(int)])
    event = cal["event_name_1"].notna().to_numpy().astype(np.float32)
    a = max(origin - ctx_len, 0)
    A = ev.A[level_idx]
    Y = np.asarray(A @ values[:, :origin], dtype=np.float32)
    size = np.asarray(A.sum(axis=1)).ravel()
    share = np.asarray(A @ onehot) / size[:, None]
    snap_g = (share @ snap).astype(np.float32)
    inputs = []
    for g in range(A.shape[0]):
        inputs.append({
            "target": Y[g, a:origin],
            "past_covariates": {"snap": snap_g[g, a:origin], "wday": wd[a:origin], "event": event[a:origin]},
            "future_covariates": {"snap": snap_g[g, origin:origin + HORIZON], "wday": wd[origin:origin + HORIZON],
                                  "event": event[origin:origin + HORIZON]},
        })
    return inputs


def stage_forecast(args, meta, values, dcols, calendar, prices, cal_full):
    from m5_aggregate import forecast
    from m5_chronos2 import MockPipeline, quantile_mean_weights
    todo = [w for w in ORDER if values.shape[1] >= ORIGINS[w] + HORIZON and not all(
        os.path.exists(os.path.join(args.out_dir, f"lvl{n}_chronos2_B_{w}.npy")) for n in (10, 11))]
    if not todo:
        print("Forecast stage: nothing to do.", flush=True)
        return
    if args.mock:
        pipe = MockPipeline()
    else:
        import torch
        from chronos import Chronos2Pipeline
        cuda = torch.cuda.is_available()
        print("Loading", args.model, "cuda" if cuda else "CPU", flush=True)
        pipe = Chronos2Pipeline.from_pretrained(args.model, device_map="cuda" if cuda else "cpu")
    weights = quantile_mean_weights(pipe.quantiles)
    med_idx = int(np.argmin(np.abs(np.asarray(pipe.quantiles) - 0.5)))
    for w in todo:
        o = ORIGINS[w]
        ev = WRMSSEEvaluator(meta, values[:, : o + HORIZON], dcols[: o + HORIZON], calendar, prices)
        for name, li in (("10", L10), ("11", L11)):
            path = os.path.join(args.out_dir, f"lvl{name}_chronos2_B_{w}.npy")
            if os.path.exists(path):
                continue
            t0 = time.time()
            inputs = build_inputs_levels(ev, meta, values, cal_full, dcols[: o + HORIZON], o, args.context_length, li)
            parts = []
            for s in range(0, len(inputs), CHUNK):
                mean, _ = forecast(pipe, inputs[s:s + CHUNK], weights, med_idx)
                parts.append(mean)
```

```
                print(f"    {w} level {name}: {min(s + CHUNK, len(inputs))}/{len(inputs)} series, "
                      f"{(time.time() - t0) / 60:.1f} min", flush=True)
            np.save(path, np.concatenate(parts, axis=0))
            print(f"  forecast {w} level {name} (origin d_{o}) done in {(time.time() - t0) / 60:.1f} min", flush=True)


def level_list(ev, winner, direct19, d10, d11, use):
    """use: set of extra levels (0-based) mixed at SHARE. Levels 1 to 9 always use direct19 when 'h' in use."""
    off = np.cumsum([0] + ev.sizes[:9])
    out = []
    for i, A in enumerate(ev.A):
        P = np.asarray(A @ winner)
        if i < 9 and "h" in use:
            P = (1 - SHARE) * P + SHARE * direct19[off[i]:off[i + 1]]
        elif i == L10 and "10" in use:
            P = (1 - SHARE) * P + SHARE * d10
        elif i == L11 and "11" in use:
            P = (1 - SHARE) * P + SHARE * d11
        out.append(P)
    return out


def stage_score(args, meta, values, dcols, calendar, prices):
    counts = make_counts(2024)
    variants = [("winner alone", set()), ("HEADLINE levels 1-9", {"h"}),
                ("HEADLINE + level 10", {"h", "10"}), ("HEADLINE + levels 10, 11", {"h", "10", "11"})]
    lines = ["EXPLORATORY: direct Chronos-2 at the item-level aggregates (level 10 item, level 11 item x state)",
             f"Fixed rule: share {SHARE:.2f} of the direct forecast, same model and settings as levels 1 to 9. Level 12 stays as the winner.",
             f"WRMSSE over 12 levels, lower is better. Bootstrap: {N_DRAWS} draws, blocks of 7 days (days only, rough guide).",
             "Decision uses the five windows before the final one; the final window is shown once and is exploratory.", ""]
    summ = {}
    for win in ORDER:
        o = ORIGINS[win]
        paths = [os.path.join(args.in_dir, f"winner_{win}.npy"),
                 os.path.join(args.in_dir, f"agg_chronos2_B_covariates_{win}.npy"),
                 os.path.join(args.out_dir, f"lvl10_chronos2_B_{win}.npy"),
                 os.path.join(args.out_dir, f"lvl11_chronos2_B_{win}.npy")]
        if values.shape[1] < o + HORIZON or not all(os.path.exists(p) for p in paths):
            lines += [f"WINDOW {win}: skipped (missing data or files).", ""]
            continue
        w, d19, d10, d11 = [np.load(p) for p in paths]
        ev = WRMSSEEvaluator(meta, values[:, : o + HORIZON], dcols[: o + HORIZON], calendar, prices)
        res = {}
        for name, use in variants:
            lv = level_list(ev, w, d19, d10, d11, use)
            errs = sq_errors(ev, lv)
            pt, per = point_score(ev, errs)
            res[name] = (pt, per, boot_scores(ev, errs, counts))
        lines.append(f"WINDOW {win} (forecast d_{o + 1}..d_{o + HORIZON})" + ("   [exploratory, scored once]" if win == "final" else ""))
        lines.append(f"{'Variant':<28}{'WRMSSE':>9}{'vs winner':>11}{'vs HEADLINE':>13}{'95% interval vs HEADLINE':>28}{'P(better)':>11}")
        base, head = res["winner alone"], res["HEADLINE levels 1-9"]
        for name, _ in variants:
            pt, _, bs = res[name]
            d = bs - head[2]
            lo, hi = np.percentile(d, [2.5, 97.5])
            extra = "" if name in ("winner alone", "HEADLINE levels 1-9") else f"{f'[{lo:+.4f}, {hi:+.4f}]':>28}{(d < 0).mean():>11.2f}"
            lines.append(f"{name:<28}{pt:>9.4f}{pt - base[0]:>+11.4f}{pt - head[0]:>+13.4f}{extra}")
        only = [np.asarray(A @ w) for A in ev.A]
        only[L10], only[L11] = d10, d11
        dl = point_score(ev, sq_errors(ev, only))[1]
        lines.append(f"  Levels 10 / 11 (WRMSSE): winner {base[1][L10]:.4f} / {base[1][L11]:.4f} | "
                     f"direct alone {dl[L10]:.4f} / {dl[L11]:.4f} | "
                     f"blend {res['HEADLINE + levels 10, 11'][1][L10]:.4f} / {res['HEADLINE + levels 10, 11'][1][L11]:.4f}")
        lines.append("")
        summ[win] = {n: res[n][0] for n, _ in variants}
    if summ:
        lines.append("SUMMARY (change = extended variant minus HEADLINE, negative is better)")
        for tag, wins in (("five windows before final", [x for x in ORDER[:-1] if x in summ]), ("all windows with results", list(summ))):
            if not wins:
                continue
            lines.append(f"  {tag}: {', '.join(wins)}")
            for n in ("HEADLINE + level 10", "HEADLINE + levels 10, 11"):
                ch = [summ[x][n] - summ[x]["HEADLINE levels 1-9"] for x in wins]
                k = sum(c < 0 for c in ch)
                lines.append(f"    {n:<28} mean change {np.mean(ch):+.4f} | improved {k} of {len(ch)} | sign test p = "
                             f"{sign_test_p(k, len(ch)):.3f} | worst {max(ch):+.4f}")
        lines += ["", "Notes: windows share training history; intervals resample days only; this is exploratory and was run after",
                  "the main result was fixed. If the five-window result is not clearly better, the headline stays unchanged."]
    text = "\n".join(lines)
    with open(os.path.join(args.out_dir, "results_lvl1011.txt"), "w") as f:
        f.write(text + "\n")
    print(text)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default="m5_data")
    ap.add_argument("--in_dir", default="inputs")
    ap.add_argument("--out_dir", default="lvl1011_out")
    ap.add_argument("--stage", default="all", choices=["all", "forecast", "score"])
    ap.add_argument("--model", default="amazon/chronos-2")
    ap.add_argument("--context_length", type=int, default=1024)
    ap.add_argument("--mock", action="store_true")
```

```
    args = ap.parse_args()
    os.makedirs(args.out_dir, exist_ok=True)
    os.environ.setdefault("M5_WINDOW", "final")
    meta, values, dcols, calendar, prices = load_m5(args.data_dir)
    dcols = list(dcols)
    cal_full = load_calendar(args.data_dir)
    print(f"Series {values.shape[0]}, days {values.shape[1]}", flush=True)
    if args.stage in ("all", "forecast"):
        stage_forecast(args, meta, values, dcols, calendar, prices, cal_full)
    if args.stage in ("all", "score"):
        stage_score(args, meta, values, dcols, calendar, prices)


if __name__ == "__main__":
    main()
```

src/m5_multiwindow.py

```
"""
E10: Does the blend gain hold across many windows? One script, two stages.

Rule fixed in advance (not tuned here): winner's six-set LightGBM average at all levels; at levels 1 to 9 mix 20 percent of
the direct Chronos-2 forecast with covariates (variant B, mean point). Levels 10 to 12 stay as the winner.
Secondary, labelled as such: the same blend followed by OLS reconciliation (all 12 levels made to add up).

Windows (28 forecast days each, train on the days before):
  cv5 d_1802..d_1829 | cv2 d_1830..d_1857 | cv3 d_1858..d_1885 | cv4 d_1886..d_1913 | public d_1914..d_1941 | final d_1942..d_1969

Stage "forecast": Chronos-2 zero-shot direct forecasts of the summed series (levels 1 to 9, 154 series) for every window
  that has no forecast file yet. About 2 minutes per window on a GPU. Existing files are never overwritten.
Stage "score": for every window that has winner_<win>.npy (30490 x 28) and the direct forecast, scores
  winner alone / blend / blend + OLS reconciliation, bootstrap interval per window (days resampled in blocks of 7),
  and a summary across windows (mean change, windows improved, sign test, bootstrap interval of the mean change).

Usage:
  python m5_multiwindow.py --data_dir m5_data --in_dir inputs --out_dir multiwindow_out [--stage all|forecast|score]
Files in --in_dir: winner_<win>.npy and agg_chronos2_B_covariates_<win>.npy (the latter is created by stage forecast).
Output: multiwindow_out/results_multiwindow.txt (plain text) and the new agg_chronos2_B_covariates_<win>.npy files.
"""
import argparse
import math
import os
import sys
import time

import numpy as np
import scipy.sparse as sp
from scipy.sparse.linalg import LinearOperator, cg

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from m5_data_and_wrmsse import HORIZON, LEVELS, WRMSSEEvaluator, load_calendar, load_m5  # noqa: E402

ORIGINS = {"cv5": 1801, "cv2": 1829, "cv3": 1857, "cv4": 1885, "public": 1913, "final": 1941}
SHARE = 0.20
SENS = [0.10, 0.30]
N_DRAWS, BLOCK, N_BLOCKS = 2000, 7, 4


# ------------------------------------------------------------- forecasts
def stage_forecast(args, meta, values, dcols, calendar, prices, cal_full):
    from m5_aggregate import build_inputs, forecast, level_slices  # noqa: F401
    from m5_chronos2 import MockPipeline, quantile_mean_weights
    todo = [w for w in ORIGINS if not os.path.exists(os.path.join(args.in_dir, f"agg_chronos2_B_covariates_{w}.npy"))
            and not os.path.exists(os.path.join(args.out_dir, f"agg_chronos2_B_covariates_{w}.npy"))
            and values.shape[1] >= ORIGINS[w] + HORIZON]
    if not todo:
        print("Forecast stage: nothing to do, all direct forecast files exist.", flush=True)
        return
    if args.mock:
        pipe = MockPipeline()
    else:
        import torch
        from chronos import Chronos2Pipeline
        cuda = torch.cuda.is_available()
        print("Loading", args.model, "cuda" if cuda else "CPU", flush=True)
        pipe = Chronos2Pipeline.from_pretrained(args.model, device_map="cuda" if cuda else "cpu")
    weights = quantile_mean_weights(pipe.quantiles)
    med_idx = int(np.argmin(np.abs(np.asarray(pipe.quantiles) - 0.5)))
    for w in todo:
        o = ORIGINS[w]
        t0 = time.time()
        ev = WRMSSEEvaluator(meta, values[:, : o + HORIZON], dcols[: o + HORIZON], calendar, prices)
        inputs = build_inputs(ev, meta, values, cal_full, dcols[: o + HORIZON], o, args.context_length, True)
        mean, _ = forecast(pipe, inputs, weights, med_idx)
        np.save(os.path.join(args.out_dir, f"agg_chronos2_B_covariates_{w}.npy"), mean)
        print(f"  forecast {w} (origin d_{o}) done in {(time.time() - t0) / 60:.1f} min", flush=True)


# ------------------------------------------------------------- scoring helpers
def mix_levels(ev, winner, direct, s):
    off = np.cumsum([0] + ev.sizes[:9])
```

```
    out = []
    for i, A in enumerate(ev.A):
        P = np.asarray(A @ winner)
        if i < 9 and s > 0:
            P = (1 - s) * P + s * direct[off[i]:off[i + 1]]
        out.append(P)
    return out


def ols_reconcile(ev, levels, init):
    """Least squares: bottom b minimising sum over all levels of (y_base - A b)^2. Matrix-free conjugate gradient."""
    S = sp.vstack([sp.csr_matrix(A) for A in ev.A]).tocsr()
    ST = S.T.tocsr()
    Y = np.vstack(levels)
    M = LinearOperator((S.shape[1], S.shape[1]), matvec=lambda x: ST @ (S @ x), dtype=np.float64)
    rhs = ST @ Y
    b = np.zeros_like(rhs)
    worst = 0.0
    for j in range(rhs.shape[1]):
        x, _ = cg(M, rhs[:, j], x0=init[:, j], rtol=1e-9, maxiter=1000)
        b[:, j] = x
        worst = max(worst, float(np.linalg.norm(M.matvec(x) - rhs[:, j]) / (np.linalg.norm(rhs[:, j]) + 1e-12)))
    b = np.clip(b, 0, None)
    return [np.asarray(A @ b) for A in ev.A], worst


def sq_errors(ev, levels):
    return [(Y - P) ** 2 for Y, P in zip(ev.y_agg, levels)]


def point_score(ev, errs):
    lv = []
    for E, sc, w in zip(errs, ev.scale, ev.weight):
        v = w > 0
        r = np.zeros(len(sc))
        r[v] = np.sqrt(E[v].mean(axis=1) / sc[v])
        lv.append(float(np.sum(w * r)))
    return float(np.mean(lv)), lv


def boot_scores(ev, errs, counts):
    tot = np.zeros(counts.shape[0])
    for E, sc, w in zip(errs, ev.scale, ev.weight):
        v = np.where(w > 0)[0]
        for s in range(0, counts.shape[0], 250):
            M = (counts[s:s + 250] / counts.shape[1]) @ E[v].T
            tot[s:s + 250] += (np.sqrt(M / sc[v]) * w[v]).sum(axis=1)
    return tot / len(ev.A)


def make_counts(seed):
    rng = np.random.default_rng(seed)
    starts = rng.integers(0, HORIZON, size=(N_DRAWS, N_BLOCKS))
    counts = np.zeros((N_DRAWS, HORIZON))
    for b in range(N_DRAWS):
        for st in starts[b]:
            for j in range(BLOCK):
                counts[b, (st + j) % HORIZON] += 1
    return counts


def sign_test_p(k, n):
    """Two-sided exact sign test p-value, k successes out of n."""
    if n == 0:
        return float("nan")
    lo = min(k, n - k)
    return min(1.0, 2 * sum(math.comb(n, i) for i in range(lo + 1)) / 2 ** n)


def stage_score(args, meta, values, dcols, calendar, prices):
    D = values.shape[1]
    lines = ["MULTI-WINDOW CHECK: winner (six-set LightGBM average) vs blend with direct Chronos-2 at levels 1 to 9",
             f"Fixed rule: share {SHARE:.2f}, direct Chronos-2 with covariates, mean point, levels 1 to 9 only. Reconciled = same blend, then OL
S reconciliation (secondary).",
             f"WRMSSE over 12 levels, lower is better. Bootstrap per window: {N_DRAWS} draws, blocks of {BLOCK} days (days only, rough guide). D
ays in data folder: {D}.", ""]
    rows = []
    for k, (win, o) in enumerate(ORIGINS.items()):
        end = o + HORIZON
        wpath = os.path.join(args.in_dir, f"winner_{win}.npy")
        dpath = next((p for p in [os.path.join(args.in_dir, f"agg_chronos2_B_covariates_{win}.npy"),
                                  os.path.join(args.out_dir, f"agg_chronos2_B_covariates_{win}.npy")] if os.path.exists(p)), None)
        if D < end or not os.path.exists(wpath) or dpath is None:
            why = "data folder too short" if D < end else ("winner file missing" if not os.path.exists(wpath) else "direct forecast missing")
            lines += [f"WINDOW {win} (d_{o + 1}..d_{end}): skipped ({why}).", ""]
            continue
        ev = WRMSSEEvaluator(meta, values[:, :end], dcols[:end], calendar, prices)
        w = np.load(wpath).astype(np.float64)
        d = np.load(dpath).astype(np.float64)
        growth = float(values[:, o:end].sum() / values[:, o - HORIZON:o].sum() - 1)
        bias = float(w.sum() / ev.y_test.sum())
        counts = make_counts(1000 + o)
        base_levels = [np.asarray(A @ w) for A in ev.A]
```

```
        blend_levels = mix_levels(ev, w, d, SHARE)
        rec_levels, resid = ols_reconcile(ev, blend_levels, w)
        res = {}
        for name, lv in [("winner alone", base_levels), ("blend (headline)", blend_levels), ("blend + OLS reconciled", rec_levels)] + \
                        [(f"blend share {s:.2f}", mix_levels(ev, w, d, s)) for s in SENS]:
            errs = sq_errors(ev, lv)
            pt, per = point_score(ev, errs)
            res[name] = (pt, per, boot_scores(ev, errs, counts))
        b_pt, b_per, b_bs = res["winner alone"]
        lines.append(f"WINDOW {win} (forecast d_{o + 1}..d_{end}) | sales growth vs previous 28 days {growth * 100:+.1f}% | winner bias {bias:.3
f}")
        lines.append(f"{'Variant':<26}{'WRMSSE':>9}{'Change':>9}{'95% interval of change':>26}{'P(better)':>11}")
        for name, (pt, per, bs) in res.items():
            dd = bs - b_bs
            lo, hi = np.percentile(dd, [2.5, 97.5])
            extra = "" if name == "winner alone" else f"{float((dd < 0).mean()):>11.2f}"
            lines.append(f"{name:<26}{pt:>9.4f}{pt - b_pt:>+9.4f}{f'[{lo:+.4f}, {hi:+.4f}]':>26}{extra}")
        lines.append(f"  OLS solver relative residual (should be tiny): {resid:.1e}")
        lines.append(f"  {'Level':<24}{'Winner':>9}{'Blend':>9}{'Reconciled':>12}")
        for i, (lv_no, lv_name, _) in enumerate(LEVELS):
            lines.append(f"  {lv_no:>2} {lv_name:<21}{b_per[i]:>9.4f}{res['blend (headline)'][1][i]:>9.4f}{res['blend + OLS reconciled'][1][i]:>
12.4f}")
        if win == "final":
            bh = res["blend (headline)"][2]
            lines.append(f"  Official M5 winner 0.5204: blend below it in {(bh < 0.5204).mean():.2f} of draws, winner alone in {(b_bs < 0.5204).
mean():.2f}")
        lines.append("")
        rows.append((win, growth, b_pt, res["blend (headline)"][0], res["blend + OLS reconciled"][0],
                     res["blend (headline)"][2] - b_bs, res["blend + OLS reconciled"][2] - b_bs))

    if rows:
        n = len(rows)
        lines.append(f"SUMMARY ACROSS {n} WINDOWS (change = variant minus winner alone, negative is better)")
        lines.append(f"{'Window':<9}{'Growth':>8}{'Winner':>9}{'Blend':>9}{'Change':>9}{'Reconciled':>12}{'Change':>9}")
        for win, g, b0, h, r, _, _ in rows:
            lines.append(f"{win:<9}{g * 100:>7.1f}%{b0:>9.4f}{h:>9.4f}{h - b0:>+9.4f}{r:>12.4f}{r - b0:>+9.4f}")
        ch_h = np.array([h - b0 for _, _, b0, h, _, _, _ in rows])
        ch_r = np.array([r - b0 for _, _, b0, _, r, _, _ in rows])
        for label, ch, bidx in [("blend (headline)", ch_h, 5), ("blend + OLS reconciled", ch_r, 6)]:
            mean_draws = np.mean([r_[bidx] for r_ in rows], axis=0)  # windows resampled independently (different seeds)
            lo, hi = np.percentile(mean_draws, [2.5, 97.5])
            kk = int((ch < 0).sum())
            lines.append(f"  {label}: mean change {ch.mean():+.4f} | windows improved {kk} of {n} | sign test p = {sign_test_p(kk, n):.3f} | "
                         f"95% interval of the mean change [{lo:+.4f}, {hi:+.4f}] | worst window {ch.max():+.4f}")
        gs = np.array([g for _, g, _, _, _, _, _ in rows])
        if n >= 4 and gs.std() > 0:
            lines.append(f"  Correlation between window sales growth and blend change: {np.corrcoef(gs, ch_h)[0, 1]:+.2f} (descriptive only)")
        lines.append("  Notes: windows do not overlap but share training history, so they are not fully independent. The interval resamples days
only.")
        lines.append("  The rule was fixed before the final window was scored; reconciliation is a secondary, labelled variant.")
    text = "\n".join(lines)
    with open(os.path.join(args.out_dir, "results_multiwindow.txt"), "w") as f:
        f.write(text + "\n")
    print(text)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default="m5_data")
    ap.add_argument("--in_dir", default="inputs")
    ap.add_argument("--out_dir", default="multiwindow_out")
    ap.add_argument("--stage", default="all", choices=["all", "forecast", "score"])
    ap.add_argument("--model", default="amazon/chronos-2")
    ap.add_argument("--context_length", type=int, default=1024)
    ap.add_argument("--mock", action="store_true", help="test the pipeline with a fake forecaster (no real forecasts)")
    args = ap.parse_args()
    os.makedirs(args.out_dir, exist_ok=True)
    os.environ.setdefault("M5_WINDOW", "final")
    meta, values, dcols, calendar, prices = load_m5(args.data_dir)
    dcols = list(dcols)
    cal_full = load_calendar(args.data_dir)
    print(f"Series {values.shape[0]}, days {values.shape[1]}", flush=True)
    if args.stage in ("all", "forecast"):
        stage_forecast(args, meta, values, dcols, calendar, prices, cal_full)
    if args.stage in ("all", "score"):
        stage_score(args, meta, values, dcols, calendar, prices)


if __name__ == "__main__":
    main()
```

src/m5_multiwindow_auto.py

```
"""
One command for PC 2, then leave it running.
  1. Makes the Chronos-2 direct forecasts for the new windows (stage forecast of m5_multiwindow.py).
  2. Waits for the PC 3 zip (send_pc3_windows.zip) to be put into the folder <root>\\drop.
     It starts scoring as soon as winner_cv3, winner_cv2 and winner_cv5 are all inside the zip.
     To score with fewer windows (for example only cv3 finished), create an empty file RUN_NOW.txt in the drop folder.
  3. Scores all windows, checks that the earlier numbers reproduce, and writes send_pc2_multiwindow.zip.
It only reads, runs and writes new files. It deletes nothing and never overwrites an input file.
```

```
usage: python m5_multiwindow_auto.py --data_dir <M5 data, 1969 days> [--root <this folder>] [--poll 60] [--mock]
"""
import argparse
import os
import re
import shutil
import subprocess
import sys
import time
import zipfile

NEED = ["winner_cv3.npy", "winner_cv2.npy", "winner_cv5.npy"]
EXPECT = {"winner": {"cv4": 0.5167, "public": 0.6415, "final": 0.5228}, "blend": {"cv4": 0.5151, "public": 0.6245, "final": 0.5195}}


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", required=True)
    ap.add_argument("--root", default=os.path.dirname(os.path.abspath(__file__)))
    ap.add_argument("--poll", type=int, default=60)
    ap.add_argument("--mock", action="store_true")
    args = ap.parse_args()
    root = os.path.abspath(args.root)
    inp, outd, drop = [os.path.join(root, x) for x in ("inputs", "multiwindow_out", "drop")]
    for d in (inp, outd, drop):
        os.makedirs(d, exist_ok=True)
    status = os.path.join(root, "AUTO_STATUS.txt")

    def log(m):
        line = f"[{time.strftime('%Y-%m-%d %H:%M:%S')}] {m}"
        print(line, flush=True)
        open(status, "a", encoding="utf-8").write(line + "\n")

    base = [sys.executable, "-u", os.path.join(root, "m5_multiwindow.py"), "--data_dir", args.data_dir, "--in_dir", inp, "--out_dir", outd]
    if args.mock:
        base.append("--mock")
    log("step 1: Chronos-2 direct forecasts for the new windows")
    rc = subprocess.run(base + ["--stage", "forecast"], cwd=root).returncode
    if rc != 0:
        log(f"STOP: forecast stage failed (exit {rc}). Send me the error text. Nothing was deleted.")
        sys.exit(1)
    zf = os.path.join(root, "send_pc2_forecasts.zip")
    if not os.path.exists(zf):
        with zipfile.ZipFile(zf, "w", zipfile.ZIP_DEFLATED) as z:
            for w in ("cv3", "cv2", "cv5"):
                p = os.path.join(outd, f"agg_chronos2_B_covariates_{w}.npy")
                if os.path.exists(p):
                    z.write(p, os.path.basename(p))
    log("step 1 done. Waiting for the PC 3 zip in: " + drop + " (put send_pc3_windows.zip there; RUN_NOW.txt there means score with what exists)
")

    handled = set()
    while True:
        run_now = os.path.exists(os.path.join(drop, "RUN_NOW.txt"))
        for fn in sorted(os.listdir(drop)):
            if fn.lower().endswith(".zip") and fn not in handled:
                p = os.path.join(drop, fn)
                try:
                    with zipfile.ZipFile(p) as z:
                        names = z.namelist()
                        got = [n for n in names if os.path.basename(n).startswith("winner_") and n.endswith(".npy")]
                        if not got:
                            continue
                        ex = os.path.join(root, "from_pc3_" + os.path.splitext(fn)[0])
                        os.makedirs(ex, exist_ok=True)
                        z.extractall(ex)
                except zipfile.BadZipFile:
                    continue  # still being copied
                for r, _, fs in os.walk(ex):
                    for f in fs:
                        if f.startswith("winner_") and f.endswith(".npy") and not os.path.exists(os.path.join(inp, f)):
                            shutil.copy2(os.path.join(r, f), os.path.join(inp, f))
                handled.add(fn)
                log(f"read {fn}: winner files now in inputs: " + ", ".join(sorted(f for f in os.listdir(inp) if f.startswith("winner_"))))
        have = [n for n in NEED if os.path.exists(os.path.join(inp, n))]
        if len(have) == len(NEED) or (run_now and have):
            break
        time.sleep(args.poll)

    for w in ("cv3", "cv2", "cv5"):  # copy the new direct forecasts next to the winner files (the scorer also reads multiwindow_out)
        p = os.path.join(outd, f"agg_chronos2_B_covariates_{w}.npy")
        q = os.path.join(inp, f"agg_chronos2_B_covariates_{w}.npy")
        if os.path.exists(p) and not os.path.exists(q):
            shutil.copy2(p, q)
    log("step 3: scoring all windows with the fixed rule")
    rc = subprocess.run(base + ["--stage", "score"], cwd=root).returncode
    if rc != 0:
        log(f"STOP: scoring failed (exit {rc}). Send me the error text.")
        sys.exit(1)
    res = open(os.path.join(outd, "results_multiwindow.txt"), encoding="utf-8").read()
    checks = []
    for m in re.finditer(r"WINDOW (\w+) .*?\n.*?\nwinner alone\s+([\d.]+).*?\nblend \(headline\)\s+([\d.]+)", res):
```

```
        win, wa, bl = m.group(1), float(m.group(2)), float(m.group(3))
        if win in EXPECT["winner"]:
            ok = abs(wa - EXPECT["winner"][win]) <= 0.0002 and abs(bl - EXPECT["blend"][win]) <= 0.0002
            checks.append(f"  {win}: winner {wa:.4f} (expected {EXPECT['winner'][win]}), blend {bl:.4f} (expected {EXPECT['blend'][win]}) -> {'O
K' if ok else 'DIFFERS'}")
    log("reproduction check of earlier numbers:\n" + ("\n".join(checks) if checks else "  no earlier window was scored (data folder too short?)"
))
    zp = os.path.join(root, "send_pc2_multiwindow.zip")
    with zipfile.ZipFile(zp, "w", zipfile.ZIP_DEFLATED) as z:
        z.write(os.path.join(outd, "results_multiwindow.txt"), "results_multiwindow_PC2.txt")
        z.write(status, "AUTO_STATUS.txt")
    log(f"ALL DONE. Send me this file: {zp}")


if __name__ == "__main__":
    main()
```

src/m5_overnight.py

```
"""
OVERNIGHT RUN: every idea to lower WRMSSE below the 0.20 headline, tested honestly. One script, plain-text output.

PART A (CPU, saved forecasts, minutes): writes results_overnight_A.txt
PART B (GPU, Chronos-2, about 15 to 40 minutes): new direct forecasts at levels 1 to 9
        ctx512 / ctx2048   : same model and covariates, 512 days or the whole history as context
        grpLevel           : cross-learning, all series of one level are forecast together
        grpAll             : cross-learning, all 154 aggregate series forecast together
PART C (CPU): repeats part A with the new forecasts as extra candidates: writes results_overnight_FINAL.txt

IDEAS TESTED (levels 1 to 9 blend with a direct foundation-model forecast; level 12 stays the winner)
  1  TOP-DOWN ALIGNMENT: the winner's item forecasts are multiplied by (blended aggregate / winner aggregate) of their parent
     series (level 1, 3 or 9; whole 28 days or per day; shrunk by a power alpha). Items then add up to the blend (second-place trick).
  2  HORIZON SHARE: the share changes linearly over the 28 days.
  3  SHRUNK correction (alpha < 1) inside idea 1.
  4  PER-GROUP / PER-LEVEL share, and a choice of direct forecast (Chronos-2, other models, averages, new GPU variants).
  5  STACK: best group blend, then the best alignment on top.
HONEST TEST: every rule picks its settings WITHOUT the window it is scored on. The five windows before the final one use the
other four of those five; the final window uses those five and is scored once. Nothing is tuned on the final window.

Usage:
  python m5_overnight.py --data_dir <M5 data with 1969 days> --in_dir inputs --fm_dir fm_out --lvl_dir lvl1011_out --out_dir overnight_out
  python m5_overnight.py --selftest        (synthetic data and a mock model, no GPU)
"""
import argparse
import glob
import itertools
import os
import re
import sys
import tempfile
import time
import traceback

import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from m5_data_and_wrmsse import HORIZON, WRMSSEEvaluator, load_calendar, load_m5, make_synthetic  # noqa: E402
from m5_multiwindow import sign_test_p  # noqa: E402

WINS = ["cv5", "cv2", "cv3", "cv4", "public", "final"]
GRID = np.round(np.arange(0.0, 0.601, 0.05), 2)
I20 = int(np.where(GRID == 0.20)[0][0])
GROUPS = [[0, 1, 2], [3, 4, 5], [6, 7, 8], [9, 10]]
GROUP_NAMES = ["L1-3", "L4-6", "L7-9", "L10-11"]
ALIGN_CFGS = [(L, mode, a) for L in (0, 2, 8) for mode in ("whole", "daily") for a in (0.5, 0.75, 1.0)]
H_GRID = [(a, b) for a in (0.10, 0.15, 0.20, 0.25, 0.30, 0.40) for b in (-0.15, -0.10, -0.05, 0.0, 0.05, 0.10, 0.15)]
LOG = None


def say(msg):
    print(msg, flush=True)
    if LOG:
        with open(LOG, "a") as f:
            f.write(time.strftime("%H:%M:%S ") + msg + "\n")


# ------------------------------------------------------------------ window container
class Win:
    def __init__(self, name, ev, winner, cands, d10, d11):
        self.name, self.ev = name, ev
        self.Wbot = winner.astype(np.float64)
        self.Wagg = [np.asarray(A @ self.Wbot) for A in ev.A]
        self.cands, self.D10, self.D11 = cands, d10.astype(np.float64), d11.astype(np.float64)
        self.off = np.cumsum([0] + ev.sizes[:9])
        self.grp = [np.asarray(A.argmax(axis=0)).ravel() for A in ev.A[:9]]
        self.cache, self.lcache = {}, {}

    def direct(self, cand, i):
        if i == 9:
            return self.D10
        if i == 10:
```

```
            return self.D11
        return self.cands[cand][self.off[i]:self.off[i + 1]].astype(np.float64)

    def score(self, i, P):
        ev = self.ev
        w = ev.weight[i]
        v = w > 0
        r = np.sqrt(np.mean((ev.y_agg[i][v] - P[v]) ** 2, axis=1) / ev.scale[i][v])
        return float((w[v] * r).sum())

    def ls(self, cand, i, k):
        """Level score of level i (0-based) with share GRID[k] of candidate `cand`."""
        if i == 11 or k == 0:
            key = ("W", i)
        else:
            key = ("C2" if i >= 9 else cand, i, k)
        if key not in self.lcache:
            if key[0] == "W":
                self.lcache[key] = self.score(i, self.Wagg[i])
            else:
                s = GRID[k]
                self.lcache[key] = self.score(i, (1 - s) * self.Wagg[i] + s * self.direct(key[0], i))
        return self.lcache[key]


def levels_blend(w, spec):
    return [w.ls(spec[i][0], i, spec[i][1]) for i in range(11)] + [w.ls("C2", 11, 0)]


def levels_h(w, a, b):
    h = np.arange(HORIZON)
    s = np.clip(a + b * (h - 13.5) / 13.5, 0, 0.6)[None, :]
    lv = [w.score(i, (1 - s) * w.Wagg[i] + s * w.direct("C2", i)) for i in range(9)]
    return lv + [w.ls("C2", 9, 0), w.ls("C2", 10, 0), w.ls("C2", 11, 0)]


def levels_align(w, spec, cfg):
    L, mode, alpha = cfg
    cand, k = spec[L]
    if k == 0:
        return levels_blend(w, spec)
    s = GRID[k]
    Bl = (1 - s) * w.Wagg[L] + s * w.direct(cand, L)
    if mode == "whole":
        r = Bl.sum(axis=1, keepdims=True) / np.maximum(w.Wagg[L].sum(axis=1, keepdims=True), 1e-9)
    else:
        r = Bl / np.maximum(w.Wagg[L], 1e-9)
    r = np.clip(r, 0.5, 2.0) ** alpha
    scaled = w.Wbot * r[w.grp[L]]
    lv = []
    for i, A in enumerate(w.ev.A):
        P = np.asarray(A @ scaled)
        if i in (9, 10) and spec[i][1] > 0:
            s2 = GRID[spec[i][1]]
            P = (1 - s2) * P + s2 * w.direct("C2", i)
        lv.append(w.score(i, P))
    return lv


def levels_cfg(w, spec, cfg):
    key = (spec, cfg)
    if key not in w.cache:
        if cfg is None:
            w.cache[key] = levels_blend(w, spec)
        elif cfg[0] == "H":
            w.cache[key] = levels_h(w, cfg[1], cfg[2])
        else:
            w.cache[key] = levels_align(w, spec, cfg)
    return w.cache[key]


def cost_cfg(w, spec, cfg):
    return float(np.mean(levels_cfg(w, spec, cfg)))


# ------------------------------------------------------------------ selection rules
def sel_groups(W, tr, groups, cands):
    spec = [None] * 11
    for g in groups:
        best = None
        cl = ["C2"] if min(g) >= 9 else cands
        for cand in cl:
            for k in range(len(GRID)):
                c = np.mean([sum(W[x].ls(cand, i, k) for i in g) for x in tr])
                if best is None or c < best[0] - 1e-12:
                    best = (c, cand, k)
        for i in g:
            spec[i] = (best[1], best[2]) if best[2] > 0 else ("C2", 0)
    return tuple(spec)


def sel_global(W, tr, upto):
    best = min(range(len(GRID)), key=lambda k: np.mean(
```

```
        [sum(W[x].ls("C2", i, k) for i in range(upto)) + sum(W[x].ls("C2", i, 0) for i in range(upto, 12)) for x in tr]))
    return tuple([("C2", best if best else 0)] * upto + [("C2", 0)] * (11 - upto))


def sel_cfg(W, tr, spec, cfgs):
    best = (np.mean([cost_cfg(W[x], spec, None) for x in tr]), None)
    for c in cfgs:
        v = np.mean([cost_cfg(W[x], spec, c) for x in tr])
        if v < best[0] - 1e-12:
            best = (v, c)
    return best[1]


def spec_text(spec):
    parts = []
    for n, g in zip(GROUP_NAMES, GROUPS):
        c, k = spec[g[0]]
        same = all(spec[i] == spec[g[0]] for i in g)
        parts.append(f"{n}={GRID[k]:.2f}" + (f"({c})" if k > 0 and c != "C2" else "") + ("" if same else "*"))
    return " ".join(parts)


def cfg_text(cfg):
    if cfg is None:
        return "no alignment"
    if cfg[0] == "H":
        return f"share a={cfg[1]:.2f} b={cfg[2]:+.2f}"
    return f"align L{cfg[0] + 1} {cfg[1]} alpha {cfg[2]}"


# ------------------------------------------------------------------ analysis
def analysis(args, meta, values, dcols, calendar, prices, D, evs, out_name, extra_gpu):
    wins_all, W, tags = [], {}, {}
    for idx, name in enumerate(WINS):
        o = D - HORIZON * (6 - idx)
        if o < 400 and not args.selftest:
            continue
        p_w = os.path.join(args.in_dir, f"winner_{name}.npy")
        p_c = os.path.join(args.in_dir, f"agg_chronos2_B_covariates_{name}.npy")
        p10 = os.path.join(args.lvl_dir, f"lvl10_chronos2_B_{name}.npy")
        p11 = os.path.join(args.lvl_dir, f"lvl11_chronos2_B_{name}.npy")
        if not all(os.path.exists(p) for p in (p_w, p_c, p10, p11)):
            say(f"window {name}: skipped (missing winner, Chronos-2 or level 10/11 file)")
            continue
        if name not in evs:
            evs[name] = WRMSSEEvaluator(meta, values[:, : o + HORIZON], dcols[: o + HORIZON], calendar, prices)
        cands = {"C2": np.load(p_c)}
        for p in glob.glob(os.path.join(args.fm_dir, f"fm_*_{name}.npy")):
            m = re.match(rf"fm_(.+)_{name}\.npy$", os.path.basename(p))
            if m and not m.group(1).endswith("median"):
                cands["FM_" + m.group(1)] = np.load(p)
        for p in glob.glob(os.path.join(args.out_dir, "gpu_fc", f"agg_chronos2_B_*_{name}.npy")):
            m = re.match(rf"agg_chronos2_B_(.+)_{name}\.npy$", os.path.basename(p))
            if m:
                cands["G_" + m.group(1)] = np.load(p)
        tags[name] = cands
        W[name] = (name, evs[name], np.load(p_w), cands, np.load(p10), np.load(p11))
        wins_all.append(name)
    if len(wins_all) < 4:
        say("Not enough windows for the analysis.")
        return None
    common = set.intersection(*[set(tags[n]) for n in wins_all])
    base = sorted(common)
    derived = {}
    fm = [c for c in base if c.startswith("FM_")]
    if fm:
        derived["AVG_ALL"] = ["C2"] + fm
    if "FM_timesfm3" in base:
        derived["AVG_C2_TFM"] = ["C2", "FM_timesfm3"]
    gx = [c for c in ("G_ctx512", "G_ctx2048") if c in base]
    if len(gx) == 2:
        derived["AVG_CTX3"] = ["C2"] + gx
    for n in wins_all:
        for dn, parts in derived.items():
            tags[n][dn] = np.mean([tags[n][p] for p in parts], axis=0)
    cands = ["C2"] + [c for c in sorted(set(base) | set(derived)) if c != "C2"]
    W = {n: Win(*W[n]) for n in wins_all}
    pre = [n for n in wins_all if n != "final"]
    say(f"[{out_name}] windows {wins_all}; candidates {cands}")

    H = tuple([("C2", I20)] * 9 + [("C2", 0)] * 2)
    EXT = tuple([("C2", I20)] * 11)
    WIN0 = tuple([("C2", 0)] * 11)
    res, desc, spc = {}, {}, {}

    def fixed(name, spec, cfg=None):
        res[name] = {n: cost_cfg(W[n], spec, cfg) for n in wins_all}
        desc[name] = {n: "" for n in wins_all}
        spc[name] = {n: (spec, cfg) for n in wins_all}

    def loo(name, rule):
        res[name], desc[name], spc[name] = {}, {}, {}
        for n in wins_all:
```

```
        tr = [x for x in pre if x != n]
        spec, cfg = rule(tr)
        res[name][n] = cost_cfg(W[n], spec, cfg)
        desc[name][n] = spec_text(spec) + ("" if cfg is None else " | " + cfg_text(cfg))
        spc[name][n] = (spec, cfg)

fixed("winner alone", WIN0)
fixed("HEADLINE 0.20, levels 1-9", H)
fixed("EXTENSION 0.20, levels 1-11", EXT)
t0 = time.time()
loo("T1 GLOBAL share, levels 1-9", lambda tr: (sel_global(W, tr, 9), None))
loo("T2 GLOBAL share, levels 1-11", lambda tr: (sel_global(W, tr, 11), None))
loo("T3 PER-GROUP share (Chronos-2)", lambda tr: (sel_groups(W, tr, GROUPS, ["C2"]), None))
loo("T4 PER-LEVEL share (Chronos-2)", lambda tr: (sel_groups(W, tr, [[i] for i in range(11)], ["C2"]), None))
if len(cands) > 1:
    loo("M1 PER-GROUP share + choice of forecast", lambda tr: (sel_groups(W, tr, GROUPS, cands), None))
say(f"  share rules done {time.time() - t0:.0f}s")

def rule_h(tr):
    best = min(H_GRID, key=lambda ab: np.mean([cost_cfg(W[x], H, ("H",) + ab) for x in tr]))
    if np.mean([cost_cfg(W[x], H, ("H",) + best) for x in tr]) >= np.mean([cost_cfg(W[x], H, None) for x in tr]) - 1e-12:
        return H, None
    return H, ("H",) + best
loo("H1 HORIZON share (levels 1-9)", rule_h)
say(f"  horizon rule done {time.time() - t0:.0f}s")
loo("A1 TOP-DOWN ALIGNMENT on the headline", lambda tr: (H, sel_cfg(W, tr, H, ALIGN_CFGS)))
say(f"  alignment rule done {time.time() - t0:.0f}s")
loo("A2 ALIGNMENT on EXTENSION (levels 1-11)", lambda tr: (EXT, sel_cfg(W, tr, EXT, ALIGN_CFGS)))
stack_src = "M1 PER-GROUP share + choice of forecast" if len(cands) > 1 else "T3 PER-GROUP share (Chronos-2)"

def rule_stack(tr):
    sp = sel_groups(W, tr, GROUPS, cands)
    return sp, sel_cfg(W, tr, sp, ALIGN_CFGS)
loo("S1 STACK: group blend + alignment", rule_stack)
say(f"  stack rule done {time.time() - t0:.0f}s")

fixed_rows = []
for c in cands:
    nm = f"(fixed 0.20, levels 1-9) forecast = {c}"
    fixed(nm, tuple([(c, I20)] * 9 + [("C2", 0)] * 2))
    fixed_rows.append(nm)
orc = {}
for n in wins_all:
    sp = sel_groups(W, [n], [[i] for i in range(11)], cands)
    orc[n] = cost_cfg(W[n], sp, None)

# ------------------------------------------------------------ write
L = ["OVERNIGHT RESULTS: lower WRMSSE than the 0.20 headline? (lower is better)",
     f"Windows with data: {', '.join(wins_all)}. Honest test: each window is scored with settings chosen WITHOUT it",
     "(five pre-final windows use the other four; the final window uses the five). mean5 = mean of the five pre-final windows.",
     f"Candidate direct forecasts for levels 1-9: {', '.join(cands)}",
     "Reference scores on the final window: official winner 0.5204, our reproduction 0.5228, preprint stretch 0.4989.", ""]
bias = ", ".join(f"{n} {W[n].Wagg[0].sum() / W[n].ev.y_agg[0].sum():.3f}" for n in wins_all)
L += [f"Winner bias (total forecast / total actual): {bias}", ""]
hdr = f"{'Rule':<46}" + "".join(f"{n:>9}" for n in wins_all) + f"{'mean5':>9}"
L += ["WRMSSE", hdr]
order = ["winner alone", "HEADLINE 0.20, levels 1-9", "EXTENSION 0.20, levels 1-11"] + \
        [k for k in res if k[:2] in ("T1", "T2", "T3", "T4", "M1", "H1", "A1", "A2", "S1")]
for k in order:
    L.append(f"{k:<46}" + "".join(f"{res[k][n]:>9.4f}" for n in wins_all) + f"{np.mean([res[k][n] for n in pre]):>9.4f}")
L.append(f"{'(reference only) per-level best ON the window':<46}" + "".join(f"{orc[n]:>9.4f}" for n in wins_all)
         + f"{np.mean([orc[n] for n in pre]):>9.4f}   not honest, upper bound")
L += ["", "FIXED RULE, WHICH DIRECT FORECAST IS BEST AT SHARE 0.20 (no selection, levels 1-9 only)", hdr]
for k in fixed_rows:
    L.append(f"{k:<46}" + "".join(f"{res[k][n]:>9.4f}" for n in wins_all) + f"{np.mean([res[k][n] for n in pre]):>9.4f}")
L += ["", "CHANGE VS HEADLINE (negative is better); 'better' counts the five pre-final windows", hdr + f"{'better':>8}{'p':>7}"]
Hh = res["HEADLINE 0.20, levels 1-9"]
for k in order[2:] + fixed_rows:
    ch = {n: res[k][n] - Hh[n] for n in wins_all}
    kk = sum(ch[n] < 0 for n in pre)
    L.append(f"{k:<46}" + "".join(f"{ch[n]:>+9.4f}" for n in wins_all) + f"{np.mean([ch[n] for n in pre]):>+9.4f}"
             f"{kk:>5}/{len(pre)}{sign_test_p(kk, len(pre)):>7.3f}")
L += ["", "SETTINGS CHOSEN FOR THE FINAL WINDOW (from the five earlier windows). * = levels in the group differ"]
for k in order[3:]:
    if "final" in desc[k] and desc[k]["final"]:
        L.append(f"  {k}: {desc[k]['final']}")
if "final" in wins_all:
    L += ["", "LEVEL SCORES ON THE FINAL WINDOW (WRMSSE per level)"]
    show = ["winner alone", "HEADLINE 0.20, levels 1-9", stack_src, "S1 STACK: group blend + alignment"]
    L.append(f"{'Level':<8}" + "".join(f"{s[:22]:>24}" for s in show))
    lvs = {s: levels_cfg(W["final"], *spc[s]["final"]) for s in show}
    for i in range(12):
        L.append(f"{'L' + str(i + 1):<8}" + "".join(f"{lvs[s][i]:>24.4f}" for s in show))
L += ["", "HOW TO READ",
      "1. A rule is worth adopting only if it beats HEADLINE on the five held-out windows (better 4/5 or 5/5) AND on the final window.",
      "2. The 'reference only' row is the most per-level tuning could ever give; the gap to the honest rows is overfitting.",
      "3. Windows share history and are only four weeks each, so p values are rough. Do not claim to beat 0.5204 unless the",
      "   final-window gain is clearly larger than the earlier bootstrap interval (about 0.01).",
      "4. Alignment makes the item forecasts add up to the blended aggregate, so those rows are coherent by construction.",
      f"Runtime of this analysis {(time.time() - t0) / 60:.1f} min"]
text = "\n".join(L)
with open(os.path.join(args.out_dir, out_name), "w") as f:
    f.write(text + "\n")
```

```
    print(text, flush=True)
    return text


# ---------------------------------------------------------------- GPU stage
def stage_gpu(args, meta, values, dcols, calendar, prices, cal_full, D, evs):
    from m5_aggregate import build_inputs
    from m5_chronos2 import MockPipeline, quantile_mean_weights, to_np
    gdir = os.path.join(args.out_dir, "gpu_fc")
    os.makedirs(gdir, exist_ok=True)
    if args.mock:
        pipe = MockPipeline()
    else:
        import torch
        from chronos import Chronos2Pipeline
        cuda = torch.cuda.is_available()
        say(f"Loading amazon/chronos-2 on {'cuda' if cuda else 'CPU'}")
        pipe = Chronos2Pipeline.from_pretrained("amazon/chronos-2", device_map="cuda" if cuda else "cpu")
    wts = quantile_mean_weights(pipe.quantiles)

    def fc(inputs, bs, cross):
        pr = pipe.predict(inputs, prediction_length=HORIZON, batch_size=bs, cross_learning=cross)
        arr = np.stack([to_np(t)[0] for t in pr])
        return np.clip(np.tensordot(arr, wts, axes=([1], [0])), 0, None).astype(np.float32)

    for idx, name in enumerate(WINS):
        o = D - HORIZON * (6 - idx)
        if o < 400 and not args.selftest:
            continue
        todo = [v for v in ("ctx512", "ctx2048", "grpLevel", "grpAll")
                if not os.path.exists(os.path.join(gdir, f"agg_chronos2_B_{v}_{name}.npy"))]
        if not todo:
            continue
        if name not in evs:
            evs[name] = WRMSSEEvaluator(meta, values[:, : o + HORIZON], dcols[: o + HORIZON], calendar, prices)
        ev = evs[name]
        sizes = ev.sizes[:9]
        for v in todo:
            t0 = time.time()
            try:
                ctx = {"ctx512": 512, "ctx2048": 2048}.get(v, 1024)
                inputs = build_inputs(ev, meta, values, cal_full, dcols[: o + HORIZON], o, ctx, True)
                if v in ("ctx512", "ctx2048"):
                    out = fc(inputs, 64, False)
                elif v == "grpAll":
                    out = fc(inputs, len(inputs), True)
                else:
                    parts, s = [], 0
                    for g in sizes:
                        parts.append(fc(inputs[s:s + g], g, True))
                        s += g
                    out = np.concatenate(parts, axis=0)
                np.save(os.path.join(gdir, f"agg_chronos2_B_{v}_{name}.npy"), out)
                say(f"  GPU {name} {v} done in {time.time() - t0:.0f}s")
            except Exception as e:  # keep going, log and skip
                say(f"  GPU {name} {v} FAILED: {e!r}")
                with open(os.path.join(args.out_dir, "gpu_errors.txt"), "a") as f:
                    f.write(f"{name} {v}\n{traceback.format_exc()}\n")


# ---------------------------------------------------------------- selftest fixtures
def make_fixtures(args, meta, values, dcols, calendar, prices, D):
    from m5_data_and_wrmsse import build_aggregators
    rng = np.random.default_rng(3)
    A = build_aggregators(meta)
    sizes = [a.shape[0] for a in A]
    for d in (args.in_dir, args.fm_dir, args.lvl_dir):
        os.makedirs(d, exist_ok=True)
    for idx, name in enumerate(WINS):
        o = D - HORIZON * (6 - idx)
        y = values[:, o:o + HORIZON]
        np.save(os.path.join(args.in_dir, f"winner_{name}.npy"), y * 0.93 * rng.uniform(0.7, 1.3, y.shape))
        agg = np.vstack([np.asarray(a @ y) for a in A[:9]])
        np.save(os.path.join(args.in_dir, f"agg_chronos2_B_covariates_{name}.npy"), agg * rng.uniform(0.85, 1.15, agg.shape))
        for tag in ("timesfm3", "t0"):
            np.save(os.path.join(args.fm_dir, f"fm_{tag}_{name}.npy"), agg * rng.uniform(0.8, 1.2, agg.shape))
        for n_, li in (("10", 9), ("11", 10)):
            z = np.asarray(A[li] @ y)
            np.save(os.path.join(args.lvl_dir, f"lvl{n_}_chronos2_B_{name}.npy"), z * rng.uniform(0.8, 1.2, z.shape))


# ---------------------------------------------------------------- main
def main():
    global LOG
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default="m5_data")
    ap.add_argument("--in_dir", default="inputs")
    ap.add_argument("--fm_dir", default="fm_out")
    ap.add_argument("--lvl_dir", default="lvl1011_out")
    ap.add_argument("--out_dir", default="overnight_out")
    ap.add_argument("--stage", default="all", choices=["all", "analysis", "gpu"])
    ap.add_argument("--mock", action="store_true")
    ap.add_argument("--selftest", action="store_true")
```

```
    args = ap.parse_args()
    if args.selftest:
        from m5_chronos2 import augment_calendar
        tmp = tempfile.mkdtemp()
        make_synthetic(tmp, n_days=420)
        augment_calendar(tmp, 420)
        args.data_dir, args.mock = tmp, True
        args.in_dir, args.fm_dir, args.lvl_dir = [os.path.join(tmp, x) for x in ("in", "fm", "lvl")]
        args.out_dir = os.path.join(tmp, "out")
    os.makedirs(args.out_dir, exist_ok=True)
    LOG = os.path.join(args.out_dir, "run_log.txt")
    os.environ.setdefault("M5_WINDOW", "final")
    meta, values, dcols, calendar, prices = load_m5(args.data_dir)
    dcols = list(dcols)
    cal_full = load_calendar(args.data_dir)
    D = values.shape[1]
    say(f"Series {values.shape[0]}, days {D}")
    if args.selftest:
        make_fixtures(args, meta, values, dcols, calendar, prices, D)
    evs = {}
    if args.stage in ("all", "analysis"):
        try:
            analysis(args, meta, values, dcols, calendar, prices, D, evs, "results_overnight_A.txt", False)
        except Exception:
            say("PART A FAILED: " + traceback.format_exc())
    if args.stage in ("all", "gpu"):
        try:
            stage_gpu(args, meta, values, dcols, calendar, prices, cal_full, D, evs)
        except Exception:
            say("PART B FAILED: " + traceback.format_exc())
    if args.stage in ("all", "analysis") and glob.glob(os.path.join(args.out_dir, "gpu_fc", "*.npy")):
        try:
            analysis(args, meta, values, dcols, calendar, prices, D, evs, "results_overnight_FINAL.txt", True)
        except Exception:
            say("PART C FAILED: " + traceback.format_exc())
    say("ALL DONE")


if __name__ == "__main__":
    main()
```

src/m5_patchtst.py

```
#!/usr/bin/env python3
"""
E2: PatchTST on M5 (global model, trained on M5 bottom-level series), with an optional covariate variant.

  P0_univariate   E2   PatchTST on sales history only (channel independent, patching, RevIN-style level scaling)
  P1_covariates   E2b  same backbone, plus known-future covariates (price ratio, SNAP, weekday, event)
                       concatenated into the forecast head

One global model per configuration and per window:
  holdout     trained on d_1..d_1913, forecasts d_1914..d_1941 (the 28-day test window)
  validation  trained on d_1..d_1885, forecasts d_1886..d_1913 (a separate training run)

The validation forecasts are what the ensemble weights are fitted on later, so nothing is chosen on the holdout.
Training settings are fixed in advance (see the config block) and are not tuned on the holdout.

Model
  input: last --context days of sales (default 224), divided by the series level (mean of the last 56 days + 0.1)
  patching: patch length 16, stride 8, transformer encoder (pre-norm), flatten head, direct 28-day output
  loss: MSE in level-scaled units, masked for days before the item was on sale
  point forecast: model output x level, clipped at zero

Outputs (in --out_dir)
  pred_patchtst_<cfg>.npy      holdout forecast, shape (30490, 28)
  predval_patchtst_<cfg>.npy   validation-window forecast, shape (30490, 28)
  results_patchtst.txt         plain-text log: ladder, 12-level table, training curves, timings

Usage
  python m5_patchtst.py --quick       # small training run to check the setup and timing
  python m5_patchtst.py               # full run (holdout and validation, both configurations)
  python m5_patchtst.py --configs P0_univariate    # one configuration only
  python m5_patchtst.py --selftest    # synthetic data, tiny model, no GPU needed

Requires m5_data_and_wrmsse.py in the same folder and a CUDA PyTorch for the full run.
"""
import argparse
import math
import os
import sys
import tempfile
import time

import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from m5_data_and_wrmsse import (  # noqa: E402
    HORIZON,
    WRMSSEEvaluator,
```

```
    find_file,
    format_table,
    load_m5,
    load_calendar,
    make_synthetic,
    seasonal_naive_forecast,
)

LEVEL_WIN = 56
LEVEL_EPS = 0.1


# ---------------------------------------------------------------------------
# Data
# ---------------------------------------------------------------------------
def augment_calendar(folder, n_days, seed=1):
    rng = np.random.default_rng(seed)
    cal = pd.read_csv(os.path.join(folder, "calendar.csv"))
    dates = pd.date_range("2015-01-01", periods=n_days)
    cal["date"] = dates.strftime("%Y-%m-%d")
    cal["wday"] = dates.dayofweek + 1
    cal["month"] = dates.month
    ev = rng.random(n_days) < 0.05
    cal["event_name_1"] = np.where(ev, "EventX", None)
    cal["event_type_1"] = np.where(ev, "Cultural", None)
    for st in ["CA", "TX", "WI"]:
        cal[f"snap_{st}"] = (rng.random(n_days) < 0.3).astype(int)
    cal.to_csv(os.path.join(folder, "calendar.csv"), index=False)


class M5Tensors:
    """Sales (with pre-launch days masked), price, and shared calendar covariates, all as arrays."""

    def __init__(self, meta, values, dcols, cal_full, prices):
        self.n, self.D = values.shape
        cal = cal_full.set_index("d").loc[dcols]
        wk = cal["wm_yr_wk"].to_numpy()
        weeks = np.unique(wk)
        widx = np.searchsorted(weeks, wk)
        p = prices[prices["wm_yr_wk"].isin(weeks)]
        key = pd.MultiIndex.from_frame(meta[["item_id", "store_id"]])
        r = key.get_indexer(pd.MultiIndex.from_frame(p[["item_id", "store_id"]]))
        c = np.searchsorted(weeks, p["wm_yr_wk"].to_numpy())
        ok = r >= 0
        Pw = np.full((self.n, len(weeks)), np.nan, dtype=np.float32)
        Pw[r[ok], c[ok]] = p["sell_price"].to_numpy(dtype=np.float32)[ok]
        P = Pw[:, widx]
        valid = ~np.isnan(P)
        first = np.where(valid.any(axis=1), valid.argmax(axis=1), 0)
        fill = P[np.arange(self.n), first]
        fill = np.where(np.isnan(fill), 0.0, fill).astype(np.float32)
        P = np.where(valid, P, fill[:, None]).astype(np.float32)
        # price relative to the item's own running level (mean of its last 365 days), always causal-ish scale
        pm = np.maximum(np.nanmean(P, axis=1, keepdims=True), 1e-3)
        self.price_rel = (P / pm).astype(np.float32)
        self.mask = valid.astype(np.float32)  # 1 where item is on sale
        self.Y = np.where(valid, values, 0.0).astype(np.float32)

        states = sorted(meta["state_id"].unique())
        self.state_code = pd.factorize(meta["state_id"], sort=True)[0]
        self.snap = np.stack([cal[f"snap_{s}"].to_numpy().astype(np.float32) for s in states])  # (3, D)
        wd = cal["wday"].to_numpy().astype(int) - 1
        self.wday_onehot = np.eye(7, dtype=np.float32)[wd]  # (D, 7)
        self.event = cal["event_name_1"].notna().to_numpy().astype(np.float32)  # (D,)

    def future_cov(self, idx, start, h=HORIZON):
        """Known-future covariates for series idx over days [start, start+h): (len(idx), h, 10)."""
        sl = slice(start, start + h)
        m = len(idx)
        price = self.price_rel[idx, sl][:, :, None]
        snap = self.snap[self.state_code[idx], sl][:, :, None]
        wd = np.broadcast_to(self.wday_onehot[sl][None], (m, h, 7))
        ev = np.broadcast_to(self.event[sl][None, :, None], (m, h, 1))
        return np.concatenate([price, snap, wd, ev], axis=2).astype(np.float32)


N_COV = 10


# ---------------------------------------------------------------------------
# Model
# ---------------------------------------------------------------------------
def build_model(cfg, use_cov):
    import torch
    import torch.nn as nn

    class PatchTST(nn.Module):
        def __init__(self):
            super().__init__()
            self.ctx, self.pl, self.st = cfg["context"], cfg["patch_len"], cfg["stride"]
            self.n_patch = (self.ctx - self.pl) // self.st + 1
            d = cfg["d_model"]
            self.embed = nn.Linear(self.pl, d)
```

```
            self.pos = nn.Parameter(torch.randn(1, self.n_patch, d) * 0.02)
            layer = nn.TransformerEncoderLayer(
                d_model=d, nhead=cfg["heads"], dim_feedforward=cfg["ff"], dropout=cfg["dropout"],
                batch_first=True, norm_first=True, activation="gelu")
            self.enc = nn.TransformerEncoder(layer, num_layers=cfg["layers"], enable_nested_tensor=False)
            self.norm = nn.LayerNorm(d)
            self.use_cov = use_cov
            self.cov_dim = HORIZON * N_COV if use_cov else 0
            self.head = nn.Sequential(
                nn.Dropout(cfg["dropout"]),
                nn.Linear(self.n_patch * d + self.cov_dim, cfg["head_hidden"]),
                nn.GELU(),
                nn.Linear(cfg["head_hidden"], HORIZON),
            )

        def forward(self, x, cov=None):
            # x: (B, ctx) level-scaled sales
            p = x.unfold(1, self.pl, self.st)  # (B, n_patch, pl)
            z = self.enc(self.embed(p) + self.pos)
            z = self.norm(z).flatten(1)
            if self.use_cov:
                z = torch.cat([z, cov.flatten(1)], dim=1)
            return self.head(z)

    return PatchTST()


# ---------------------------------------------------------------------------
# Train / predict for one window
# ---------------------------------------------------------------------------
def make_batch(T_, idx, ends, ctx, use_cov, torch, dev):
    """Gather contexts ending at `ends` (exclusive) and the 28-day targets after them."""
    pos_c = ends[:, None] + np.arange(-ctx, 0)[None, :]
    pos_t = ends[:, None] + np.arange(HORIZON)[None, :]
    Yc = T_.Y[idx[:, None], np.clip(pos_c, 0, None)] * (pos_c >= 0)
    Mc = T_.mask[idx[:, None], np.clip(pos_c, 0, None)] * (pos_c >= 0)
    lv = (Yc[:, -LEVEL_WIN:].sum(1) / np.maximum(Mc[:, -LEVEL_WIN:].sum(1), 1.0)) + LEVEL_EPS
    out = {"x": Yc / lv[:, None], "lv": lv}
    if pos_t.max() < T_.D:
        out["y"] = T_.Y[idx[:, None], pos_t] / lv[:, None]
        out["m"] = T_.mask[idx[:, None], pos_t]
    if use_cov:
        out["cov"] = _cov_batch(T_, idx, ends)
    return {k: torch.from_numpy(np.ascontiguousarray(v)).float().to(dev) for k, v in out.items()}


def _cov_batch(T_, idx, ends):
    pos_t = ends[:, None] + np.arange(HORIZON)[None, :]
    price = T_.price_rel[idx[:, None], pos_t][:, :, None]
    snap = T_.snap[T_.state_code[idx][:, None], pos_t][:, :, None]
    wd = T_.wday_onehot[pos_t]
    ev = T_.event[pos_t][:, :, None]
    return np.concatenate([price, snap, wd, ev], axis=2).astype(np.float32)


def train_and_predict(T_, origin, use_cov, cfg, steps, dev, log, seed=0):
    """Train on data before `origin`, forecast origin..origin+28. Returns (n, 28) forecast."""
    import torch

    torch.manual_seed(seed)
    rng = np.random.default_rng(seed)
    ctx = cfg["context"]
    model = build_model(cfg, use_cov).to(dev)
    n_params = sum(p.numel() for p in model.parameters())
    log(f"    model parameters: {n_params / 1e6:.2f} M")
    opt = torch.optim.AdamW(model.parameters(), lr=cfg["lr"], weight_decay=cfg["wd"])
    warm = max(1, int(0.05 * steps))
    sched = torch.optim.lr_scheduler.LambdaLR(
        opt, lambda s: min(1.0, (s + 1) / warm) * 0.5 * (1 + math.cos(math.pi * min(1.0, s / steps))))
    amp = dev.type == "cuda"
    scaler = torch.amp.GradScaler("cuda", enabled=amp)

    # training end points: forecast target must lie fully before `origin`
    last_end = origin - HORIZON
    first_end = max(ctx // 2, last_end - cfg["train_span"])
    n = T_.n
    curve = []
    t0 = time.time()
    model.train()
    run = 0.0
    for step in range(steps):
        idx = rng.integers(0, n, size=cfg["batch"])
        ends = rng.integers(first_end, last_end + 1, size=cfg["batch"])
        b = make_batch(T_, idx, ends, ctx, use_cov, torch, dev)
        with torch.autocast(device_type=dev.type, dtype=torch.bfloat16, enabled=amp):
            out = model(b["x"], b.get("cov"))
        out = out.float()
        err = (out - b["y"]) ** 2 * b["m"]
        loss = err.sum() / b["m"].sum().clamp(min=1.0)
        opt.zero_grad(set_to_none=True)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step()
```

```
        sched.step()
        run = 0.98 * run + 0.02 * loss.item() if step else loss.item()
        if (step + 1) % max(1, steps // 10) == 0:
            curve.append((step + 1, run))
            log(f"    step {step + 1}/{steps} | smoothed train loss {run:.4f} | {time.time() - t0:.0f} s")

    model.eval()
    preds = np.zeros((n, HORIZON), dtype=np.float32)
    with torch.no_grad():
        for s in range(0, n, 2048):
            idx = np.arange(s, min(s + 2048, n))
            ends = np.full(len(idx), origin)
            b = make_batch(T_, idx, ends, ctx, use_cov, torch, dev)
            with torch.autocast(device_type=dev.type, dtype=torch.bfloat16, enabled=amp):
                o = model(b["x"], b.get("cov"))
            preds[idx] = (o.float() * b["lv"][:, None]).cpu().numpy()
    return np.clip(preds, 0, None), curve


# ---------------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------------
CONFIG = dict(context=224, patch_len=16, stride=8, d_model=128, heads=8, ff=256, layers=3,
              dropout=0.1, head_hidden=512, lr=5e-4, wd=1e-2, batch=512, train_span=1100)
ALL_CONFIGS = {"P0_univariate": False, "P1_covariates": True}


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default=None)
    ap.add_argument("--out_dir", default="m5_outputs")
    ap.add_argument("--steps", type=int, default=12000, help="training steps per model")
    ap.add_argument("--configs", default="P0_univariate,P1_covariates")
    ap.add_argument("--seed", type=int, default=0)
    ap.add_argument("--no_val", action="store_true")
    ap.add_argument("--quick", action="store_true", help="400 steps, holdout only, prints a time estimate")
    ap.add_argument("--selftest", action="store_true")
    args = ap.parse_args()

    t0 = time.time()
    log = lambda m: print(m, flush=True)  # noqa: E731
    cfg = dict(CONFIG)

    if args.selftest:
        tmp = tempfile.mkdtemp()
        make_synthetic(tmp, n_days=240)
        augment_calendar(tmp, 240)
        data_dir = tmp
        cfg.update(context=64, d_model=32, heads=4, ff=64, layers=1, head_hidden=64, batch=128, train_span=100)
        args.steps = 60
    elif args.data_dir:
        data_dir = args.data_dir
    elif os.path.isdir("m5_data"):
        data_dir = "m5_data"
    else:
        import kagglehub
        data_dir = kagglehub.competition_download("m5-forecasting-accuracy")
    log(f"Data folder: {data_dir}")

    import torch
    dev = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    dev_note = f"cuda: {torch.cuda.get_device_name(0)}" if dev.type == "cuda" else "CPU (no CUDA found, this will be slow)"
    log(f"Device: {dev_note}")

    meta, values, dcols, calendar, prices = load_m5(data_dir)
    cal_full = load_calendar(data_dir)
    n, D = values.shape
    T = D - HORIZON
    log(f"Series: {n} | days: {D} | holdout origin: {T} | validation origin: {T - HORIZON}")
    T_ = M5Tensors(meta, values, dcols, cal_full, prices)

    names = [c.strip() for c in args.configs.split(",") if c.strip()]
    for c in names:
        if c not in ALL_CONFIGS:
            raise SystemExit(f"Unknown config {c}. Choose from {list(ALL_CONFIGS)}")

    if args.quick:
        steps = 400
        for c in names:
            t1 = time.time()
            f, curve = train_and_predict(T_, T, ALL_CONFIGS[c], cfg, steps, dev, log, args.seed)
            dt = time.time() - t1
            log(f"{c}: {steps} steps in {dt:.0f} s | NaN count {int(np.isnan(f).sum())} | "
                f"estimated {dt / steps * args.steps / 60:.1f} min per model at {args.steps} steps "
                f"({'x2 with validation' if not args.no_val else 'holdout only'})")
        return

    ev_hold = WRMSSEEvaluator(meta, values, dcols, calendar, prices)
    ev_val = None if args.no_val else WRMSSEEvaluator(meta, values[:, :T], dcols[:T], calendar, prices)

    os.makedirs(args.out_dir, exist_ok=True)
    results, ladder, curves = {}, [], {}
    for c in names:
```

```
        t1 = time.time()
        use_cov = ALL_CONFIGS[c]
        log(f"\n{c}: holdout model (trained to d_{T})")
        f, curve = train_and_predict(T_, T, use_cov, cfg, args.steps, dev, log, args.seed)
        s = ev_hold.score(f)
        np.save(os.path.join(args.out_dir, f"pred_patchtst_{c}.npy"), f)
        curves[c] = curve
        vs = None
        if ev_val is not None:
            log(f"{c}: validation model (trained to d_{T - HORIZON})")
            fv, _ = train_and_predict(T_, T - HORIZON, use_cov, cfg, args.steps, dev, log, args.seed)
            vs = ev_val.score(fv)
            np.save(os.path.join(args.out_dir, f"predval_patchtst_{c}.npy"), fv)
        mins = (time.time() - t1) / 60
        results[c] = s
        ladder.append((c, s[0], None if vs is None else vs[0], mins))
        log(f"{c}: holdout WRMSSE {s[0]:.4f} | validation {'n/a' if vs is None else f'{vs[0]:.4f}'} | {mins:.1f} min")

    table = {"SeasonalNaive": ev_hold.score(seasonal_naive_forecast(ev_hold.X_train))}
    table.update(results)
    lines = [f"{'Config':<20}{'Holdout':>10}{'Validation':>12}{'Min':>7}"]
    for c, h, v, m in ladder:
        lines.append(f"{c:<20}{h:>10.4f}{('n/a' if v is None else f'{v:.4f}'):>12}{m:>7.1f}")
    curve_lines = []
    for c, cv in curves.items():
        curve_lines.append(f"  {c}: " + ", ".join(f"{s}:{l:.4f}" for s, l in cv))

    report = [
        "PATCHTST ON M5 (global model trained on M5)",
        f"Data folder: {data_dir}",
        f"Device: {dev_note} | steps per model: {args.steps} | seed: {args.seed}",
        "Settings (fixed in advance, not tuned on the holdout): " + ", ".join(f"{k}={v}" for k, v in cfg.items()),
        f"Training window: d_1 to d_{T} | Holdout (28 days): d_{T + 1} to d_{D}",
        "Validation models are trained separately on data up to d_%d and forecast d_%d to d_%d." % (T - HORIZON, T - HORIZON + 1, T),
        "",
        "LADDER (WRMSSE, lower is better)",
        *lines,
        "",
        "TRAINING CURVES (step:smoothed loss, holdout model)",
        *curve_lines,
        "",
        "WRMSSE BY LEVEL (holdout)",
        format_table(ev_hold, table),
        "",
        "REFERENCE VALUES (published, window to be confirmed)",
        "  Original M5 winner: 0.520 | Yang et al. Top-1 reproduction: 0.5230 | Yang et al. LightGBM + PatchTST: 0.4989",
        "  Our E1b L3 (LightGBM, unweighted, all features): 0.5422",
        "",
        f"Runtime: {(time.time() - t0) / 60:.1f} min",
    ]
    text = "\n".join(report)
    path = os.path.join(args.out_dir, "results_patchtst.txt")
    with open(path, "w") as fh:
        fh.write(text + "\n")
    print("\n" + text)
    print(f"\nSaved: {path}")


if __name__ == "__main__":
    main()
```

src/m5_shap_nr.py

```
"""
Explainability of the winner's non-recursive per-store LightGBM models (set nr_store) with exact TreeSHAP (LightGBM's own pred_contrib).
READ ONLY on the winner's folder: nothing there is changed, no model is retrained, nothing is deleted. Results go to --out_dir.

What it does, for a few stores:
  1. Reuses the winner's own prepare_data() from its PREDICT script (executed up to the "Predict" section), so the features are exactly the ones
used for the forecast.
  2. Takes a random sample of rows from the forecast window, loads the saved model non_recur_model_<store>.bin and computes SHAP values.
     The objective is Tweedie (log link), so SHAP values are contributions to the log of the forecast, summed over features they give log(predic
tion).
  3. Writes mean |SHAP| per feature (global importance), feature families (lags, rolling means, price, calendar, encodings, ...),
     the sign of the effect (correlation of feature value and SHAP value), and the model's own gain importance for comparison.

usage: python m5_shap_nr.py --root C:\\A1_lean --stores CA_1,TX_1,WI_1 --n_rows 4000 --cv private --out_dir C:\\A1_shap
  --cv private = the original final window d_1942..d_1969 (default). Other keys exist in the winner's own table (public, cv4, ...).
"""
import argparse
import os
import pickle
import re
import sys
import time
import warnings

import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")
```

```
def find_predict_script(root):
    for dp, _, fs in os.walk(root):
        for f in fs:
            if f == "2-1. nonrecursive_store_PREDICT.py":
                return os.path.join(dp, f)
    raise SystemExit("2-1. nonrecursive_store_PREDICT.py not found under " + root)


def family(name):
    n = name.lower()
    if "lag" in n and "roll" not in n:
        return "lags"
    if "roll" in n:
        return "rolling means and stds"
    if "enc" in n or n.startswith("enc_"):
        return "target encodings"
    if "price" in n or n.startswith("pr_"):
        return "price"
    if n.startswith("tm_") or "snap" in n or "event" in n or "wday" in n or "month" in n or "year" in n:
        return "calendar, events, SNAP"
    if n.endswith("_id") or n in ("item_id", "dept_id", "cat_id", "state_id", "store_id"):
        return "item and group identity"
    return "other"


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--root", required=True, help="winner's folder that contains '3. code', '5. models' and '2. data'")
    ap.add_argument("--stores", default="CA_1,TX_1,WI_1")
    ap.add_argument("--n_rows", type=int, default=4000)
    ap.add_argument("--cv", default="private")
    ap.add_argument("--out_dir", required=True)
    args = ap.parse_args()
    args.out_dir = os.path.abspath(args.out_dir)
    os.makedirs(args.out_dir, exist_ok=True)

    script = find_predict_script(args.root)
    root = os.path.dirname(os.path.dirname(os.path.dirname(script)))
    os.environ["A1_DIR"] = root
    os.chdir(root)
    src = open(script, encoding="utf-8").read()
    cut = src.find("########################## Predict")
    if cut < 0:
        raise SystemExit("could not find the Predict section in the winner's script")
    ns = {"__name__": "winner_defs"}
    exec(compile(src[:cut], script, "exec"), ns)  # imports, parameters and prepare_data only; nothing is trained or predicted
    prepare_data, remove_feature, validation = ns["prepare_data"], ns["remove_feature"], ns["validation"]
    lo, hi = validation[args.cv]
    model_dir = ns["model_dir"]
    rng = np.random.default_rng(0)

    per_store, gain_store, sign_store, notes = {}, {}, {}, []
    for store in [s.strip() for s in args.stores.split(",") if s.strip()]:
        t0 = time.time()
        grid_df = prepare_data(store)
        model_var = list(grid_df.columns[~grid_df.columns.isin(remove_feature)])
        vl = (grid_df["d"] > lo) & (grid_df["d"] <= hi)
        X = grid_df.loc[vl, model_var]
        n = min(args.n_rows, len(X))
        X = X.iloc[np.sort(rng.choice(len(X), size=n, replace=False))]
        model = pickle.load(open(os.path.join(model_dir, f"non_recur_model_{store}.bin"), "rb"))
        names = model.feature_name()
        assert names == model_var or set(names) == set(model_var), "feature names of the saved model differ from the prepared data"
        X = X[names]
        contrib = model.predict(X, pred_contrib=True)  # (n, n_features + 1), last column is the base value
        sv = np.asarray(contrib)[:, :-1]
        base = float(np.asarray(contrib)[:, -1].mean())
        per_store[store] = pd.Series(np.abs(sv).mean(axis=0), index=names)
        gain_store[store] = pd.Series(model.feature_importance("gain"), index=names)
        Xv = X.apply(pd.to_numeric, errors="coerce").to_numpy(dtype=float)
        sg = {}
        for j, nm in enumerate(names):
            col = Xv[:, j]
            ok = np.isfinite(col)
            sg[nm] = float(np.corrcoef(col[ok], sv[ok, j])[0, 1]) if ok.sum() > 50 and col[ok].std() > 0 and sv[ok, j].std() > 0 else np.nan
        sign_store[store] = pd.Series(sg)
        notes.append(f"{store}: {n} rows sampled from d_{lo + 1}..d_{hi}, {len(names)} features, base value (log scale) {base:.3f}, mean total |
SHAP| {np.abs(sv).sum(axis=1).mean():.3f}, {(time.time() - t0) / 60:.1f} min")
        print(notes[-1], flush=True)
        del grid_df, X, contrib, sv, model

    df = pd.DataFrame(per_store)
    df["mean_abs_shap"] = df.mean(axis=1)
    df["share_percent"] = 100 * df["mean_abs_shap"] / df["mean_abs_shap"].sum()
    df["gain_share_percent"] = 100 * pd.DataFrame(gain_store).apply(lambda c: c / c.sum()).mean(axis=1)
    df["effect_sign_corr"] = pd.DataFrame(sign_store).mean(axis=1)
    df["family"] = [family(i) for i in df.index]
    df = df.sort_values("mean_abs_shap", ascending=False)
    df.to_csv(os.path.join(args.out_dir, "shap_nr_store_features.csv"))

    L = ["SHAP EXPLAINABILITY OF THE WINNER'S NON-RECURSIVE PER-STORE LIGHTGBM MODELS (set nr_store)",
```

```
        f"Window key '{args.cv}' (days d_{lo + 1}..d_{hi}). Exact TreeSHAP from LightGBM (pred_contrib). Values are on the log scale of the for
ecast (Tweedie, log link).",
        "Mean |SHAP| = the average size of a feature's push on the log forecast. Share = that feature's part of the total. 'Effect sign' = corr
elation of the feature value with its SHAP value "
        "(positive: higher value raises the forecast; this is a rough direction, not a causal effect).", ""] + notes + [""]
    L.append("FEATURE FAMILIES (share of total mean |SHAP|)")
    fam = df.groupby("family")["share_percent"].sum().sort_values(ascending=False)
    for k, v in fam.items():
        L.append(f"  {k:<28}{v:>7.1f}%")
    L += ["", "TOP 25 FEATURES (average over the stores)", f"  {'Feature':<34}{'Family':<26}{'Share %':>8}{'Gain %':>8}{'Effect sign':>13}"]
    for nm, r in df.head(25).iterrows():
        L.append(f"  {nm:<34}{r['family']:<26}{r['share_percent']:>8.1f}{r['gain_share_percent']:>8.1f}{r['effect_sign_corr']:>+13.2f}")
    L += ["", "TOP 10 PER STORE (mean |SHAP|)"]
    for store in per_store:
        top = df[store].sort_values(ascending=False).head(10)
        L.append(f"  {store}: " + ", ".join(f"{k} ({v:.3f})" for k, v in top.items()))
    L += ["", "Limits: only the nr_store set (one of six); a sample of rows; SHAP explains the model, not the world. Recursive sets are not cove
red."]
    text = "\n".join(L)
    open(os.path.join(args.out_dir, "results_shap.txt"), "w", encoding="utf-8").write(text + "\n")
    print("\n" + text)
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        top = df.head(20).iloc[::-1]
        plt.figure(figsize=(7, 6))
        plt.barh(top.index, top["mean_abs_shap"])
        plt.xlabel("mean |SHAP| (log scale of the forecast)")
        plt.title("nr_store: top 20 features")
        plt.tight_layout()
        plt.savefig(os.path.join(args.out_dir, "shap_nr_store_top20.png"), dpi=150)
    except Exception as e:  # noqa: BLE001
        print("plot skipped:", repr(e)[:100])


if __name__ == "__main__":
    main()
```

src/m5_tune.py

```
"""
Tuned blend shares, tested honestly. Uses forecasts that are already saved. No model is run.

Idea: WRMSSE is the plain mean of 12 level scores, and each level score depends only on that level's forecast.
So we score every window once at a grid of shares (0 to 0.60) for each level, store the table, and then every
tuning rule is just a lookup. Nothing is tuned on the window being scored:
  - for each of the five windows before the final one: the share is chosen on the other four of those five;
  - for the final window: the share is chosen on those five windows only, and the final is scored once.

Needs, per window: winner_<w>.npy, agg_chronos2_B_covariates_<w>.npy (in --in_dir),
                   lvl10_chronos2_B_<w>.npy, lvl11_chronos2_B_<w>.npy (in --lvl_dir).
Usage:
  python m5_tune.py --data_dir <M5 data, 1969 days> --in_dir inputs --lvl_dir lvl1011_out --out_dir tune_out
Output: tune_out/results_tune.txt  (plain text) and tune_out/level_table.npz (the cached table, reused on rerun).
"""
import argparse
import os
import sys

import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from m5_data_and_wrmsse import HORIZON, WRMSSEEvaluator, load_m5  # noqa: E402
from m5_multiwindow import point_score, sign_test_p, sq_errors  # noqa: E402

ORDER = ["cv5", "cv2", "cv3", "cv4", "public", "final"]
ORIGINS = {"cv5": 1801, "cv2": 1829, "cv3": 1857, "cv4": 1885, "public": 1913, "final": 1941}
GRID = np.round(np.arange(0.0, 0.601, 0.05), 2)
I20 = int(np.where(GRID == 0.20)[0][0])
GROUPS = [("L1-3", [0, 1, 2]), ("L4-6", [3, 4, 5]), ("L7-9", [6, 7, 8]), ("L10-11", [9, 10])]


def build_table(args):
    meta, values, dcols, calendar, prices = load_m5(args.data_dir)
    dcols = list(dcols)
    tab, have = {}, []
    for w in ORDER:
        o = ORIGINS[w]
        paths = [os.path.join(args.in_dir, f"winner_{w}.npy"), os.path.join(args.in_dir, f"agg_chronos2_B_covariates_{w}.npy"),
                 os.path.join(args.lvl_dir, f"lvl10_chronos2_B_{w}.npy"), os.path.join(args.lvl_dir, f"lvl11_chronos2_B_{w}.npy")]
        if values.shape[1] < o + HORIZON or not all(os.path.exists(p) for p in paths):
            print(f"window {w}: skipped (missing files or days)", flush=True)
            continue
        win, d19, d10, d11 = [np.load(p) for p in paths]
        ev = WRMSSEEvaluator(meta, values[:, : o + HORIZON], dcols[: o + HORIZON], calendar, prices)
        off = np.cumsum([0] + ev.sizes[:9])
        T = np.zeros((len(GRID), 12))
        for k, s in enumerate(GRID):
            lv = []
            for i, A in enumerate(ev.A):
```

```
                P = np.asarray(A @ win)
                if i < 9:
                    P = (1 - s) * P + s * d19[off[i]:off[i + 1]]
                elif i == 9:
                    P = (1 - s) * P + s * d10
                elif i == 10:
                    P = (1 - s) * P + s * d11
                lv.append(P)
            T[k] = point_score(ev, sq_errors(ev, lv))[1]
            print(f"  {w} share {s:.2f}: WRMSSE (levels 1-11 at this share) {T[k].mean():.4f}", flush=True)
        tab[w] = T
        have.append(w)
    np.savez(os.path.join(args.out_dir, "level_table.npz"), windows=np.array(have), **{w: tab[w] for w in have})
    return tab


def load_table(args):
    p = os.path.join(args.out_dir, "level_table.npz")
    if os.path.exists(p) and not args.rebuild:
        z = np.load(p)
        print("Using cached", p, flush=True)
        return {w: z[w] for w in z["windows"]}
    return build_table(args)


def cost(T, shares):
    """WRMSSE of a window given a share index per level (12 entries; level 12 always index 0)."""
    return float(np.mean([T[shares[l], l] for l in range(12)]))


def fixed(idx_l1_9, idx_l10_11=0):
    return [idx_l1_9] * 9 + [idx_l10_11] * 2 + [0]


def pick_global(tr, tab, with_item):
    best = min(range(len(GRID)), key=lambda k: np.mean([cost(tab[w], fixed(k, k if with_item else 0)) for w in tr]))
    return fixed(best, best if with_item else 0), f"{GRID[best]:.2f}"


def pick_perlevel(tr, tab):
    sh = []
    for l in range(11):
        sh.append(int(np.argmin(np.mean([tab[w][:, l] for w in tr], axis=0))))
    return sh + [0], " ".join(f"{GRID[k]:.2f}" for k in sh)


def pick_group(tr, tab):
    sh = [0] * 12
    for _, ls in GROUPS:
        k = int(np.argmin(np.mean([[tab[w][j, ls].sum() for j in range(len(GRID))] for w in tr], axis=0)))
        for l in ls:
            sh[l] = k
    return sh, " ".join(f"{n}={GRID[sh[ls[0]]]:.2f}" for n, ls in GROUPS)


def pick_shrunk(tr, tab):
    sh, _ = pick_perlevel(tr, tab)
    out = [int(round((sh[l] + I20) / 2)) if l < 11 else 0 for l in range(12)]
    return out, " ".join(f"{GRID[k]:.2f}" for k in out[:11])


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default="m5_data")
    ap.add_argument("--in_dir", default="inputs")
    ap.add_argument("--lvl_dir", default="lvl1011_out")
    ap.add_argument("--out_dir", default="tune_out")
    ap.add_argument("--rebuild", action="store_true")
    args = ap.parse_args()
    os.makedirs(args.out_dir, exist_ok=True)
    tab = load_table(args)
    wins = [w for w in ORDER if w in tab]
    pre = [w for w in wins if w != "final"]
    if len(pre) < 3:
        print("Not enough windows.")
        return
    rules = [("GLOBAL share, levels 1-9", lambda tr: pick_global(tr, tab, False)),
             ("GLOBAL share, levels 1-11", lambda tr: pick_global(tr, tab, True)),
             ("PER-LEVEL share, levels 1-11", lambda tr: pick_perlevel(tr, tab)),
             ("PER-GROUP share (L1-3, L4-6, L7-9, L10-11)", lambda tr: pick_group(tr, tab)),
             ("PER-LEVEL shrunk halfway to 0.20", lambda tr: pick_shrunk(tr, tab))]
    fixed_rules = [("winner alone", fixed(0)), ("HEADLINE 0.20, levels 1-9", fixed(I20)),
                   ("EXTENSION 0.20, levels 1-11", fixed(I20, I20))]
    res, chosen = {}, {}
    for n, sh in fixed_rules:
        res[n] = {w: cost(tab[w], sh) for w in wins}
    for n, f in rules:
        res[n] = {}
        for w in wins:
            tr = [x for x in pre if x != w]
            sh, txt = f(tr)
            res[n][w] = cost(tab[w], sh)
            if w == "final" or (w == pre[-1] and "final" not in wins):
```

```
                chosen[n] = txt
    orc = {}
    for w in wins:
        sh, _ = pick_perlevel([w], tab)
        orc[w] = cost(tab[w], sh)
    L = ["TUNED BLEND SHARES, HONEST TEST (no model run, saved forecasts only)",
         "WRMSSE, lower is better. Each window is scored with shares chosen WITHOUT looking at that window:",
         "windows before the final one use the other four of those five; the final window uses those five.",
         f"Share grid: {GRID[0]:.2f} to {GRID[-1]:.2f} step 0.05. Windows with data: {', '.join(wins)}.", ""]
    L.append(f"{'Rule':<46}" + "".join(f"{w:>9}" for w in wins) + f"{'mean5':>9}")
    for n in res:
        L.append(f"{n:<46}" + "".join(f"{res[n][w]:>9.4f}" for w in wins) + f"{np.mean([res[n][w] for w in pre]):>9.4f}")
    L.append(f"{'(reference only) per-level best ON the window':<46}" + "".join(f"{orc[w]:>9.4f}" for w in wins)
             + f"{np.mean([orc[w] for w in pre]):>9.4f}   <- not honest, upper bound")
    L += ["", "CHANGE VS HEADLINE (negative is better)"]
    L.append(f"{'Rule':<46}" + "".join(f"{w:>9}" for w in wins) + f"{'mean5':>9}{'better':>8}{'p':>7}")
    H = res["HEADLINE 0.20, levels 1-9"]
    for n in res:
        if n.startswith("winner") or n.startswith("HEADLINE"):
            continue
        ch = {w: res[n][w] - H[w] for w in wins}
        k = sum(ch[w] < 0 for w in pre)
        L.append(f"{n:<46}" + "".join(f"{ch[w]:>+9.4f}" for w in wins) + f"{np.mean([ch[w] for w in pre]):>+9.4f}"
                 f"{k:>5}/{len(pre)}{sign_test_p(k, len(pre)):>7.3f}")
    L += ["", "SHARES CHOSEN FOR THE LAST WINDOW (trained on the windows before it)"]
    for n, t in chosen.items():
        L.append(f"  {n}: {t}")
    L += ["", "HOW TO READ: a rule is only worth using if it beats HEADLINE on the five held-out windows AND on the final window.",
          "If it wins on the five but not the final, treat it as unproven. Windows share history, so p values are rough.",
          "The 'reference only' row shows the most any per-level tuning could ever give; the gap to the honest rows is overfitting."]
    t = "\n".join(L)
    open(os.path.join(args.out_dir, "results_tune.txt"), "w").write(t + "\n")
    print(t)


if __name__ == "__main__":
    main()
```

src/recon_test.py

```
import os, sys, time
import numpy as np, scipy.sparse as sp
from scipy.sparse.linalg import cg, LinearOperator
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
os.environ.setdefault("M5_WINDOW", "public")
from m5_data_and_wrmsse import HORIZON, WRMSSEEvaluator, load_m5
data_dir = sys.argv[1]
ORIGIN = {"cv4": 1885, "public": 1913}
meta, values, dcols, calendar, prices = load_m5(data_dir)
dcols = list(dcols)
SH = 0.20
for win, o in ORIGIN.items():
    end = o + HORIZON
    ev = WRMSSEEvaluator(meta, values[:, :end], dcols[:end], calendar, prices)
    w = np.load(f"inputs/winner_{win}.npy").astype(np.float64)
    d = np.load(f"inputs/agg_chronos2_B_covariates_{win}.npy").astype(np.float64)
    off = np.cumsum([0] + ev.sizes[:9])
    S = sp.vstack([sp.csr_matrix(A) for A in ev.A]).tocsr()
    base = []
    for i, A in enumerate(ev.A):
        P = np.asarray(A @ w)
        if i < 9:
            P = (1 - SH) * P + SH * d[off[i]:off[i + 1]]
        base.append(P)
    Y = np.vstack(base)
    def lev_score(b):
        return ev.score(np.clip(b, 0, None))
    def levels_from_bottom(b):
        return ev.score(np.clip(b, 0, None))
    # reference: blended levels (incoherent) from the earlier script
    from itertools import chain
    def score_levels(levels):
        out = []
        for (P, sc, wt, Yt) in zip(levels, ev.scale, ev.weight, ev.y_agg):
            v = wt > 0
            r = np.zeros(len(sc)); r[v] = np.sqrt(np.mean((Yt[v] - P[v]) ** 2, axis=1) / sc[v])
            out.append(float(np.sum(wt * r)))
        return float(np.mean(out)), out
    print(f"\n{win}: winner alone {ev.score(w)[0]:.4f} | blend incoherent {score_levels(base)[0]:.4f}")
    size = np.asarray(S.sum(axis=1)).ravel()
    for name, wts in [("OLS", np.ones(S.shape[0])), ("structural", 1.0 / size)]:
        Wd = sp.diags(wts)
        ST = S.T.tocsr()
        M = LinearOperator((S.shape[1], S.shape[1]), matvec=lambda x, S=S, ST=ST, wts=wts: ST @ (wts * (S @ x)), dtype=np.float64)
        rhs = S.T @ (wts[:, None] * Y)
        t0 = time.time()
        b = np.zeros((S.shape[1], HORIZON))
        # warm start from the winner's bottom forecast
        for j in range(HORIZON):
            x, info = cg(M, rhs[:, j], x0=w[:, j], rtol=1e-8, maxiter=500)
            b[:, j] = x
```

```
        rec = ev.score(np.clip(b, 0, None))
        print(f"  {name:<11} reconciled {rec[0]:.4f} (change vs blend {rec[0] - score_levels(base)[0]:+.4f}, vs winner {rec[0] - ev.score(w)[0]:
+.4f}) [{time.time() - t0:.0f}s]")
        print("   levels:", " ".join(f"{x:.3f}" for x in rec[1]))
```

src/sweep_lowo.py

```
import os, sys, numpy as np
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
os.environ.setdefault("M5_WINDOW", "public")
from m5_data_and_wrmsse import HORIZON, WRMSSEEvaluator, load_m5
from m5_multiwindow import mix_levels, sq_errors, point_score, ORIGINS
meta, values, dcols, calendar, prices = load_m5(sys.argv[1]); dcols = list(dcols)
SH = [0.0, 0.05, 0.10, 0.15, 0.20, 0.30, 0.40, 0.50, 0.60]
tab = {}
for win, o in ORIGINS.items():
    if o + HORIZON > values.shape[1]: continue
    ev = WRMSSEEvaluator(meta, values[:, :o + HORIZON], dcols[:o + HORIZON], calendar, prices)
    w = np.load(f"inputs_all/winner_{win}.npy").astype(float); d = np.load(f"inputs_all/agg_chronos2_B_covariates_{win}.npy").astype(float)
    tab[win] = np.array([point_score(ev, sq_errors(ev, mix_levels(ev, w, d, s)))[0] for s in SH])
wins = list(tab)
print("Change vs winner alone (WRMSSE) by share"); print(f"{'window':<8}" + "".join(f"{s:>8.2f}" for s in SH[1:]) + "   best")
for k in wins:
    c = tab[k] - tab[k][0]; print(f"{k:<8}" + "".join(f"{x:>+8.4f}" for x in c[1:]) + f"   {SH[int(np.argmin(c))]:.2f}")
print("\nLeave-one-window-out: pick the share that minimises the mean change on the OTHER windows, apply it to the held-out one")
for k in wins:
    oth = [j for j in wins if j != k]
    m = np.mean([tab[j] - tab[j][0] for j in oth], axis=0); i = int(np.argmin(m))
    fixed = SH.index(0.20)
    print(f"held-out {k:<7} learned share {SH[i]:.2f}: change {tab[k][i] - tab[k][0]:+.4f} | fixed 0.20: change {tab[k][fixed] - tab[k][0]:+.4f}
")
m = np.mean([tab[j] - tab[j][0] for j in wins], axis=0)
print("\nPooled mean change over the", len(wins), "windows by share:", " ".join(f"{s:.2f}:{x:+.4f}" for s, x in zip(SH[1:], m[1:])))
```

src/winner_runner/gen_full.py

```
import sys, os, numpy as np, pandas as pd
# usage: gen_full.py OUTDIR ITEMS_PER_STORE LAYOUT(kaggle|organiser) [SEED]
out, ips, layout = sys.argv[1], int(sys.argv[2]), sys.argv[3]
seed = int(sys.argv[4]) if len(sys.argv) > 4 else 0
rng = np.random.default_rng(seed)
os.makedirs(out, exist_ok=True)
D, DC = 1941, 1969
stores = {"CA_1":"CA","CA_2":"CA","CA_3":"CA","CA_4":"CA","TX_1":"TX","TX_2":"TX","TX_3":"TX","WI_1":"WI","WI_2":"WI","WI_3":"WI"}
depts = {"HOBBIES_1":"HOBBIES","HOBBIES_2":"HOBBIES","FOODS_1":"FOODS","FOODS_2":"FOODS","FOODS_3":"FOODS","HOUSEHOLD_1":"HOUSEHOLD","HOUSEHOLD_
2":"HOUSEHOLD"}
dept_list = list(depts)
rows = []
for s, st in stores.items():
    for j in range(ips):
        dept = dept_list[j % len(dept_list)]
        rows.append(dict(item_id=f"{dept}_{j:03d}", dept_id=dept, cat_id=depts[dept], store_id=s, state_id=st))
meta = pd.DataFrame(rows)
lam = rng.uniform(0, 3, len(meta)) * (rng.uniform(0, 1, len(meta)) > 0.3)
sales = rng.poisson(lam[:, None], size=(len(meta), D)).astype(np.int64)
sales_df = pd.concat([meta, pd.DataFrame(sales, columns=[f"d_{k+1}" for k in range(D)])], axis=1)
if layout == "kaggle":
    sales_df = pd.concat([pd.DataFrame({"id": meta["item_id"] + "_" + meta["store_id"] + "_evaluation"}), sales_df], axis=1)
sales_df.to_csv(os.path.join(out, "sales_train_evaluation.csv"), index=False)
# calendar
dates = pd.date_range("2011-01-29", periods=DC)
cal = pd.DataFrame({"date": dates.strftime("%Y-%m-%d"),
                    "wm_yr_wk": 11101 + np.arange(DC) // 7,
                    "weekday": dates.day_name(), "wday": dates.dayofweek + 1,
                    "month": dates.month, "year": dates.year})
if layout == "kaggle":
    cal["d"] = [f"d_{i+1}" for i in range(DC)]
cal["event_name_1"] = pd.Series(np.where(rng.uniform(0,1,DC) > 0.97, "Holiday", None), dtype=object)
cal["event_type_1"] = pd.Series(np.where(cal["event_name_1"].notna(), "National", None), dtype=object)
cal["event_name_2"] = np.nan; cal["event_type_2"] = np.nan
cal["snap_CA"] = rng.integers(0,2,DC); cal["snap_TX"] = rng.integers(0,2,DC); cal["snap_WI"] = rng.integers(0,2,DC)
cal = cal[[c for c in ["date","wm_yr_wk","weekday","wday","month","year","d","event_name_1","event_type_1","event_name_2","event_type_2","snap_C
A","snap_TX","snap_WI"] if c in cal.columns]]
cal.to_csv(os.path.join(out, "calendar.csv"), index=False)
# prices: weekly, some series start later (release)
weeks = np.unique(cal["wm_yr_wk"])
pr = []
for i, r in meta.iterrows():
    start = rng.integers(0, len(weeks)//2) if rng.uniform() < 0.3 else 0
    base = rng.uniform(1, 20)
    p = np.round(base + rng.normal(0, 0.5, len(weeks)).cumsum() * 0.05, 2).clip(0.5)
    for w in range(start, len(weeks)):
        pr.append((r["store_id"], r["item_id"], weeks[w], p[w]))
pd.DataFrame(pr, columns=["store_id","item_id","wm_yr_wk","sell_price"]).to_csv(os.path.join(out, "sell_prices.csv"), index=False)
# sample submission (Kaggle layout)
ids = list(meta["item_id"] + "_" + meta["store_id"])
ss = pd.DataFrame({"id": [i + "_validation" for i in ids] + [i + "_evaluation" for i in ids]})
for k in range(1, 29): ss[f"F{k}"] = 0
ss.to_csv(os.path.join(out, "sample_submission.csv"), index=False)
```

```
print("generated", out, layout, len(meta), "series")
```

src/winner_runner/make_extra_pkgs.py

```
"""Builds the winner's package (M5 1st place, folder A1) for extra earlier windows from the finished cv4 package.
Only the window selector changes (which days are train and which are forecast). No model setting, no training step.
Windows (28 forecast days each):  cv3 = train to d_1857, forecast d_1858..d_1885   (winner's own 'cv3')
                                  cv2 = train to d_1829, forecast d_1830..d_1857   (winner's own 'cv2')
                                  cv5 = train to d_1801, forecast d_1802..d_1829   (added by us, same rule)
usage: python make_extra_pkgs.py BASE_CV4_DIR OUT_DIR
"""
import os, re, shutil, sys
base, out = sys.argv[1], sys.argv[2]
WIN = {"cv3": 3, "cv2": 4, "cv5": 5}  # KKK: origin = 1941 - 28*KKK
for W, K in WIN.items():
    origin = 1941 - 28 * K
    end = origin + 28
    dst = os.path.join(out, f"pkg_{W}")
    if os.path.exists(dst):
        shutil.rmtree(dst)
    shutil.copytree(base, dst, ignore=shutil.ignore_patterns("2. data", "4. logs", "5. models", "6. submissions", "run_logs", "m5_outputs_A1", "
drv.txt", "__pycache__"))
    log = []
    def sub(path, pat, rep, exp=None, regex=True):
        t = open(path, encoding="utf-8").read()
        t2, n = (re.subn(pat, rep, t, flags=re.M) if regex else (t.replace(pat, rep), t.count(pat)))
        if exp is not None and n != exp:
            raise SystemExit(f"{path}: {pat!r} expected {exp} got {n}")
        open(path, "w", encoding="utf-8").write(t2)
        log.append(f"{os.path.relpath(path, dst)}: {pat[:40]!r} x{n}")
    code = os.path.join(dst, "3. code")
    for sd in os.listdir(code):
        for f in os.listdir(os.path.join(code, sd)):
            p = os.path.join(code, sd, f)
            if not f.endswith(".py"):
                continue
            if "nonrecursive" in f and ("TRAIN" in f or "PREDICT" in f):
                sub(p, r"^cvs = \['cv4'\].*$", f"cvs = ['{W}']  # train to d_{origin}, forecast d_{origin + 1}..d_{end}", 1)
                if W == "cv5":
                    sub(p, r"^(\s*)'cv4' : \[1885,1913\],", r"\1'cv4' : [1885,1913],\n\1'cv5' : [1801,1829],", 1)
            if "recursive_" in f and "nonrecursive" not in f and ("TRAIN" in f or "PREDICT" in f):
                sub(p, r"^ver, KKK = 'cv4', 2.*$", f"ver, KKK = '{W}', {K}  # END_TRAIN = 1941 - 28*{K} = {origin}", 1)
    sub(os.path.join(code, "1. preprocessing", "1. preprocessing.py"), r"^grid_df\.loc\[grid_df\['d'\]>\(1941-56\), 'sales'\] = np\.nan.*$",
        f"grid_df.loc[grid_df['d']>(1941-{28 * K}), 'sales'] = np.nan  # {W}: target encodings may only use days up to d_{origin}", 1)
    r = os.path.join(dst, "run_winner_a1.py")
    sub(r, "values[:, :1913]", f"values[:, :{end}]", regex=False)
    sub(r, "list(dcols)[:1913]", f"list(dcols)[:{end}]", regex=False)
    sub(r, "d_1886..d_1913, training history up to d_1885", f"d_{origin + 1}..d_{end}, training history up to d_{origin}", regex=False)
    sub(r, "fold ending d_1913", f"fold ending d_{end}", regex=False)
    sub(r, "up to d_1885, forecast d_1886 to d_1913", f"up to d_{origin}, forecast d_{origin + 1} to d_{end}", regex=False)
    sub(r, "cv4 = d_1886..d_1913 (package built in cv4 mode)", f"{W} = d_{origin + 1}..d_{end} (package built in {W} mode)", regex=False)
    sub(r, "target encodings rebuilt on days up to d_1885", f"target encodings rebuilt on days up to d_{origin}", regex=False)
    sub(r, "cv4", W, regex=False)
    t = open(r, encoding="utf-8").read()
    assert "1886" not in t and "1885" not in t.replace(f"{end}", "") or W == "cv3", "stray cv4 numbers left"
    assert "d_1886" not in t and "[:1913]" not in t
    open(os.path.join(dst, "README_" + W.upper() + ".txt"), "w").write(
        f"Winner's LightGBM (M5 1st place, A1), earlier fold {W}: train to d_{origin}, forecast d_{origin + 1}..d_{end}.\n"
        f"Built from the cv4 package; only the window selector changes. Target encodings use days up to d_{origin}.\n"
        f"Run: python run_winner_a1.py --data_dir <folder with calendar, sell_prices, sales_train_evaluation> --window {W}\n"
        f"Outputs (m5_outputs_A1): pred{W}_A1_<set>.npy, pred{W}_A1_winner.npy, results_winner_A1_{W}.txt\n")
    open(os.path.join(dst, "PATCH_LOG_" + W.upper() + ".txt"), "w").write("\n".join(log) + "\n")
    print(W, "built:", dst)
```

src/winner_runner/run_extra_windows.py

```
"""
Runs the winner's six LightGBM sets (M5 1st place, folder A1) on the extra earlier windows, one window after another.
Windows (28 forecast days each): cv3 train to d_1857, forecast d_1858..d_1885 | cv2 d_1830..d_1857 | cv5 d_1802..d_1829.
Nothing is deleted or changed. Finished sets are skipped on a rerun (run_winner_a1.py remembers them).
Before every window it checks free disk space and stops (without touching anything) if it is below --min_free_gb.
At the end (or after every finished window) it copies the small result files into <root>/send and zips them.

usage: python run_extra_windows.py --data_dir <folder with calendar.csv, sell_prices.csv, sales_train_evaluation.csv>
       [--root <this folder>] [--windows cv3,cv2,cv5] [--min_free_gb 55]
"""
import argparse
import os
import shutil
import subprocess
import sys
import time
import zipfile

SETS = ["nr_store", "nr_store_cat", "nr_store_dept", "r_store", "r_store_cat", "r_store_dept"]


def main():
    ap = argparse.ArgumentParser()
```

```
    ap.add_argument("--data_dir", required=True)
    ap.add_argument("--root", default=os.path.dirname(os.path.abspath(__file__)))
    ap.add_argument("--windows", default="cv3,cv2,cv5")
    ap.add_argument("--min_free_gb", type=float, default=55.0)
    args = ap.parse_args()
    root = os.path.abspath(args.root)
    send = os.path.join(root, "send")
    os.makedirs(send, exist_ok=True)
    status = os.path.join(root, "EXTRA_STATUS.txt")

    def log(m):
        line = f"[{time.strftime('%Y-%m-%d %H:%M:%S')}] {m}"
        print(line, flush=True)
        with open(status, "a", encoding="utf-8") as f:
            f.write(line + "\n")

    def make_zip():
        zp = os.path.join(root, "send_pc3_windows.zip")
        with zipfile.ZipFile(zp, "w", zipfile.ZIP_DEFLATED) as z:
            for fn in sorted(os.listdir(send)):
                z.write(os.path.join(send, fn), fn)
            if os.path.exists(status):
                z.write(status, "EXTRA_STATUS.txt")
        log(f"zip written: {zp}")

    for W in [w.strip() for w in args.windows.split(",") if w.strip()]:
        pkg = os.path.join(root, f"pkg_{W}")
        out = os.path.join(pkg, "m5_outputs_A1")
        done = all(os.path.exists(os.path.join(out, f"pred{W}_A1_{s}.npy")) for s in SETS) and \
            os.path.exists(os.path.join(out, f"pred{W}_A1_winner.npy"))
        if not done:
            free = shutil.disk_usage(root).free / 1e9
            log(f"{W}: free disk {free:.0f} GB (need {args.min_free_gb:.0f})")
            if free < args.min_free_gb:
                log(f"STOP before {W}: not enough free disk. Nothing was deleted. Free space or move old model folders yourself, then rerun the
same command.")
                make_zip()
                sys.exit(2)
            t0 = time.time()
            log(f"{W}: start (six sets, about 11 hours)")
            with open(os.path.join(root, f"driver_{W}.txt"), "a", encoding="utf-8", errors="replace") as lf:
                rc = subprocess.run([sys.executable, "-u", "run_winner_a1.py", "--data_dir", args.data_dir, "--window", W],
                                    cwd=pkg, stdout=lf, stderr=subprocess.STDOUT).returncode
            log(f"{W}: finished with exit code {rc} after {(time.time() - t0) / 3600:.1f} h")
            if rc != 0:
                log(f"STOP: {W} failed. See driver_{W}.txt and {pkg}\\run_logs. Do not delete anything; send me the last 40 lines of driver_{W}.
txt.")
                make_zip()
                sys.exit(1)
        else:
            log(f"{W}: already finished, skipping the run")
        missing = [s for s in SETS if not os.path.exists(os.path.join(out, f"pred{W}_A1_{s}.npy"))]
        if missing:
            log(f"STOP: {W} has no forecast file for {missing}")
            make_zip()
            sys.exit(1)
        shutil.copy2(os.path.join(out, f"pred{W}_A1_winner.npy"), os.path.join(send, f"winner_{W}.npy"))
        rp = os.path.join(out, f"results_winner_A1_{W}.txt")
        if os.path.exists(rp):
            shutil.copy2(rp, os.path.join(send, f"results_winner_A1_{W}.txt"))
        log(f"{W}: result files copied to {send}")
        make_zip()
    log("ALL WINDOWS DONE")


if __name__ == "__main__":
    main()
```

src/winner_runner/run_winner_a1_cv2.py

```
#!/usr/bin/env python3
"""
Run the 1st place M5 Accuracy solution (YeonJun In, M5-methods folder A1) on the competition's own final window, then score it with our evaluato
r.

What this does
  The winner's own scripts are run in order (preprocessing, then for each of six model sets: train and predict),
  with only small compatibility fixes (this package keeps the winner's own final window: train to d_1941,
  forecast d_1942 to d_1969, scored on the released actuals in sales_test_evaluation.csv):
    - small compatibility fixes for current pandas and LightGBM (see PATCH_LOG.txt); no model setting is touched
  The six sets are: non-recursive and recursive models trained per store (10 models each), per store x category
  (30) and per store x department (70). The winner averaged all six with equal weights.
  Afterwards every finished set and the equal-weight average of the finished sets is scored with the same WRMSSE
  evaluator as the rest of the project.

Time and memory (honest expectations)
  Needs about 16 GB RAM free and about 15 GB disk. Preprocessing takes tens of minutes. Training uses 3000 trees per
  model on CPU. The non-recursive sets are the cheaper ones; the recursive sets use much larger trees and are the
  slowest. The full job of 220 models can take days on an ordinary CPU, so use --sets to run it in stages.
  Finished sets are remembered; a rerun skips them.
```

```
Usage
  python run_winner_a1.py --sets nr_store           # cheapest start: non-recursive, one model per store
  python run_winner_a1.py --sets nr_store,nr_store_cat,nr_store_dept
  python run_winner_a1.py                           # all six sets, in order of cost
  python run_winner_a1.py --score_only              # rescore whatever has finished
  Default window is the original final one (d_1942 to d_1969); this needs sales_test_evaluation.csv next to the other
  data files. For d_1914 to d_1941 build the package in public mode and add --window public.

Needs the three M5 files calendar.csv, sell_prices.csv, sales_train_evaluation.csv and sample_submission.csv
(found automatically under --data_dir). Packages: pandas, numpy, scipy, lightgbm, psutil, scikit-learn, tqdm.

Outputs (in --out_dir, default m5_outputs_A1)
  pred_A1_<set>.npy        forecast of one set, shape (30490, 28), same row order as sales_train_evaluation.csv
  pred_A1_winner.npy       equal-weight average of the finished sets
  results_winner_A1.txt    plain-text log with scores, bias and the 12-level table
"""
import argparse
import importlib
import os
import shutil
import subprocess
import sys
import time

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)

SETS = {
    "nr_store": ("2-1. nonrecursive_store", "submission_kaggle_nonrecursive_store.csv"),
    "nr_store_cat": ("2-2. nonrecursive_store_cat", "submission_kaggle_nonrecursive_store_cat.csv"),
    "nr_store_dept": ("2-3. nonrecursive_store_dept", "submission_kaggle_nonrecursive_store_dept.csv"),
    "r_store": ("1-1. recursive_store", "submission_kaggle_recursive_store.csv"),
    "r_store_cat": ("1-2. recursive_store_cat", "submission_kaggle_recursive_store_cat.csv"),
    "r_store_dept": ("1-3. recursive_store_dept", "submission_kaggle_recursive_store_dept.csv"),
}
RAW_FILES = ["sales_train_evaluation.csv", "sell_prices.csv", "calendar.csv", "sample_submission.csv"]


def find_file(root, name):
    for dirpath, _, files in os.walk(root):
        if name in files:
            return os.path.join(dirpath, name)
    return None


def run_script(path, cwd, env, log_path):
    t0 = time.time()
    with open(log_path, "w", encoding="utf-8", errors="replace") as lf:
        rc = subprocess.run([sys.executable, "-u", path], cwd=cwd, env=env, stdout=lf, stderr=subprocess.STDOUT).returncode
    mins = (time.time() - t0) / 60
    return rc, mins


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default=None, help="folder holding the M5 csv files (searched recursively)")
    ap.add_argument("--work_dir", default=HERE, help="where the winner's folder structure is created")
    ap.add_argument("--out_dir", default=None)
    ap.add_argument("--sets", default="all", help="comma list of: " + ",".join(SETS))
    ap.add_argument("--score_only", action="store_true")
    ap.add_argument("--window", default="cv2", choices=["final", "public", "cv2"],
                    help="final = the winner's own final window d_1942..d_1969 (needs sales_test_evaluation.csv to score); "
                         "public = d_1914..d_1941 (package built in public mode); cv2 = d_1830..d_1857 (package built in cv2 mode)")
    args = ap.parse_args()

    work = os.path.abspath(args.work_dir)
    out_dir = args.out_dir or os.path.join(work, "m5_outputs_A1")
    os.makedirs(out_dir, exist_ok=True)
    # file names: final window -> pred_A1_*.npy / results_winner_A1.txt ; public window (validation forecasts) -> predval_A1_*.npy / results_win
ner_A1_public.txt
    PREFIX = {"final": "pred", "public": "predval", "cv2": "predcv2"}[args.window]
    RESULTS_NAME = {"final": "results_winner_A1.txt", "public": "results_winner_A1_public.txt", "cv2": "results_winner_A1_cv2.txt"}[args.window]
    chosen = list(SETS) if args.sets == "all" else [s.strip() for s in args.sets.split(",") if s.strip()]
    for s in chosen:
        if s not in SETS:
            raise SystemExit(f"Unknown set {s}. Choose from {list(SETS)}")

    # ---- packages
    missing = []
    for mod in ["pandas", "numpy", "scipy", "lightgbm", "psutil", "sklearn", "tqdm"]:
        try:
            importlib.import_module(mod)
        except ImportError:
            missing.append({"sklearn": "scikit-learn"}.get(mod, mod))
    if missing:
        raise SystemExit("Missing packages: " + ", ".join(missing) + "\nInstall with: pip install " + " ".join(missing))

    # ---- data
    data_dir = args.data_dir
    if data_dir is None:
        for cand in [os.path.join(HERE, "m5_data"), os.path.join(HERE, "..", "m5_data"), os.path.join(os.getcwd(), "m5_data")]:
            if os.path.isdir(cand):
                data_dir = cand
```

```
            break
if data_dir is None:
    raise SystemExit("Could not find the M5 data. Pass --data_dir <folder with the M5 csv files>.")
raw = os.path.join(work, "2. data")
for d in ["2. data/processed", "4. logs", "5. models", "6. submissions/before_ensemble", "run_logs"]:
    os.makedirs(os.path.join(work, d), exist_ok=True)
for f in RAW_FILES:
    dst = os.path.join(raw, f)
    if not os.path.exists(dst):
        src = find_file(data_dir, f)
        if src is None and f == "sample_submission.csv":
            # not in some copies of the data: rebuild it (ids of the 30,490 series for validation, then evaluation; F1..F28 = 0)
            import pandas as pd
            ids = pd.read_csv(os.path.join(raw, "sales_train_evaluation.csv"), usecols=["id"])["id"]
            ids = ids.str.replace("_evaluation", "_validation")
            ss = pd.DataFrame({"id": list(ids) + [i.replace("_validation", "_evaluation") for i in ids]})
            for k in range(1, 29):
                ss[f"F{k}"] = 0
            ss.to_csv(dst, index=False)
            print("sample_submission.csv not found, rebuilt it from the sales ids", flush=True)
            continue
        if src is None:
            raise SystemExit(f"{f} not found under {data_dir}")
        print(f"Copying {f} ...", flush=True)
        shutil.copy(src, dst)

env = dict(os.environ, A1_DIR=work)
code = os.path.join(work, "3. code")
t_start = time.time()
status = []

if not args.score_only:
    marker = os.path.join(work, "run_logs", "done_preprocessing")
    if not os.path.exists(marker):
        print("Preprocessing (tens of minutes, needs lots of RAM) ...", flush=True)
        rc, mins = run_script(os.path.join(code, "1. preprocessing", "1. preprocessing.py"), work, env,
                              os.path.join(work, "run_logs", "preprocessing.log"))
        if rc != 0:
            raise SystemExit(f"Preprocessing failed (exit {rc}). See run_logs/preprocessing.log")
        open(marker, "w").write("ok\n")
        status.append(f"preprocessing: {mins:.1f} min")
    else:
        status.append("preprocessing: already done")

    for s in chosen:
        stem, csv = SETS[s]
        csv_path = os.path.join(work, "6. submissions", "before_ensemble", csv)
        if os.path.exists(csv_path):
            status.append(f"{s}: already done")
            print(f"{s}: already done, skipping", flush=True)
            continue
        for kind in ("train", "predict"):
            sub = "2. train" if kind == "train" else "3. predict"
            fname = f"{stem}_{kind.upper()}.py"
            print(f"{s}: {kind} ({fname}) ...", flush=True)
            rc, mins = run_script(os.path.join(code, sub, fname), work, env,
                                  os.path.join(work, "run_logs", f"{s}_{kind}.log"))
            status.append(f"{s} {kind}: {mins:.1f} min (exit {rc})")
            print(f"  {s} {kind} finished in {mins:.1f} min (exit {rc})", flush=True)
            if rc != 0:
                raise SystemExit(f"{s} {kind} failed. See run_logs/{s}_{kind}.log")
        if not os.path.exists(csv_path):
            raise SystemExit(f"{s}: expected output not found: {csv_path}")

# ---- scoring
import numpy as np
import pandas as pd
from m5_data_and_wrmsse import WRMSSEEvaluator, format_table, load_m5, seasonal_naive_forecast

meta, values, dcols, calendar, prices = load_m5(raw)
if args.window == "final":
    tpath = None
    for base in [data_dir, work, HERE]:
        try:
            tpath = find_file(base, "sales_test_evaluation.csv")
        except Exception:
            tpath = None
        if tpath:
            break
    if tpath is None:
        raise SystemExit("Final window needs sales_test_evaluation.csv (actual sales d_1942..d_1969) under --data_dir.")
    tdf = pd.read_csv(tpath)
    tcols = sorted([c for c in tdf.columns if c.startswith("d_")], key=lambda c: int(c[2:]))
    assert len(tcols) == 28, f"expected 28 test days, found {len(tcols)}"
    key = meta["item_id"] + "|" + meta["store_id"]
    tdf = tdf.set_index(tdf["item_id"] + "|" + tdf["store_id"])
    tv = tdf.reindex(key)[tcols].to_numpy(dtype=np.float64)
    assert not np.isnan(tv).any(), "test file does not cover every series"
    values = np.concatenate([values, tv], axis=1)
    dcols = list(dcols) + tcols
if args.window == "cv2":
    # fold ending d_1857: holdout d_1830..d_1857, training history up to d_1829 (the M5 protocol on that fold)
    values = values[:, :1857]
```

```
        dcols = list(dcols)[:1857]
    ev = WRMSSEEvaluator(meta, values, dcols, calendar, prices)
    total_actual = ev.y_test.sum()
    preds, res = {}, {}
    for s in SETS:
        csv_path = os.path.join(work, "6. submissions", "before_ensemble", SETS[s][1])
        if not os.path.exists(csv_path):
            continue
        sub = pd.read_csv(csv_path).drop_duplicates("id").set_index("id")
        fcols = [f"F{i}" for i in range(1, 29)]
        arr = sub.reindex(meta["id"])[fcols].fillna(0).to_numpy(dtype=np.float64)
        arr = np.clip(arr, 0, None)
        preds[s] = arr
        res[s] = ev.score(arr)
        np.save(os.path.join(out_dir, f"{PREFIX}_A1_{s}.npy"), arr.astype(np.float32))
    if not preds:
        raise SystemExit("No finished set to score yet.")
    avg = np.mean(list(preds.values()), axis=0)
    res["average"] = ev.score(avg)
    np.save(os.path.join(out_dir, f"{PREFIX}_A1_winner.npy"), avg.astype(np.float32))

    lines = [f"{'Set':<16}{'WRMSSE':>9}{'Bias':>8}"]
    for s in list(preds) + ["average"]:
        a = avg if s == "average" else preds[s]
        lines.append(f"{s:<16}{res[s][0]:>9.4f}{a.sum() / total_actual:>8.3f}")
    pub = args.window == "public"
    table = {"SeasonalNaive": ev.score(seasonal_naive_forecast(ev.X_train))}
    for s in list(preds)[:3]:
        table[s] = res[s]
    table["average"] = res["average"]
    n_done = len(preds)
    report = [
        "WINNER'S CODE (M5 1st place, folder A1) RUN ON " + {"final": "THE ORIGINAL FINAL WINDOW", "public": "OUR PUBLIC WINDOW", "cv2": "AN EAR
LIER VALIDATION FOLD (cv2)"}[args.window],
        ("Trained on days up to d_1941, forecast d_1942 to d_1969, scored on the released actuals (the competition's final window)."
         if args.window == "final" else
         ("Trained on days up to d_1913, forecast d_1914 to d_1941 (the winner's own 'public' setting)."
          if args.window == "public" else
          "Trained on days up to d_1829, forecast d_1830 to d_1857 (the winner's own 'cv2' fold; target encodings rebuilt on days up to d_1829).
")),
        "Settings of the winner's models are unchanged. Compatibility patches are listed in PATCH_LOG.txt.",
        f"Sets finished: {n_done} of 6 ({', '.join(preds)}). The winner averaged all six with equal weights;",
        "the 'average' row is the equal-weight average of the sets finished so far.",
        "Bias = total forecast / total actual (1.000 is unbiased).",
        "",
        "RESULTS (WRMSSE on the holdout, lower is better)",
        *lines,
        "",
        "WRMSSE BY LEVEL (first three finished sets and the average)",
        format_table(ev, table),
        "",
        "REFERENCE VALUES (published, window to be confirmed)",
        "  Original M5 winner (final window d_1942 to d_1969): 0.520",
        "  Yang et al. Kaggle Top-1 reproduction: 0.5230 | Yang et al. LightGBM + PatchTST: 0.4989",
        "  Our numbers on d_1914 to d_1941 (a different window): E1b L3 0.5422 | E1d non-recursive per-store port 0.6959 raw, 0.5992 calibrated"
,
        "",
        "STEPS: " + " | ".join(status),
        f"Runtime of this invocation: {(time.time() - t_start) / 60:.1f} min",
    ]
    text = "\n".join(report)
    with open(os.path.join(out_dir, RESULTS_NAME), "w") as f:
        f.write(text + "\n")
    print("\n" + text)
    print(f"\nSaved: {os.path.join(out_dir, RESULTS_NAME)}")


if __name__ == "__main__":
    main()
```

src/winner_runner/run_winner_a1_cv3.py

```
#!/usr/bin/env python3
"""
Run the 1st place M5 Accuracy solution (YeonJun In, M5-methods folder A1) on the competition's own final window, then score it with our evaluato
r.

What this does
  The winner's own scripts are run in order (preprocessing, then for each of six model sets: train and predict),
  with only small compatibility fixes (this package keeps the winner's own final window: train to d_1941,
  forecast d_1942 to d_1969, scored on the released actuals in sales_test_evaluation.csv):
    - small compatibility fixes for current pandas and LightGBM (see PATCH_LOG.txt); no model setting is touched
  The six sets are: non-recursive and recursive models trained per store (10 models each), per store x category
  (30) and per store x department (70). The winner averaged all six with equal weights.
  Afterwards every finished set and the equal-weight average of the finished sets is scored with the same WRMSSE
  evaluator as the rest of the project.

Time and memory (honest expectations)
  Needs about 16 GB RAM free and about 15 GB disk. Preprocessing takes tens of minutes. Training uses 3000 trees per
  model on CPU. The non-recursive sets are the cheaper ones; the recursive sets use much larger trees and are the
  slowest. The full job of 220 models can take days on an ordinary CPU, so use --sets to run it in stages.
```

```
  Finished sets are remembered; a rerun skips them.

Usage
  python run_winner_a1.py --sets nr_store           # cheapest start: non-recursive, one model per store
  python run_winner_a1.py --sets nr_store,nr_store_cat,nr_store_dept
  python run_winner_a1.py                           # all six sets, in order of cost
  python run_winner_a1.py --score_only              # rescore whatever has finished
  Default window is the original final one (d_1942 to d_1969); this needs sales_test_evaluation.csv next to the other
  data files. For d_1914 to d_1941 build the package in public mode and add --window public.

Needs the three M5 files calendar.csv, sell_prices.csv, sales_train_evaluation.csv and sample_submission.csv
(found automatically under --data_dir). Packages: pandas, numpy, scipy, lightgbm, psutil, scikit-learn, tqdm.

Outputs (in --out_dir, default m5_outputs_A1)
  pred_A1_<set>.npy        forecast of one set, shape (30490, 28), same row order as sales_train_evaluation.csv
  pred_A1_winner.npy       equal-weight average of the finished sets
  results_winner_A1.txt    plain-text log with scores, bias and the 12-level table
"""
import argparse
import importlib
import os
import shutil
import subprocess
import sys
import time

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)

SETS = {
    "nr_store": ("2-1. nonrecursive_store", "submission_kaggle_nonrecursive_store.csv"),
    "nr_store_cat": ("2-2. nonrecursive_store_cat", "submission_kaggle_nonrecursive_store_cat.csv"),
    "nr_store_dept": ("2-3. nonrecursive_store_dept", "submission_kaggle_nonrecursive_store_dept.csv"),
    "r_store": ("1-1. recursive_store", "submission_kaggle_recursive_store.csv"),
    "r_store_cat": ("1-2. recursive_store_cat", "submission_kaggle_recursive_store_cat.csv"),
    "r_store_dept": ("1-3. recursive_store_dept", "submission_kaggle_recursive_store_dept.csv"),
}
RAW_FILES = ["sales_train_evaluation.csv", "sell_prices.csv", "calendar.csv", "sample_submission.csv"]


def find_file(root, name):
    for dirpath, _, files in os.walk(root):
        if name in files:
            return os.path.join(dirpath, name)
    return None


def run_script(path, cwd, env, log_path):
    t0 = time.time()
    with open(log_path, "w", encoding="utf-8", errors="replace") as lf:
        rc = subprocess.run([sys.executable, "-u", path], cwd=cwd, env=env, stdout=lf, stderr=subprocess.STDOUT).returncode
    mins = (time.time() - t0) / 60
    return rc, mins


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default=None, help="folder holding the M5 csv files (searched recursively)")
    ap.add_argument("--work_dir", default=HERE, help="where the winner's folder structure is created")
    ap.add_argument("--out_dir", default=None)
    ap.add_argument("--sets", default="all", help="comma list of: " + ",".join(SETS))
    ap.add_argument("--score_only", action="store_true")
    ap.add_argument("--window", default="cv3", choices=["final", "public", "cv3"],
                    help="final = the winner's own final window d_1942..d_1969 (needs sales_test_evaluation.csv to score); "
                         "public = d_1914..d_1941 (package built in public mode); cv3 = d_1858..d_1885 (package built in cv3 mode)")
    args = ap.parse_args()

    work = os.path.abspath(args.work_dir)
    out_dir = args.out_dir or os.path.join(work, "m5_outputs_A1")
    os.makedirs(out_dir, exist_ok=True)
    # file names: final window -> pred_A1_*.npy / results_winner_A1.txt ; public window (validation forecasts) -> predval_A1_*.npy / results_win
ner_A1_public.txt
    PREFIX = {"final": "pred", "public": "predval", "cv3": "predcv3"}[args.window]
    RESULTS_NAME = {"final": "results_winner_A1.txt", "public": "results_winner_A1_public.txt", "cv3": "results_winner_A1_cv3.txt"}[args.window]
    chosen = list(SETS) if args.sets == "all" else [s.strip() for s in args.sets.split(",") if s.strip()]
    for s in chosen:
        if s not in SETS:
            raise SystemExit(f"Unknown set {s}. Choose from {list(SETS)}")

    # ---- packages
    missing = []
    for mod in ["pandas", "numpy", "scipy", "lightgbm", "psutil", "sklearn", "tqdm"]:
        try:
            importlib.import_module(mod)
        except ImportError:
            missing.append({"sklearn": "scikit-learn"}.get(mod, mod))
    if missing:
        raise SystemExit("Missing packages: " + ", ".join(missing) + "\nInstall with: pip install " + " ".join(missing))

    # ---- data
    data_dir = args.data_dir
    if data_dir is None:
        for cand in [os.path.join(HERE, "m5_data"), os.path.join(HERE, "..", "m5_data"), os.path.join(os.getcwd(), "m5_data")]:
```

```
        if os.path.isdir(cand):
            data_dir = cand
            break
if data_dir is None:
    raise SystemExit("Could not find the M5 data. Pass --data_dir <folder with the M5 csv files>.")
raw = os.path.join(work, "2. data")
for d in ["2. data/processed", "4. logs", "5. models", "6. submissions/before_ensemble", "run_logs"]:
    os.makedirs(os.path.join(work, d), exist_ok=True)
for f in RAW_FILES:
    dst = os.path.join(raw, f)
    if not os.path.exists(dst):
        src = find_file(data_dir, f)
        if src is None and f == "sample_submission.csv":
            # not in some copies of the data: rebuild it (ids of the 30,490 series for validation, then evaluation; F1..F28 = 0)
            import pandas as pd
            ids = pd.read_csv(os.path.join(raw, "sales_train_evaluation.csv"), usecols=["id"])["id"]
            ids = ids.str.replace("_evaluation", "_validation")
            ss = pd.DataFrame({"id": list(ids) + [i.replace("_validation", "_evaluation") for i in ids]})
            for k in range(1, 29):
                ss[f"F{k}"] = 0
            ss.to_csv(dst, index=False)
            print("sample_submission.csv not found, rebuilt it from the sales ids", flush=True)
            continue
        if src is None:
            raise SystemExit(f"{f} not found under {data_dir}")
        print(f"Copying {f} ...", flush=True)
        shutil.copy(src, dst)

env = dict(os.environ, A1_DIR=work)
code = os.path.join(work, "3. code")
t_start = time.time()
status = []

if not args.score_only:
    marker = os.path.join(work, "run_logs", "done_preprocessing")
    if not os.path.exists(marker):
        print("Preprocessing (tens of minutes, needs lots of RAM) ...", flush=True)
        rc, mins = run_script(os.path.join(code, "1. preprocessing", "1. preprocessing.py"), work, env,
                              os.path.join(work, "run_logs", "preprocessing.log"))
        if rc != 0:
            raise SystemExit(f"Preprocessing failed (exit {rc}). See run_logs/preprocessing.log")
        open(marker, "w").write("ok\n")
        status.append(f"preprocessing: {mins:.1f} min")
    else:
        status.append("preprocessing: already done")

    for s in chosen:
        stem, csv = SETS[s]
        csv_path = os.path.join(work, "6. submissions", "before_ensemble", csv)
        if os.path.exists(csv_path):
            status.append(f"{s}: already done")
            print(f"{s}: already done, skipping", flush=True)
            continue
        for kind in ("train", "predict"):
            sub = "2. train" if kind == "train" else "3. predict"
            fname = f"{stem}_{kind.upper()}.py"
            print(f"{s}: {kind} ({fname}) ...", flush=True)
            rc, mins = run_script(os.path.join(code, sub, fname), work, env,
                                  os.path.join(work, "run_logs", f"{s}_{kind}.log"))
            status.append(f"{s} {kind}: {mins:.1f} min (exit {rc})")
            print(f"  {s} {kind} finished in {mins:.1f} min (exit {rc})", flush=True)
            if rc != 0:
                raise SystemExit(f"{s} {kind} failed. See run_logs/{s}_{kind}.log")
        if not os.path.exists(csv_path):
            raise SystemExit(f"{s}: expected output not found: {csv_path}")

# ---- scoring
import numpy as np
import pandas as pd
from m5_data_and_wrmsse import WRMSSEEvaluator, format_table, load_m5, seasonal_naive_forecast

meta, values, dcols, calendar, prices = load_m5(raw)
if args.window == "final":
    tpath = None
    for base in [data_dir, work, HERE]:
        try:
            tpath = find_file(base, "sales_test_evaluation.csv")
        except Exception:
            tpath = None
        if tpath:
            break
    if tpath is None:
        raise SystemExit("Final window needs sales_test_evaluation.csv (actual sales d_1942..d_1969) under --data_dir.")
    tdf = pd.read_csv(tpath)
    tcols = sorted([c for c in tdf.columns if c.startswith("d_")], key=lambda c: int(c[2:]))
    assert len(tcols) == 28, f"expected 28 test days, found {len(tcols)}"
    key = meta["item_id"] + "|" + meta["store_id"]
    tdf = tdf.set_index(tdf["item_id"] + "|" + tdf["store_id"])
    tv = tdf.reindex(key)[tcols].to_numpy(dtype=np.float64)
    assert not np.isnan(tv).any(), "test file does not cover every series"
    values = np.concatenate([values, tv], axis=1)
    dcols = list(dcols) + tcols
if args.window == "cv3":
```

```
        # fold ending d_1885: holdout d_1858..d_1885, training history up to d_1857 (the M5 protocol on that fold)
        values = values[:, :1885]
        dcols = list(dcols)[:1885]
    ev = WRMSSEEvaluator(meta, values, dcols, calendar, prices)
    total_actual = ev.y_test.sum()
    preds, res = {}, {}
    for s in SETS:
        csv_path = os.path.join(work, "6. submissions", "before_ensemble", SETS[s][1])
        if not os.path.exists(csv_path):
            continue
        sub = pd.read_csv(csv_path).drop_duplicates("id").set_index("id")
        fcols = [f"F{i}" for i in range(1, 29)]
        arr = sub.reindex(meta["id"])[fcols].fillna(0).to_numpy(dtype=np.float64)
        arr = np.clip(arr, 0, None)
        preds[s] = arr
        res[s] = ev.score(arr)
        np.save(os.path.join(out_dir, f"{PREFIX}_A1_{s}.npy"), arr.astype(np.float32))
    if not preds:
        raise SystemExit("No finished set to score yet.")
    avg = np.mean(list(preds.values()), axis=0)
    res["average"] = ev.score(avg)
    np.save(os.path.join(out_dir, f"{PREFIX}_A1_winner.npy"), avg.astype(np.float32))

    lines = [f"{'Set':<16}{'WRMSSE':>9}{'Bias':>8}"]
    for s in list(preds) + ["average"]:
        a = avg if s == "average" else preds[s]
        lines.append(f"{s:<16}{res[s][0]:>9.4f}{a.sum() / total_actual:>8.3f}")
    pub = args.window == "public"
    table = {"SeasonalNaive": ev.score(seasonal_naive_forecast(ev.X_train))}
    for s in list(preds)[:3]:
        table[s] = res[s]
    table["average"] = res["average"]
    n_done = len(preds)
    report = [
        "WINNER'S CODE (M5 1st place, folder A1) RUN ON " + {"final": "THE ORIGINAL FINAL WINDOW", "public": "OUR PUBLIC WINDOW", "cv3": "AN EAR
LIER VALIDATION FOLD (cv3)"}[args.window],
        ("Trained on days up to d_1941, forecast d_1942 to d_1969, scored on the released actuals (the competition's final window)."
         if args.window == "final" else
         ("Trained on days up to d_1913, forecast d_1914 to d_1941 (the winner's own 'public' setting)."
          if args.window == "public" else
          "Trained on days up to d_1857, forecast d_1858 to d_1885 (the winner's own 'cv3' fold; target encodings rebuilt on days up to d_1857).
")),
        "Settings of the winner's models are unchanged. Compatibility patches are listed in PATCH_LOG.txt.",
        f"Sets finished: {n_done} of 6 ({', '.join(preds)}). The winner averaged all six with equal weights;",
        "the 'average' row is the equal-weight average of the sets finished so far.",
        "Bias = total forecast / total actual (1.000 is unbiased).",
        "",
        "RESULTS (WRMSSE on the holdout, lower is better)",
        *lines,
        "",
        "WRMSSE BY LEVEL (first three finished sets and the average)",
        format_table(ev, table),
        "",
        "REFERENCE VALUES (published, window to be confirmed)",
        "  Original M5 winner (final window d_1942 to d_1969): 0.520",
        "  Yang et al. Kaggle Top-1 reproduction: 0.5230 | Yang et al. LightGBM + PatchTST: 0.4989",
        "  Our numbers on d_1914 to d_1941 (a different window): E1b L3 0.5422 | E1d non-recursive per-store port 0.6959 raw, 0.5992 calibrated"
,
        "",
        "STEPS: " + " | ".join(status),
        f"Runtime of this invocation: {(time.time() - t_start) / 60:.1f} min",
    ]
    text = "\n".join(report)
    with open(os.path.join(out_dir, RESULTS_NAME), "w") as f:
        f.write(text + "\n")
    print("\n" + text)
    print(f"\nSaved: {os.path.join(out_dir, RESULTS_NAME)}")


if __name__ == "__main__":
    main()
```

src/winner_runner/run_winner_a1_cv4.py

```
#!/usr/bin/env python3
"""
Run the 1st place M5 Accuracy solution (YeonJun In, M5-methods folder A1) on the competition's own final window, then score it with our evaluato
r.

What this does
  The winner's own scripts are run in order (preprocessing, then for each of six model sets: train and predict),
  with only small compatibility fixes (this package keeps the winner's own final window: train to d_1941,
  forecast d_1942 to d_1969, scored on the released actuals in sales_test_evaluation.csv):
    - small compatibility fixes for current pandas and LightGBM (see PATCH_LOG.txt); no model setting is touched
  The six sets are: non-recursive and recursive models trained per store (10 models each), per store x category
  (30) and per store x department (70). The winner averaged all six with equal weights.
  Afterwards every finished set and the equal-weight average of the finished sets is scored with the same WRMSSE
  evaluator as the rest of the project.

Time and memory (honest expectations)
  Needs about 16 GB RAM free and about 15 GB disk. Preprocessing takes tens of minutes. Training uses 3000 trees per
```

```
  model on CPU. The non-recursive sets are the cheaper ones; the recursive sets use much larger trees and are the
  slowest. The full job of 220 models can take days on an ordinary CPU, so use --sets to run it in stages.
  Finished sets are remembered; a rerun skips them.

Usage
  python run_winner_a1.py --sets nr_store            # cheapest start: non-recursive, one model per store
  python run_winner_a1.py --sets nr_store,nr_store_cat,nr_store_dept
  python run_winner_a1.py                            # all six sets, in order of cost
  python run_winner_a1.py --score_only               # rescore whatever has finished
  Default window is the original final one (d_1942 to d_1969); this needs sales_test_evaluation.csv next to the other
  data files. For d_1914 to d_1941 build the package in public mode and add --window public.

Needs the three M5 files calendar.csv, sell_prices.csv, sales_train_evaluation.csv and sample_submission.csv
(found automatically under --data_dir). Packages: pandas, numpy, scipy, lightgbm, psutil, scikit-learn, tqdm.

Outputs (in --out_dir, default m5_outputs_A1)
  pred_A1_<set>.npy        forecast of one set, shape (30490, 28), same row order as sales_train_evaluation.csv
  pred_A1_winner.npy       equal-weight average of the finished sets
  results_winner_A1.txt    plain-text log with scores, bias and the 12-level table
"""
import argparse
import importlib
import os
import shutil
import subprocess
import sys
import time

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)

SETS = {
    "nr_store": ("2-1. nonrecursive_store", "submission_kaggle_nonrecursive_store.csv"),
    "nr_store_cat": ("2-2. nonrecursive_store_cat", "submission_kaggle_nonrecursive_store_cat.csv"),
    "nr_store_dept": ("2-3. nonrecursive_store_dept", "submission_kaggle_nonrecursive_store_dept.csv"),
    "r_store": ("1-1. recursive_store", "submission_kaggle_recursive_store.csv"),
    "r_store_cat": ("1-2. recursive_store_cat", "submission_kaggle_recursive_store_cat.csv"),
    "r_store_dept": ("1-3. recursive_store_dept", "submission_kaggle_recursive_store_dept.csv"),
}
RAW_FILES = ["sales_train_evaluation.csv", "sell_prices.csv", "calendar.csv", "sample_submission.csv"]


def find_file(root, name):
    for dirpath, _, files in os.walk(root):
        if name in files:
            return os.path.join(dirpath, name)
    return None


def run_script(path, cwd, env, log_path):
    t0 = time.time()
    with open(log_path, "w", encoding="utf-8", errors="replace") as lf:
        rc = subprocess.run([sys.executable, "-u", path], cwd=cwd, env=env, stdout=lf, stderr=subprocess.STDOUT).returncode
    mins = (time.time() - t0) / 60
    return rc, mins


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default=None, help="folder holding the M5 csv files (searched recursively)")
    ap.add_argument("--work_dir", default=HERE, help="where the winner's folder structure is created")
    ap.add_argument("--out_dir", default=None)
    ap.add_argument("--sets", default="all", help="comma list of: " + ",".join(SETS))
    ap.add_argument("--score_only", action="store_true")
    ap.add_argument("--window", default="cv4", choices=["final", "public", "cv4"],
                    help="final = the winner's own final window d_1942..d_1969 (needs sales_test_evaluation.csv to score); "
                         "public = d_1914..d_1941 (package built in public mode); cv4 = d_1886..d_1913 (package built in cv4 mode)")
    args = ap.parse_args()

    work = os.path.abspath(args.work_dir)
    out_dir = args.out_dir or os.path.join(work, "m5_outputs_A1")
    os.makedirs(out_dir, exist_ok=True)
    # file names: final window -> pred_A1_*.npy / results_winner_A1.txt ; public window (validation forecasts) -> predval_A1_*.npy / results_win
ner_A1_public.txt
    PREFIX = {"final": "pred", "public": "predval", "cv4": "predcv4"}[args.window]
    RESULTS_NAME = {"final": "results_winner_A1.txt", "public": "results_winner_A1_public.txt", "cv4": "results_winner_A1_cv4.txt"}[args.window]
    chosen = list(SETS) if args.sets == "all" else [s.strip() for s in args.sets.split(",") if s.strip()]
    for s in chosen:
        if s not in SETS:
            raise SystemExit(f"Unknown set {s}. Choose from {list(SETS)}")

    # ---- packages
    missing = []
    for mod in ["pandas", "numpy", "scipy", "lightgbm", "psutil", "sklearn", "tqdm"]:
        try:
            importlib.import_module(mod)
        except ImportError:
            missing.append({"sklearn": "scikit-learn"}.get(mod, mod))
    if missing:
        raise SystemExit("Missing packages: " + ", ".join(missing) + "\nInstall with: pip install " + " ".join(missing))

    # ---- data
    data_dir = args.data_dir
```

```
if data_dir is None:
    for cand in [os.path.join(HERE, "m5_data"), os.path.join(HERE, "..", "m5_data"), os.path.join(os.getcwd(), "m5_data")]:
        if os.path.isdir(cand):
            data_dir = cand
            break
if data_dir is None:
    raise SystemExit("Could not find the M5 data. Pass --data_dir <folder with the M5 csv files>.")
raw = os.path.join(work, "2. data")
for d in ["2. data/processed", "4. logs", "5. models", "6. submissions/before_ensemble", "run_logs"]:
    os.makedirs(os.path.join(work, d), exist_ok=True)
for f in RAW_FILES:
    dst = os.path.join(raw, f)
    if not os.path.exists(dst):
        src = find_file(data_dir, f)
        if src is None and f == "sample_submission.csv":
            # not in some copies of the data: rebuild it (ids of the 30,490 series for validation, then evaluation; F1..F28 = 0)
            import pandas as pd
            ids = pd.read_csv(os.path.join(raw, "sales_train_evaluation.csv"), usecols=["id"])["id"]
            ids = ids.str.replace("_evaluation", "_validation")
            ss = pd.DataFrame({"id": list(ids) + [i.replace("_validation", "_evaluation") for i in ids]})
            for k in range(1, 29):
                ss[f"F{k}"] = 0
            ss.to_csv(dst, index=False)
            print("sample_submission.csv not found, rebuilt it from the sales ids", flush=True)
            continue
        if src is None:
            raise SystemExit(f"{f} not found under {data_dir}")
        print(f"Copying {f} ...", flush=True)
        shutil.copy(src, dst)

env = dict(os.environ, A1_DIR=work)
code = os.path.join(work, "3. code")
t_start = time.time()
status = []

if not args.score_only:
    marker = os.path.join(work, "run_logs", "done_preprocessing")
    if not os.path.exists(marker):
        print("Preprocessing (tens of minutes, needs lots of RAM) ...", flush=True)
        rc, mins = run_script(os.path.join(code, "1. preprocessing", "1. preprocessing.py"), work, env,
                              os.path.join(work, "run_logs", "preprocessing.log"))
        if rc != 0:
            raise SystemExit(f"Preprocessing failed (exit {rc}). See run_logs/preprocessing.log")
        open(marker, "w").write("ok\n")
        status.append(f"preprocessing: {mins:.1f} min")
    else:
        status.append("preprocessing: already done")

    for s in chosen:
        stem, csv = SETS[s]
        csv_path = os.path.join(work, "6. submissions", "before_ensemble", csv)
        if os.path.exists(csv_path):
            status.append(f"{s}: already done")
            print(f"{s}: already done, skipping", flush=True)
            continue
        for kind in ("train", "predict"):
            sub = "2. train" if kind == "train" else "3. predict"
            fname = f"{stem}_{kind.upper()}.py"
            print(f"{s}: {kind} ({fname}) ...", flush=True)
            rc, mins = run_script(os.path.join(code, sub, fname), work, env,
                                  os.path.join(work, "run_logs", f"{s}_{kind}.log"))
            status.append(f"{s} {kind}: {mins:.1f} min (exit {rc})")
            print(f"  {s} {kind} finished in {mins:.1f} min (exit {rc})", flush=True)
            if rc != 0:
                raise SystemExit(f"{s} {kind} failed. See run_logs/{s}_{kind}.log")
        if not os.path.exists(csv_path):
            raise SystemExit(f"{s}: expected output not found: {csv_path}")

# ---- scoring
import numpy as np
import pandas as pd
from m5_data_and_wrmsse import WRMSSEEvaluator, format_table, load_m5, seasonal_naive_forecast

meta, values, dcols, calendar, prices = load_m5(raw)
if args.window == "final":
    tpath = None
    for base in [data_dir, work, HERE]:
        try:
            tpath = find_file(base, "sales_test_evaluation.csv")
        except Exception:
            tpath = None
        if tpath:
            break
    if tpath is None:
        raise SystemExit("Final window needs sales_test_evaluation.csv (actual sales d_1942..d_1969) under --data_dir.")
    tdf = pd.read_csv(tpath)
    tcols = sorted([c for c in tdf.columns if c.startswith("d_")], key=lambda c: int(c[2:]))
    assert len(tcols) == 28, f"expected 28 test days, found {len(tcols)}"
    key = meta["item_id"] + "|" + meta["store_id"]
    tdf = tdf.set_index(tdf["item_id"] + "|" + tdf["store_id"])
    tv = tdf.reindex(key)[tcols].to_numpy(dtype=np.float64)
    assert not np.isnan(tv).any(), "test file does not cover every series"
    values = np.concatenate([values, tv], axis=1)
```

```
        dcols = list(dcols) + tcols
    if args.window == "cv4":
        # fold ending d_1913: holdout d_1886..d_1913, training history up to d_1885 (the M5 protocol on that fold)
        values = values[:, :1913]
        dcols = list(dcols)[:1913]
    ev = WRMSSEEvaluator(meta, values, dcols, calendar, prices)
    total_actual = ev.y_test.sum()
    preds, res = {}, {}
    for s in SETS:
        csv_path = os.path.join(work, "6. submissions", "before_ensemble", SETS[s][1])
        if not os.path.exists(csv_path):
            continue
        sub = pd.read_csv(csv_path).drop_duplicates("id").set_index("id")
        fcols = [f"F{i}" for i in range(1, 29)]
        arr = sub.reindex(meta["id"])[fcols].fillna(0).to_numpy(dtype=np.float64)
        arr = np.clip(arr, 0, None)
        preds[s] = arr
        res[s] = ev.score(arr)
        np.save(os.path.join(out_dir, f"{PREFIX}_A1_{s}.npy"), arr.astype(np.float32))
    if not preds:
        raise SystemExit("No finished set to score yet.")
    avg = np.mean(list(preds.values()), axis=0)
    res["average"] = ev.score(avg)
    np.save(os.path.join(out_dir, f"{PREFIX}_A1_winner.npy"), avg.astype(np.float32))

    lines = [f"{'Set':<16}{'WRMSSE':>9}{'Bias':>8}"]
    for s in list(preds) + ["average"]:
        a = avg if s == "average" else preds[s]
        lines.append(f"{s:<16}{res[s][0]:>9.4f}{a.sum() / total_actual:>8.3f}")
    pub = args.window == "public"
    table = {"SeasonalNaive": ev.score(seasonal_naive_forecast(ev.X_train))}
    for s in list(preds)[:3]:
        table[s] = res[s]
    table["average"] = res["average"]
    n_done = len(preds)
    report = [
        "WINNER'S CODE (M5 1st place, folder A1) RUN ON " + {"final": "THE ORIGINAL FINAL WINDOW", "public": "OUR PUBLIC WINDOW", "cv4": "AN EAR
LIER VALIDATION FOLD (cv4)"}[args.window],
        ("Trained on days up to d_1941, forecast d_1942 to d_1969, scored on the released actuals (the competition's final window)."
         if args.window == "final" else
         ("Trained on days up to d_1913, forecast d_1914 to d_1941 (the winner's own 'public' setting)."
          if args.window == "public" else
          "Trained on days up to d_1885, forecast d_1886 to d_1913 (the winner's own 'cv4' fold; target encodings rebuilt on days up to d_1885).
")),
        "Settings of the winner's models are unchanged. Compatibility patches are listed in PATCH_LOG.txt.",
        f"Sets finished: {n_done} of 6 ({', '.join(preds)}). The winner averaged all six with equal weights;",
        "the 'average' row is the equal-weight average of the sets finished so far.",
        "Bias = total forecast / total actual (1.000 is unbiased).",
        "",
        "RESULTS (WRMSSE on the holdout, lower is better)",
        *lines,
        "",
        "WRMSSE BY LEVEL (first three finished sets and the average)",
        format_table(ev, table),
        "",
        "REFERENCE VALUES (published, window to be confirmed)",
        "  Original M5 winner (final window d_1942 to d_1969): 0.520",
        "  Yang et al. Kaggle Top-1 reproduction: 0.5230 | Yang et al. LightGBM + PatchTST: 0.4989",
        "  Our numbers on d_1914 to d_1941 (a different window): E1b L3 0.5422 | E1d non-recursive per-store port 0.6959 raw, 0.5992 calibrated"
,
        "",
        "STEPS: " + " | ".join(status),
        f"Runtime of this invocation: {(time.time() - t_start) / 60:.1f} min",
    ]
    text = "\n".join(report)
    with open(os.path.join(out_dir, RESULTS_NAME), "w") as f:
        f.write(text + "\n")
    print("\n" + text)
    print(f"\nSaved: {os.path.join(out_dir, RESULTS_NAME)}")


if __name__ == "__main__":
    main()
```

src/winner_runner/run_winner_a1_cv5.py

```
#!/usr/bin/env python3
"""
Run the 1st place M5 Accuracy solution (YeonJun In, M5-methods folder A1) on the competition's own final window, then score it with our evaluato
r.

What this does
  The winner's own scripts are run in order (preprocessing, then for each of six model sets: train and predict),
  with only small compatibility fixes (this package keeps the winner's own final window: train to d_1941,
  forecast d_1942 to d_1969, scored on the released actuals in sales_test_evaluation.csv):
    - small compatibility fixes for current pandas and LightGBM (see PATCH_LOG.txt); no model setting is touched
  The six sets are: non-recursive and recursive models trained per store (10 models each), per store x category
  (30) and per store x department (70). The winner averaged all six with equal weights.
  Afterwards every finished set and the equal-weight average of the finished sets is scored with the same WRMSSE
  evaluator as the rest of the project.
```

```
Time and memory (honest expectations)
  Needs about 16 GB RAM free and about 15 GB disk. Preprocessing takes tens of minutes. Training uses 3000 trees per
  model on CPU. The non-recursive sets are the cheaper ones; the recursive sets use much larger trees and are the
  slowest. The full job of 220 models can take days on an ordinary CPU, so use --sets to run it in stages.
  Finished sets are remembered; a rerun skips them.

Usage
  python run_winner_a1.py --sets nr_store           # cheapest start: non-recursive, one model per store
  python run_winner_a1.py --sets nr_store,nr_store_cat,nr_store_dept
  python run_winner_a1.py                           # all six sets, in order of cost
  python run_winner_a1.py --score_only              # rescore whatever has finished
  Default window is the original final one (d_1942 to d_1969); this needs sales_test_evaluation.csv next to the other
  data files. For d_1914 to d_1941 build the package in public mode and add --window public.

Needs the three M5 files calendar.csv, sell_prices.csv, sales_train_evaluation.csv and sample_submission.csv
(found automatically under --data_dir). Packages: pandas, numpy, scipy, lightgbm, psutil, scikit-learn, tqdm.

Outputs (in --out_dir, default m5_outputs_A1)
  pred_A1_<set>.npy        forecast of one set, shape (30490, 28), same row order as sales_train_evaluation.csv
  pred_A1_winner.npy       equal-weight average of the finished sets
  results_winner_A1.txt    plain-text log with scores, bias and the 12-level table
"""
import argparse
import importlib
import os
import shutil
import subprocess
import sys
import time

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)

SETS = {
    "nr_store": ("2-1. nonrecursive_store", "submission_kaggle_nonrecursive_store.csv"),
    "nr_store_cat": ("2-2. nonrecursive_store_cat", "submission_kaggle_nonrecursive_store_cat.csv"),
    "nr_store_dept": ("2-3. nonrecursive_store_dept", "submission_kaggle_nonrecursive_store_dept.csv"),
    "r_store": ("1-1. recursive_store", "submission_kaggle_recursive_store.csv"),
    "r_store_cat": ("1-2. recursive_store_cat", "submission_kaggle_recursive_store_cat.csv"),
    "r_store_dept": ("1-3. recursive_store_dept", "submission_kaggle_recursive_store_dept.csv"),
}
RAW_FILES = ["sales_train_evaluation.csv", "sell_prices.csv", "calendar.csv", "sample_submission.csv"]


def find_file(root, name):
    for dirpath, _, files in os.walk(root):
        if name in files:
            return os.path.join(dirpath, name)
    return None


def run_script(path, cwd, env, log_path):
    t0 = time.time()
    with open(log_path, "w", encoding="utf-8", errors="replace") as lf:
        rc = subprocess.run([sys.executable, "-u", path], cwd=cwd, env=env, stdout=lf, stderr=subprocess.STDOUT).returncode
    mins = (time.time() - t0) / 60
    return rc, mins


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", default=None, help="folder holding the M5 csv files (searched recursively)")
    ap.add_argument("--work_dir", default=HERE, help="where the winner's folder structure is created")
    ap.add_argument("--out_dir", default=None)
    ap.add_argument("--sets", default="all", help="comma list of: " + ",".join(SETS))
    ap.add_argument("--score_only", action="store_true")
    ap.add_argument("--window", default="cv5", choices=["final", "public", "cv5"],
                    help="final = the winner's own final window d_1942..d_1969 (needs sales_test_evaluation.csv to score); "
                         "public = d_1914..d_1941 (package built in public mode); cv5 = d_1802..d_1829 (package built in cv5 mode)")
    args = ap.parse_args()

    work = os.path.abspath(args.work_dir)
    out_dir = args.out_dir or os.path.join(work, "m5_outputs_A1")
    os.makedirs(out_dir, exist_ok=True)
    # file names: final window -> pred_A1_*.npy / results_winner_A1.txt ; public window (validation forecasts) -> predval_A1_*.npy / results_win
ner_A1_public.txt
    PREFIX = {"final": "pred", "public": "predval", "cv5": "predcv5"}[args.window]
    RESULTS_NAME = {"final": "results_winner_A1.txt", "public": "results_winner_A1_public.txt", "cv5": "results_winner_A1_cv5.txt"}[args.window]
    chosen = list(SETS) if args.sets == "all" else [s.strip() for s in args.sets.split(",") if s.strip()]
    for s in chosen:
        if s not in SETS:
            raise SystemExit(f"Unknown set {s}. Choose from {list(SETS)}")

    # ---- packages
    missing = []
    for mod in ["pandas", "numpy", "scipy", "lightgbm", "psutil", "sklearn", "tqdm"]:
        try:
            importlib.import_module(mod)
        except ImportError:
            missing.append({"sklearn": "scikit-learn"}.get(mod, mod))
    if missing:
        raise SystemExit("Missing packages: " + ", ".join(missing) + "\nInstall with: pip install " + " ".join(missing))
```

```
# ---- data
data_dir = args.data_dir
if data_dir is None:
    for cand in [os.path.join(HERE, "m5_data"), os.path.join(HERE, "..", "m5_data"), os.path.join(os.getcwd(), "m5_data")]:
        if os.path.isdir(cand):
            data_dir = cand
            break
if data_dir is None:
    raise SystemExit("Could not find the M5 data. Pass --data_dir <folder with the M5 csv files>.")
raw = os.path.join(work, "2. data")
for d in ["2. data/processed", "4. logs", "5. models", "6. submissions/before_ensemble", "run_logs"]:
    os.makedirs(os.path.join(work, d), exist_ok=True)
for f in RAW_FILES:
    dst = os.path.join(raw, f)
    if not os.path.exists(dst):
        src = find_file(data_dir, f)
        if src is None and f == "sample_submission.csv":
            # not in some copies of the data: rebuild it (ids of the 30,490 series for validation, then evaluation; F1..F28 = 0)
            import pandas as pd
            ids = pd.read_csv(os.path.join(raw, "sales_train_evaluation.csv"), usecols=["id"])["id"]
            ids = ids.str.replace("_evaluation", "_validation")
            ss = pd.DataFrame({"id": list(ids) + [i.replace("_validation", "_evaluation") for i in ids]})
            for k in range(1, 29):
                ss[f"F{k}"] = 0
            ss.to_csv(dst, index=False)
            print("sample_submission.csv not found, rebuilt it from the sales ids", flush=True)
            continue
        if src is None:
            raise SystemExit(f"{f} not found under {data_dir}")
        print(f"Copying {f} ...", flush=True)
        shutil.copy(src, dst)

env = dict(os.environ, A1_DIR=work)
code = os.path.join(work, "3. code")
t_start = time.time()
status = []

if not args.score_only:
    marker = os.path.join(work, "run_logs", "done_preprocessing")
    if not os.path.exists(marker):
        print("Preprocessing (tens of minutes, needs lots of RAM) ...", flush=True)
        rc, mins = run_script(os.path.join(code, "1. preprocessing", "1. preprocessing.py"), work, env,
                              os.path.join(work, "run_logs", "preprocessing.log"))
        if rc != 0:
            raise SystemExit(f"Preprocessing failed (exit {rc}). See run_logs/preprocessing.log")
        open(marker, "w").write("ok\n")
        status.append(f"preprocessing: {mins:.1f} min")
    else:
        status.append("preprocessing: already done")

    for s in chosen:
        stem, csv = SETS[s]
        csv_path = os.path.join(work, "6. submissions", "before_ensemble", csv)
        if os.path.exists(csv_path):
            status.append(f"{s}: already done")
            print(f"{s}: already done, skipping", flush=True)
            continue
        for kind in ("train", "predict"):
            sub = "2. train" if kind == "train" else "3. predict"
            fname = f"{stem}_{kind.upper()}.py"
            print(f"{s}: {kind} ({fname}) ...", flush=True)
            rc, mins = run_script(os.path.join(code, sub, fname), work, env,
                                  os.path.join(work, "run_logs", f"{s}_{kind}.log"))
            status.append(f"{s} {kind}: {mins:.1f} min (exit {rc})")
            print(f"  {s} {kind} finished in {mins:.1f} min (exit {rc})", flush=True)
            if rc != 0:
                raise SystemExit(f"{s} {kind} failed. See run_logs/{s}_{kind}.log")
        if not os.path.exists(csv_path):
            raise SystemExit(f"{s}: expected output not found: {csv_path}")

# ---- scoring
import numpy as np
import pandas as pd
from m5_data_and_wrmsse import WRMSSEEvaluator, format_table, load_m5, seasonal_naive_forecast

meta, values, dcols, calendar, prices = load_m5(raw)
if args.window == "final":
    tpath = None
    for base in [data_dir, work, HERE]:
        try:
            tpath = find_file(base, "sales_test_evaluation.csv")
        except Exception:
            tpath = None
        if tpath:
            break
    if tpath is None:
        raise SystemExit("Final window needs sales_test_evaluation.csv (actual sales d_1942..d_1969) under --data_dir.")
    tdf = pd.read_csv(tpath)
    tcols = sorted([c for c in tdf.columns if c.startswith("d_")], key=lambda c: int(c[2:]))
    assert len(tcols) == 28, f"expected 28 test days, found {len(tcols)}"
    key = meta["item_id"] + "|" + meta["store_id"]
    tdf = tdf.set_index(tdf["item_id"] + "|" + tdf["store_id"])
    tv = tdf.reindex(key)[tcols].to_numpy(dtype=np.float64)
```

```
        assert not np.isnan(tv).any(), "test file does not cover every series"
        values = np.concatenate([values, tv], axis=1)
        dcols = list(dcols) + tcols
    if args.window == "cv5":
        # fold ending d_1829: holdout d_1802..d_1829, training history up to d_1801 (the M5 protocol on that fold)
        values = values[:, :1829]
        dcols = list(dcols)[:1829]
    ev = WRMSSEEvaluator(meta, values, dcols, calendar, prices)
    total_actual = ev.y_test.sum()
    preds, res = {}, {}
    for s in SETS:
        csv_path = os.path.join(work, "6. submissions", "before_ensemble", SETS[s][1])
        if not os.path.exists(csv_path):
            continue
        sub = pd.read_csv(csv_path).drop_duplicates("id").set_index("id")
        fcols = [f"F{i}" for i in range(1, 29)]
        arr = sub.reindex(meta["id"])[fcols].fillna(0).to_numpy(dtype=np.float64)
        arr = np.clip(arr, 0, None)
        preds[s] = arr
        res[s] = ev.score(arr)
        np.save(os.path.join(out_dir, f"{PREFIX}_A1_{s}.npy"), arr.astype(np.float32))
    if not preds:
        raise SystemExit("No finished set to score yet.")
    avg = np.mean(list(preds.values()), axis=0)
    res["average"] = ev.score(avg)
    np.save(os.path.join(out_dir, f"{PREFIX}_A1_winner.npy"), avg.astype(np.float32))

    lines = [f"{'Set':<16}{'WRMSSE':>9}{'Bias':>8}"]
    for s in list(preds) + ["average"]:
        a = avg if s == "average" else preds[s]
        lines.append(f"{s:<16}{res[s][0]:>9.4f}{a.sum() / total_actual:>8.3f}")
    pub = args.window == "public"
    table = {"SeasonalNaive": ev.score(seasonal_naive_forecast(ev.X_train))}
    for s in list(preds)[:3]:
        table[s] = res[s]
    table["average"] = res["average"]
    n_done = len(preds)
    report = [
        "WINNER'S CODE (M5 1st place, folder A1) RUN ON " + {"final": "THE ORIGINAL FINAL WINDOW", "public": "OUR PUBLIC WINDOW", "cv5": "AN EAR
LIER VALIDATION FOLD (cv5)"}[args.window],
        ("Trained on days up to d_1941, forecast d_1942 to d_1969, scored on the released actuals (the competition's final window)."
         if args.window == "final" else
         ("Trained on days up to d_1913, forecast d_1914 to d_1941 (the winner's own 'public' setting)."
          if args.window == "public" else
          "Trained on days up to d_1801, forecast d_1802 to d_1829 (the winner's own 'cv5' fold; target encodings rebuilt on days up to d_1801).
")),
        "Settings of the winner's models are unchanged. Compatibility patches are listed in PATCH_LOG.txt.",
        f"Sets finished: {n_done} of 6 ({', '.join(preds)}). The winner averaged all six with equal weights;",
        "the 'average' row is the equal-weight average of the sets finished so far.",
        "Bias = total forecast / total actual (1.000 is unbiased).",
        "",
        "RESULTS (WRMSSE on the holdout, lower is better)",
        *lines,
        "",
        "WRMSSE BY LEVEL (first three finished sets and the average)",
        format_table(ev, table),
        "",
        "REFERENCE VALUES (published, window to be confirmed)",
        "  Original M5 winner (final window d_1942 to d_1969): 0.520",
        "  Yang et al. Kaggle Top-1 reproduction: 0.5230 | Yang et al. LightGBM + PatchTST: 0.4989",
        "  Our numbers on d_1914 to d_1941 (a different window): E1b L3 0.5422 | E1d non-recursive per-store port 0.6959 raw, 0.5992 calibrated"
,
        "",
        "STEPS: " + " | ".join(status),
        f"Runtime of this invocation: {(time.time() - t_start) / 60:.1f} min",
    ]
    text = "\n".join(report)
    with open(os.path.join(out_dir, RESULTS_NAME), "w") as f:
        f.write(text + "\n")
    print("\n" + text)
    print(f"\nSaved: {os.path.join(out_dir, RESULTS_NAME)}")


if __name__ == "__main__":
    main()
```

scripts/prepare_inputs.py

```
"""Build inputs/ (the folder the scoring scripts expect) from the saved forecasts in forecasts/.
Copies files only. Run from the repository root:  python scripts/prepare_inputs.py
"""
import shutil, glob, os
root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
out = os.path.join(root, "inputs"); os.makedirs(out, exist_ok=True)
n = 0
for pat in ["forecasts/winner_lightgbm/winner_*.npy", "forecasts/chronos2_aggregate/agg_chronos2_B_covariates_*.npy"]:
    for f in glob.glob(os.path.join(root, pat)):
        shutil.copy(f, os.path.join(out, os.path.basename(f))); n += 1
print(f"copied {n} files to {out}")
```

Appendix F. All raw result files

results/01_baselines_and_early_models/results_E1_lightgbm.txt

```
E1 GLOBAL LIGHTGBM RESULTS
Data folder: m5_data
Training window: d_1 to d_1913 | Holdout (28 days): d_1914 to d_1941
Training origins: 52 weekly steps | horizons sampled per series per origin: 6
Training rows: 9,512,880 | features: 34
Objective: tweedie (variance power 1.1) | best iteration: 187 | validation RMSE: 2.0492
Early stopping validation uses an origin whose targets lie before the holdout (no test leakage).

WRMSSE BY LEVEL
Level Name                    Series   SeasonalNaive     E1_LightGBM
----------------------------------------------------------------------
1     Total                        1          0.7514          0.4691
2     State                        3          0.7544          0.4870
3     Store                       10          0.7993          0.5541
4     Category                     3          0.7620          0.4841
5     Department                   7          0.8171          0.5381
6     State x Category             9          0.7830          0.5254
7     State x Department          21          0.8304          0.5777
8     Store x Category            30          0.8393          0.5997
9     Store x Department          70          0.8919          0.6601
10    Item                      3049          1.0322          0.8224
11    Item x State              9147          1.0750          0.8256
12    Item x Store             30490          1.1004          0.8264
----------------------------------------------------------------------
      OVERALL WRMSSE           42840          0.8697          0.6141

REFERENCE VALUES (published, window to be confirmed)
  Original M5 winner (hidden test window): 0.520
  Yang et al. Kaggle Top-1 reproduction: 0.5230
  Yang et al. LightGBM + PatchTST: 0.4989
  This run, E1 LightGBM: 0.6141

TOP FEATURES BY GAIN
  mean28                  68.2%
  meannz28                 6.5%
  mean14                   5.1%
  mean112                  3.6%
  std112                   3.2%
  mean7                    2.8%
  mean56                   2.3%
  days_since_sale          1.0%
  price                    0.9%
  lag364                   0.8%
  wday                     0.8%
  month                    0.5%
  price_ratio_origin       0.5%
  sw_mean                  0.5%
  dept                     0.4%

Runtime: 1.8 min
```

results/01_baselines_and_early_models/results_E1b_lightgbm.txt

```
E1b GLOBAL LIGHTGBM RESULTS (ablation ladder)
Data folder: m5_data
Training window: d_1 to d_1913 | Holdout (28 days): d_1914 to d_1941
Training origins: 52 weekly steps | horizons sampled per series per origin: 6
Training rows: 9,512,880 | total features available: 49
Objective: tweedie (variance power 1.1). Early stopping on a validation origin before the holdout.
Final model is L4, fixed in advance. L0 to L3 holdout scores are for ablation only.
Bias = total forecast / total actual over the holdout (1.000 means unbiased overall).

ABLATION LADDER (each step adds to the previous one)
Step           Feats   WRMSSE   Change  BestIt  ValRMSE    Bias   Min
L0_base           34   0.6141             187   2.0492   0.952   1.5
L1_+price         38   0.5779  -0.0362    365   2.0394   0.961   2.1
L2_+event         43   0.5597  -0.0183    327   2.0457   0.969   2.1
L3_+context       49   0.5422  -0.0174    247   2.0390   0.972   2.5
L4_+weights       49   0.5888  +0.0465    118   4.1440   0.973   1.4

WRMSSE BY LEVEL
Level Name                    Series   SeasonalNaive         L0_base        L4_final
-------------------------------------------------------------------------------------
1     Total                        1          0.7514          0.4691          0.3813
2     State                        3          0.7544          0.4870          0.4251
3     Store                       10          0.7993          0.5541          0.5014
4     Category                     3          0.7620          0.4841          0.4552
5     Department                   7          0.8171          0.5381          0.5229
6     State x Category             9          0.7830          0.5254          0.5072
7     State x Department          21          0.8304          0.5777          0.5713
8     Store x Category            30          0.8393          0.5997          0.5829
9     Store x Department          70          0.8919          0.6601          0.6525
10    Item                      3049          1.0322          0.8224          0.8165
11    Item x State              9147          1.0750          0.8256          0.8229
12    Item x Store             30490          1.1004          0.8264          0.8258
-------------------------------------------------------------------------------------
      OVERALL WRMSSE           42840          0.8697          0.6141          0.5888

REFERENCE VALUES (published, window to be confirmed)
  Original M5 winner (hidden test window): 0.520
  Yang et al. Kaggle Top-1 reproduction: 0.5230
  Yang et al. LightGBM + PatchTST: 0.4989
  E1 (previous script, full run): 0.6141
  This run, E1b final (L4): 0.5888

TOP FEATURES BY GAIN (final model)
  meannz28                51.1%
```

```
  mean28                 17.4%
  mean14                 13.6%
  sw_mean                 4.3%
  mean7                   3.6%
  mean112                 1.7%
  mean56                  1.2%
  lag364                  1.2%
  ctx_item_ratio7_28      0.5%
  ctx_group_dow           0.4%
  std112                  0.4%
  event1                  0.4%
  days_since_sale         0.3%
  ctx_item_mean28         0.3%
  wday                    0.3%
  price                   0.3%
  price_mean365_ratio     0.3%
  sw_lag1                 0.3%
  last1                   0.2%
  store                   0.2%

Runtime: 10.0 min
```

results/01_baselines_and_early_models/results_chronos2_final_window.txt

```
CHRONOS-2 ON M5 (zero-shot ablation ladder)
Data folder: m5_data
Device: cuda: NVIDIA GeForce RTX 3090 | model: amazon/chronos-2
Context length: 1024 days | cross-learning tasks per batch: about 100
Training window: d_1 to d_1941 | Holdout (28 days): d_1942 to d_1969
Point forecast: mean implied by the quantiles (primary, fixed in advance); median also reported.
No training on M5 was done. Whether M5 or similar retail data was in the Chronos-2 pretraining corpus
is not checked here and should be addressed in the limitations section.

LADDER (WRMSSE, lower is better; validation window is d_1914 to d_1941)
Config                        Holdout mean  Holdout median  Validation mean    Min
A_univariate                        0.6525          1.7639           0.7501    3.8
B_covariates                        0.6528          1.7277           0.6991   11.0
C_crosslearn_store_dept             0.6320          1.8192           0.7104    7.0

WRMSSE BY LEVEL (holdout, mean point)
Level Name                    Series   SeasonalNaive    A_univariate    B_covariates C_cl_store_dept
-------------------------------------------------------------------------------------------------
1     Total                        1          0.5600          0.3875          0.3832          0.3678
2     State                        3          0.6731          0.4819          0.4781          0.4404
3     Store                       10          0.7176          0.5593          0.5607          0.5223
4     Category                     3          0.6230          0.4497          0.4402          0.4345
5     Department                   7          0.7082          0.5437          0.5254          0.5215
6     State x Category             9          0.7600          0.5604          0.5581          0.5281
7     State x Department          21          0.8292          0.6492          0.6406          0.6180
8     Store x Category            30          0.8008          0.6298          0.6334          0.6034
9     Store x Department          70          0.8881          0.7175          0.7162          0.6915
10    Item                      3049          1.2233          0.9975          1.0171          1.0010
11    Item x State              9147          1.2045          0.9516          0.9666          0.9526
12    Item x Store             30490          1.1764          0.9014          0.9143          0.9026
-------------------------------------------------------------------------------------------------
      OVERALL WRMSSE           42840          0.8470          0.6525          0.6528          0.6320

REFERENCE VALUES (published, window to be confirmed)
  Original M5 winner: 0.520 | Yang et al. Top-1 reproduction: 0.5230 | Yang et al. LightGBM + PatchTST: 0.4989
  Our E1b L3 (LightGBM, unweighted, all features): 0.5422

Runtime: 22.9 min
```

results/01_baselines_and_early_models/results_chronos2_public_window.txt

```
CHRONOS-2 ON M5 (zero-shot ablation ladder)
Data folder: m5_data
Device: cuda: NVIDIA GeForce RTX 3090 | model: amazon/chronos-2
Context length: 1024 days | cross-learning tasks per batch: about 100
Training window: d_1 to d_1913 | Holdout (28 days): d_1914 to d_1941
Point forecast: mean implied by the quantiles (primary, fixed in advance); median also reported.
No training on M5 was done. Whether M5 or similar retail data was in the Chronos-2 pretraining corpus
is not checked here and should be addressed in the limitations section.

LADDER (WRMSSE, lower is better; validation window is d_1886 to d_1913)
Config                        Holdout mean  Holdout median  Validation mean    Min
A_univariate                        0.7501          1.9437           0.7331    1.5
B_covariates                        0.6991          1.8931           0.6921    7.6
C_crosslearn_store_dept             0.7104          2.0210           0.7406    6.9
C_crosslearn_store_cat              0.7093          2.0192           0.7410    7.0
C_crosslearn_store                  0.7096          2.0199           0.7415    7.0
C_crosslearn_item                   0.7046          2.0017           0.7065    7.3
C_crosslearn_cat_state              0.7096          2.0199           0.7411    7.0

WRMSSE BY LEVEL (holdout, mean point)
Level Name                    Series   SeasonalNaive    A_univariate    B_covariates C_cl_store_dept  C_cl_store_cat      C_cl_store       C_cl_item  C_cl_cat_s
tate
-------------------------------------------------------------------------------------------------------------------------------------------------------------------
----
1     Total                        1          0.7514          0.6663          0.5745          0.6135          0.6116          0.6122          0.5902          0.
6122
2     State                        3          0.7544          0.6776          0.5958          0.6122          0.6105          0.6111          0.6110          0.
6110
3     Store                       10          0.7993          0.7127          0.6482          0.6537          0.6529          0.6525          0.6570          0.
6531
4     Category                     3          0.7620          0.6886          0.5990          0.6380          0.6363          0.6369          0.6158          0.
6369
5     Department                   7          0.8171          0.7594          0.6878          0.7139          0.7124          0.7129          0.6919          0.
7127
6     State x Category             9          0.7830          0.7045          0.6281          0.6458          0.6442          0.6449          0.6433          0.
6447
7     State x Department          21          0.8304          0.7608          0.7013          0.7099          0.7086          0.7092          0.7061          0.
7089
```

```
8     Store x Category            30          0.8393          0.7348          0.6785          0.6859          0.6851          0.6849          0.6877          0.
6852
9     Store x Department          70          0.8919          0.7880          0.7441          0.7461          0.7456          0.7454          0.7473          0.
7456
10    Item                      3049          1.0322          0.8360          0.8440          0.8351          0.8348          0.8347          0.8336          0.
8349
11    Item x State              9147          1.0750          0.8375          0.8449          0.8353          0.8349          0.8350          0.8360          0.
8352
12    Item x Store             30490          1.1004          0.8348          0.8431          0.8353          0.8351          0.8351          0.8347          0.
8352
-------------------------------------------------------------------------------------------------------------------------------------------------------------------
----
      OVERALL WRMSSE           42840          0.8697          0.7501          0.6991          0.7104          0.7093          0.7096          0.7046          0.
7096

REFERENCE VALUES (published, window to be confirmed)
  Original M5 winner: 0.520 | Yang et al. Top-1 reproduction: 0.5230 | Yang et al. LightGBM + PatchTST: 0.4989
  Our E1b L3 (LightGBM, unweighted, all features): 0.5422

Runtime: 45.0 min
```

results/01_baselines_and_early_models/results_patchtst_final_window.txt

```
PATCHTST ON M5 (global model trained on M5)
Data folder: m5_data
Device: cuda: NVIDIA GeForce RTX 3090 | steps per model: 12000 | seed: 0
Settings (fixed in advance, not tuned on the holdout): context=224, patch_len=16, stride=8, d_model=128, heads=8, ff=256, layers=3, dropout=0.1, head_hidden=512
, lr=0.0005, wd=0.01, batch=512, train_span=1100
Training window: d_1 to d_1941 | Holdout (28 days): d_1942 to d_1969
Validation models are trained separately on data up to d_1913 and forecast d_1914 to d_1941.

LADDER (WRMSSE, lower is better)
Config                Holdout  Validation    Min
P0_univariate          0.7721      0.8962    8.0
P1_covariates          0.6613      0.7588    8.4

TRAINING CURVES (step:smoothed loss, holdout model)
  P0_univariate: 1200:15.6879, 2400:22.3370, 3600:13.4789, 4800:20.6656, 6000:16.9809, 7200:18.5904, 8400:18.3979, 9600:15.6432, 10800:16.1868, 12000:17.2715
  P1_covariates: 1200:15.6768, 2400:22.3563, 3600:13.4827, 4800:20.6997, 6000:16.9654, 7200:18.5494, 8400:18.3755, 9600:15.6083, 10800:16.1846, 12000:17.2457

WRMSSE BY LEVEL (holdout)
Level Name                   Series   SeasonalNaive   P0_univariate   P1_covariates
-------------------------------------------------------------------------------------
1     Total                       1          0.5600          0.6194          0.3752
2     State                       3          0.6731          0.6636          0.4630
3     Store                      10          0.7176          0.7000          0.5852
4     Category                    3          0.6230          0.6485          0.4477
5     Department                  7          0.7082          0.7430          0.5795
6     State x Category            9          0.7600          0.7110          0.5583
7     State x Department         21          0.8292          0.7821          0.6671
8     Store x Category           30          0.8008          0.7397          0.6538
9     Store x Department         70          0.8881          0.8089          0.7448
10    Item                     3049          1.2233          1.0054          1.0091
11    Item x State             9147          1.2045          0.9508          0.9549
12    Item x Store            30490          1.1764          0.8934          0.8973
-------------------------------------------------------------------------------------
      OVERALL WRMSSE          42840          0.8470          0.7721          0.6613

REFERENCE VALUES (published, window to be confirmed)
  Original M5 winner: 0.520 | Yang et al. Top-1 reproduction: 0.5230 | Yang et al. LightGBM + PatchTST: 0.4989
  Our E1b L3 (LightGBM, unweighted, all features): 0.5422

Runtime: 17.2 min
```

results/01_baselines_and_early_models/results_patchtst_public_window.txt

```
PATCHTST ON M5 (global model trained on M5)
Data folder: m5_data
Device: cuda: NVIDIA GeForce RTX 3090 | steps per model: 12000 | seed: 0
Settings (fixed in advance, not tuned on the holdout): context=224, patch_len=16, stride=8, d_model=128, heads=8, ff=256, layers=3, dropout=0.1, head_hidden=512
, lr=0.0005, wd=0.01, batch=512, train_span=1100
Training window: d_1 to d_1913 | Holdout (28 days): d_1914 to d_1941
Validation models are trained separately on data up to d_1885 and forecast d_1886 to d_1913.

LADDER (WRMSSE, lower is better)
Config                Holdout  Validation    Min
P0_univariate          0.8962      0.8507    7.5
P1_covariates          0.7460      0.6793    7.9

TRAINING CURVES (step:smoothed loss, holdout model)
  P0_univariate: 1200:25.0059, 2400:15.2808, 3600:16.2736, 4800:14.5140, 6000:18.4790, 7200:18.4036, 8400:25.7132, 9600:13.0851, 10800:20.3799, 12000:15.5654
  P1_covariates: 1200:24.9975, 2400:15.2124, 3600:16.2674, 4800:14.4815, 6000:18.4764, 7200:18.4332, 8400:25.6355, 9600:13.0510, 10800:20.3599, 12000:15.5269

WRMSSE BY LEVEL (holdout)
Level Name                   Series   SeasonalNaive   P0_univariate   P1_covariates
-------------------------------------------------------------------------------------
1     Total                       1          0.7514          0.9052          0.6055
2     State                       3          0.7544          0.8958          0.6562
3     Store                      10          0.7993          0.9076          0.7389
4     Category                    3          0.7620          0.9068          0.6494
5     Department                  7          0.8171          0.9476          0.7248
6     State x Category            9          0.7830          0.9004          0.7041
7     State x Department         21          0.8304          0.9264          0.7615
8     Store x Category           30          0.8393          0.8984          0.7701
9     Store x Department         70          0.8919          0.9146          0.8183
10    Item                     3049          1.0322          0.8651          0.8474
11    Item x State             9147          1.0750          0.8500          0.8422
12    Item x Store            30490          1.1004          0.8368          0.8342
-------------------------------------------------------------------------------------
      OVERALL WRMSSE          42840          0.8697          0.8962          0.7460

REFERENCE VALUES (published, window to be confirmed)
  Original M5 winner: 0.520 | Yang et al. Top-1 reproduction: 0.5230 | Yang et al. LightGBM + PatchTST: 0.4989
```

```
  Our E1b L3 (LightGBM, unweighted, all features): 0.5422

Runtime: 16.0 min
```

results/01_baselines_and_early_models/results_phase1_baselines.txt

```
M5 DATA AND WRMSSE PIPELINE RESULTS (Phase 1 and 2)
Data folder: m5_data
Bottom series: 30490 | Days: 1941
Training window: d_1 to d_1913
Holdout window (28 days): d_1914 to d_1941
Weight window: last 28 training days, dollar sales (units x price)
Overall WRMSSE = mean of the 12 level scores

SANITY CHECKS
All sanity checks passed (weights sum to 1 per level, perfect forecast = 0, 42,840 series across 12 levels).

WRMSSE BY LEVEL
Level Name                     Series           Naive   SeasonalNaive
-----------------------------------------------------------------------
1     Total                         1          1.4879          0.7514
2     State                         3          1.5172          0.7544
3     Store                        10          1.5346          0.7993
4     Category                      3          1.5200          0.7620
5     Department                    7          1.6055          0.8171
6     State x Category              9          1.5405          0.7830
7     State x Department           21          1.6143          0.8304
8     Store x Category             30          1.5388          0.8393
9     Store x Department           70          1.5616          0.8919
10    Item                       3049          1.2708          1.0322
11    Item x State               9147          1.2107          1.0750
12    Item x Store              30490          1.1645          1.1004
-----------------------------------------------------------------------
      OVERALL WRMSSE            42840          1.4639          0.8697

Runtime: 10.4 s
```

results/02_winner_reproduction/PATCH_LOG_cv4.txt

```
1. preprocessing/1. preprocessing.py: directory from environment x1
1. preprocessing/1. preprocessing.py: .dt.week -> isocalendar().week x1
1. preprocessing/1. preprocessing.py: chained assignment (pandas 3) x1
2. train/1-1. recursive_store_TRAIN.py: directory from environment x1
2. train/1-1. recursive_store_TRAIN.py: KKK 0 -> 2 x1
2. train/1-1. recursive_store_TRAIN.py: verbose_eval -> callbacks (LightGBM 4) x1
2. train/1-1. recursive_store_TRAIN.py: writable copy (pandas 3) x1
2. train/1-2. recursive_store_cat_TRAIN.py: directory from environment x1
2. train/1-2. recursive_store_cat_TRAIN.py: KKK 0 -> 2 x1
2. train/1-2. recursive_store_cat_TRAIN.py: verbose_eval -> callbacks (LightGBM 4) x1
2. train/1-2. recursive_store_cat_TRAIN.py: writable copy (pandas 3) x1
2. train/1-3. recursive_store_dept_TRAIN.py: directory from environment x1
2. train/1-3. recursive_store_dept_TRAIN.py: KKK 0 -> 2 x1
2. train/1-3. recursive_store_dept_TRAIN.py: verbose_eval -> callbacks (LightGBM 4) x1
2. train/1-3. recursive_store_dept_TRAIN.py: writable copy (pandas 3) x1
2. train/2-1. nonrecursive_store_TRAIN.py: directory from environment x1
2. train/2-1. nonrecursive_store_TRAIN.py: cv private -> cv4 x1
2. train/2-1. nonrecursive_store_TRAIN.py: verbose_eval -> callbacks (LightGBM 4) x1
2. train/2-2. nonrecursive_store_cat_TRAIN.py: directory from environment x1
2. train/2-2. nonrecursive_store_cat_TRAIN.py: cv private -> cv4 x1
2. train/2-2. nonrecursive_store_cat_TRAIN.py: verbose_eval -> callbacks (LightGBM 4) x1
2. train/2-3. nonrecursive_store_dept_TRAIN.py: directory from environment x1
2. train/2-3. nonrecursive_store_dept_TRAIN.py: cv private -> cv4 x1
2. train/2-3. nonrecursive_store_dept_TRAIN.py: verbose_eval -> callbacks (LightGBM 4) x1
3. predict/1-1. recursive_store_PREDICT.py: directory from environment x1
3. predict/1-1. recursive_store_PREDICT.py: KKK 0 -> 2 x1
3. predict/1-1. recursive_store_PREDICT.py: chained assignment (pandas 3) x1
3. predict/1-2. recursive_store_cat_PREDICT.py: directory from environment x1
3. predict/1-2. recursive_store_cat_PREDICT.py: KKK 0 -> 2 x1
3. predict/1-2. recursive_store_cat_PREDICT.py: chained assignment (pandas 3) x1
3. predict/1-3. recursive_store_dept_PREDICT.py: directory from environment x1
3. predict/1-3. recursive_store_dept_PREDICT.py: KKK 0 -> 2 x1
3. predict/1-3. recursive_store_dept_PREDICT.py: chained assignment (pandas 3) x1
3. predict/2-1. nonrecursive_store_PREDICT.py: directory from environment x1
3. predict/2-1. nonrecursive_store_PREDICT.py: cv private -> cv4 x1
3. predict/2-2. nonrecursive_store_cat_PREDICT.py: directory from environment x1
3. predict/2-2. nonrecursive_store_cat_PREDICT.py: cv private -> cv4 x1
3. predict/2-3. nonrecursive_store_dept_PREDICT.py: directory from environment x1
3. predict/2-3. nonrecursive_store_dept_PREDICT.py: cv private -> cv4 x1
3. predict/3-1. Final ensemble.py: directory from environment x1
```

results/02_winner_reproduction/PATCH_LOG_final.txt

```
1. preprocessing/1. preprocessing.py: directory from environment x1
1. preprocessing/1. preprocessing.py: .dt.week -> isocalendar().week x1
1. preprocessing/1. preprocessing.py: chained assignment (pandas 3) x1
2. train/1-1. recursive_store_TRAIN.py: directory from environment x1
2. train/1-1. recursive_store_TRAIN.py: verbose_eval -> callbacks (LightGBM 4) x1
2. train/1-1. recursive_store_TRAIN.py: writable copy (pandas 3) x1
2. train/1-2. recursive_store_cat_TRAIN.py: directory from environment x1
2. train/1-2. recursive_store_cat_TRAIN.py: verbose_eval -> callbacks (LightGBM 4) x1
2. train/1-2. recursive_store_cat_TRAIN.py: writable copy (pandas 3) x1
2. train/1-3. recursive_store_dept_TRAIN.py: directory from environment x1
2. train/1-3. recursive_store_dept_TRAIN.py: verbose_eval -> callbacks (LightGBM 4) x1
2. train/1-3. recursive_store_dept_TRAIN.py: writable copy (pandas 3) x1
2. train/2-1. nonrecursive_store_TRAIN.py: directory from environment x1
2. train/2-1. nonrecursive_store_TRAIN.py: verbose_eval -> callbacks (LightGBM 4) x1
2. train/2-2. nonrecursive_store_cat_TRAIN.py: directory from environment x1
2. train/2-2. nonrecursive_store_cat_TRAIN.py: verbose_eval -> callbacks (LightGBM 4) x1
2. train/2-3. nonrecursive_store_dept_TRAIN.py: directory from environment x1
2. train/2-3. nonrecursive_store_dept_TRAIN.py: verbose_eval -> callbacks (LightGBM 4) x1
```

```
3. predict/1-1. recursive_store_PREDICT.py: directory from environment x1
3. predict/1-1. recursive_store_PREDICT.py: chained assignment (pandas 3) x1
3. predict/1-2. recursive_store_cat_PREDICT.py: directory from environment x1
3. predict/1-2. recursive_store_cat_PREDICT.py: chained assignment (pandas 3) x1
3. predict/1-3. recursive_store_dept_PREDICT.py: directory from environment x1
3. predict/1-3. recursive_store_dept_PREDICT.py: chained assignment (pandas 3) x1
3. predict/2-1. nonrecursive_store_PREDICT.py: directory from environment x1
3. predict/2-2. nonrecursive_store_cat_PREDICT.py: directory from environment x1
3. predict/2-3. nonrecursive_store_dept_PREDICT.py: directory from environment x1
3. predict/3-1. Final ensemble.py: directory from environment x1
LEAN PREPROCESSING (1. preprocessing.py replaced by hand, same outputs, verified):
- index columns converted to category before the melt (no 60M-row string columns)
- lag and rolling features written into one float16 matrix, computed in chunks of 3000 series
- week-of-month computed as (day+6)//7 (identical to ceil(day/7))
- full organiser release accepted: id column and calendar d column are rebuilt when missing
- verified on synthetic data (3000 series, both layouts): all five output pickles identical to the original
```

results/02_winner_reproduction/PATCH_LOG_public.txt

```
1. preprocessing/1. preprocessing.py: directory from environment x1
1. preprocessing/1. preprocessing.py: .dt.week -> isocalendar().week x1
1. preprocessing/1. preprocessing.py: chained assignment (pandas 3) x1
2. train/1-1. recursive_store_TRAIN.py: directory from environment x1
2. train/1-1. recursive_store_TRAIN.py: KKK 0 -> 1 x1
2. train/1-1. recursive_store_TRAIN.py: verbose_eval -> callbacks (LightGBM 4) x1
2. train/1-1. recursive_store_TRAIN.py: writable copy (pandas 3) x1
2. train/1-2. recursive_store_cat_TRAIN.py: directory from environment x1
2. train/1-2. recursive_store_cat_TRAIN.py: KKK 0 -> 1 x1
2. train/1-2. recursive_store_cat_TRAIN.py: verbose_eval -> callbacks (LightGBM 4) x1
2. train/1-2. recursive_store_cat_TRAIN.py: writable copy (pandas 3) x1
2. train/1-3. recursive_store_dept_TRAIN.py: directory from environment x1
2. train/1-3. recursive_store_dept_TRAIN.py: KKK 0 -> 1 x1
2. train/1-3. recursive_store_dept_TRAIN.py: verbose_eval -> callbacks (LightGBM 4) x1
2. train/1-3. recursive_store_dept_TRAIN.py: writable copy (pandas 3) x1
2. train/2-1. nonrecursive_store_TRAIN.py: directory from environment x1
2. train/2-1. nonrecursive_store_TRAIN.py: cv private -> public x1
2. train/2-1. nonrecursive_store_TRAIN.py: verbose_eval -> callbacks (LightGBM 4) x1
2. train/2-2. nonrecursive_store_cat_TRAIN.py: directory from environment x1
2. train/2-2. nonrecursive_store_cat_TRAIN.py: cv private -> public x1
2. train/2-2. nonrecursive_store_cat_TRAIN.py: verbose_eval -> callbacks (LightGBM 4) x1
2. train/2-3. nonrecursive_store_dept_TRAIN.py: directory from environment x1
2. train/2-3. nonrecursive_store_dept_TRAIN.py: cv private -> public x1
2. train/2-3. nonrecursive_store_dept_TRAIN.py: verbose_eval -> callbacks (LightGBM 4) x1
3. predict/1-1. recursive_store_PREDICT.py: directory from environment x1
3. predict/1-1. recursive_store_PREDICT.py: KKK 0 -> 1 x1
3. predict/1-1. recursive_store_PREDICT.py: chained assignment (pandas 3) x1
3. predict/1-2. recursive_store_cat_PREDICT.py: directory from environment x1
3. predict/1-2. recursive_store_cat_PREDICT.py: KKK 0 -> 1 x1
3. predict/1-2. recursive_store_cat_PREDICT.py: chained assignment (pandas 3) x1
3. predict/1-3. recursive_store_dept_PREDICT.py: directory from environment x1
3. predict/1-3. recursive_store_dept_PREDICT.py: KKK 0 -> 1 x1
3. predict/1-3. recursive_store_dept_PREDICT.py: chained assignment (pandas 3) x1
3. predict/2-1. nonrecursive_store_PREDICT.py: directory from environment x1
3. predict/2-1. nonrecursive_store_PREDICT.py: cv private -> public x1
3. predict/2-2. nonrecursive_store_cat_PREDICT.py: directory from environment x1
3. predict/2-2. nonrecursive_store_cat_PREDICT.py: cv private -> public x1
3. predict/2-3. nonrecursive_store_dept_PREDICT.py: directory from environment x1
3. predict/2-3. nonrecursive_store_dept_PREDICT.py: cv private -> public x1
3. predict/3-1. Final ensemble.py: directory from environment x1
```

results/02_winner_reproduction/STATUS_cv3_cv2_cv5.txt

```
[2026-10-14 20:48:55] cv3: free disk 685 GB (need 55)
[2026-10-14 20:48:55] cv3: start (six sets, about 11 hours)
[2026-10-15 07:47:53] cv3: finished with exit code 0 after 11.0 h
[2026-10-15 07:47:53] cv3: result files copied to C:\A1_extra\send
[2026-10-15 07:47:53] zip written: C:\A1_extra\send_pc3_windows.zip
[2026-10-15 07:47:53] cv2: free disk 650 GB (need 55)
[2026-10-15 07:47:53] cv2: start (six sets, about 11 hours)
[2026-10-15 18:46:18] cv2: finished with exit code 0 after 11.0 h
[2026-10-15 18:46:18] cv2: result files copied to C:\A1_extra\send
[2026-10-15 18:46:18] zip written: C:\A1_extra\send_pc3_windows.zip
[2026-10-15 18:46:18] cv5: free disk 615 GB (need 55)
[2026-10-15 18:46:18] cv5: start (six sets, about 11 hours)
[2026-10-16 05:33:25] cv5: finished with exit code 0 after 10.8 h
[2026-10-16 05:33:25] cv5: result files copied to C:\A1_extra\send
```

results/02_winner_reproduction/STATUS_cv4.txt

```
CV4 STATUS - M5 winner (A1) on validation fold cv4 (train to d_1885, forecast d_1886..d_1913)
Created: 2026-10-13 16:13

SETUP
  Package: A1_cv4_pkg.zip (52,989 bytes, 2026-10-13 16:12) from the project folder, unzipped into C:\A1_cv4 -> C:\A1_cv4\pkg_cv4
  Python: existing venv C:\A1_lean\venv\Scripts\python.exe (nothing created or installed)
  Monitor: existing C:\A1_lean\monitor_a1.py (only executed; nothing in C:\A1_lean or C:\A1_public changed)
  Data in C:\A1_cv4\data (copied, not moved, from C:\A1_lean\m5_data); sales_test_evaluation.csv NOT copied:
  calendar.csv                      112,477 bytes
  sales_train_evaluation.csv     121,168,898 bytes
  sample_submission.csv            5,228,786 bytes
  sell_prices.csv                237,073,689 bytes
  NOTE: sample_submission.csv was added to the three requested files. The predict scripts read it, and the driver's
        fallback that rebuilds it needs an 'id' column that this sales file lacks, so call a would fail without it.
        It contains only series ids and zeros (no sales), so it leaks nothing about d_1886..d_1913. No code changed.
  Package note (from README_CV4.txt, not a change to the model): target encodings rebuilt on days up to d_1885.
  Free RAM at start: 20.5 GB; free disk C: 672.7 GB

ORCHESTRATION
  C:\A1_cv4\orchestrate_cv4.py runs the requested calls one after another, never two at once,
  all with --window cv4 --data_dir C:\A1_cv4\data. Started now with call a ONLY (nr_store); b, c, d wait for the user.
```

```
  Logs: C:\A1_cv4\run_logs (driver_<call>.txt, monitor_<call>.txt); step logs in pkg_cv4\run_logs (driver's work folder).
  Every 20 min a status line is appended here. After each finished call, copies go to <project>\m5_outputs_final as
  <call>__predcv4_A1_<set>.npy, <call>__predcv4_A1_winner.npy, <call>__results_winner_A1_cv4.txt and
  CV4_STATUS_after_<call>.txt (a _2 suffix is added if a name already exists).

LOG
2026-10-13 16:13:22 orchestrator start (pid 13240); calls in order: cv4_a_nr_store
2026-10-13 16:13:22 cv4_a_nr_store: started (--sets nr_store), driver pid 42440, log run_logs\driver_cv4_a_nr_store.txt
2026-10-13 16:33:22 cv4_a_nr_store: elapsed 20.0 min | free RAM 17.25 GB | nr_store: train (2-1. nonrecursive_store_TRAIN.py) ...
2026-10-13 16:53:22 cv4_a_nr_store: elapsed 40.0 min | free RAM 17.09 GB | nr_store: train (2-1. nonrecursive_store_TRAIN.py) ...
2026-10-13 17:13:22 cv4_a_nr_store: elapsed 60.0 min | free RAM 17.08 GB | nr_store: train (2-1. nonrecursive_store_TRAIN.py) ...
2026-10-13 17:29:52 cv4_a_nr_store: FINISHED (exit 0) in 76.5 min
    nr_store           0.5496   0.970
    average            0.5496   0.970
    copied to m5_outputs_final: cv4_a_nr_store__predcv4_A1_nr_store.npy, cv4_a_nr_store__predcv4_A1_winner.npy, cv4_a_nr_store__results_winner_A1_cv4.txt
2026-10-13 17:29:52 orchestrator: requested calls finished (cv4_a_nr_store)
2026-10-13 22:48:47 orchestrator start (pid 32272); calls in order: cv4_b_nr_store_cat_dept, cv4_c_r_store_cat_dept, cv4_d_r_store
2026-10-13 22:48:47 cv4_b_nr_store_cat_dept: started (--sets nr_store_cat,nr_store_dept), driver pid 22216, log run_logs\driver_cv4_b_nr_store_cat_dept.txt
2026-10-13 23:08:47 cv4_b_nr_store_cat_dept: elapsed 20.0 min | free RAM 17.65 GB | nr_store_cat: train (2-2. nonrecursive_store_cat_TRAIN.py) ...
2026-10-13 23:28:47 cv4_b_nr_store_cat_dept: elapsed 40.0 min | free RAM 17.82 GB | nr_store_cat: train (2-2. nonrecursive_store_cat_TRAIN.py) ...
2026-10-13 23:48:47 cv4_b_nr_store_cat_dept: elapsed 60.0 min | free RAM 17.79 GB | nr_store_cat: train (2-2. nonrecursive_store_cat_TRAIN.py) ...
2026-10-14 00:08:47 cv4_b_nr_store_cat_dept: elapsed 80.0 min | free RAM 18.05 GB | nr_store_dept: train (2-3. nonrecursive_store_dept_TRAIN.py) ...
2026-10-14 00:28:47 cv4_b_nr_store_cat_dept: elapsed 100.0 min | free RAM 18.07 GB | nr_store_dept: train (2-3. nonrecursive_store_dept_TRAIN.py) ...
2026-10-14 00:48:47 cv4_b_nr_store_cat_dept: elapsed 120.0 min | free RAM 18.00 GB | nr_store_dept: train (2-3. nonrecursive_store_dept_TRAIN.py) ...
2026-10-14 01:08:47 cv4_b_nr_store_cat_dept: elapsed 140.0 min | free RAM 18.00 GB | nr_store_dept: train (2-3. nonrecursive_store_dept_TRAIN.py) ...
2026-10-14 01:28:47 cv4_b_nr_store_cat_dept: elapsed 160.0 min | free RAM 17.97 GB | nr_store_dept: train (2-3. nonrecursive_store_dept_TRAIN.py) ...
2026-10-14 01:48:47 cv4_b_nr_store_cat_dept: elapsed 180.0 min | free RAM 18.03 GB | nr_store_dept: predict (2-3. nonrecursive_store_dept_PREDICT.py) ...
2026-10-14 01:53:47 cv4_b_nr_store_cat_dept: FINISHED (exit 0) in 185.0 min
    nr_store_cat       0.5863   0.958
    nr_store_dept      0.6514   0.938
    average            0.5887   0.955
    copied to m5_outputs_final: cv4_b_nr_store_cat_dept__predcv4_A1_nr_store_cat.npy, cv4_b_nr_store_cat_dept__predcv4_A1_nr_store_dept.npy, cv4_b_nr_store_cat_
dept__predcv4_A1_winner.npy, cv4_b_nr_store_cat_dept__results_winner_A1_cv4.txt
2026-10-14 01:53:47 cv4_c_r_store_cat_dept: started (--sets r_store_cat,r_store_dept), driver pid 41508, log run_logs\driver_cv4_c_r_store_cat_dept.txt
2026-10-14 02:13:47 cv4_c_r_store_cat_dept: elapsed 20.0 min | free RAM 17.84 GB | r_store_cat: train (1-2. recursive_store_cat_TRAIN.py) ...
2026-10-14 02:33:47 cv4_c_r_store_cat_dept: elapsed 40.0 min | free RAM 18.01 GB | r_store_cat: train (1-2. recursive_store_cat_TRAIN.py) ...
2026-10-14 02:53:47 cv4_c_r_store_cat_dept: elapsed 60.0 min | free RAM 17.85 GB | r_store_cat: train (1-2. recursive_store_cat_TRAIN.py) ...
2026-10-14 03:13:47 cv4_c_r_store_cat_dept: elapsed 80.0 min | free RAM 18.08 GB | r_store_cat: predict (1-2. recursive_store_cat_PREDICT.py) ...
2026-10-14 03:33:47 cv4_c_r_store_cat_dept: elapsed 100.0 min | free RAM 16.80 GB | r_store_cat: predict (1-2. recursive_store_cat_PREDICT.py) ...
2026-10-14 03:53:47 cv4_c_r_store_cat_dept: elapsed 120.0 min | free RAM 18.15 GB | r_store_dept: train (1-3. recursive_store_dept_TRAIN.py) ...
2026-10-14 04:13:48 cv4_c_r_store_cat_dept: elapsed 140.0 min | free RAM 18.15 GB | r_store_dept: train (1-3. recursive_store_dept_TRAIN.py) ...
2026-10-14 04:33:48 cv4_c_r_store_cat_dept: elapsed 160.0 min | free RAM 18.00 GB | r_store_dept: train (1-3. recursive_store_dept_TRAIN.py) ...
2026-10-14 04:53:48 cv4_c_r_store_cat_dept: elapsed 180.0 min | free RAM 17.92 GB | r_store_dept: train (1-3. recursive_store_dept_TRAIN.py) ...
2026-10-14 05:13:48 cv4_c_r_store_cat_dept: elapsed 200.0 min | free RAM 17.97 GB | r_store_dept: train (1-3. recursive_store_dept_TRAIN.py) ...
2026-10-14 05:33:48 cv4_c_r_store_cat_dept: elapsed 220.0 min | free RAM 16.77 GB | r_store_dept: predict (1-3. recursive_store_dept_PREDICT.py) ...
2026-10-14 05:53:48 cv4_c_r_store_cat_dept: elapsed 240.0 min | free RAM 17.02 GB | r_store_dept: predict (1-3. recursive_store_dept_PREDICT.py) ...
2026-10-14 06:13:48 cv4_c_r_store_cat_dept: elapsed 260.0 min | free RAM 16.71 GB | r_store_dept: predict (1-3. recursive_store_dept_PREDICT.py) ...
2026-10-14 06:14:48 cv4_c_r_store_cat_dept: FINISHED (exit 0) in 261.0 min
    r_store_cat        0.6083   1.051
    r_store_dept       0.5584   1.023
    average            0.5264   0.988
    copied to m5_outputs_final: cv4_c_r_store_cat_dept__predcv4_A1_r_store_cat.npy, cv4_c_r_store_cat_dept__predcv4_A1_r_store_dept.npy, cv4_c_r_store_cat_dept_
_predcv4_A1_winner.npy, cv4_c_r_store_cat_dept__results_winner_A1_cv4.txt
2026-10-14 06:14:48 cv4_d_r_store: started (--sets r_store), driver pid 5996, log run_logs\driver_cv4_d_r_store.txt
2026-10-14 06:34:48 cv4_d_r_store: elapsed 20.0 min | free RAM 16.20 GB | r_store: train (1-1. recursive_store_TRAIN.py) ...
2026-10-14 06:54:48 cv4_d_r_store: elapsed 40.0 min | free RAM 16.22 GB | r_store: train (1-1. recursive_store_TRAIN.py) ...
2026-10-14 07:14:48 cv4_d_r_store: elapsed 60.0 min | free RAM 16.27 GB | r_store: train (1-1. recursive_store_TRAIN.py) ...
2026-10-14 07:34:48 cv4_d_r_store: elapsed 80.0 min | free RAM 15.94 GB | r_store: train (1-1. recursive_store_TRAIN.py) ...
2026-10-14 07:54:48 cv4_d_r_store: elapsed 100.0 min | free RAM 16.13 GB | r_store: train (1-1. recursive_store_TRAIN.py) ...
2026-10-14 08:14:48 cv4_d_r_store: elapsed 120.0 min | free RAM 17.10 GB | r_store: predict (1-1. recursive_store_PREDICT.py) ...
2026-10-14 08:34:48 cv4_d_r_store: elapsed 140.0 min | free RAM 16.60 GB | r_store: predict (1-1. recursive_store_PREDICT.py) ...
2026-10-14 08:45:03 cv4_d_r_store: FINISHED (exit 0) in 150.3 min
    r_store            0.6504   1.073
    r_store_cat        0.6083   1.051
    r_store_dept       0.5584   1.023
    average            0.5167   1.002
    copied to m5_outputs_final: cv4_d_r_store__predcv4_A1_r_store.npy, cv4_d_r_store__predcv4_A1_winner.npy, cv4_d_r_store__results_winner_A1_cv4.txt
2026-10-14 08:45:03 orchestrator: requested calls finished (cv4_b_nr_store_cat_dept, cv4_c_r_store_cat_dept, cv4_d_r_store)
```

results/02_winner_reproduction/STATUS_final.txt

```
PC3 STATUS - M5 winner (A1) lean package
Last updated: 2026-10-09 04:32 (ALL 6 SETS FINISHED - see SUMMARY at the end)

SETUP
  Package: A1_lean_pkg.zip (51,339 bytes, 2026-10-08 16:43) from the project folder, unzipped into C:\A1_lean
           (zip has a top folder A1_lean_pkg\; its contents were copied up to C:\A1_lean so run_winner_a1.py runs from there;
            the extracted A1_lean_pkg\ folder is kept as is)
  Data copied (not moved) from <project>\m5_data to C:\A1_lean\m5_data:
  calendar.csv                       112,477 bytes
  sales_test_evaluation.csv        3,260,569 bytes
  sales_train_evaluation.csv     121,168,898 bytes
  sample_submission.csv            5,228,786 bytes
  sell_prices.csv                237,073,689 bytes
  sales_train_evaluation.csv: 1946 columns, last day column d_1941 (as expected); no 'id' column (organiser layout, lean package handles this)
  sales_test_evaluation.csv: d_1942 .. d_1969

MACHINE
  CPU: 13th Gen Intel Core i7-13700, 16 cores / 24 logical processors
  RAM: 31.7 GB total, 19.5 GB free at setup
  Disk C: 733 GB free (need 40 GB) - OK

PYTHON
  Base: C:\ProgramData\anaconda3\python.exe (Anaconda base env untouched)
  venv: C:\A1_lean\venv  -> Python 3.13.9 (Anaconda build)
  Installed in venv with pip: pandas 3.0.6, numpy 2.5.3, scipy 1.18.1, lightgbm 4.7.0, psutil 7.2.2,
                              scikit-learn 1.9.1, tqdm 4.70.1
  Fixes applied to package code: none

MONITORING
  C:\A1_lean\monitor_a1.py samples every 30 s the memory of the whole process tree of the run
  (cmd -> venv launcher -> interpreter -> step scripts), free RAM and the current step; log in run_logs\monitor_<set>.txt.
  If free RAM stays under 1 GB for 5 minutes it stops only that process tree and writes why.

RUNS
  nr_store: started 2026-10-08 16:49:41, root pid 24696 (cmd wrapper), monitor pid 26512
```

```
    check 16:54: elapsed 5.0 min | tree RSS 4.80 GB | peak 5.15 GB | free RAM 15.96 GB | preprocessing (lags/rollings done, encoding)
    check 17:00: elapsed    10.5 min | tree RSS   1.89 GB | peak   5.15 GB | free RAM  19.21 GB | nr_store: train (2-1. nonrecursive_store_TRAIN.py) ... || late
st step log nr_store_train.log: [1900]  training's rmse: 2.00757        valid_0's rmse: 3.27147
    preprocessing finished 16:55:35 (about 5.9 min, peak tree RSS 5.15 GB); nr_store train started, store CA_1 at iteration 1900/3000 at 17:00 -> roughly 7 min
per store model, 10 stores
    check 17:08: elapsed    18.5 min | tree RSS   1.84 GB | peak   5.15 GB | free RAM  19.66 GB | nr_store: train (2-1. nonrecursive_store_TRAIN.py) ... || late
st step log nr_store_train.log: [2900]  training's rmse: 1.61304        valid_0's rmse: 3.0232
    check 17:23: elapsed    34.0 min | tree RSS   1.91 GB | peak   5.15 GB | free RAM  19.65 GB | nr_store: train (2-1. nonrecursive_store_TRAIN.py) ... || late
st step log nr_store_train.log: [600]   training's rmse: 1.84637        valid_0's rmse: 2.75406
    check 17:44: elapsed    54.0 min | tree RSS   1.92 GB | peak   5.15 GB | free RAM  19.40 GB | nr_store: train (2-1. nonrecursive_store_TRAIN.py) ... || late
st step log nr_store_train.log: [500]   training's rmse: 1.64144        valid_0's rmse: 2.38218
    check 18:04: elapsed    74.0 min | tree RSS   1.95 GB | peak   5.15 GB | free RAM  19.46 GB | nr_store: train (2-1. nonrecursive_store_TRAIN.py) ... || late
st step log nr_store_predict.log: CA_1 start
    nr_store train finished in 68.3 min (exit 0); predict started
    check 18:09: 2026-10-08 18:07:43 elapsed 78.0 min | PEAK tree RSS 5.15 GB || latest step log nr_store_predict.log: WI_3 start
  nr_store: FINISHED 2026-10-08 18:09 (driver exit OK)
    preprocessing 5.9 min | nr_store train 68.3 min (10 models, ~6.8 min each) | nr_store predict 3.5 min | total 77.8 min
    peak memory of the run (whole process tree, peak during preprocessing): 5.15 GB; free RAM never below ~15 GB
    SCORE (results_winner_A1.txt): nr_store WRMSSE 0.5282, bias 0.987 (seasonal naive 0.8470; published winner 0.520)
    Fixes needed: none

  ESTIMATE for the remaining sets (ESTIMATE, extrapolated from nr_store only; settings read from the train scripts):
    nr_store_cat  : 30 models, same tree settings (255 leaves, 3000 trees), each on ~1/3 of a store's rows
                    -> train ~70-110 min, predict ~5 min                                        ESTIMATE
    nr_store_dept : 70 models, same settings, each on ~1/7 of a store's rows
                    -> train ~80-150 min, predict ~8 min                                        ESTIMATE
    r_store       : 10 models, 2047 leaves (8x nr_store) -> train ~2-4 h; recursive day-by-day predict ~0.5-1.5 h  ESTIMATE
    r_store_cat   : 30 models, 255 leaves -> train ~70-110 min; recursive predict ~0.5-1.5 h     ESTIMATE
    r_store_dept  : 70 models, 255 leaves -> train ~80-150 min; recursive predict ~0.5-1.5 h     ESTIMATE
    Total remaining: roughly 10-17 hours                                                        ESTIMATE
  Note: "nr_store_cat and nr_store_dept together" is run as ONE driver call (--sets nr_store_cat,nr_store_dept), which
        does them one after the other; two parallel runs on the same work folder were judged too risky for RAM.
  nr_store_cat_dept (--sets nr_store_cat,nr_store_dept): started 2026-10-08 18:09:57, root pid 1956 (cmd wrapper, rc 0), monitor pid 28076; driver log run_logs\
driver_nr_store_cat_dept.txt
    check 18:30: elapsed    20.0 min | tree RSS   0.97 GB | peak   2.42 GB | free RAM  20.25 GB | nr_store_cat: train (2-2. nonrecursive_store_cat_TRAIN.py) ...
|| latest step log nr_store_cat_train.log: [2600]       training's rmse: 1.70758        valid_0's rmse: 3.17651
    check 19:00: elapsed    50.0 min | tree RSS   0.75 GB | peak   4.06 GB | free RAM  20.91 GB | nr_store_cat: train (2-2. nonrecursive_store_cat_TRAIN.py) ...
|| latest step log nr_store_cat_train.log: [3000]       training's rmse: 0.935562       valid_0's rmse: 1.58323
    check 19:35: elapsed    85.0 min | tree RSS   1.93 GB | peak   4.20 GB | free RAM  19.25 GB | nr_store_cat: predict (2-2. nonrecursive_store_cat_PREDICT.py)
... || latest step log nr_store_cat_predict.log: WI_3 HOUSEHOLD start
    nr_store_cat train finished in 80.7 min (exit 0, 30 models ~2.7 min each); predict running at 19:35
    check 20:05: elapsed   115.5 min | tree RSS   0.69 GB | peak   4.20 GB | free RAM  20.92 GB | nr_store_dept: train (2-3. nonrecursive_store_dept_TRAIN.py) .
.. || latest step log nr_store_dept_train.log: [1800]   training's rmse: 1.3557 valid_0's rmse: 1.6624
    nr_store_cat predict finished in 4.9 min (exit 0) -> nr_store_cat FINISHED ~19:40 (score comes when this driver call ends); nr_store_dept train started ~19:
40, ~1.2 min per model
    check 20:45: elapsed   155.6 min | tree RSS   0.56 GB | peak   4.20 GB | free RAM  20.97 GB | nr_store_dept: train (2-3. nonrecursive_store_dept_TRAIN.py) .
.. || latest step log nr_store_dept_train.log: WI_1 HOUSEHOLD_1 start
    check 21:15: elapsed   185.6 min | tree RSS   1.00 GB | peak   4.20 GB | free RAM  20.52 GB | nr_store_dept: predict (2-3. nonrecursive_store_dept_PREDICT.p
y) ... || latest step log nr_store_dept_predict.log: CA_3 FOODS_3 start
    nr_store_dept train finished in 97.6 min (exit 0, 70 models ~1.4 min each); predict running at 21:15
    check 21:26: 2026-10-08 21:22:32 elapsed 192.6 min | PEAK tree RSS 4.20 GB || latest step log nr_store_dept_predict.log: WI_3 FOODS_3 start
  nr_store_cat + nr_store_dept: FINISHED 2026-10-08 21:26 (driver exit OK, total 192.4 min)
    nr_store_cat  train 80.7 min | predict 4.9 min   -> WRMSSE 0.5535, bias 0.973
    nr_store_dept train 97.6 min | predict 9.0 min   -> WRMSSE 0.6064, bias 0.952
    equal-weight average of the 3 finished sets      -> WRMSSE 0.5525, bias 0.970
    peak memory of the run (whole process tree): 4.20 GB; free RAM never below ~16.9 GB. Fixes needed: none
  Updated ESTIMATE for the recursive sets (ESTIMATE): r_store ~2.5-5.5 h, r_store_cat ~1.7-3 h, r_store_dept ~2-4 h
  r_store (--sets r_store): started 2026-10-08 21:26:19, root pid 9656 (cmd wrapper, rc 0), monitor pid 27204; driver log run_logs\driver_r_store.txt
    check 21:41: elapsed    15.0 min | tree RSS   2.15 GB | peak   3.07 GB | free RAM  19.36 GB | r_store: train (1-1. recursive_store_TRAIN.py) ... || latest s
tep log r_store_train.log: [100]        valid_0's rmse: 2.11441
    r_store: CA_1 trained in ~13 min (21:27-21:40) -> train ESTIMATE ~2.2 h for 10 stores; peak 3.07 GB so far
    check 22:21: elapsed    55.0 min | tree RSS   2.36 GB | peak   3.07 GB | free RAM  19.11 GB | r_store: train (1-1. recursive_store_TRAIN.py) ... || latest s
tep log r_store_train.log: [100]        valid_0's rmse: 1.78794
    check 23:06: elapsed   100.1 min | tree RSS   2.41 GB | peak   3.07 GB | free RAM  19.90 GB | r_store: train (1-1. recursive_store_TRAIN.py) ... || latest s
tep log r_store_train.log: [1100]       valid_0's rmse: 1.55692
    check 23:46: elapsed   140.6 min | tree RSS   1.72 GB | peak   3.07 GB | free RAM  20.53 GB | r_store: predict (1-1. recursive_store_PREDICT.py) ... || late
st step log r_store_predict.log: Predict | Day: 5
    r_store train finished in 136.7 min (exit 0); recursive predict running, ~0.87 min per forecast day -> ~25 min ESTIMATE
    check 00:12: 2026-10-09 00:07:54 elapsed 161.6 min | PEAK tree RSS 3.07 GB || latest step log r_store_predict.log: ##########  0.87 min round |  24.40 min t
otal |  49994.63 day sales |
  r_store: FINISHED 2026-10-09 00:08 (driver exit OK, total 161.4 min)
    r_store train 136.7 min (10 models, ~13.7 min each) | predict 24.6 min (recursive, 28 days) -> WRMSSE 0.6835, bias 1.066
    equal-weight average of the 4 finished sets -> WRMSSE 0.5168, bias 0.994
    peak memory of the run (whole process tree): 3.07 GB; free RAM never below ~18 GB. Fixes needed: none
  Updated ESTIMATE: r_store_cat ~1.5-2 h, r_store_dept ~2-2.5 h (train like nr_*_cat/dept + ~25-40 min recursive predict)   ESTIMATE
  r_store_cat (--sets r_store_cat): started 2026-10-09 00:12:21, root pid 13404 (cmd wrapper, rc 0), monitor pid 21500; driver log run_logs\driver_r_store_cat.t
xt
    check 00:52: elapsed    40.0 min | tree RSS   1.07 GB | peak   4.48 GB | free RAM  21.14 GB | r_store_cat: train (1-2. recursive_store_cat_TRAIN.py) ... ||
latest step log r_store_cat_train.log: [400]    valid_0's rmse: 1.79677
    check 01:35: elapsed    82.9 min | tree RSS   1.08 GB | peak   4.77 GB | free RAM  21.07 GB | r_store_cat: train (1-2. recursive_store_cat_TRAIN.py) ... ||
latest step log r_store_cat_train.log: [1300]   valid_0's rmse: 2.08643
    check 02:05: elapsed   113.0 min | tree RSS   2.01 GB | peak   4.77 GB | free RAM  20.18 GB | r_store_cat: predict (1-2. recursive_store_cat_PREDICT.py) ...
|| latest step log r_store_cat_predict.log: ##########  0.99 min round |  27.77 min total |  47150.83 day sales |
  r_store_cat: FINISHED 2026-10-09 02:05 (driver exit OK, total 113.4 min)
    r_store_cat train 85.3 min (30 models) | predict 28.0 min (recursive) -> WRMSSE 0.5802, bias 1.029
    equal-weight average of the 5 finished sets -> WRMSSE 0.5198, bias 1.001
    peak memory of the run (whole process tree): 4.77 GB; free RAM never below ~16 GB. Fixes needed: none
  ESTIMATE r_store_dept: train ~100 min (like nr_store_dept) + recursive predict ~30-40 min = ~2.2-2.5 h   ESTIMATE
  r_store_dept (--sets r_store_dept): started 2026-10-09 02:06:55, root pid 15916 (cmd wrapper, rc 0), monitor pid 26140; driver log run_logs\driver_r_store_dep
t.txt
    check 02:52: elapsed    45.0 min | tree RSS   0.68 GB | peak   4.06 GB | free RAM  21.46 GB | r_store_dept: train (1-3. recursive_store_dept_TRAIN.py) ... |
| latest step log r_store_dept_train.log: [200] valid_0's rmse: 1.52464
    check 03:52: elapsed   105.0 min | tree RSS   0.83 GB | peak   4.79 GB | free RAM  21.27 GB | r_store_dept: train (1-3. recursive_store_dept_TRAIN.py) ... |
| latest step log r_store_dept_train.log: [700] valid_0's rmse: 2.48491
    check 04:32: 2026-10-09 04:29:28 elapsed 142.6 min | PEAK tree RSS 4.79 GB || latest step log r_store_dept_predict.log: ##########  1.24 min round |  34.89
min total |  46354.37 day sales |
  r_store_dept: FINISHED 2026-10-09 04:29 (driver exit OK, total 142.2 min)
    r_store_dept train 106.9 min (70 models) | predict 35.1 min (recursive) -> r_store_dept       0.5526   1.011
    equal-weight average of ALL 6 sets -> average           0.5228   1.003
    peak memory of the run (whole process tree): 4.79 GB; free RAM never below ~16 GB. Fixes needed: none

SUMMARY: ALL 6 SETS FINISHED (nr_store, nr_store_cat, nr_store_dept, r_store, r_store_cat, r_store_dept)
  Measured times (train + predict): nr_store 68.3+3.5 | nr_store_cat 80.7+4.9 | nr_store_dept 97.6+9.0 |
    r_store 136.7+24.6 | r_store_cat 85.3+28.0 | r_store_dept 106.9+35.1 min; preprocessing 5.9 min.
    Total compute ~11.5 h (2026-10-08 16:49 -> 2026-10-09 04:29, including gaps between runs).
  Highest peak memory of any run: 5.15 GB (preprocessing). No package code was changed; no compatibility fix was needed.
  Last updated: 2026-10-09 04:32
```

results/02_winner_reproduction/STATUS_public.txt

```
PC3 PUBLIC STATUS - M5 winner (A1) on the PUBLIC window (train to d_1913, forecast d_1914..d_1941)
Created: 2026-10-12 18:38

SETUP
  Package: A1_public_pkg.zip (52,804 bytes, 2026-10-12 18:36) from the project folder (identical SHA-256 to the copy in Downloads),
           unzipped into C:\A1_public -> C:\A1_public\pkg_public (holds run_winner_a1.py; runs start from there)
  Python: existing venv C:\A1_lean\venv\Scripts\python.exe (nothing new created or installed)
  Monitor: existing C:\A1_lean\monitor_a1.py (only executed; nothing in C:\A1_lean changed)
  Data in C:\A1_public\data (copied, not moved, from C:\A1_lean\m5_data):
  calendar.csv                      112,477 bytes
  sales_test_evaluation.csv       3,260,569 bytes
  sales_train_evaluation.csv    121,168,898 bytes
  sample_submission.csv           5,228,786 bytes
  sell_prices.csv               237,073,689 bytes
  NOTE: sample_submission.csv was added to the four requested files. The predict scripts read it, and the driver's fallback
        that rebuilds it needs an 'id' column that this organiser-layout sales_train_evaluation.csv does not have, so the
        first call would have failed. This is a data copy only; no code or model setting was changed.
  Free RAM at start: 20.7 GB; free disk C: 706.1 GB

ORCHESTRATION
  C:\A1_public\orchestrate_public.py runs the calls one after another, never two at once:
    a_nr_store (--sets nr_store) -> b_nr_store_cat_dept (nr_store_cat,nr_store_dept) -> c_r_store_cat_dept (r_store_cat,r_store_dept) -> d_r_store (r_store)
  all with --window public --data_dir C:\A1_public\data; logs in C:\A1_public\run_logs (driver_<call>.txt, monitor_<call>.txt, step logs).
  (The driver writes its step logs and preprocessing marker to pkg_public\run_logs, its own work folder.)
  Every 20 min a status line (call, elapsed, free RAM, current step) is appended here. On a failed call it stops.
  After each finished call, copies go to <project>\m5_outputs_final as <call>__predval_A1_<set>.npy, <call>__predval_A1_winner.npy,
  <call>__results_winner_A1_public.txt and PC3_PUBLIC_STATUS_after_<call>.txt (a _2 suffix is added if a name already exists).

LOG
2026-10-12 18:38:09 orchestrator start (pid 36264); calls in order: a_nr_store, b_nr_store_cat_dept, c_r_store_cat_dept, d_r_store
2026-10-12 18:38:09 a_nr_store: started (--sets nr_store), driver pid 35344, log run_logs\driver_a_nr_store.txt
2026-10-12 18:58:09 a_nr_store: elapsed 20.0 min | free RAM 19.39 GB | nr_store: train (2-1. nonrecursive_store_TRAIN.py) ...
2026-10-12 19:18:09 a_nr_store: elapsed 40.0 min | free RAM 19.30 GB | nr_store: train (2-1. nonrecursive_store_TRAIN.py) ...
2026-10-12 19:38:10 a_nr_store: elapsed 60.0 min | free RAM 19.12 GB | nr_store: train (2-1. nonrecursive_store_TRAIN.py) ...
2026-10-12 19:58:10 a_nr_store: elapsed 80.0 min | free RAM 19.30 GB | nr_store: train (2-1. nonrecursive_store_TRAIN.py) ...
2026-10-12 20:05:10 a_nr_store: FINISHED (exit 0) in 87.0 min
    nr_store           0.6950   0.920
    average            0.6950   0.920
    copied to m5_outputs_final: a_nr_store__predval_A1_nr_store.npy, a_nr_store__predval_A1_winner.npy, a_nr_store__results_winner_A1_public.txt
2026-10-12 20:05:10 b_nr_store_cat_dept: started (--sets nr_store_cat,nr_store_dept), driver pid 16944, log run_logs\driver_b_nr_store_cat_dept.txt
2026-10-12 20:25:10 b_nr_store_cat_dept: elapsed 20.0 min | free RAM 20.18 GB | nr_store_cat: train (2-2. nonrecursive_store_cat_TRAIN.py) ...
2026-10-12 20:45:10 b_nr_store_cat_dept: elapsed 40.0 min | free RAM 20.36 GB | nr_store_cat: train (2-2. nonrecursive_store_cat_TRAIN.py) ...
2026-10-12 21:05:10 b_nr_store_cat_dept: elapsed 60.0 min | free RAM 18.17 GB | nr_store_cat: train (2-2. nonrecursive_store_cat_TRAIN.py) ...
2026-10-12 21:25:10 b_nr_store_cat_dept: elapsed 80.0 min | free RAM 20.38 GB | nr_store_cat: predict (2-2. nonrecursive_store_cat_PREDICT.py) ...
2026-10-12 21:45:10 b_nr_store_cat_dept: elapsed 100.0 min | free RAM 20.00 GB | nr_store_dept: train (2-3. nonrecursive_store_dept_TRAIN.py) ...
2026-10-12 22:05:10 b_nr_store_cat_dept: elapsed 120.0 min | free RAM 20.42 GB | nr_store_dept: train (2-3. nonrecursive_store_dept_TRAIN.py) ...
2026-10-12 22:25:10 b_nr_store_cat_dept: elapsed 140.0 min | free RAM 20.44 GB | nr_store_dept: train (2-3. nonrecursive_store_dept_TRAIN.py) ...
2026-10-12 22:45:10 b_nr_store_cat_dept: elapsed 160.0 min | free RAM 20.34 GB | nr_store_dept: train (2-3. nonrecursive_store_dept_TRAIN.py) ...
2026-10-12 23:05:10 b_nr_store_cat_dept: elapsed 180.0 min | free RAM 16.89 GB | nr_store_dept: train (2-3. nonrecursive_store_dept_TRAIN.py) ...
2026-10-12 23:18:10 b_nr_store_cat_dept: FINISHED (exit 0) in 193.0 min
    nr_store_cat       0.7678   0.905
    nr_store_dept      0.8629   0.883
    average            0.7717   0.903
    copied to m5_outputs_final: b_nr_store_cat_dept__predval_A1_nr_store_cat.npy, b_nr_store_cat_dept__predval_A1_nr_store_dept.npy, b_nr_store_cat_dept__predva
l_A1_winner.npy, b_nr_store_cat_dept__results_winner_A1_public.txt
2026-10-12 23:18:10 c_r_store_cat_dept: started (--sets r_store_cat,r_store_dept), driver pid 36116, log run_logs\driver_c_r_store_cat_dept.txt
2026-10-12 23:38:10 c_r_store_cat_dept: elapsed 20.0 min | free RAM 20.15 GB | r_store_cat: train (1-2. recursive_store_cat_TRAIN.py) ...
2026-10-12 23:58:11 c_r_store_cat_dept: elapsed 40.0 min | free RAM 19.94 GB | r_store_cat: train (1-2. recursive_store_cat_TRAIN.py) ...
2026-10-13 00:18:11 c_r_store_cat_dept: elapsed 60.0 min | free RAM 19.93 GB | r_store_cat: train (1-2. recursive_store_cat_TRAIN.py) ...
2026-10-13 00:38:11 c_r_store_cat_dept: elapsed 80.0 min | free RAM 20.28 GB | r_store_cat: train (1-2. recursive_store_cat_TRAIN.py) ...
2026-10-13 00:58:11 c_r_store_cat_dept: elapsed 100.0 min | free RAM 19.09 GB | r_store_cat: predict (1-2. recursive_store_cat_PREDICT.py) ...
2026-10-13 01:18:11 c_r_store_cat_dept: elapsed 120.0 min | free RAM 20.31 GB | r_store_dept: train (1-3. recursive_store_dept_TRAIN.py) ...
2026-10-13 01:38:11 c_r_store_cat_dept: elapsed 140.0 min | free RAM 20.28 GB | r_store_dept: train (1-3. recursive_store_dept_TRAIN.py) ...
2026-10-13 01:58:11 c_r_store_cat_dept: elapsed 160.0 min | free RAM 20.42 GB | r_store_dept: train (1-3. recursive_store_dept_TRAIN.py) ...
2026-10-13 02:18:11 c_r_store_cat_dept: elapsed 180.0 min | free RAM 16.15 GB | r_store_dept: train (1-3. recursive_store_dept_TRAIN.py) ...
2026-10-13 02:38:11 c_r_store_cat_dept: elapsed 200.0 min | free RAM 20.14 GB | r_store_dept: train (1-3. recursive_store_dept_TRAIN.py) ...
2026-10-13 02:58:11 c_r_store_cat_dept: elapsed 220.0 min | free RAM 20.30 GB | r_store_dept: train (1-3. recursive_store_dept_TRAIN.py) ...
2026-10-13 03:18:11 c_r_store_cat_dept: elapsed 240.0 min | free RAM 19.00 GB | r_store_dept: predict (1-3. recursive_store_dept_PREDICT.py) ...
2026-10-13 03:37:56 c_r_store_cat_dept: FINISHED (exit 0) in 259.8 min
    r_store_cat        0.5440   0.963
    r_store_dept       0.6184   0.942
    average            0.6869   0.923
    copied to m5_outputs_final: c_r_store_cat_dept__predval_A1_r_store_cat.npy, c_r_store_cat_dept__predval_A1_r_store_dept.npy, c_r_store_cat_dept__predval_A1_
winner.npy, c_r_store_cat_dept__results_winner_A1_public.txt
2026-10-13 03:37:56 d_r_store: started (--sets r_store), driver pid 32544, log run_logs\driver_d_r_store.txt
2026-10-13 03:57:56 d_r_store: elapsed 20.0 min | free RAM 18.51 GB | r_store: train (1-1. recursive_store_TRAIN.py) ...
2026-10-13 04:17:56 d_r_store: elapsed 40.0 min | free RAM 18.66 GB | r_store: train (1-1. recursive_store_TRAIN.py) ...
2026-10-13 04:37:57 d_r_store: elapsed 60.0 min | free RAM 18.25 GB | r_store: train (1-1. recursive_store_TRAIN.py) ...
2026-10-13 04:57:57 d_r_store: elapsed 80.0 min | free RAM 18.52 GB | r_store: train (1-1. recursive_store_TRAIN.py) ...
2026-10-13 05:17:57 d_r_store: elapsed 100.0 min | free RAM 18.24 GB | r_store: train (1-1. recursive_store_TRAIN.py) ...
2026-10-13 05:37:57 d_r_store: elapsed 120.0 min | free RAM 18.45 GB | r_store: train (1-1. recursive_store_TRAIN.py) ...
2026-10-13 05:57:57 d_r_store: elapsed 140.0 min | free RAM 19.02 GB | r_store: predict (1-1. recursive_store_PREDICT.py) ...
2026-10-13 06:11:57 d_r_store: FINISHED (exit 0) in 154.0 min
    r_store            0.4734   0.987
    r_store_cat        0.5440   0.963
    r_store_dept       0.6184   0.942
    average            0.6415   0.933
    copied to m5_outputs_final: d_r_store__predval_A1_r_store.npy, d_r_store__predval_A1_winner.npy, d_r_store__results_winner_A1_public.txt
2026-10-13 06:11:57 orchestrator: ALL FOUR CALLS FINISHED
```

results/02_winner_reproduction/results_winner_A1_cv2.txt

```
WINNER'S CODE (M5 1st place, folder A1) RUN ON AN EARLIER VALIDATION FOLD (cv2)
Trained on days up to d_1829, forecast d_1830 to d_1857 (the winner's own 'cv2' fold; target encodings rebuilt on days up to d_1829).
Settings of the winner's models are unchanged. Compatibility patches are listed in PATCH_LOG.txt.
Sets finished: 6 of 6 (nr_store, nr_store_cat, nr_store_dept, r_store, r_store_cat, r_store_dept). The winner averaged all six with equal weights;
the 'average' row is the equal-weight average of the sets finished so far.
Bias = total forecast / total actual (1.000 is unbiased).

RESULTS (WRMSSE on the holdout, lower is better)
Set                WRMSSE    Bias
nr_store           0.7502   0.918
```

```
nr_store_cat       0.8097   0.903
nr_store_dept      0.8898   0.883
r_store            0.6185   1.034
r_store_cat        0.5966   0.997
r_store_dept       0.6170   0.969
average            0.6477   0.951

WRMSSE BY LEVEL (first three finished sets and the average)
Level Name                    Series   SeasonalNaive        nr_store    nr_store_cat   nr_store_dept         average
-------------------------------------------------------------------------------------------------------------------
1     Total                        1          0.9298          0.6850          0.7801          0.9181          0.4973
2     State                        3          0.9375          0.6732          0.7533          0.8752          0.5182
3     Store                       10          0.9477          0.6919          0.7604          0.8643          0.5662
4     Category                     3          0.9637          0.6575          0.7630          0.8853          0.5098
5     Department                   7          1.0332          0.7005          0.7896          0.8987          0.5755
6     State x Category             9          0.9647          0.6632          0.7420          0.8448          0.5411
7     State x Department          21          1.0229          0.7120          0.7791          0.8619          0.6096
8     Store x Category            30          0.9777          0.7019          0.7622          0.8445          0.6042
9     Store x Department          70          1.0376          0.7611          0.8112          0.8771          0.6787
10    Item                      3049          1.1480          0.9470          0.9561          0.9717          0.9090
11    Item x State              9147          1.1511          0.9213          0.9277          0.9367          0.8943
12    Item x Store             30490          1.1388          0.8876          0.8920          0.8992          0.8691
-------------------------------------------------------------------------------------------------------------------
      OVERALL WRMSSE           42840          1.0211          0.7502          0.8097          0.8898          0.6477

REFERENCE VALUES (published, window to be confirmed)
  Original M5 winner (final window d_1942 to d_1969): 0.520
  Yang et al. Kaggle Top-1 reproduction: 0.5230 | Yang et al. LightGBM + PatchTST: 0.4989
  Our numbers on d_1914 to d_1941 (a different window): E1b L3 0.5422 | E1d non-recursive per-store port 0.6959 raw, 0.5992 calibrated

STEPS: preprocessing: 6.7 min | nr_store train: 64.1 min (exit 0) | nr_store predict: 3.6 min (exit 0) | nr_store_cat train: 71.0 min (exit 0) | nr_store_cat pr
edict: 5.4 min (exit 0) | nr_store_dept train: 93.7 min (exit 0) | nr_store_dept predict: 10.2 min (exit 0) | r_store train: 115.4 min (exit 0) | r_store predic
t: 30.0 min (exit 0) | r_store_cat train: 77.8 min (exit 0) | r_store_cat predict: 33.8 min (exit 0) | r_store_dept train: 103.9 min (exit 0) | r_store_dept pre
dict: 42.6 min (exit 0)
Runtime of this invocation: 658.4 min
```

results/02_winner_reproduction/results_winner_A1_cv3.txt

```
WINNER'S CODE (M5 1st place, folder A1) RUN ON AN EARLIER VALIDATION FOLD (cv3)
Trained on days up to d_1857, forecast d_1858 to d_1885 (the winner's own 'cv3' fold; target encodings rebuilt on days up to d_1857).
Settings of the winner's models are unchanged. Compatibility patches are listed in PATCH_LOG.txt.
Sets finished: 6 of 6 (nr_store, nr_store_cat, nr_store_dept, r_store, r_store_cat, r_store_dept). The winner averaged all six with equal weights;
the 'average' row is the equal-weight average of the sets finished so far.
Bias = total forecast / total actual (1.000 is unbiased).

RESULTS (WRMSSE on the holdout, lower is better)
Set                WRMSSE    Bias
nr_store           0.7297   0.942
nr_store_cat       0.7735   0.924
nr_store_dept      0.8511   0.901
r_store            0.6955   1.050
r_store_cat        0.6629   1.022
r_store_dept       0.6578   0.997
average            0.6562   0.973

WRMSSE BY LEVEL (first three finished sets and the average)
Level Name                    Series   SeasonalNaive        nr_store    nr_store_cat   nr_store_dept         average
-------------------------------------------------------------------------------------------------------------------
1     Total                        1          0.6582          0.6295          0.7223          0.8598          0.5189
2     State                        3          0.7091          0.6521          0.7255          0.8402          0.5628
3     Store                       10          0.8158          0.7229          0.7898          0.8862          0.6339
4     Category                     3          0.7453          0.6229          0.6941          0.8150          0.5215
5     Department                   7          0.8472          0.6302          0.6970          0.8101          0.5330
6     State x Category             9          0.7991          0.6643          0.7108          0.8068          0.5770
7     State x Department          21          0.8816          0.6802          0.7201          0.8067          0.6033
8     Store x Category            30          0.9107          0.7459          0.7826          0.8570          0.6634
9     Store x Department          70          0.9762          0.7674          0.7970          0.8581          0.6980
10    Item                      3049          1.0914          0.8994          0.8998          0.9150          0.8638
11    Item x State              9147          1.1116          0.8831          0.8827          0.8924          0.8575
12    Item x Store             30490          1.1211          0.8589          0.8600          0.8656          0.8419
-------------------------------------------------------------------------------------------------------------------
      OVERALL WRMSSE           42840          0.8889          0.7297          0.7735          0.8511          0.6562

REFERENCE VALUES (published, window to be confirmed)
  Original M5 winner (final window d_1942 to d_1969): 0.520
  Yang et al. Kaggle Top-1 reproduction: 0.5230 | Yang et al. LightGBM + PatchTST: 0.4989
  Our numbers on d_1914 to d_1941 (a different window): E1b L3 0.5422 | E1d non-recursive per-store port 0.6959 raw, 0.5992 calibrated

STEPS: preprocessing: 6.2 min | nr_store train: 64.7 min (exit 0) | nr_store predict: 3.6 min (exit 0) | nr_store_cat train: 71.6 min (exit 0) | nr_store_cat pr
edict: 5.4 min (exit 0) | nr_store_dept train: 94.5 min (exit 0) | nr_store_dept predict: 10.1 min (exit 0) | r_store train: 117.2 min (exit 0) | r_store predic
t: 29.2 min (exit 0) | r_store_cat train: 77.7 min (exit 0) | r_store_cat predict: 33.0 min (exit 0) | r_store_dept train: 104.0 min (exit 0) | r_store_dept pre
dict: 41.7 min (exit 0)
Runtime of this invocation: 658.9 min
```

results/02_winner_reproduction/results_winner_A1_cv4.txt

```
WINNER'S CODE (M5 1st place, folder A1) RUN ON AN EARLIER VALIDATION FOLD (cv4)
Trained on days up to d_1885, forecast d_1886 to d_1913 (the winner's own 'cv4' fold; target encodings rebuilt on days up to d_1885).
Settings of the winner's models are unchanged. Compatibility patches are listed in PATCH_LOG.txt.
Sets finished: 6 of 6 (nr_store, nr_store_cat, nr_store_dept, r_store, r_store_cat, r_store_dept). The winner averaged all six with equal weights;
the 'average' row is the equal-weight average of the sets finished so far.
Bias = total forecast / total actual (1.000 is unbiased).

RESULTS (WRMSSE on the holdout, lower is better)
Set                WRMSSE    Bias
nr_store           0.5496   0.970
nr_store_cat       0.5863   0.958
nr_store_dept      0.6514   0.938
r_store            0.6504   1.073
r_store_cat        0.6083   1.051
r_store_dept       0.5584   1.023
average            0.5167   1.002
```

```
WRMSSE BY LEVEL (first three finished sets and the average)
Level Name                    Series   SeasonalNaive        nr_store    nr_store_cat   nr_store_dept         average
-------------------------------------------------------------------------------------------------------------------
1     Total                        1          0.6677          0.3321          0.4015          0.5285          0.2320
2     State                        3          0.7245          0.3821          0.4254          0.5200          0.3286
3     Store                       10          0.8252          0.4815          0.5039          0.5690          0.4448
4     Category                     3          0.7543          0.3628          0.4402          0.5510          0.3181
5     Department                   7          0.9738          0.4324          0.5066          0.6086          0.4114
6     State x Category             9          0.8347          0.4396          0.4880          0.5622          0.4177
7     State x Department          21          0.9844          0.5019          0.5491          0.6202          0.4858
8     Store x Category            30          0.9205          0.5480          0.5736          0.6243          0.5254
9     Store x Department          70          1.0290          0.6029          0.6265          0.6819          0.5842
10    Item                      3049          1.1143          0.8337          0.8367          0.8527          0.8061
11    Item x State              9147          1.1192          0.8419          0.8454          0.8537          0.8228
12    Item x Store             30490          1.1259          0.8359          0.8383          0.8453          0.8239
-------------------------------------------------------------------------------------------------------------------
      OVERALL WRMSSE           42840          0.9228          0.5496          0.5863          0.6514          0.5167

REFERENCE VALUES (published, window to be confirmed)
  Original M5 winner (final window d_1942 to d_1969): 0.520
  Yang et al. Kaggle Top-1 reproduction: 0.5230 | Yang et al. LightGBM + PatchTST: 0.4989
  Our numbers on d_1914 to d_1941 (a different window): E1b L3 0.5422 | E1d non-recursive per-store port 0.6959 raw, 0.5992 calibrated

STEPS: preprocessing: already done | r_store train: 119.6 min (exit 0) | r_store predict: 30.4 min (exit 0)
Runtime of this invocation: 150.1 min
```

results/02_winner_reproduction/results_winner_A1_cv5.txt

```
WINNER'S CODE (M5 1st place, folder A1) RUN ON AN EARLIER VALIDATION FOLD (cv5)
Trained on days up to d_1801, forecast d_1802 to d_1829 (the winner's own 'cv5' fold; target encodings rebuilt on days up to d_1801).
Settings of the winner's models are unchanged. Compatibility patches are listed in PATCH_LOG.txt.
Sets finished: 6 of 6 (nr_store, nr_store_cat, nr_store_dept, r_store, r_store_cat, r_store_dept). The winner averaged all six with equal weights;
the 'average' row is the equal-weight average of the sets finished so far.
Bias = total forecast / total actual (1.000 is unbiased).

RESULTS (WRMSSE on the holdout, lower is better)
Set               WRMSSE    Bias
nr_store          0.8232   0.884
nr_store_cat      0.9155   0.862
nr_store_dept     0.9757   0.848
r_store           0.5451   1.030
r_store_cat       0.5650   0.975
r_store_dept      0.6142   0.952
average           0.6730   0.925

WRMSSE BY LEVEL (first three finished sets and the average)
Level Name                    Series   SeasonalNaive        nr_store    nr_store_cat   nr_store_dept         average
-------------------------------------------------------------------------------------------------------------------
1     Total                        1          0.9933          0.8309          0.9734          1.0660          0.5700
2     State                        3          1.0305          0.8077          0.9332          1.0159          0.5876
3     Store                       10          1.0483          0.8046          0.9166          0.9896          0.6222
4     Category                     3          1.0721          0.7895          0.9374          1.0180          0.5714
5     Department                   7          1.1279          0.8190          0.9580          1.0556          0.6120
6     State x Category             9          1.1172          0.7808          0.9021          0.9716          0.5964
7     State x Department          21          1.1479          0.8031          0.9149          0.9943          0.6412
8     Store x Category            30          1.1306          0.7981          0.8926          0.9536          0.6505
9     Store x Department          70          1.1566          0.8205          0.9067          0.9700          0.6999
10    Item                      3049          1.1791          0.8978          0.9117          0.9220          0.8531
11    Item x State              9147          1.1746          0.8750          0.8822          0.8895          0.8418
12    Item x Store             30490          1.1668          0.8510          0.8574          0.8630          0.8298
-------------------------------------------------------------------------------------------------------------------
      OVERALL WRMSSE           42840          1.1121          0.8232          0.9155          0.9757          0.6730

REFERENCE VALUES (published, window to be confirmed)
  Original M5 winner (final window d_1942 to d_1969): 0.520
  Yang et al. Kaggle Top-1 reproduction: 0.5230 | Yang et al. LightGBM + PatchTST: 0.4989
  Our numbers on d_1914 to d_1941 (a different window): E1b L3 0.5422 | E1d non-recursive per-store port 0.6959 raw, 0.5992 calibrated

STEPS: preprocessing: 6.7 min | nr_store train: 62.3 min (exit 0) | nr_store predict: 3.6 min (exit 0) | nr_store_cat train: 69.6 min (exit 0) | nr_store_cat pr
edict: 5.5 min (exit 0) | nr_store_dept train: 92.5 min (exit 0) | nr_store_dept predict: 10.3 min (exit 0) | r_store train: 111.3 min (exit 0) | r_store predic
t: 30.1 min (exit 0) | r_store_cat train: 75.9 min (exit 0) | r_store_cat predict: 34.7 min (exit 0) | r_store_dept train: 101.3 min (exit 0) | r_store_dept pre
dict: 43.1 min (exit 0)
Runtime of this invocation: 647.1 min
```

results/02_winner_reproduction/results_winner_A1_final.txt

```
WINNER'S CODE (M5 1st place, folder A1) RUN ON THE ORIGINAL FINAL WINDOW
Trained on days up to d_1941, forecast d_1942 to d_1969, scored on the released actuals (the competition's final window).
Settings of the winner's models are unchanged. Compatibility patches are listed in PATCH_LOG.txt.
Sets finished: 6 of 6 (nr_store, nr_store_cat, nr_store_dept, r_store, r_store_cat, r_store_dept). The winner averaged all six with equal weights;
the 'average' row is the equal-weight average of the sets finished so far.
Bias = total forecast / total actual (1.000 is unbiased).

RESULTS (WRMSSE on the holdout, lower is better)
Set               WRMSSE    Bias
nr_store          0.5282   0.987
nr_store_cat      0.5535   0.973
nr_store_dept     0.6064   0.952
r_store           0.6835   1.066
r_store_cat       0.5802   1.029
r_store_dept      0.5526   1.011
average           0.5228   1.003

WRMSSE BY LEVEL (first three finished sets and the average)
Level Name                    Series   SeasonalNaive        nr_store    nr_store_cat   nr_store_dept         average
-------------------------------------------------------------------------------------------------------------------
1     Total                        1          0.5600          0.2032          0.2609          0.3886          0.2006
2     State                        3          0.6731          0.3117          0.3418          0.4309          0.3118
3     Store                       10          0.7176          0.4139          0.4375          0.5099          0.3994
4     Category                     3          0.6230          0.2603          0.3168          0.4211          0.2839
5     Department                   7          0.7082          0.3766          0.4234          0.4710          0.3720
6     State x Category             9          0.7600          0.3820          0.4073          0.4725          0.3948
7     State x Department          21          0.8292          0.4867          0.5103          0.5424          0.4782
```

```
8     Store x Category            30          0.8008          0.4852          0.5044          0.5555          0.4816
9     Store x Department          70          0.8881          0.5855          0.6028          0.6292          0.5737
10    Item                      3049          1.2233          0.9919          0.9919          0.9995          0.9653
11    Item x State              9147          1.2045          0.9459          0.9473          0.9527          0.9287
12    Item x Store             30490          1.1764          0.8958          0.8981          0.9039          0.8841
-------------------------------------------------------------------------------------------------------------------
      OVERALL WRMSSE           42840          0.8470          0.5282          0.5535          0.6064          0.5228

REFERENCE VALUES (published, window to be confirmed)
  Original M5 winner (final window d_1942 to d_1969): 0.520
  Yang et al. Kaggle Top-1 reproduction: 0.5230 | Yang et al. LightGBM + PatchTST: 0.4989
  Our numbers on d_1914 to d_1941 (a different window): E1b L3 0.5422 | E1d non-recursive per-store port 0.6959 raw, 0.5992 calibrated

STEPS: preprocessing: already done | r_store_dept train: 106.9 min (exit 0) | r_store_dept predict: 35.1 min (exit 0)
Runtime of this invocation: 142.2 min
```

results/02_winner_reproduction/results_winner_A1_public.txt

```
WINNER'S CODE (M5 1st place, folder A1) RUN ON OUR PUBLIC WINDOW
Trained on days up to d_1913, forecast d_1914 to d_1941 (the winner's own 'public' setting).
Settings of the winner's models are unchanged. Compatibility patches are listed in PATCH_LOG.txt.
Sets finished: 6 of 6 (nr_store, nr_store_cat, nr_store_dept, r_store, r_store_cat, r_store_dept). The winner averaged all six with equal weights;
the 'average' row is the equal-weight average of the sets finished so far.
Bias = total forecast / total actual (1.000 is unbiased).

RESULTS (WRMSSE on the holdout, lower is better)
Set               WRMSSE    Bias
nr_store          0.6950   0.920
nr_store_cat      0.7678   0.905
nr_store_dept     0.8629   0.883
r_store           0.4734   0.987
r_store_cat       0.5440   0.963
r_store_dept      0.6184   0.942
average           0.6415   0.933

WRMSSE BY LEVEL (first three finished sets and the average)
Level Name                    Series   SeasonalNaive        nr_store    nr_store_cat   nr_store_dept         average
-------------------------------------------------------------------------------------------------------------------
1     Total                        1          0.7514          0.6408          0.7606          0.9178          0.5513
2     State                        3          0.7544          0.6284          0.7303          0.8710          0.5517
3     Store                       10          0.7993          0.6586          0.7407          0.8613          0.5845
4     Category                     3          0.7620          0.6221          0.7476          0.8896          0.5521
5     Department                   7          0.8171          0.6267          0.7397          0.8742          0.5654
6     State x Category             9          0.7830          0.6278          0.7255          0.8478          0.5664
7     State x Department          21          0.8304          0.6389          0.7252          0.8376          0.5907
8     Store x Category            30          0.8393          0.6694          0.7394          0.8377          0.6092
9     Store x Department          70          0.8919          0.6983          0.7532          0.8445          0.6513
10    Item                      3049          1.0322          0.8471          0.8579          0.8681          0.8230
11    Item x State              9147          1.0750          0.8431          0.8504          0.8574          0.8261
12    Item x Store             30490          1.1004          0.8387          0.8435          0.8482          0.8260
-------------------------------------------------------------------------------------------------------------------
      OVERALL WRMSSE           42840          0.8697          0.6950          0.7678          0.8629          0.6415

REFERENCE VALUES (published, window to be confirmed)
  Original M5 winner (final window d_1942 to d_1969): 0.520
  Yang et al. Kaggle Top-1 reproduction: 0.5230 | Yang et al. LightGBM + PatchTST: 0.4989
  Our numbers on d_1914 to d_1941 (a different window): E1b L3 0.5422 | E1d non-recursive per-store port 0.6959 raw, 0.5992 calibrated

STEPS: preprocessing: already done | r_store train: 128.9 min (exit 0) | r_store predict: 24.7 min (exit 0)
Runtime of this invocation: 153.8 min
```

results/03_chronos2_aggregate/results_aggregate.txt

```
AGGREGATE-LEVEL DIRECT FORECASTS (Chronos-2 zero-shot), WRMSSE per level, lower is better
Origins: {'final': 1941, 'public': 1913, 'cv4': 1885} | context 1024 | days available 1969
Direct rows come from forecasting the summed series; bottomup rows come from the supplied bottom forecasts.
Hypothesis test only; no weights fitted here.

WINDOW: final (forecast d_1942..d_1969)
Level                           bottomup_chronosC         bottomup_patchtstP1         direct_A_univariate         direct_B_covariates
-------------------------------------------------------------------------------------------------------------------------------------
 1 Total                                   0.3678                      0.3752                      0.4299                      0.3510
 2 State                                   0.4404                      0.4630                      0.4885                      0.4453
 3 Store                                   0.5223                      0.5852                      0.5079                      0.4901
 4 Category                                0.4345                      0.4477                      0.4715                      0.3364
 5 Department                              0.5215                      0.5795                      0.5427                      0.4319
 6 State x Category                        0.5281                      0.5583                      0.5714                      0.4949
 7 State x Department                      0.6180                      0.6671                      0.6241                      0.5688
 8 Store x Category                        0.6034                      0.6538                      0.5983                      0.5629
 9 Store x Department                      0.6915                      0.7448                      0.6788                      0.6424
mean of L1..L9                             0.5253                      0.5638                      0.5459                      0.4804

WINDOW: public (forecast d_1914..d_1941)
Level                           bottomup_chronosC         bottomup_patchtstP1         direct_A_univariate         direct_B_covariates
-------------------------------------------------------------------------------------------------------------------------------------
 1 Total                                   0.6135                      0.6298                      0.5539                      0.4643
 2 State                                   0.6122                      0.6767                      0.5654                      0.5218
 3 Store                                   0.6537                      0.7575                      0.6318                      0.5790
 4 Category                                0.6380                      0.6679                      0.5965                      0.5190
 5 Department                              0.7139                      0.7413                      0.6211                      0.5700
 6 State x Category                        0.6458                      0.7197                      0.6200                      0.5728
 7 State x Department                      0.7099                      0.7757                      0.6730                      0.6314
 8 Store x Category                        0.6859                      0.7830                      0.6543                      0.6252
 9 Store x Department                      0.7461                      0.8290                      0.7135                      0.6892
mean of L1..L9                             0.6688                      0.7312                      0.6255                      0.5747

WINDOW: cv4 (forecast d_1886..d_1913)
Level                           bottomup_chronosC         bottomup_patchtstP1         direct_A_univariate         direct_B_covariates
-------------------------------------------------------------------------------------------------------------------------------------
 1 Total                                   0.6358                      0.4796                      0.3563                      0.3182
 2 State                                   0.6541                      0.5556                      0.4122                      0.3955
 3 Store                                   0.6752                      0.6384                      0.4797                      0.4727
 4 Category                                0.6912                      0.5401                      0.4470                      0.3830
```

```
 5 Department                          0.7536                  0.6447                  0.5427                  0.4619
 6 State x Category                    0.7047                  0.6204                  0.5023                  0.4757
 7 State x Department                  0.7507                  0.6983                  0.5894                  0.5610
 8 Store x Category                    0.7302                  0.6980                  0.5697                  0.5443
 9 Store x Department                  0.7663                  0.7546                  0.6433                  0.6165
mean of L1..L9                         0.7069                  0.6255                  0.5047                  0.4699

Runtime 2.1 min
```

results/04_six_window_blend/README_MULTIWINDOW.txt

```
Multi-window check of the blend (winner six-set LightGBM + direct Chronos-2 at levels 1 to 9).
Part 1: python m5_multiwindow.py --data_dir <M5 data, 1969 days> --in_dir inputs --out_dir multiwindow_out --stage forecast
Part 2 (after the PC 3 winner files are in inputs): the same command with --stage score
Output: multiwindow_out/results_multiwindow.txt
Rule fixed in advance: share 0.20, direct Chronos-2 with covariates, mean point, levels 1 to 9. OLS reconciliation is a labelled secondary variant.
```

results/04_six_window_blend/results_multiwindow.txt

```
MULTI-WINDOW CHECK: winner (six-set LightGBM average) vs blend with direct Chronos-2 at levels 1 to 9
Fixed rule: share 0.20, direct Chronos-2 with covariates, mean point, levels 1 to 9 only. Reconciled = same blend, then OLS reconciliation (secondary).
WRMSSE over 12 levels, lower is better. Bootstrap per window: 2000 draws, blocks of 7 days (days only, rough guide). Days in data folder: 1941.

WINDOW cv5 (forecast d_1802..d_1829) | sales growth vs previous 28 days +8.5% | winner bias 0.925
Variant                       WRMSSE   Change    95% interval of change  P(better)
winner alone                  0.6730  +0.0000        [+0.0000, +0.0000]
blend (headline)              0.6545  -0.0185        [-0.0285, -0.0078]       1.00
blend + OLS reconciled        0.6577  -0.0153        [-0.0241, -0.0061]       1.00
blend share 0.10              0.6628  -0.0102        [-0.0153, -0.0048]       1.00
blend share 0.30              0.6483  -0.0247        [-0.0396, -0.0087]       1.00
  OLS solver relative residual (should be tiny): 1.0e-09
  Level                       Winner    Blend  Reconciled
   1 Total                    0.5700   0.5575      0.5522
   2 State                    0.5876   0.5703      0.5728
   3 Store                    0.6222   0.5886      0.6028
   4 Category                 0.5714   0.5487      0.5491
   5 Department               0.6120   0.5871      0.5870
   6 State x Category         0.5964   0.5710      0.5790
   7 State x Department       0.6412   0.6184      0.6211
   8 Store x Category         0.6505   0.6185      0.6288
   9 Store x Department       0.6999   0.6688      0.6757
  10 Item                     0.8531   0.8531      0.8524
  11 Item x State             0.8418   0.8418      0.8416
  12 Item x Store             0.8298   0.8298      0.8296

WINDOW cv2 (forecast d_1830..d_1857) | sales growth vs previous 28 days +7.5% | winner bias 0.951
Variant                       WRMSSE   Change    95% interval of change  P(better)
winner alone                  0.6477  +0.0000        [+0.0000, +0.0000]
blend (headline)              0.6379  -0.0098        [-0.0156, -0.0022]       0.99
blend + OLS reconciled        0.6361  -0.0116        [-0.0169, -0.0047]       1.00
blend share 0.10              0.6417  -0.0061        [-0.0092, -0.0020]       1.00
blend share 0.30              0.6366  -0.0112        [-0.0195, -0.0004]       0.98
  OLS solver relative residual (should be tiny): 9.3e-10
  Level                       Winner    Blend  Reconciled
   1 Total                    0.4973   0.4825      0.4865
   2 State                    0.5182   0.5085      0.5041
   3 Store                    0.5662   0.5588      0.5517
   4 Category                 0.5098   0.4964      0.4960
   5 Department               0.5755   0.5511      0.5553
   6 State x Category         0.5411   0.5324      0.5263
   7 State x Department       0.6096   0.5991      0.5931
   8 Store x Category         0.6042   0.5907      0.5888
   9 Store x Department       0.6787   0.6632      0.6613
  10 Item                     0.9090   0.9090      0.9080
  11 Item x State             0.8943   0.8943      0.8938
  12 Item x Store             0.8691   0.8691      0.8689

WINDOW cv3 (forecast d_1858..d_1885) | sales growth vs previous 28 days +0.0% | winner bias 0.973
Variant                       WRMSSE   Change    95% interval of change  P(better)
winner alone                  0.6562  +0.0000        [+0.0000, +0.0000]
blend (headline)              0.6291  -0.0272        [-0.0538, -0.0030]       1.00
blend + OLS reconciled        0.6281  -0.0282        [-0.0569, -0.0036]       1.00
blend share 0.10              0.6414  -0.0149        [-0.0287, -0.0024]       1.00
blend share 0.30              0.6196  -0.0367        [-0.0751, -0.0020]       0.98
  OLS solver relative residual (should be tiny): 9.9e-10
  Level                       Winner    Blend  Reconciled
   1 Total                    0.5189   0.4625      0.4664
   2 State                    0.5628   0.5188      0.5199
   3 Store                    0.6339   0.5982      0.5960
   4 Category                 0.5215   0.4803      0.4760
   5 Department               0.5330   0.4938      0.4925
   6 State x Category         0.5770   0.5445      0.5425
   7 State x Department       0.6033   0.5747      0.5729
   8 Store x Category         0.6634   0.6384      0.6362
   9 Store x Department       0.6980   0.6745      0.6747
  10 Item                     0.8638   0.8638      0.8618
  11 Item x State             0.8575   0.8575      0.8566
  12 Item x Store             0.8419   0.8419      0.8417

WINDOW cv4 (forecast d_1886..d_1913) | sales growth vs previous 28 days +0.6% | winner bias 1.002
Variant                       WRMSSE   Change    95% interval of change  P(better)
winner alone                  0.5167  +0.0000        [+0.0000, +0.0000]
blend (headline)              0.5151  -0.0016        [-0.0074, +0.0031]       0.72
blend + OLS reconciled        0.5144  -0.0023        [-0.0079, +0.0024]       0.81
blend share 0.10              0.5153  -0.0014        [-0.0043, +0.0009]       0.86
blend share 0.30              0.5162  -0.0005        [-0.0090, +0.0063]       0.53
  OLS solver relative residual (should be tiny): 9.9e-10
  Level                       Winner    Blend  Reconciled
   1 Total                    0.2320   0.2363      0.2370
   2 State                    0.3286   0.3301      0.3304
   3 Store                    0.4448   0.4359      0.4331
   4 Category                 0.3181   0.3205      0.3211
   5 Department               0.4114   0.4102      0.4089
```

```
   6 State x Category        0.4177   0.4158      0.4163
   7 State x Department      0.4858   0.4882      0.4830
   8 Store x Category        0.5254   0.5149      0.5150
   9 Store x Department      0.5842   0.5766      0.5754
  10 Item                    0.8061   0.8061      0.8062
  11 Item x State            0.8228   0.8228      0.8229
  12 Item x Store            0.8239   0.8239      0.8239

WINDOW public (forecast d_1914..d_1941) | sales growth vs previous 28 days +4.1% | winner bias 0.933
Variant                      WRMSSE   Change    95% interval of change  P(better)
winner alone                 0.6415  +0.0000        [+0.0000, +0.0000]
blend (headline)             0.6245  -0.0169        [-0.0280, -0.0085]       1.00
blend + OLS reconciled       0.6229  -0.0186        [-0.0307, -0.0090]       1.00
blend share 0.10             0.6321  -0.0094        [-0.0153, -0.0046]       1.00
blend share 0.30             0.6189  -0.0225        [-0.0389, -0.0118]       1.00
  OLS solver relative residual (should be tiny): 9.6e-10
  Level                      Winner    Blend  Reconciled
   1 Total                   0.5513   0.5118      0.5161
   2 State                   0.5517   0.5211      0.5243
   3 Store                   0.5845   0.5608      0.5614
   4 Category                0.5521   0.5301      0.5188
   5 Department              0.5654   0.5448      0.5397
   6 State x Category        0.5664   0.5458      0.5417
   7 State x Department      0.5907   0.5748      0.5728
   8 Store x Category        0.6092   0.5919      0.5891
   9 Store x Department      0.6513   0.6383      0.6370
  10 Item                    0.8230   0.8230      0.8222
  11 Item x State            0.8261   0.8261      0.8260
  12 Item x Store            0.8260   0.8260      0.8260

WINDOW final (d_1942..d_1969): skipped (data folder too short).

SUMMARY ACROSS 5 WINDOWS (change = variant minus winner alone, negative is better)
Window     Growth   Winner    Blend   Change  Reconciled   Change
cv5          8.5%   0.6730   0.6545  -0.0185      0.6577  -0.0153
cv2          7.5%   0.6477   0.6379  -0.0098      0.6361  -0.0116
cv3          0.0%   0.6562   0.6291  -0.0272      0.6281  -0.0282
cv4          0.6%   0.5167   0.5151  -0.0016      0.5144  -0.0023
public       4.1%   0.6415   0.6245  -0.0169      0.6229  -0.0186
  blend (headline): mean change -0.0148 | windows improved 5 of 5 | sign test p = 0.062 | 95% interval of the mean change [-0.0211, -0.0083] | worst window -0.0
016
  blend + OLS reconciled: mean change -0.0152 | windows improved 5 of 5 | sign test p = 0.062 | 95% interval of the mean change [-0.0215, -0.0087] | worst windo
w -0.0023
  Correlation between window sales growth and blend change: +0.04 (descriptive only)
  Notes: windows do not overlap but share training history, so they are not fully independent. The interval resamples days only.
  The rule was fixed before the final window was scored; reconciliation is a secondary, labelled variant.
```

results/05_foundation_models/fm_t0_log.txt

```
[2026-10-17 12:30:11] model t0, series 30490, days 1969, calendar days 1969
model loaded in 1.6 min
  cv5 (origin d_1801) done in 0.3 min | saved as fm_t0_cv5.npy | level 1 forecast total / previous 28 days actual 1.037
  cv2 (origin d_1829) done in 0.3 min | saved as fm_t0_cv2.npy | level 1 forecast total / previous 28 days actual 1.029
  cv3 (origin d_1857) done in 0.3 min | saved as fm_t0_cv3.npy | level 1 forecast total / previous 28 days actual 0.979
  cv4 (origin d_1885) done in 0.3 min | saved as fm_t0_cv4.npy | level 1 forecast total / previous 28 days actual 1.020
  public (origin d_1913) done in 0.3 min | saved as fm_t0_public.npy | level 1 forecast total / previous 28 days actual 1.002
  final (origin d_1941) done in 0.3 min | saved as fm_t0_final.npy | level 1 forecast total / previous 28 days actual 1.031
done
```

results/05_foundation_models/fm_timesfm3_log.txt

```
[2026-10-17 12:49:41] model timesfm3, series 30490, days 1969, calendar days 1969
model loaded in 2.2 min
  cv5 (origin d_1801) done in 0.4 min | saved as fm_timesfm3_cv5.npy | level 1 forecast total / previous 28 days actual 1.063
  cv2 (origin d_1829) done in 0.4 min | saved as fm_timesfm3_cv2.npy | level 1 forecast total / previous 28 days actual 1.025
  cv3 (origin d_1857) done in 0.4 min | saved as fm_timesfm3_cv3.npy | level 1 forecast total / previous 28 days actual 0.989
  cv4 (origin d_1885) done in 0.4 min | saved as fm_timesfm3_cv4.npy | level 1 forecast total / previous 28 days actual 1.003
  public (origin d_1913) done in 0.4 min | saved as fm_timesfm3_public.npy | level 1 forecast total / previous 28 days actual 0.980
  final (origin d_1941) done in 0.4 min | saved as fm_timesfm3_final.npy | level 1 forecast total / previous 28 days actual 1.031
done
```

results/05_foundation_models/fm_timestral_log.txt

```
[2026-10-17 12:34:00] model timestral, series 30490, days 1969, calendar days 1969
model loaded in 1.4 min
[2026-10-17 12:36:14] model timestral, series 30490, days 1969, calendar days 1969
model loaded in 0.1 min
  cv5 (origin d_1801) done in 0.3 min | saved as fm_timestral_cv5.npy | level 1 forecast total / previous 28 days actual 1.026
  cv2 (origin d_1829) done in 0.3 min | saved as fm_timestral_cv2.npy | level 1 forecast total / previous 28 days actual 1.032
  cv3 (origin d_1857) done in 0.4 min | saved as fm_timestral_cv3.npy | level 1 forecast total / previous 28 days actual 0.999
  cv4 (origin d_1885) done in 0.4 min | saved as fm_timestral_cv4.npy | level 1 forecast total / previous 28 days actual 1.019
  public (origin d_1913) done in 0.4 min | saved as fm_timestral_public.npy | level 1 forecast total / previous 28 days actual 1.005
  final (origin d_1941) done in 0.4 min | saved as fm_timestral_final.npy | level 1 forecast total / previous 28 days actual 1.040
done
```

results/05_foundation_models/fm_tirex2g_log.txt

```
[2026-10-17 12:38:54] model tirex2g, series 30490, days 1969, calendar days 1969
model loaded in 0.8 min
[2026-10-17 12:41:35] model tirex2g, series 30490, days 1969, calendar days 1969
model loaded in 0.1 min
  cv5 (origin d_1801) done in 1.4 min | saved as fm_tirex2g_cv5.npy | level 1 forecast total / previous 28 days actual 1.049
  cv2 (origin d_1829) done in 1.2 min | saved as fm_tirex2g_cv2.npy | level 1 forecast total / previous 28 days actual 1.017
  cv3 (origin d_1857) done in 1.2 min | saved as fm_tirex2g_cv3.npy | level 1 forecast total / previous 28 days actual 1.014
  cv4 (origin d_1885) done in 1.3 min | saved as fm_tirex2g_cv4.npy | level 1 forecast total / previous 28 days actual 1.001
  public (origin d_1913) done in 1.2 min | saved as fm_tirex2g_public.npy | level 1 forecast total / previous 28 days actual 0.991
  final (origin d_1941) done in 1.3 min | saved as fm_tirex2g_final.npy | level 1 forecast total / previous 28 days actual 1.020
```

```
done
```

results/05_foundation_models/results_fm_PC2.txt

```
FOUNDATION MODELS: standalone quality, then blend into the winner
Models found: t0, timesfm3, timestral, tirex2g | Chronos-2 B is the earlier reference. Days in data folder: 1969.
Blend rule fixed in advance: share 0.20, levels 1 to 9 only, levels 10 to 12 stay as the winner. Lower WRMSSE is better.

PART 1: STANDALONE (direct forecasts of levels 1 to 9, scored as in the earlier Chronos-2 table)

WINDOW cv5 (forecast d_1802..d_1829), standalone WRMSSE per level, direct forecasts of the summed series
Level                          winner     Chronos-2 B              t0        timesfm3       timestral         tirex2g
-------------------------------------------------------------------------------------------------------------------
 1 Total                       0.5700          0.5715          0.5170          0.3895          0.6329          0.4639
 2 State                       0.5876          0.5754          0.5474          0.4543          0.5921          0.5688
 3 Store                       0.6222          0.5540          0.5293          0.4833          0.5787          0.5633
 4 Category                    0.5714          0.6087          0.6252          0.5131          0.6715          0.5424
 5 Department                  0.6120          0.6144          0.6053          0.5518          0.6581          0.5812
 6 State x Category            0.5964          0.6352          0.6077          0.5311          0.6505          0.6075
 7 State x Department          0.6412          0.6737          0.6284          0.5888          0.6504          0.6617
 8 Store x Category            0.6505          0.6497          0.6180          0.5798          0.6537          0.6329
 9 Store x Department          0.6999          0.6893          0.6634          0.6358          0.6902          0.6966
mean of L1..L9                 0.6168          0.6191          0.5935          0.5253          0.6420          0.5909

WINDOW cv2 (forecast d_1830..d_1857), standalone WRMSSE per level, direct forecasts of the summed series
Level                          winner     Chronos-2 B              t0        timesfm3       timestral         tirex2g
-------------------------------------------------------------------------------------------------------------------
 1 Total                       0.4973          0.5337          0.5721          0.5302          0.5204          0.5954
 2 State                       0.5182          0.5856          0.6115          0.5563          0.5615          0.6739
 3 Store                       0.5662          0.6370          0.6248          0.5989          0.6050          0.6999
 4 Category                    0.5098          0.5676          0.6554          0.5536          0.5779          0.6443
 5 Department                  0.5755          0.6108          0.6899          0.6142          0.6274          0.6636
 6 State x Category            0.5411          0.6161          0.6538          0.5918          0.6076          0.7172
 7 State x Department          0.6096          0.6852          0.7056          0.6712          0.6908          0.7704
 8 Store x Category            0.6042          0.6571          0.6685          0.6390          0.6493          0.7250
 9 Store x Department          0.6787          0.7240          0.7374          0.7161          0.7189          0.7829
mean of L1..L9                 0.5667          0.6241          0.6577          0.6079          0.6176          0.6970

WINDOW cv3 (forecast d_1858..d_1885), standalone WRMSSE per level, direct forecasts of the summed series
Level                          winner     Chronos-2 B              t0        timesfm3       timestral         tirex2g
-------------------------------------------------------------------------------------------------------------------
 1 Total                       0.5189          0.4424          0.4446          0.3999          0.4307          0.4748
 2 State                       0.5628          0.5041          0.5338          0.4778          0.5013          0.5220
 3 Store                       0.6339          0.6047          0.6100          0.5651          0.5766          0.6299
 4 Category                    0.5215          0.4841          0.5301          0.4635          0.4766          0.5310
 5 Department                  0.5330          0.5247          0.5585          0.5056          0.5170          0.5730
 6 State x Category            0.5770          0.5677          0.5797          0.5371          0.5553          0.5923
 7 State x Department          0.6033          0.6092          0.6125          0.5721          0.5906          0.6323
 8 Store x Category            0.6634          0.6801          0.6687          0.6333          0.6459          0.7039
 9 Store x Department          0.6980          0.7168          0.7060          0.6777          0.6856          0.7354
mean of L1..L9                 0.5902          0.5704          0.5826          0.5369          0.5533          0.5994

WINDOW cv4 (forecast d_1886..d_1913), standalone WRMSSE per level, direct forecasts of the summed series
Level                          winner     Chronos-2 B              t0        timesfm3       timestral         tirex2g
-------------------------------------------------------------------------------------------------------------------
 1 Total                       0.2320          0.3182          0.3786          0.2612          0.3457          0.3101
 2 State                       0.3286          0.3955          0.4187          0.3629          0.4006          0.3970
 3 Store                       0.4448          0.4727          0.4613          0.4256          0.4525          0.4733
 4 Category                    0.3181          0.3830          0.4544          0.3395          0.3687          0.4000
 5 Department                  0.4114          0.4619          0.5031          0.4079          0.4371          0.4691
 6 State x Category            0.4177          0.4757          0.5011          0.4302          0.4630          0.4780
 7 State x Department          0.4858          0.5610          0.5555          0.4975          0.5218          0.5533
 8 Store x Category            0.5254          0.5443          0.5379          0.5004          0.5392          0.5609
 9 Store x Department          0.5842          0.6165          0.5983          0.5603          0.5958          0.6338
mean of L1..L9                 0.4164          0.4699          0.4899          0.4206          0.4583          0.4751

WINDOW public (forecast d_1914..d_1941), standalone WRMSSE per level, direct forecasts of the summed series
Level                          winner     Chronos-2 B              t0        timesfm3       timestral         tirex2g
-------------------------------------------------------------------------------------------------------------------
 1 Total                       0.5513          0.4643          0.5170          0.5798          0.4107          0.6000
 2 State                       0.5517          0.5218          0.5530          0.5694          0.4992          0.5979
 3 Store                       0.5845          0.5790          0.5990          0.5945          0.5481          0.6533
 4 Category                    0.5521          0.5190          0.5392          0.5543          0.4850          0.6240
 5 Department                  0.5654          0.5700          0.5846          0.5854          0.5215          0.6513
 6 State x Category            0.5664          0.5728          0.5756          0.5820          0.5290          0.6277
 7 State x Department          0.5907          0.6314          0.6239          0.6194          0.5720          0.6835
 8 Store x Category            0.6092          0.6252          0.6202          0.6080          0.5819          0.6733
 9 Store x Department          0.6513          0.6892          0.6736          0.6622          0.6431          0.7234
mean of L1..L9                 0.5803          0.5747          0.5873          0.5950          0.5323          0.6483

WINDOW final (forecast d_1942..d_1969), standalone WRMSSE per level, direct forecasts of the summed series
Level                          winner     Chronos-2 B              t0        timesfm3       timestral         tirex2g
-------------------------------------------------------------------------------------------------------------------
 1 Total                       0.2006          0.3510          0.3829          0.3823          0.4615          0.4567
 2 State                       0.3118          0.4453          0.4667          0.4329          0.4354          0.5197
 3 Store                       0.3994          0.4901          0.4899          0.4749          0.4629          0.5392
 4 Category                    0.2839          0.3364          0.4231          0.3974          0.3822          0.4302
 5 Department                  0.3720          0.4319          0.4771          0.4569          0.4536          0.4818
 6 State x Category            0.3948          0.4949          0.5053          0.4861          0.4718          0.5499
 7 State x Department          0.4782          0.5688          0.5616          0.5456          0.5431          0.5936
 8 Store x Category            0.4816          0.5629          0.5510          0.5469          0.5264          0.5935
 9 Store x Department          0.5737          0.6424          0.6228          0.6292          0.6105          0.6570
mean of L1..L9                 0.3884          0.4804          0.4978          0.4836          0.4830          0.5357

PART 2: BLEND INTO THE WINNER (change in 12-level WRMSSE vs the winner alone, negative is better)

Chronos-2 B
  Window     Winner    Blend   Change    95% interval of change  P(better)
  cv5        0.6730   0.6545  -0.0185        [-0.0285, -0.0078]       1.00
  cv2        0.6477   0.6379  -0.0098        [-0.0156, -0.0022]       0.99
  cv3        0.6562   0.6291  -0.0272        [-0.0538, -0.0030]       1.00
  cv4        0.5167   0.5151  -0.0016        [-0.0074, +0.0031]       0.72
  public     0.6415   0.6245  -0.0169        [-0.0280, -0.0085]       1.00
  final      0.5228   0.5195  -0.0033        [-0.0135, +0.0060]       0.74
  Across 6 windows: mean change -0.0129 | improved 6 of 6 | sign test p = 0.031 | 95% interval of the mean change [-0.0182, -0.0072] | worst window -0.0016
```

```
t0
  Window     Winner    Blend   Change    95% interval of change  P(better)
  cv5        0.6730   0.6502  -0.0228        [-0.0342, -0.0111]       1.00
  cv2        0.6477   0.6401  -0.0076        [-0.0190, +0.0044]       0.86
  cv3        0.6562   0.6340  -0.0223        [-0.0502, +0.0013]       0.91
  cv4        0.5167   0.5191  +0.0023        [-0.0026, +0.0077]       0.19
  public     0.6415   0.6293  -0.0122        [-0.0312, +0.0021]       0.95
  final      0.5228   0.5236  +0.0008        [-0.0123, +0.0122]       0.45
  Across 6 windows: mean change -0.0103 | improved 4 of 6 | sign test p = 0.688 | 95% interval of the mean change [-0.0166, -0.0037] | worst window +0.0023

timesfm3
  Window     Winner    Blend   Change    95% interval of change  P(better)
  cv5        0.6730   0.6402  -0.0328        [-0.0415, -0.0242]       1.00
  cv2        0.6477   0.6375  -0.0102        [-0.0164, -0.0030]       1.00
  cv3        0.6562   0.6304  -0.0258        [-0.0552, -0.0024]       1.00
  cv4        0.5167   0.5122  -0.0045        [-0.0080, -0.0014]       1.00
  public     0.6415   0.6345  -0.0070        [-0.0181, +0.0013]       0.93
  final      0.5228   0.5218  -0.0011        [-0.0115, +0.0084]       0.59
  Across 6 windows: mean change -0.0136 | improved 6 of 6 | sign test p = 0.031 | 95% interval of the mean change [-0.0190, -0.0078] | worst window -0.0011

timestral
  Window     Winner    Blend   Change    95% interval of change  P(better)
  cv5        0.6730   0.6563  -0.0167        [-0.0223, -0.0117]       1.00
  cv2        0.6477   0.6400  -0.0077        [-0.0151, +0.0014]       0.94
  cv3        0.6562   0.6341  -0.0221        [-0.0468, +0.0003]       0.97
  cv4        0.5167   0.5160  -0.0007        [-0.0047, +0.0046]       0.63
  public     0.6415   0.6248  -0.0166        [-0.0264, -0.0089]       1.00
  final      0.5228   0.5210  -0.0018        [-0.0118, +0.0059]       0.62
  Across 6 windows: mean change -0.0109 | improved 6 of 6 | sign test p = 0.031 | 95% interval of the mean change [-0.0155, -0.0057] | worst window -0.0007

tirex2g
  Window     Winner    Blend   Change    95% interval of change  P(better)
  cv5        0.6730   0.6511  -0.0219        [-0.0378, -0.0069]       1.00
  cv2        0.6477   0.6459  -0.0019        [-0.0119, +0.0081]       0.67
  cv3        0.6562   0.6316  -0.0246        [-0.0524, -0.0000]       0.98
  cv4        0.5167   0.5149  -0.0018        [-0.0059, +0.0030]       0.79
  public     0.6415   0.6340  -0.0075        [-0.0224, +0.0053]       0.86
  final      0.5228   0.5226  -0.0002        [-0.0105, +0.0113]       0.53
  Across 6 windows: mean change -0.0097 | improved 6 of 6 | sign test p = 0.031 | 95% interval of the mean change [-0.0161, -0.0030] | worst window -0.0002

AVERAGE of Chronos-2 B + all models (extra)
  Window     Winner    Blend   Change    95% interval of change  P(better)
  cv5        0.6730   0.6497  -0.0233        [-0.0322, -0.0134]       1.00
  cv2        0.6477   0.6396  -0.0082        [-0.0150, +0.0000]       0.97
  cv3        0.6562   0.6311  -0.0251        [-0.0522, -0.0021]       1.00
  cv4        0.5167   0.5147  -0.0021        [-0.0055, +0.0009]       0.91
  public     0.6415   0.6288  -0.0127        [-0.0260, -0.0023]       1.00
  final      0.5228   0.5204  -0.0024        [-0.0129, +0.0073]       0.66
  Across 6 windows: mean change -0.0123 | improved 6 of 6 | sign test p = 0.031 | 95% interval of the mean change [-0.0176, -0.0066] | worst window -0.0021

Notes: models are reported separately and none is picked by its result. The AVERAGE row is an extra, not the main claim. Windows share training history, so the
intervals (days resampled in blocks of 7) are a rough guide.
```

results/06_ablation_bootstrap/results_final_analysis.txt

```
ABLATION, UNCERTAINTY AND SECONDARY OUTCOMES FOR THE BLEND (winner + direct Chronos-2 at levels 1 to 9)
Fixed share 0.20 unless stated. WRMSSE over 12 levels, lower is better. Bootstrap: 2000 draws, circular blocks of 7 days, 4 blocks per draw (days only, rough gu
ide).
Days in data folder: 1969.
Note: the bottom-up rows are also mixed at levels 1 to 9 only, so they differ slightly from the earlier all-level mix.

WINDOW cv4 (forecast d_1886..d_1913)
Variant                          WRMSSE   Change    95% interval of change  P(better)
winner alone                     0.5167  +0.0000        [+0.0000, +0.0000]
HEADLINE direct B mean 20%       0.5151  -0.0016        [-0.0075, +0.0030]       0.72
direct B mean 10%                0.5153  -0.0014        [-0.0043, +0.0010]       0.86
direct B mean 30%                0.5162  -0.0005        [-0.0092, +0.0062]       0.53
direct B median 20%              0.5144  -0.0023        [-0.0078, +0.0021]       0.83
direct A (no covariates) 20%     0.5193  +0.0026        [-0.0062, +0.0095]       0.26
bottom-up Chronos-2 C 20%        0.5342  +0.0175        [-0.0010, +0.0299]       0.03
bottom-up PatchTST P1 20%        0.5253  +0.0086        [+0.0004, +0.0146]       0.02
  Winner alone WRMSSE 95% interval [0.4859, 0.5345] | headline blend [0.4799, 0.5359]
  Level 1 bias (total forecast / total actual): winner 1.002 | headline blend 1.004
  Level                     Winner  Headline   Change
   1 Total                  0.2320    0.2363  +0.0043
   2 State                  0.3286    0.3301  +0.0015
   3 Store                  0.4448    0.4359  -0.0089
   4 Category               0.3181    0.3205  +0.0024
   5 Department             0.4114    0.4102  -0.0012
   6 State x Category       0.4177    0.4158  -0.0019
   7 State x Department     0.4858    0.4882  +0.0023
   8 Store x Category       0.5254    0.5149  -0.0104
   9 Store x Department     0.5842    0.5766  -0.0076
  10 Item                   0.8061    0.8061  +0.0000
  11 Item x State           0.8228    0.8228  +0.0000
  12 Item x Store           0.8239    0.8239  +0.0000

WINDOW public (forecast d_1914..d_1941)
Variant                          WRMSSE   Change    95% interval of change  P(better)
winner alone                     0.6415  +0.0000        [+0.0000, +0.0000]
HEADLINE direct B mean 20%       0.6245  -0.0169        [-0.0276, -0.0087]       1.00
direct B mean 10%                0.6321  -0.0094        [-0.0151, -0.0046]       1.00
direct B mean 30%                0.6189  -0.0225        [-0.0376, -0.0119]       1.00
direct B median 20%              0.6260  -0.0155        [-0.0260, -0.0072]       1.00
direct A (no covariates) 20%     0.6325  -0.0089        [-0.0234, +0.0024]       0.92
bottom-up Chronos-2 C 20%        0.6302  -0.0113        [-0.0282, +0.0048]       0.92
bottom-up PatchTST P1 20%        0.6424  +0.0009        [-0.0220, +0.0224]       0.48
  Winner alone WRMSSE 95% interval [0.5963, 0.6779] | headline blend [0.5731, 0.6675]
  Level 1 bias (total forecast / total actual): winner 0.933 | headline blend 0.940
  Level                     Winner  Headline   Change
   1 Total                  0.5513    0.5118  -0.0395
   2 State                  0.5517    0.5211  -0.0306
   3 Store                  0.5845    0.5608  -0.0237
   4 Category               0.5521    0.5301  -0.0220
   5 Department             0.5654    0.5448  -0.0206
   6 State x Category       0.5664    0.5458  -0.0206
```

```
   7 State x Department      0.5907    0.5748  -0.0159
   8 Store x Category        0.6092    0.5919  -0.0172
   9 Store x Department      0.6513    0.6383  -0.0131
  10 Item                    0.8230    0.8230  +0.0000
  11 Item x State            0.8261    0.8261  +0.0000
  12 Item x Store            0.8260    0.8260  +0.0000

WINDOW final (forecast d_1942..d_1969)
Variant                           WRMSSE   Change    95% interval of change  P(better)
winner alone                      0.5228  +0.0000        [+0.0000, +0.0000]
HEADLINE direct B mean 20%        0.5195  -0.0033        [-0.0132, +0.0061]       0.73
direct B mean 10%                 0.5199  -0.0029        [-0.0083, +0.0021]       0.85
direct B mean 30%                 0.5215  -0.0013        [-0.0147, +0.0121]       0.58
direct B median 20%               0.5194  -0.0034        [-0.0129, +0.0059]       0.75
direct A (no covariates) 20%      0.5252  +0.0023        [-0.0071, +0.0100]       0.28
bottom-up Chronos-2 C 20%         0.5133  -0.0096        [-0.0224, +0.0014]       0.95
bottom-up PatchTST P1 20%         0.5174  -0.0055        [-0.0138, +0.0036]       0.89
  Winner alone WRMSSE 95% interval [0.4912, 0.5439] | headline blend [0.4825, 0.5469]
  Share of draws where the headline blend is below the official 0.5204: 0.61 (the official score comes from a different run of the winner's code, so this is a r
ough reference)
  Share of draws where the winner alone is below the official 0.5204: 0.53
  Level 1 bias (total forecast / total actual): winner 1.003 | headline blend 1.008
  Level                     Winner  Headline   Change
   1 Total                  0.2006    0.2009  +0.0003
   2 State                  0.3118    0.3153  +0.0035
   3 Store                  0.3994    0.3958  -0.0036
   4 Category               0.2839    0.2693  -0.0146
   5 Department             0.3720    0.3576  -0.0144
   6 State x Category       0.3948    0.3944  -0.0004
   7 State x Department     0.4782    0.4743  -0.0038
   8 Store x Category       0.4816    0.4789  -0.0028
   9 Store x Department     0.5737    0.5695  -0.0042
  10 Item                   0.9653    0.9653  +0.0000
  11 Item x State           0.9287    0.9287  +0.0000
  12 Item x Store           0.8841    0.8841  +0.0000

STABILITY ACROSS WINDOWS (headline blend minus winner alone, WRMSSE; negative is better)
  cv4      winner 0.5167 | blend 0.5151 | change -0.0016
  public   winner 0.6415 | blend 0.6245 | change -0.0169
  final    winner 0.5228 | blend 0.5195 | change -0.0033
  Windows improved: 3 of 3. Worst-case change: -0.0016.
```

results/07_reconciliation/results_reconciliation_test.txt

```
cv4: winner alone 0.5167 | blend incoherent 0.5151
  OLS         reconciled 0.5144 (change vs blend -0.0007, vs winner -0.0023) [1s]
   levels: 0.237 0.330 0.433 0.321 0.409 0.416 0.483 0.515 0.575 0.806 0.823 0.824
  structural  reconciled 0.5155 (change vs blend +0.0004, vs winner -0.0012) [0s]
   levels: 0.236 0.330 0.436 0.322 0.411 0.418 0.485 0.518 0.578 0.806 0.823 0.824

public: winner alone 0.6415 | blend incoherent 0.6245
  OLS         reconciled 0.6229 (change vs blend -0.0016, vs winner -0.0186) [1s]
   levels: 0.516 0.524 0.561 0.519 0.540 0.542 0.573 0.589 0.637 0.822 0.826 0.826
  structural  reconciled 0.6279 (change vs blend +0.0033, vs winner -0.0136) [0s]
   levels: 0.526 0.532 0.568 0.529 0.546 0.549 0.577 0.594 0.640 0.822 0.826 0.826
exit 0
```

results/08_shap/results_shap.txt

```
SHAP EXPLAINABILITY OF THE WINNER'S NON-RECURSIVE PER-STORE LIGHTGBM MODELS (set nr_store)
Window key 'private' (days d_1942..d_1969). Exact TreeSHAP from LightGBM (pred_contrib). Values are on the log scale of the forecast (Tweedie, log link).
Mean |SHAP| = the average size of a feature's push on the log forecast. Share = that feature's part of the total. 'Effect sign' = correlation of the feature val
ue with its SHAP value (positive: higher value raises the forecast; this is a rough direction, not a causal effect).

CA_1: 4000 rows sampled from d_1942..d_1969, 58 features, base value (log scale) -0.458, mean total |SHAP| 1.522, 1.3 min
TX_1: 4000 rows sampled from d_1942..d_1969, 58 features, base value (log scale) -0.905, mean total |SHAP| 1.587, 1.2 min
WI_1: 4000 rows sampled from d_1942..d_1969, 58 features, base value (log scale) -0.695, mean total |SHAP| 1.549, 1.1 min

FEATURE FAMILIES (share of total mean |SHAP|)
  rolling means and stds         34.8%
  target encodings               25.8%
  calendar, events, SNAP         19.4%
  item and group identity        10.9%
  lags                            4.4%
  price                           4.0%
  other                           0.7%

TOP 25 FEATURES (average over the stores)
  Feature                          Family                     Share %  Gain %  Effect sign
  enc_item_id_state_id_mean        target encodings              18.5    12.5        +0.60
  rolling_mean_30                  rolling means and stds        12.4    28.1        +0.71
  item_id                          item and group identity       10.2    13.2         +nan
  enc_item_id_state_id_std         target encodings               6.8     1.6        +0.55
  tm_dw                            calendar, events, SNAP         6.0     0.9        +0.78
  rolling_mean_14                  rolling means and stds         6.0     6.9        +0.61
  rolling_mean_60                  rolling means and stds         4.2    16.0        +0.77
  tm_y                             calendar, events, SNAP         3.5     0.2         +nan
  rolling_mean_7                   rolling means and stds         2.8     1.8        +0.64
  tm_w_end                         calendar, events, SNAP         2.8     0.4        +0.99
  rolling_std_30                   rolling means and stds         2.2     1.7        +0.37
  rolling_mean_180                 rolling means and stds         2.1     4.3        +0.38
  tm_w                             calendar, events, SNAP         1.8     0.9        +0.07
  rolling_std_180                  rolling means and stds         1.7     1.1        +0.49
  rolling_std_14                   rolling means and stds         1.6     0.5        -0.01
  event_name_1                     calendar, events, SNAP         1.5     0.9         +nan
  price_momentum_m                 price                          1.5     0.6        -0.78
  sales_lag_28                     lags                           1.4     0.6        +0.65
  tm_d                             calendar, events, SNAP         1.3     0.3        -0.59
  tm_m                             calendar, events, SNAP         1.1     0.4        +0.07
  rolling_std_7                    rolling means and stds         0.9     0.4        +0.09
  rolling_std_60                   rolling means and stds         0.9     1.4        +0.34
  snap_TX                          calendar, events, SNAP         0.6     0.0        +0.42
  price_momentum_y                 price                          0.6     0.4        -0.46
```

```
  dept_id                          item and group identity        0.5     0.2         +nan

TOP 10 PER STORE (mean |SHAP|)
  CA_1: enc_item_id_state_id_mean (0.288), rolling_mean_30 (0.178), item_id (0.148), rolling_mean_14 (0.100), tm_dw (0.099), enc_item_id_state_id_std (0.093), r
olling_mean_60 (0.052), tm_w_end (0.047), rolling_mean_7 (0.044), tm_y (0.040)
  TX_1: enc_item_id_state_id_mean (0.352), rolling_mean_30 (0.176), item_id (0.162), enc_item_id_state_id_std (0.135), rolling_mean_60 (0.087), rolling_mean_14
(0.082), tm_dw (0.070), tm_y (0.044), rolling_mean_7 (0.043), tm_w (0.035)
  WI_1: rolling_mean_30 (0.223), enc_item_id_state_id_mean (0.220), item_id (0.164), tm_dw (0.109), rolling_mean_14 (0.096), enc_item_id_state_id_std (0.089), t
m_y (0.081), rolling_mean_60 (0.059), tm_w_end (0.048), rolling_mean_7 (0.043)

Limits: only the nr_store set (one of six); a sample of rows; SHAP explains the model, not the world. Recursive sets are not covered.
```

results/09_levels_10_11/results_lvl1011.txt

```
EXPLORATORY: direct Chronos-2 at the item-level aggregates (level 10 item, level 11 item x state)
Fixed rule: share 0.20 of the direct forecast, same model and settings as levels 1 to 9. Level 12 stays as the winner.
WRMSSE over 12 levels, lower is better. Bootstrap: 2000 draws, blocks of 7 days (days only, rough guide).
Decision uses the five windows before the final one; the final window is shown once and is exploratory.

WINDOW cv5 (forecast d_1802..d_1829)
Variant                        WRMSSE  vs winner  vs HEADLINE    95% interval vs HEADLINE  P(better)
winner alone                   0.6730    +0.0000      +0.0185
HEADLINE levels 1-9            0.6545    -0.0185      +0.0000
HEADLINE + level 10            0.6525    -0.0205      -0.0020          [-0.0022, -0.0017]       1.00
HEADLINE + levels 10, 11       0.6511    -0.0219      -0.0034          [-0.0038, -0.0029]       1.00
  Levels 10 / 11 (WRMSSE): winner 0.8531 / 0.8418 | direct alone 0.8225 / 0.8284 | blend 0.8296 / 0.8247

WINDOW cv2 (forecast d_1830..d_1857)
Variant                        WRMSSE  vs winner  vs HEADLINE    95% interval vs HEADLINE  P(better)
winner alone                   0.6477    +0.0000      +0.0098
HEADLINE levels 1-9            0.6379    -0.0098      +0.0000
HEADLINE + level 10            0.6368    -0.0109      -0.0011          [-0.0014, -0.0009]       1.00
HEADLINE + levels 10, 11       0.6361    -0.0116      -0.0018          [-0.0022, -0.0015]       1.00
  Levels 10 / 11 (WRMSSE): winner 0.9090 / 0.8943 | direct alone 0.9173 / 0.9073 | blend 0.8954 / 0.8860

WINDOW cv3 (forecast d_1858..d_1885)
Variant                        WRMSSE  vs winner  vs HEADLINE    95% interval vs HEADLINE  P(better)
winner alone                   0.6562    +0.0000      +0.0272
HEADLINE levels 1-9            0.6291    -0.0272      +0.0000
HEADLINE + level 10            0.6280    -0.0283      -0.0011          [-0.0015, -0.0004]       1.00
HEADLINE + levels 10, 11       0.6273    -0.0289      -0.0018          [-0.0024, -0.0007]       1.00
  Levels 10 / 11 (WRMSSE): winner 0.8638 / 0.8575 | direct alone 0.8748 / 0.8736 | blend 0.8507 / 0.8496

WINDOW cv4 (forecast d_1886..d_1913)
Variant                        WRMSSE  vs winner  vs HEADLINE    95% interval vs HEADLINE  P(better)
winner alone                   0.5167    +0.0000      +0.0016
HEADLINE levels 1-9            0.5151    -0.0016      +0.0000
HEADLINE + level 10            0.5146    -0.0022      -0.0006          [-0.0007, -0.0004]       1.00
HEADLINE + levels 10, 11       0.5142    -0.0026      -0.0010          [-0.0012, -0.0007]       1.00
  Levels 10 / 11 (WRMSSE): winner 0.8061 / 0.8228 | direct alone 0.8385 / 0.8469 | blend 0.7993 / 0.8182

WINDOW public (forecast d_1914..d_1941)
Variant                        WRMSSE  vs winner  vs HEADLINE    95% interval vs HEADLINE  P(better)
winner alone                   0.6415    +0.0000      +0.0169
HEADLINE levels 1-9            0.6245    -0.0169      +0.0000
HEADLINE + level 10            0.6234    -0.0181      -0.0011          [-0.0014, -0.0008]       1.00
HEADLINE + levels 10, 11       0.6228    -0.0186      -0.0017          [-0.0022, -0.0013]       1.00
  Levels 10 / 11 (WRMSSE): winner 0.8230 / 0.8261 | direct alone 0.8268 / 0.8455 | blend 0.8096 / 0.8191

WINDOW final (forecast d_1942..d_1969)   [exploratory, scored once]
Variant                        WRMSSE  vs winner  vs HEADLINE    95% interval vs HEADLINE  P(better)
winner alone                   0.5228    +0.0000      +0.0033
HEADLINE levels 1-9            0.5195    -0.0033      +0.0000
HEADLINE + level 10            0.5192    -0.0036      -0.0003          [-0.0006, -0.0001]       1.00
HEADLINE + levels 10, 11       0.5190    -0.0038      -0.0005          [-0.0008, -0.0002]       1.00
  Levels 10 / 11 (WRMSSE): winner 0.9653 / 0.9287 | direct alone 1.0105 / 0.9629 | blend 0.9617 / 0.9264

SUMMARY (change = extended variant minus HEADLINE, negative is better)
  five windows before final: cv5, cv2, cv3, cv4, public
    HEADLINE + level 10          mean change -0.0012 | improved 5 of 5 | sign test p = 0.062 | worst -0.0006
    HEADLINE + levels 10, 11     mean change -0.0019 | improved 5 of 5 | sign test p = 0.062 | worst -0.0010
  all windows with results: cv5, cv2, cv3, cv4, public, final
    HEADLINE + level 10          mean change -0.0010 | improved 6 of 6 | sign test p = 0.031 | worst -0.0003
    HEADLINE + levels 10, 11     mean change -0.0017 | improved 6 of 6 | sign test p = 0.031 | worst -0.0005

Notes: windows share training history; intervals resample days only; this is exploratory and was run after
the main result was fixed. If the five-window result is not clearly better, the headline stays unchanged.
```

results/09_levels_10_11/console_forecast_20261019_0600.log

```
Series 30490, days 1969
Loading amazon/chronos-2 cuda
    cv5 level 10: 1500/3049 series, 0.1 min
    cv5 level 10: 3000/3049 series, 0.3 min
    cv5 level 10: 3049/3049 series, 0.3 min
  forecast cv5 level 10 (origin d_1801) done in 0.3 min
    cv5 level 11: 1500/9147 series, 0.1 min
    cv5 level 11: 3000/9147 series, 0.3 min
    cv5 level 11: 4500/9147 series, 0.4 min
    cv5 level 11: 6000/9147 series, 0.5 min
    cv5 level 11: 7500/9147 series, 0.7 min
    cv5 level 11: 9000/9147 series, 0.8 min
    cv5 level 11: 9147/9147 series, 0.8 min
  forecast cv5 level 11 (origin d_1801) done in 0.8 min
    cv2 level 10: 1500/3049 series, 0.1 min
    cv2 level 10: 3000/3049 series, 0.3 min
    cv2 level 10: 3049/3049 series, 0.3 min
  forecast cv2 level 10 (origin d_1829) done in 0.3 min
    cv2 level 11: 1500/9147 series, 0.1 min
    cv2 level 11: 3000/9147 series, 0.3 min
    cv2 level 11: 4500/9147 series, 0.4 min
    cv2 level 11: 6000/9147 series, 0.5 min
    cv2 level 11: 7500/9147 series, 0.7 min
```

```
    cv2 level 11: 9000/9147 series, 0.8 min
    cv2 level 11: 9147/9147 series, 0.8 min
  forecast cv2 level 11 (origin d_1829) done in 0.8 min
    cv3 level 10: 1500/3049 series, 0.1 min
    cv3 level 10: 3000/3049 series, 0.3 min
    cv3 level 10: 3049/3049 series, 0.3 min
  forecast cv3 level 10 (origin d_1857) done in 0.3 min
    cv3 level 11: 1500/9147 series, 0.1 min
    cv3 level 11: 3000/9147 series, 0.3 min
    cv3 level 11: 4500/9147 series, 0.4 min
    cv3 level 11: 6000/9147 series, 0.5 min
    cv3 level 11: 7500/9147 series, 0.7 min
    cv3 level 11: 9000/9147 series, 0.8 min
    cv3 level 11: 9147/9147 series, 0.8 min
  forecast cv3 level 11 (origin d_1857) done in 0.8 min
    cv4 level 10: 1500/3049 series, 0.1 min
    cv4 level 10: 3000/3049 series, 0.3 min
    cv4 level 10: 3049/3049 series, 0.3 min
  forecast cv4 level 10 (origin d_1885) done in 0.3 min
    cv4 level 11: 1500/9147 series, 0.1 min
    cv4 level 11: 3000/9147 series, 0.3 min
    cv4 level 11: 4500/9147 series, 0.4 min
    cv4 level 11: 6000/9147 series, 0.6 min
    cv4 level 11: 7500/9147 series, 0.7 min
    cv4 level 11: 9000/9147 series, 0.8 min
    cv4 level 11: 9147/9147 series, 0.8 min
  forecast cv4 level 11 (origin d_1885) done in 0.8 min
    public level 10: 1500/3049 series, 0.1 min
    public level 10: 3000/3049 series, 0.3 min
    public level 10: 3049/3049 series, 0.3 min
  forecast public level 10 (origin d_1913) done in 0.3 min
    public level 11: 1500/9147 series, 0.1 min
    public level 11: 3000/9147 series, 0.3 min
    public level 11: 4500/9147 series, 0.4 min
    public level 11: 6000/9147 series, 0.6 min
    public level 11: 7500/9147 series, 0.7 min
    public level 11: 9000/9147 series, 0.8 min
    public level 11: 9147/9147 series, 0.8 min
  forecast public level 11 (origin d_1913) done in 0.8 min
    final level 10: 1500/3049 series, 0.1 min
    final level 10: 3000/3049 series, 0.3 min
    final level 10: 3049/3049 series, 0.3 min
  forecast final level 10 (origin d_1941) done in 0.3 min
    final level 11: 1500/9147 series, 0.1 min
    final level 11: 3000/9147 series, 0.3 min
    final level 11: 4500/9147 series, 0.4 min
    final level 11: 6000/9147 series, 0.6 min
    final level 11: 7500/9147 series, 0.7 min
    final level 11: 9000/9147 series, 0.8 min
    final level 11: 9147/9147 series, 0.9 min
  forecast final level 11 (origin d_1941) done in 0.9 min
```

results/09_levels_10_11/console_score_20261019_0600.log

```
Series 30490, days 1969
EXPLORATORY: direct Chronos-2 at the item-level aggregates (level 10 item, level 11 item x state)
Fixed rule: share 0.20 of the direct forecast, same model and settings as levels 1 to 9. Level 12 stays as the winner.
WRMSSE over 12 levels, lower is better. Bootstrap: 2000 draws, blocks of 7 days (days only, rough guide).
Decision uses the five windows before the final one; the final window is shown once and is exploratory.

WINDOW cv5 (forecast d_1802..d_1829)
Variant                      WRMSSE  vs winner  vs HEADLINE    95% interval vs HEADLINE  P(better)
winner alone                 0.6730    +0.0000      +0.0185
HEADLINE levels 1-9          0.6545    -0.0185      +0.0000
HEADLINE + level 10          0.6525    -0.0205      -0.0020          [-0.0022, -0.0017]       1.00
HEADLINE + levels 10, 11     0.6511    -0.0219      -0.0034          [-0.0038, -0.0029]       1.00
  Levels 10 / 11 (WRMSSE): winner 0.8531 / 0.8418 | direct alone 0.8225 / 0.8284 | blend 0.8296 / 0.8247

WINDOW cv2 (forecast d_1830..d_1857)
Variant                      WRMSSE  vs winner  vs HEADLINE    95% interval vs HEADLINE  P(better)
winner alone                 0.6477    +0.0000      +0.0098
HEADLINE levels 1-9          0.6379    -0.0098      +0.0000
HEADLINE + level 10          0.6368    -0.0109      -0.0011          [-0.0014, -0.0009]       1.00
HEADLINE + levels 10, 11     0.6361    -0.0116      -0.0018          [-0.0022, -0.0015]       1.00
  Levels 10 / 11 (WRMSSE): winner 0.9090 / 0.8943 | direct alone 0.9173 / 0.9073 | blend 0.8954 / 0.8860

WINDOW cv3 (forecast d_1858..d_1885)
Variant                      WRMSSE  vs winner  vs HEADLINE    95% interval vs HEADLINE  P(better)
winner alone                 0.6562    +0.0000      +0.0272
HEADLINE levels 1-9          0.6291    -0.0272      +0.0000
HEADLINE + level 10          0.6280    -0.0283      -0.0011          [-0.0015, -0.0004]       1.00
HEADLINE + levels 10, 11     0.6273    -0.0289      -0.0018          [-0.0024, -0.0007]       1.00
  Levels 10 / 11 (WRMSSE): winner 0.8638 / 0.8575 | direct alone 0.8748 / 0.8736 | blend 0.8507 / 0.8496

WINDOW cv4 (forecast d_1886..d_1913)
Variant                      WRMSSE  vs winner  vs HEADLINE    95% interval vs HEADLINE  P(better)
winner alone                 0.5167    +0.0000      +0.0016
HEADLINE levels 1-9          0.5151    -0.0016      +0.0000
HEADLINE + level 10          0.5146    -0.0022      -0.0006          [-0.0007, -0.0004]       1.00
HEADLINE + levels 10, 11     0.5142    -0.0026      -0.0010          [-0.0012, -0.0007]       1.00
  Levels 10 / 11 (WRMSSE): winner 0.8061 / 0.8228 | direct alone 0.8385 / 0.8469 | blend 0.7993 / 0.8182

WINDOW public (forecast d_1914..d_1941)
Variant                      WRMSSE  vs winner  vs HEADLINE    95% interval vs HEADLINE  P(better)
winner alone                 0.6415    +0.0000      +0.0169
HEADLINE levels 1-9          0.6245    -0.0169      +0.0000
HEADLINE + level 10          0.6234    -0.0181      -0.0011          [-0.0014, -0.0008]       1.00
HEADLINE + levels 10, 11     0.6228    -0.0186      -0.0017          [-0.0022, -0.0013]       1.00
  Levels 10 / 11 (WRMSSE): winner 0.8230 / 0.8261 | direct alone 0.8268 / 0.8455 | blend 0.8096 / 0.8191

WINDOW final (forecast d_1942..d_1969)   [exploratory, scored once]
Variant                      WRMSSE  vs winner  vs HEADLINE    95% interval vs HEADLINE  P(better)
winner alone                 0.5228    +0.0000      +0.0033
HEADLINE levels 1-9          0.5195    -0.0033      +0.0000
HEADLINE + level 10          0.5192    -0.0036      -0.0003          [-0.0006, -0.0001]       1.00
HEADLINE + levels 10, 11     0.5190    -0.0038      -0.0005          [-0.0008, -0.0002]       1.00
```

```
  Levels 10 / 11 (WRMSSE): winner 0.9653 / 0.9287 | direct alone 1.0105 / 0.9629 | blend 0.9617 / 0.9264

SUMMARY (change = extended variant minus HEADLINE, negative is better)
  five windows before final: cv5, cv2, cv3, cv4, public
    HEADLINE + level 10          mean change -0.0012 | improved 5 of 5 | sign test p = 0.062 | worst -0.0006
    HEADLINE + levels 10, 11     mean change -0.0019 | improved 5 of 5 | sign test p = 0.062 | worst -0.0010
  all windows with results: cv5, cv2, cv3, cv4, public, final
    HEADLINE + level 10          mean change -0.0010 | improved 6 of 6 | sign test p = 0.031 | worst -0.0003
    HEADLINE + levels 10, 11     mean change -0.0017 | improved 6 of 6 | sign test p = 0.031 | worst -0.0005

Notes: windows share training history; intervals resample days only; this is exploratory and was run after
the main result was fixed. If the five-window result is not clearly better, the headline stays unchanged.
```